



CODE BRIGHT

DESENVOLVIMENTO DE APLICAÇÕES
WEB COM O FRAMEWORK **LARAVEL 4**
PARA INICIANTES



POR DAYLE REES
& LUÍS DALMOLIN

Laravel: Code Bright (PT-BR)

Desenvolvimento de aplicações web com o framework Laravel 4 para iniciantes.

Dayle Rees, Luís Dalmolin, eAntonio Laguna

Esse livro está à venda em <http://leanpub.com/codebright-ptbr>

Essa versão foi publicada em 2014-06-06



This is a [Leanpub](#) book. Leanpub empowers authors and publishers with the Lean Publishing process. [Lean Publishing](#) is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

©2013 - 2014 Dayle Rees

Tweet Sobre Esse Livro!

Por favor ajude Dayle Rees, Luís Dalmolin, e Antonio Laguna a divulgar esse livro no [Twitter](#)!

O tweet sugerido para esse livro é:

Eu acabei de comprar o livro: Code Bright by @daylerees. <http://leanpub.com/codebright-ptbr>
#laravel

A hashtag sugerida para esse livro é [#codebright](#).

Descubra o que as outras pessoas estão falando sobre esse livro clicando nesse link para buscar a hashtag no Twitter:

<https://twitter.com/search?q=#codebright>

Outras Obras Desses Autores

Livros De Dayle Rees

Laravel: Code Happy

Laravel: Code Happy (ES)

Laravel: Code Happy (JP)

Laravel: Code Bright

Code Happy (ITA)

Laravel: Code Bright (ES)

Laravel: Code Bright (SR)

Laravel: Code Bright (JP)

Laravel: Code Bright (HU)

Laravel: Code Bright (IT)

Laravel: Code Bright (TR) Türkçe Çevirisi

Livros De Antonio Laguna

Laravel: Code Happy (ES)

Descubriendo Node.js y Express

Laravel: Code Bright (ES)

Laravel: Code Bright (JP)

Conteúdo

Agradecimentos	i
Errata	ii
Feedback	iii
Traduções	iv
Introdução	v
Iniciando	1
Namespaces	1
JSON	7
Composer	13
Arquitetura	34
O Container	34
Facades	36
Flexibilidade	38
Robustez	39
Começando	40
Requerimentos	40
Instalação	41
Configuração do Servidor Web	43
Estrutura do Projeto	47
Roteamento Básico	53
Roteamento Básico	53
Parâmetros de Rotas	58
Tipos de Respostas	61
Views	62
Dados nas Views	63
Redirecionamentos	65
Respostas Personalizadas	67

CONTEÚDO

Atalhos de Respostas	70
Filtros	73
Filtros Básicos	73
Filtros Múltiplos	77
Parâmetros nos Filtros	78
Classes de Filtros	82
Filtros Globais	83
Filtros Padrões	84
Filtros em Múltiplas Rotas	85
Controllers	87
Criando Controllers	87
Roteamento de Controllers	89
Controllers RESTful	91
Blade	93
Criando Templates	93
Saída PHP	94
Estruturas de Controle	96
Templates	99
Herança de Templates	100
Comentários	107
Roteamento avançado	108
Rotas Nomeadas	108
Rotas Seguras	110
Restrições de Parâmetros	111
Grupos de Rotas	112
Prefixo de Rotas	113
Roteamento de Domínios	114
Geração de URLs	117
A URL Atual	117
Gerando URLs do Framework	119
URLs para Assets	123
Atalhos de Geração	125
Dados da Requisição	128
Recebendo	129
Dados da Requisição Anterior	135
Upload de Arquivos	141
Cookies	149
Formulários	153

CONTEÚDO

Abrindo Formulários	154
Campos de Formulário	159
Botões de Formulário	170
Form Macros	172
Segurança de Formulários	174
Validação	177
Validações Simples	179
Regras de Validação	187
Mensagens de Erro	197
Regras de Validação Personalizadas	205
Mensagens de Validação Customizadas	208
Banco de Dados	212
Abstração	212
Configuração	213
Preparando	219
Schema Builder	220
Criando Tabelas	220
Tipos de Coluna	222
Tipos de Colunas Especiais	233
Modificadores de Colunas	234
Atualizando Tabelas	240
Removendo Tabelas	246
Truques Finais	247
Migrações	251
Conceito Básico	251
Criando Migrações	252
Rodando Migrações	256
Revertendo Migrações	261
Truques de Migrações	262
Eloquent ORM	264
Criando Novos Modelos	267
Lendo Registros Existentes	275
Atualizando Modelos Já Existentes	275
Excluindo Modelos Existentes	278
Consultas com o Eloquent	281
Preparação	281
Eloquent para String	285
Estrutura das Consultas	290

CONTEÚDO

Métodos de Consulta	292
Restrições de Consulta	303
Magic Where Queries	321
Escopo de Consulta	324
Coleções do Eloquent	327
A Classe de Coleções	327
Métodos da Coleção	327
Melhores Práticas	349
Relacionamentos do Eloquent	351
Introdução aos Relacionamentos	352
Implementando as Relações	357
Relacionando e Buscando	362
Monte uma Aplicação 1: Coleção de Jogos de Playstation	367
Vamos Pensar Juntos	367
Hora de Programar!	368
Banco de Dados	370
Controller	373
Routes	374
Views	376
Lógica da Aplicação	383
Relaxe	388
Lição de Casa	389
Autenticação	390
Onde Vai Isto?	405
Códgo Estrutural	407
Código Orientado a Objetos	410
Eventos	416
Conceito	416
Disparando Eventos	417
Observando Eventos	418
Registrando Eventos	421
Eventos Globais	422
Casos de Uso	423
O Container	424
Inversão de Controle	425
Injeção de Dependências	435
Em Breve	440

Agradecimentos

Antes de tudo, eu gostaria de agradecer à minha namorada Emma, não só por estar junto em todas as minhas aventuras nerds, mas também por ter tirado as duas surpreendentes fotos dos pandas vermelhos em ambos os livros! Te amo Emma!

Taylor Otwell, este último ano foi incrível, obrigado por ter me dado a oportunidade de fazer parte do time, e pela sua amizade. Obrigado por fazer um framework que é um verdadeiro prazer de usar, que faz com que o nosso código seja lido como poesia, e por dedicar tanto tempo e paixão em seu desenvolvimento. Eu realmente gostei de trabalhar com você nesta nova versão do Laravel, e espero trabalhar com você novamente em futuros projetos!

Eric Barnes, Phill Sparks, Shawn McCool, Jason Lewis, Ian Landsman, obrigado por todo o suporte com o framework e por serem bons amigos.

Obrigado aos meus parentes, que vem apoiando os meus esforços nerds por mais ou menos vinte e oito anos! Também obrigado por comprar um bilhão de cópias do primeiro livro, ou quase isto, membros da família!

Obrigado a todos que compraram o primeiro livro, Code Happy, e a todos da comunidade Laravel. Sem o apoio de vocês um segundo livro nunca teria acontecido.

Errata

Este pode ser o meu segundo livro e minha escrita pode ter melhorado desde o último livro, mas eu garanto que haverá muitos, muitos erros. Você pode ajudar o livro enviando um email com qualquer erro que você encontrou para luis.nh@gmail.com¹ juntamente com o título da seção.

Os erros serão corrigidos à medida que forem descobertos. Correções serão liberadas nas futuras edições do livro.

¹<mailto:luis.nh@gmail.com>

Feedback

Você também pode enviar qualquer feedback que você tenha sobre o conteúdo do livro ou qualquer outra coisa enviando um email para me@daylerees.com² ou um tweet para @daylerees. Vou me esforçar para responder todos os emails que recebo.

²<mailto:me@daylerees.com>

Traduções

Se você gostaria de traduzir o livro Code Bright para sua língua, por favor envie um email para me@daylerees.com³ com suas intenções. Eu vou oferecer uma divisão de 50/50 dos lucros da cópia traduzida, que terá o mesmo preço que a versão em inglês.

Por favor, tenha em mente que o livro é escrito no formato markdown.

³<mailto:me@daylerees.com>

Introdução

Bem, já se passou um bom tempo desde que eu escrevi um capítulo de livro. Code Happy foi publicado praticamente doze meses atrás e chegou a um total de quase três mil cópias vendidas. Vamos ver se eu ainda lembro como fazer essa coisa de “escrever”.

Se você leu a versão anterior do livro, você já sabe que eu sou primeiramente um desenvolvedor, e depois um escritor. Por isto você não vai encontrar longas frases neste livro. Nada que vá impressionar Shakespeare (além dos erros ortográficos). O que você **vai** encontrar é uma conversa direta e informações simples de entender sobre o framework Laravel. Você também vai se apaixonar. Não aquela paixão ardente, mas um grande entusiasmo com o framework, que não pode ser comparado. Eu gosto de escrever meus livros como se eu estivesse bem na sua frente tendo uma conversa. Na verdade, se você realmente quiser ter uma conversa, venha me procurar no canal do Laravel do IRC.

Agora é a hora dos pequenos parágrafos ‘sobre o autor’. Ninguém realmente quer ler isto, mas não machuca ninguém inflar um pouco o ego, certo?

Meu nome é Dayle Rees (isto já está dito na capa!) e eu sou um desenvolvedor web e um entusiasta de design. Eu venho de uma pequena cidade na costa do País de Gales chamada Aberystwyth. Quando eu escrevi meu último livro ‘Code Happy’, eu trabalhava para a Biblioteca Nacional do País de Gales em Aberystwyth, que é uma das três bibliotecas de direitos autorais do Reino Unido.

Desde então eu me mudei para Cardiff, que é a capital do País de Gales e comecei a trabalhar na BoxUK. A BoxUK é uma empresa de consultoria e desenvolvimento em internet, na qual eu posso trabalhar com um time de desenvolvedores que são apaixonados pelo mundo do desenvolvimento web.

Desenvolvimento web não é somente o meu trabalho, mas também o meu hobby. Eu gosto de achar códigos úteis e interessantes ou belos designs. Eu acredito que as nossas habilidades nos permitem fazer coisas maravilhosamente criativas, e eu amo ver ideias se tornando reais.

Pouco mais de um ano atrás eu comecei a ajudar a comunidade Laravel com pacotes de código, web designs e ajudando da maneira que eu podia. Desde então meu envolvimento vem crescendo. O Laravel é o meu primeiro projeto de código aberto e agora eu sou um membro do time de desenvolvimento do core do framework.

Com o Laravel 4 (codinome Illuminate) meu envolvimento chegou em um novo nível. Eu venho trabalhando juntamente com Taylor Otwell para fazer desta versão o melhor framework que você já usou. Mas não acredite em mim palavra. Comece a usar e nos agradeça depois, quando você não conseguir tirar aquele sorriso do rosto enquanto programa.

O Laravel é um exemplo de como até mesmo uma ferramenta de desenvolvimento pode ser criativa. A sintaxe extremamente bela vinda diretamente da mente genial do Taylor Otwell não pode ser

comparada. Ela nos permite escrever códigos que são lidos como poesia nerd, e nos permite desfrutar das nossas tarefas de codificação.

Então Dayle, o que mudou desde a última versão do framework?

A resposta é simples, porém confusa. É tudo e nada!

O Laravel 4 foi reescrito do zero, permitindo aumentar a flexibilidade e testabilidade junto com o seu bilhão (não inteiramente preciso, não contei elas) de novas funcionalidades. Enquanto o Laravel 3 te dava certa liberdade em relação a como estruturar o seu código, o Laravel 4 permite que hackers vão à loucura e mudem toda a estrutura do framework para atender as suas necessidades.

Quando eu escuto que algo melhorou, eu sempre estou procurando por uma falha. Mas com o Laravel 4 não existe uma. E ele ainda possui a excelente e expressiva sintaxe que você adora; Você vai até acabar descobrindo que você o adora mais ainda!

Então por que você escrever um novo livro?

Code Happy cobriu as versões a partir da 3.0 até a 3.2.x, e com quase três mil cópias vendidas, eu devo ter feito alguma coisa certa. Claro, eu poderia fazer um retrabalho no livro inteiro para adaptá-lo para o Laravel 4. Mas, essa versão do framework é uma reinvenção. Se eu tivesse que atualizar o livro, você perderia todas as informações sobre a versão 3, que eu considero um grande framework. Muitas pessoas possuem grandes projetos baseados no Laravel 3, e eu penso que elas devem ter acesso as informações no Code Happy, se precisarem.

E tem as minhas próprias experiências. Eu aprendi muito sobre como escrever um livro desde que eu terminei o Code Happy. Aprendi sobre erros comuns, que agora eu posso evitar. Eu posso melhorar o que eu já fiz, e espero fazer.

Eu não li o Code Happy! Devo ler ele primeiro?

Você pode se quiser, eu coloquei algumas boas piadas nele. Porém, este livro é para iniciantes, então nós vamos começar desde o básico. Se você já vem utilizando o Laravel, vá em frente e pule para as partes interessantes para ver o que mudou. Se você é novo no framework, então eu sugiro que você continue comigo e leia do início ao fim. Não se preocupe! Eu vou tentar mantê-lo interessante. Logo você vai estar construindo aplicações PHP expressivas e sensacionais com o Laravel.

Quando o livro vai estar pronto?

Assim como o meu livro anterior, este livro é uma publicação em progresso. Isso significa que você vai receber cada capítulo assim que eu escrever. No estado atual o livro pode não estar completo, mas assim que eu for adicionando novos capítulos você vai receber um e-mail e poderá fazer o download das atualizações, de graça.

Eu acho que este método de escrita oferece uma grande flexibilidade. Eu posso ficar relaxado com minha escrita, sabendo que eu posso alterar alguma coisa facilmente se estiver errada. Sem me apressar para cumprir um prazo, eu posso escrever um livro que eu sinta que vai ser de grande qualidade. Eu posso atualizar o livro para novas versões, ou para dar mais foco para informações importantes. Você poderá acessar o conteúdo rapidamente. Isto também me permite publicar o livro juntamente com o lançamento de uma nova versão do framework.

Eu estou sem mais perguntas...

Que bom! Você já deve estar ansioso para começar o seu processo de aprendizado. Comece agora mesmo e já comece a gostar do Laravel. Sinta-se à vontade para me enviar um tweet ou uma mensagem no IRC se você quiser conversar.

Iniciando

Hey, este capítulo não estava no Code Happy!?

Como um grande fã do Code Happy, eu simplesmente não consigo deixar nada escapar de você! Muito bom meu caro leitor.

O Laravel 4 utiliza um grande número de novas tecnologias. Estas tecnologias podem ser facilmente ensinadas sozinhas, e também juntamente com o framework. Com isto em mente eu achei melhor abrir um capítulo sobre estas novas tecnologias para sua experiência de aprendizado.

Desenvolvedores web mais experientes já podem ter visto sobre essas tecnologias, ou talvez até já venham utilizando elas. Sinta-se livre para pular qualquer um dos capítulos iniciais se você quiser. Eu não vou ficar triste. É sério, vai lá. Você não precisa mais de mim.

Se você ainda está lendo eu vou assumir que você é meu amigo. Não é um daqueles traidores que pularam direto para o capítulo sobre rotas.

Vamos começar agora mesmo o nosso primeiro capítulo para falar sobre os namespaces do PHP.

Namespaces

Na versão 5.3 do PHP, uma nova funcionalidade conhecida como namespaces foi adicionada na linguagem. Muitas das linguagens modernas já possuíam essa funcionalidade há algum tempo, mas o PHP ficou um pouco atrasado em relação a elas. E não por acaso, cada nova funcionalidade tem um propósito. Vamos descobrir porque os namespaces do PHP podem beneficiar a nossa aplicação.

No PHP, você não pode ter duas classes que tenham o mesmo nome. Elas precisam ser únicas. O problema com esta restrição é que se você está usando alguma biblioteca de terceiros que tenha uma classe chamada `User`, você não pode criar sua própria classe chamada `User`. E isto é uma pena, porque este é um nome muito conveniente para uma classe, certo?

Os namespaces do PHP nos permitem contornar este problema. E na verdade, nós podemos ter quantas classes chamadas `User` nós quisermos. E não só isso, mas nós também podemos utilizar os namespaces para organizar nossos códigos em pequenos pacotes, ou até mesmo para mostrar a quem pertencem.

Vamos dar uma olhada em uma classe normal. Sim... eu sei que você já usou elas antes. Apenas confie em mim nisto, ok?

Namespace Global

Aqui está uma classe realmente muito simples.


```
1 <?php
2
3 // app/models/Eddard.php
4
5 class Eddard
6 {
7
8 }
```

Não existe nada de especial nela, e se nós quisermos usar ela nós podemos fazer isso.

```
1 <?php
2
3 // app/routes.php
4
5 $edward = new Eddard();
```

Dayle, eu sei um pouco de PHP...

Ok, ok, me desculpe. Basicamente, nós podemos pensar nesta classe como pertencendo ao namespace global. Eu não sei se este é o termo certo para isto, mas parece bem razoável para mim. Essencialmente significa que a classe existe sem nenhum namespace. É simplesmente uma classe normal.

Namespaces simples

Vamos criar uma outra classe além da nossa classe original, a global Eddard.

```
1 <?php
2
3 namespace Stark;
4
5 // app/models/another.php
6
7 class Eddard
8 {
9
10 }
```

Aqui nós temos uma outra classe chamada Eddard, com uma pequena mudança. A adição da diretiva namespace. A linha `namespace Stark;` diz pro PHP que tudo que fizemos é relativo ao namespace Stark. Isso também significa que as classes criadas dentro deste arquivo vão pertencer ao namespace ‘Stark’.

Agora, quando nós tentamos utilizar a classe ‘Eddard’ novamente.

```
1 <?php
2
3 // app/routes.php
4
5 $eddard = new Eddard();
```

Novamente, nós temos uma instância da primeira classe que nós criamos na seção anterior. Não a classe com namespace o 'Stark'. Agora vamos tentar criar uma instância da classe 'Eddard' pertencente ao namespace 'Stark'.

```
1 <?php
2
3 // app/routes.php
4
5 $eddard = new Stark\Eddard();
```

Nós podemos instanciar uma classe pertencente a um namespace, prefixando ela com o nome do namespace, e separando os dois com uma barra invertida (\). Agora nós temos uma instância da classe 'Eddard' pertencente ao namespace 'Stark'. Não somos mágicos!?

É importante saber que os namespaces podem ter quantos níveis de hierarquia eles precisarem. Por exemplo:

```
1 This\Namespace\And\Class\Combination\Is\Silly\But\Works
```

Teoria da Relatividade

Lembra como eu falei que o PHP sempre responde **relativamente** ao namespace atual? Bem, vamos dar uma olhada nisto na prática.

```
1 <?php
2
3 namespace Stark;
4
5 // app/routes.php
6
7 $eddard = new Eddard();
```

Adicionando a diretiva do namespace ao exemplo, nós mudamos a execução do script PHP para o namespace 'Stark'. Agora, nós estamos no mesmo namespace que nós colocamos a classe 'Eddard', e recebemos a classe 'Eddard' pertencente ao namespace 'Stark'. Viu como é tudo relativo?

Agora que nós trocamos o namespace, nós criamos um pequeno problema. Você pode adivinhar o que é? Como nós instanciamos a classe ‘Eddard’ original? A que não pertence a nenhum namespace. Felizmente, o PHP possui um truque para referenciar classes que estão localizadas no namespace global. Nós simplesmente precisamos adicionar uma barra invertida (\).

```
1 <?php
2
3 // app/routes.php
4
5 $edward = new \Eddard();
```

Com a barra invertida (\), o PHP sabe que nós estamos nos referindo a classe ‘Eddard’ pertencente ao namespace global, e cria uma instância desta classe.

Use sua imaginação um pouco, como o Barney te mostrou. Imagine que você possui um outro namespace chamado Tully\Edmure. Agora nós queremos usar esta classe a dentro do namespace Stark. Como podemos fazer isto?

```
1 <?php
2
3 namespace Stark;
4
5 // app/routes.php
6
7 $edmure = new \Tully\Edmure();
```

Novamente, nós precisamos utilizar a barra invertida para nos trazer para o namespace global, antes de instanciar a classe do namespace ‘Tully’.

Isso pode ser cansativo, ficar fazendo referência para outros namespaces com toda a sua hierarquia todas as vezes. Felizmente, existe um belo atalho que nós podemos usar. Vamos ver em ação.

```
1 <?php
2
3 namespace Stark;
4
5 use Tully\Edmure;
6
7 // app/routes.php
8
9 $edmure = new Edmure();
```

Utilizando o `use`, nós trazemos uma classe de outro namespace, para o namespace atual. Nos permitindo instanciar novas classes apenas pelo seu nome. Agora não me pergunte porque neste caso não precisamos usar uma barra invertida, porque eu simplesmente não sei. Esta é a única exceção que eu sei. Desculpe por isso! Você pode prefixar com uma barra se você quiser, mas você não precisa.

Para mascarar essa horrível inconsistência, deixe-me mostrar um outro truque. Nós podemos dar as nossas classes importadas pequenos apelidos. Como os que nós usamos no playground PHP. Deixe-me mostrar para você.

```
1  <?php
2
3  namespace Stark;
4
5  use Tully\Brynden as Blackfish;
6
7  // app/routes.php
8
9  $edmure = new Blackfish();
```

Utilizando a palavra chave `as`, nós damos à nossa classe ‘Tully/Brynden’ o apelido ‘Blackfish’, permitindo que possamos usar o novo apelido para identificar a classe dentro do namespace atual. Belo truque, não é? Isto também é muito útil se você precisa usar duas classes com o mesmo nome dentro do mesmo namespace, por exemplo:

```
1  <?php
2
3  namespace Targaryen;
4
5  use Dothraki\Daenerys as Khaleesi;
6
7  // app/routes.php
8
9  class Daenerys
10 {
11
12 }
13
14 // Targaryen\Daenerys
15 $daenerys = new Daenerys();
16
17 // Dothraki\Daenerys
18 $khaleesi = new Khaleesi();
```

Ao dar à classe ‘Daenerys’, pertencente ao namespace ‘Dothraki’ o apelido de ‘Khaleesi’, nós podemos usar as duas classes ‘Daenerys’ apenas pelo nome. Conveniente, não? O ponto aqui é evitar conflitos, e agrupar os objetos pelos seus propósitos.

Você pode usar o `use` quantas vezes precisar.

```
1 <?php
2
3 namespace Targaryen;
4
5 use Dothraki\Daenerys;
6 use Stark\Eddard;
7 use Lannister\Tyrion;
8 use Snow\Jon as Bastard;
```

Estrutura

Namespaces não são somente para evitar conflitos, nós também podemos usá-los para organização e dizer a quem pertencem. Deixe-me explicar com um outro exemplo.

Vamos dizer que eu quero criar uma biblioteca de código aberto. Eu iria adorar que os outros utilizassem o meu código, ia ser demais! O problema é que eu não quero causar nenhum conflito com os nomes das classes para a pessoa que vai usar o meu código. Isto seria terrivelmente inconveniente. E assim é como eu posso evitar o aborrecimento do indivíduo engajado com open source.

```
1 Dayle\Blog\Content\Post
2 Dayle\Blog\Content\Page
3 Dayle\Blog\Tag
```

Aqui nós utilizamos o meu nome, para mostrar que eu criei o código original, e para separar o meu código do código do código da pessoa que está usando a biblioteca. Dentro do namespace base, eu criei alguns sub-namespaces para organizar a estrutura interna da minha aplicação.

Na seção que fala sobre o Composer, você vai aprender como usar namespaces para simplificar o carregamento das classes. Eu fortemente recomendo que você dê uma olhada nesta seção.

Limitações

Na verdade, eu me sinto um pouco culpado por chamar esta seção de ‘Limitações’. O que eu vou falar agora não é na verdade um erro.

Em outras linguagens, os namespaces são implementados de uma maneira similar. Porém, estas outras linguagens nos oferecem uma funcionalidade adicional ao trabalhar com namespaces.

Em Java, por exemplo, você pode importar todas as classes para o namespace atual utilizando a declaração ‘import’ como um genérico. Em Java, ‘import’ é equivalente ao ‘use’ do PHP, e utiliza pontos para separar os namespaces (ou pacotes). Abaixo temos um exemplo.

```
1 import dayle.blog.*;
```

Isto vai importar todas as classes que estão localizadas dentro do pacote ‘dayle.blog’.

No PHP, você não pode fazer isto. Você precisa importar cada classe separadamente. Desculpe. Na verdade, por que eu estou pedindo desculpas? Reclame com o time de desenvolvimento do PHP. Mas, vá gentilmente. Eles já fizeram muitas coisas boas recentemente.

No entanto, existe um pequeno truque que você pode usar. Imagine que nós temos um namespace e uma estrutura de classes, como a do exemplo anterior.

```
1 Dayle\Blog\Content\Post
2 Dayle\Blog\Content\Page
3 Dayle\Blog\Tag
```

Nós podemos dar um apelido ao namespace, para usar suas classes filhas. Abaixo temos um exemplo.

```
1 <?php
2
3 namespace Baratheon;
4
5 use Dayle\Blog as Cms;
6
7 // app/routes.php
8
9 $post = new Cms\Content\Post;
10 $page = new Cms\Content\Page;
11 $tag = new Cms\Tag;
```

Isto é muito útil se você precisa usar muitas classes do mesmo namespace. Aproveite!

Agora nós vamos aprender sobre Jason. Não, não o Jason Lewis, mas strings JSON. Apenas vire a página e você vai ver o que eu quero dizer!

JSON

O que é JSON?

JSON significa Javascript Object Notation. Foi nomeado desta maneira porque Javascript foi a primeira linguagem a utilizar as vantagens desta linguagem.

Essencialmente, JSON é um método mais humano para armazenar arrays e objetos com valores como strings. É usado primeiramente para transferência de dados, e é muito menos verboso do que outras opções, como XML.

Geralmente é usado quando a parte de front-end da aplicação precisa de alguma informação do back-end sem ter que recarregar a página. Isso é normalmente feito usando Javascript e uma requisição AJAX.

Muitas APIs de software também entregam conteúdo usando este formato. O Twitter é um belo exemplo.

Desde a versão 5.2.0, o PHP é capaz de serializar objetos e arrays para JSON. E isto é uma coisa que eu venho usando mais ou menos um bilhão de vezes e foi uma grande adição para a linguagem.

Se você vem trabalhando com PHP por um tempo, você já pode ter usado o método `serialize()` para representar um objeto PHP como uma string. Você pode então usar o método `unserialize()` para transformar a string em uma nova instância contendo o valor original.

É basicamente para isso que usamos o JSON. Porém, a vantagem é que o JSON pode ser interpretado por uma grande variedade de linguagens, visto que strings geradas com a função `serialize()` do PHP, só podem ser interpretadas pelo próprio PHP. A vantagem adicional é que nós (como humanos, e pandas) podemos ler strings JSON, e as strings geradas pelo PHP método `serialize` parecem lixo.

Chega de história, vamos dar uma olhada em um pouco de JSON.

JSON Syntax

```
1 { "name": "Lushui", "species": "Panda", "diet": "Green Things", "age": 7, "colours": [ "red" \
2  , "brown", "white" ] }
```

Oba, JSON! Ok, quando eu disse que era legível para humanos eu posso ter me esquecido de mencionar alguma coisa. Por padrão JSON é armazenado sem nenhum espaço em branco entre seus valores, o que pode tornar a leitura um pouco difícil.

Isto é normalmente feito para poupar largura de banda quando estamos transferindo dados. Sem todos os espaços em branco, a string JSON fica muito menor e pesa menos bytes para a transferência.

A boa notícia é que JSON não se importa com espaço em branco ou quebras de linhas entre suas chaves e valores. Então vai lá! Adicione todos os espaços em branco que você quiser para deixar o código mais legível.

Claro, nós podemos fazer isso manualmente (não vamos), mas existem muitas ferramentas na web para deixar o JSON mais legível. Eu não vou escolher para você. Vá procurar! Você pode até mesmo achar extensões para o seu navegador que permitem ler respostas JSON de servidores web mais facilmente. Eu recomendo fortemente você procurar uma dessas!

Vamos adicionar um espaço em branco no JSON do exemplo anterior para deixar mais legível. (Honestamente, eu fiz isso manualmente. Não tente fazer isso em casa.)

```
1 {  
2     "name":      "Lushui",  
3     "species":   "Panda",  
4     "diet":      "Green Things",  
5     "age":       7,  
6     "colours":   ["red", "brown", "white"]  
7 }
```

Aha! Agora sim. Nós temos uma string representando o panda vermelho que vive na capa dos meus livros. O Lushui guarda seu conhecimento no Laravel em curiosos olhos.

Como você pode ver no exemplo, nós temos um número de pares chave-valor. Dentro de um dos pares nós temos um array. Honestamente, se você já usou algum Javascript antes você pode estar se perguntando o que mudou aqui. Na verdade, vamos dar uma olhada em como isto seria representado em JavaScript.

```
1 var lushui = {  
2     name:      'Lushui',  
3     species:   'Panda',  
4     diet:      'Green Things',  
5     age:       7,  
6     colours:   ['red', 'brown', 'white']  
7 };
```

Felizmente, você vai perceber muitas similaridades entre o JavaScript e as strings JSON.

A snippet JavaScript está atribuindo um objeto literal para dentro de uma variável, assim:

```
1 var lushui = { .. };
```

Bem, JSON é um formato de transferência de dados, e não uma linguagem. Não tem um conceito de variáveis. Por isto que você não precisa de atribuições na snippet JSON.

O método de representação de um objeto literal é muito parecido. E não é nenhuma coincidência! Como eu falei antes, JSON foi originalmente inventado para se usar com JavaScript.

Nos dois, JSON e JavaScript, objetos são contidos com { duas chaves } e consistem em pares de dados de chave-valor. No JavaScript, as chaves não precisam de aspas porque representam variáveis. Mas nós acabamos de ver que JSON não possui variáveis. Porém, nós também podemos usar strings como chaves, e isto é exatamente o que o JSON faz para contornar este problema.

Você também pode ter percebido que eu usei aspas simples nos valores no JavaScript. Isto é uma armadilha! O comportamento é perfeitamente aceitável com JavaScript, mas as strings em JSON **precisam** estar entre aspas duplas. Tenha isto em mente, jovem panda!

Nos JavaScript e no JSON, os pares de chave-valor precisam estar separados com dois pontos (:), e o conjunto da chave-valor deve estar separado por uma vírgula (,) no final.

O JSON suporta strings e números. Você pode ver que o valor da idade do Lushui's é um número inteiro com o valor 7.

```
1 age: 7,
```

O JSON nos permite usar os seguintes tipos.

- Double
- Float
- String
- Boolean
- Array
- Object
- Null

Valores numéricos são representados sem aspas. Cuidado quando escolher se vai usar ou não aspas em um valor. Por exemplo, CEPs dos Estados Unidos possuem cinco números. Porém, se você não usar aspas em um CEP, então 07702 vai se comportar como um número inteiro e será interpretado como 7702. E isto é um erro que já pegou muitos aventureiros na web.

Valores booleanos são representados com as palavras `true` e `false`, sem aspas, como no próprio PHP. E como eu falei antes, valores do tipo string estão contidos com aspas duplas e **não aspas simples**.

O valor 'null' funciona de uma maneira muito parecida com o PHP, e é representado pela palavra `null`, sem aspas. Isto deve ser fácil para você lembrar.

Nós já vimos os objetos. Da mesma maneira que o principal objeto do JSON, eles são representados com chaves e podem conter todos os tipos de valores dentro.

Os arrays também se parecem muito com os arrays do JavaScript.

```
1 // JavaScript
2 ['red', 'brown', 'white']
3
4
5 ["red", "brown", "white"]
```

Você vai perceber que eu não adicionei nenhum comentário no exemplo do JSON anterior. Isto é porque o JSON não suporta comentários, visto que é usado para transferência de dados. Lembre-se disto!

Como você pode ver, os arrays estão contidos com [colchetes], e contém uma lista de vírgulas (,) separando valores sem índices. De novo a única diferença é que aspas duplas precisam ser usadas em strings dentro do JSON. Você já está ficando chateado comigo falando isso?

Como eu mencionei antes, os valores de um JSON incluem objetos e arrays. Os mais espertos entre os meus leitores (todos vocês) podem ter percebido que o JSON também suporta objetos e arrays dentro um do outro. Vamos dar uma olhada nisto em ação!

```
1 {
2     "an_object": {
3         "an_array_of_objects": [
4             { "The": "secret" },
5             { "is": "that" },
6             { "I": "still" },
7             { "love": "shoes!" }
8         ]
9     }
10 }
```

Ok, respire fundo. Aqui nós temos um objeto JSON, que contém um objeto, que contém um array de objetos. Isto é perfeitamente normal e aceitável, e vai permitir que o JSON represente coleções complexas de dados.

JSON e PHP

Como eu falei antes, desde a versão 5.2.0 o PHP tem suporte para serializar e deserializar dados para e do formato JSON. Vamos dar uma olhada misto em ação.

Serializando um array PHP para JSON

Para serializar um valor em PHP, nós apenas precisamos utilizar o método `json_encode()`. Assim:

```
1 <?php
2
3 $truth = array('panda' => 'Awesome!');
4 echo json_encode($truth);
```

O resultado deste código var ser uma string JSON contendo o seguinte valor.

```
1 { "panda": "Awesome!" }
```

Ótimo! Agora vamos nos certificar que nós conseguimos converter de volta estes dados para um formato que possa ser entendido pelo PHP.

Deserializar um array PHP de uma string JSON

Aqui, nós vamos utilizar o método `json_decode()`. Eu aposto que você não viu esta vindo?

```
1 <?php
2
3 $truth = json_decode('{"panda":"Awesome!"}');
4 echo $truth['panda'];
```

Demais, nós vamos... ops, o quê?

```
1 Fatal error: Cannot use object of type stdClass as array in ...
```

Seguinte, o método `json_decode()` não retorna o nosso JSON como um array PHP; Ele utiliza um objeto do tipo `stdClass` para representar os nossos dados. Vamos acessar a chave do nosso objeto como um atributo de um objeto.

```
1 <?php
2
3 $truth = json_decode('{"panda":"Awesome!"}');
4 echo $truth->panda;
5
6 // Awesome!
```

Agora sim! Era isso que nós queríamos. Se nós quisermos um array, o PHP oferece muitas maneiras de converter um objeto em um array, mas felizmente o `json_decode` possui um outro truque debaixo da manga!

Se você adicionar um segundo parâmetro, com o valor `true`, nós vamos receber o nosso array em PHP exatamente como nós queríamos. Obrigado `json_decode()`!

```
1 <?php
2
3 $truth = json_decode('{"panda":"Awesome!"}', true);
4 echo $truth['panda'];
5
6 // Awesome!
```

Agora sim!

Você deve estar se perguntando porque eu escrevi um capítulo gigante sobre JSON dentro de um livro sobre Laravel. E além disto, você provavelmente deve estar se perguntando por que eu escolhi responder essa pergunta no final do capítulo!?

É simplesmente mais divertido assim.

Na próxima seção nós vamos dar uma olhada em um novo gerenciador de pacotes para o PHP. Quando nós começarmos a ver o Composer você vai perceber porque o conhecimento em JSON é tão importante.

Composer

O Composer é uma coisa muito especial no mundo do PHP. Ele mudou a maneira com que nós gerenciemos as dependências das nossas aplicações, e reprimiu as lágrimas de muitos desenvolvedores PHP.

Nos velhos tempos, quando você queria construir uma aplicação que se baseava em dependências de terceiros, você precisava instalar elas com PEAR ou PECL. Esses dois gerenciadores de dependências possuem pacotes desatualizados e vem sendo um espinho na coluna dos desenvolvedores PHP por um bom tempo.

Quando um pacote é finalmente disponibilizado, você pode fazer o download de uma versão específica, e ela vai ser instalada no seu sistema. Porém, a dependência é instalada globalmente, e não apenas em seu projeto. Isto significa que se você tiver duas aplicações que precisam de versões diferentes da mesma dependência... Bem, você vai ter algumas complicações.

Então veio o Composer, o rei dos gerenciadores de pacotes. Primeiro vamos pensar sobre pacotes, o que são eles?

Primeiro de tudo, vamos esquecer o conceito de ‘aplicações’ e ‘projetos’ por enquanto. A ferramenta que você está construindo é um pacote. Imagine uma pequena caixa contendo tudo o que você precisa para rodar a sua aplicação, e de um nome para ela.

Esta caixa precisa apenas um pedaço de papel (um arquivo) dentro dela para ser registrada como um pacote.

Configuração

Você aprendeu sobre JSON no último capítulo, certo? Então você está preparado para isto! Lembra que eu falei que JSON é usado para transferência de dados entre aplicações web? Bem, eu menti. Não é que eu seja desagradável, apenas fica mais fácil para ensinar sobre determinado tópico com um escopo menor sobre suas capacidades. Eu faço bastante isso, espere bastantes mentiras de minha parte.

Você lembra como o JSON representa um número complexo de informações? Bem, e por que nós não podemos usar isto dentro de arquivos de texto para fornecer configurações? E isso é exatamente o que os criadores do Composer pensaram. Quem somos nós para discutir com eles?

Arquivos JSON utilizam a extensão `.json`. O Composer espera que o arquivo de configuração esteja na raiz da aplicação, com o nome de `composer.json`. Lembre-se disto! O Laravel vai utilizar bastante este arquivo.

Vamos abrir este arquivo e começar a digitar algumas informações sobre o nosso pacote.

```
1 {
2     "name":          "marvel/xmen",
3     "description":   "Mutants saving the world for people who hate them.",
4     "keywords":      ["mutant", "superhero", "bald", "guy"],
5     "homepage":      "http://marvel.com/xmen",
6     "time":          "1963-09-01",
7     "license":       "MIT",
8     "authors": [
9         {
10            "name":      "Stan Lee",
11            "email":      "stan@marvel.com",
12            "homepage":   "http://marvel.com",
13            "role":       "Genius"
14        }
15    ]
16 }
```

Certo, aqui nós temos o arquivo `composer.json` na raiz do pacote X-Men. Por que os x-Men? Porque eles são demais.

Verdade seja dita, todas as **opções** (chaves) neste arquivo são opcionais. Normalmente você vai fornecer as informações acima se você pretende distribuir o seu pacote.

Para ser bem honesto com você, eu normalmente preencho estas informações de qualquer maneira. Não vai prejudicar. A configuração acima é usada para identificar o pacote. Eu escondi algumas chaves que eu senti que estão reservadas para circunstâncias especiais. Se você está curioso sobre qualquer configuração adicional, eu recomendo olhar [o site do Composer](http://getcomposer.org/)⁴ que contém uma grande riqueza de informações e documentações.

Eu também encontrei [este belo guia](http://composer.json.jolicode.com/)⁵ online, que pode ser bem útil para novatos no Composer que estão criando novos pacotes. É só passar o mouse em cada linha para ver mais informações sobre cada item de configuração.

De qualquer maneira, vamos dar uma olhada mais de perto nas configurações que nós criamos para o pacote X-Men.

```
1 "name": "marvel/xmen",
```

Este é o nome do pacote. Se você já usou o [Github](http://github.com)⁶, então o formato do nome vai ser bem familiar para você, mas eu vou explicar mesmo assim.

⁴<http://getcomposer.org/>

⁵<http://composer.json.jolicode.com/>

⁶<http://github.com>

O nome do pacote consiste em duas palavras separadas por uma barra (/). A parte antes da barra representa o dono do pacote. Na maioria das circunstâncias os desenvolvedores tendem a utilizar os seus nomes de usuário do Github, e eu concordo completamente com esta abordagem. Você pode, porém, usar qualquer nome que você queira. Apenas tenha certeza de manter uma consistência entre todos os pacotes que pertencem a você.

A segunda parte da string é o nome do pacote. Mantenha simples e descritivo. Novamente, muitos desenvolvedores escolhem usar o nome do repositório para o pacote hospedado no Github, e novamente eu concordo com esta abordagem.

```
1 "description": "Mutants saving the world for people who hate them.",
```

Fornece uma breve descrição sobre a funcionalidade do pacote, Lembre-se, mantenha simples. Se o pacote é destinado para código aberto, então você poder criar descrições mais detalhadas dentro do arquivo README do seu pacote. Se você quiser manter alguma documentação, este não é o lugar para isto. Talvez você possa tatuar nas suas costas, e levar sempre um espelho na mão no bolso. Faz mais sentido para mim. Post-its também funcionam muito bem.

```
1 "keywords": ["mutant", "superhero", "bald", "guy"],
```

Estas palavras chaves são um array de strings usadas para representar o seu pacote. Elas são similares as tags dentro de um blog, e servem essencialmente para o mesmo propósito. As tags vão prover informações úteis de busca para quando seu pacote for listado dentro de um repositório.

```
1 "homepage": "http://marvel.com/xmen",
```

A configuração 'homepage' é útil para pacotes destinados a serem de código aberto. Você pode usar a página do seu projeto, ou talvez o repositório do Github. O que você achar que é mais informativo.

Novamente eu devo lembrá-lo que todas essas opções de configuração são opcionais. Sinta-se à vontade para não preenchê-las se elas não fizerem sentido para o seu pacote.

```
1 "time": "1963-09-01",
```

Esta opção eu não vejo muito. De acordo com a documentação, representa a data de publicação da sua aplicação ou biblioteca. Eu imagino que não seja obrigatório na maioria das circunstâncias justamente porque a maioria dos pacotes estão hospedados no Github, ou em algum outro site de controle de versões. E estes sites normalmente já possuem datas em cada commit, tag ou evento.

Os formatos aceitos para a configuração do tempo são YYYY-MM-DD e YYYY-MM-DD HH:MM:SS. Vá em frente e forneça estes valores se você achar que deve!

```
1  "license": "MIT",
```

Se o seu pacote é destinado para ser distribuído, então você vai querer fornecer uma licença com ele. Sem uma licença, muitos desenvolvedores não poderão usar o seu pacote por questões de restrições legais. Escolha uma licença que se encaixe em suas necessidades, mas que não seja muito restritiva para aqueles que querem usar o seu código. O projeto Laravel utiliza a licença MIT, que fornece uma grande liberdade.

Muitas licenças exigem que você mantenha uma cópia da licença juntamente com o código fonte do repositório, mas se você fornecer esta configuração dentro do arquivo `composer.json`, então o repositório do pacote vai ser capaz de listar o pacote pela sua licença.

A seção dos autores nas configurações fornece informações sobre os criadores do pacote, e pode ser útil para usuários do pacote que queiram fazer contato com eles.

Note que a seção dos autores permite um array de autores para pacotes colaborativos. Vamos dar uma olhada nas opções passadas.

```
1  "authors": [  
2      {  
3          "name":      "Stan Lee",  
4          "email":     "stan@marvel.com",  
5          "homepage":  "http://marvel.com",  
6          "role":      "Genius"  
7      }  
8  ]
```

Utiliza um objeto para representar cada autor. Nosso exemplo tem apenas um autor. Vamos dar uma olhada no Stan Lee. Ele não só tem uma pequena participação em cada filme da Marvel, como também conseguiu tempo para estar no meu livro. Que velho atrevido!

```
1  "name": "Stan Lee",
```

Eu não sei realmente como simplificar esta linha. Se você está tendo problemas para entender ela talvez você deva considerar fechar este livro, e ao invés disto buscar uma carreira com fantoches de meia.

```
1  "email": "stan@marvel.com",
```

Tenha certeza de fornecer um email válido para que você possa ser contatado se o pacote estiver com erros.

```
1 "homepage": "http://marvel.com",
```

Aqui o seu site pode ser fornecido, vá em frente e ganhe alguns acessos!

```
1 "role": "Genius"
```

A opção 'role' define o papel do autor no projeto. Por exemplo, desenvolvedor, designer, ou até mesmo um artista de fantoches de meia. Se você não consegue pensar em algo preciso, coloque algo engraçado.

Isso é tudo o que você precisa para descrever o seu pacote, agora vamos dar uma olhada em algo mais interessante. Gerenciamento de dependências!

Gerenciamento de dependências

Você tem uma caixa que contém o X-Men. Só que não existem mutantes nesta caixa ainda. Pra montar um grande time de super-heróis (aplicação) você vai precisar registrar outros mutantes (dependências de terceiros). Vamos dar uma olhada em como o Composer vai nos ajudar a fazer isto.

```
1 {
2     "name":          "marvel/xmen",
3     "description":   "Mutants saving the world for people who hate them."
4     "keywords":      ["mutant", "superhero", "bald", "guy"],
5     "homepage":      "http://marvel.com/xmen",
6     "time":          "1963-09-01",
7     "license":       "MIT",
8     "authors": [
9         {
10            "name":      "Stan Lee",
11            "email":      "stan@marvel.com",
12            "homepage":   "http://marvel.com",
13            "role":       "Genius"
14        }
15    ],
16    "require": {
17
18    }
19 }
```

Agora nós temos uma nova seção dentro do nosso arquivo `composer.json` chamada 'require'. Ela é usada para listar as nossas dependênc... mutantes. De agra em diante eu vou omitir o resto das

configurações, e apenas mostrar o bloco ‘require’ com pequenos exemplos. Tenha certeza que você sabe onde ele realmente fica!

Nós sabemos que os X-Men dependem de:

- Wolverine
- Cyclops
- Storm
- Gambit

Existem muitos outros, mas estes caras são demais. Nós vamos usar eles por enquanto. Assim, nós podemos copiar os arquivos destes personagens para dentro de nossa aplicação diretamente, mas assim nós teríamos que atualizá-los manualmente com qualquer alteração. E isso pode ficar muito chato. Vamos adicioná-los no bloco `require` para que o Composer gerencie eles para nós.

```
1 "require": {  
2     "xmen/wolverine": "1.0.0",  
3     "xmen/cyclops": "1.0.1",  
4     "xmen/storm": "1.2.0",  
5     "xmen/gambit": "1.0.0"  
6 }
```

Aqui nós estamos listando os pacotes para as nossas dependências mutantes, e as versões que gostaríamos de usar. Neste exemplo, eles todos pertencem ao mesmo dono, o pacote X-Men, mas eles poderiam facilmente ser de outra pessoa.

*Muitos pacotes que são distribuídos estão hospedados em algum site de controle de versões como o [Github](http://github.com)⁷ ou o [Bitbucket](http://bitbucket.org)⁸. Repositórios de controle de versão oferecem um sistema de tags onde nós podemos definir versões estáveis para nossa aplicação. Por exemplo com o git nós podemos usar esse comando:

```
1 git tag -a 1.0.0 -m 'First version.'
```

Com isto nós criamos a versão 1.0.0 para nossa aplicação. Isto é uma distribuição estável em que as pessoas podem depender.

Vamos dar uma olhada na dependência do Gambit.

⁷<http://github.com>

⁸<http://bitbucket.org>

```
1 "xmen/gambit": "1.0.0"
```

Você também deve saber agora que os nomes dos pacotes do Composer consistem em um dono e um apelido separados por uma barra (/). Com essa informação nós sabemos que este é o pacote `gambit` escrito pelo usuário `xmen`.

Dentro do bloco `require`, a chave para cada item é o seu nome, e o valor representa a versão desejada.

No caso do `Gambit`, o número da versão combina com a tag disponível no repositório do Github onde o código é versionado. Você vê como as versões das dependências são específicas da nossa aplicação, e não de todo o sistema?

Você pode adicionar quantas dependências você quiser no seu projeto. Vá em frente, adicione um bilhão! Prove que estou errado.

Escute, você quer saber um segredo? Você promete não contar? Bem, vamos lá. Mais perto, deixe-me sussurrar no seu ouvido. Diga as palavras que você deseja ouvir...

Suas dependências podem ter suas próprias dependências.

Isso mesmo! Suas dependências também são pacotes do Composer. Elas tem seu próprio arquivo `composer.json`. Isso significa que elas tem sua própria seção `require` com uma lista de dependências, e estas dependências podem ter ainda mais dependências.

Uma notícia melhor ainda, é que o Composer gerencia e instala essas dependências para você. O quão fantástico é isso? O `Wolverine` pode precisar de `tools/claws`, `tools/yellow-mask` e `power/regeneration`, mas você não precisa se preocupar com isto. Enquanto sua seção `require` tiver o pacote `xmen/wolverine` o Composer vai tomar conta do resto.

Quanto as versões das dependências, elas podem ter diferentes formas. Por exemplo, você pode não se importar com pequenas atualizações em um componente. Nesse caso, você pode usar um genérico dentro da versão, assim:

```
1 "xmen/gambit": "1.0.*"
```

Agora o Composer vai instalar a última versão que começar com `1.0`. Por exemplo, se o `Gambit` tiver as versões `1.0.0` e `1.0.1`, então a versão `1.0.1` vai ser instalada.

Seu pacote também pode ter um número mínimo ou máximo para as versões. Isso pode ser definido usando os operadores `maior` `que` e `menor` `que`.

```
1 "xmen/gambit": ">1.0.0"
```

O exemplo acima vai ficar satisfeito com qualquer versão do pacote `xmen/gambit` que tenha uma versão maior que `1.0.0`.

```
1 "xmen/gambit": "<1.0.0"
```

Similarmente, o operador `menor` que fica satisfeito com pacotes que tenha a versão menor do que `1.0.0`. Permitindo que o pacote seja especificado ao máximo as versões das dependências.

```
1 "xmen/gambit": "=>1.0.0"  
2 "xmen/gambit": "=<1.0.0"
```

Incluindo um sinal de igual `=` juntamente com o sinal de comparação vai resultar na comparação da versão digitada juntamente com as versões aceitáveis da dependência.

Ocasionalmente, você pode querer digitar mais de uma versão, ou prover um alcance determinado para as versões de um pacote. Mais de uma definição de versão pode ser adicionada para separando cada definição com uma vírgula `(,)`. Por exemplo:

```
1 "xmen/gambit": ">1.0.0, <1.0.2"
```

O exemplo acima vai ficar satisfeito com a versão `1.0.1`.

Se você não quiser instalar uma versão estável, por exemplo, você pode ser o tipo que curte bungee jump ou saltar de paraquedas, então você pode querer usar versões em desenvolvimento.

```
1 "xmen/gambit": "dev-branchname"
```

Por exemplo, se você quiser usar o código atual do branch `develop` do projeto do Gambit no Github, então você pode usar a definição da versão `dev-develop`.

```
1 "xmen/gambit": "dev-develop"
```

Estas definições de versões não vão funcionar se você não tiver a configuração de estabilidade mínima para o seu pacote. Por padrão o Composer usa a definição de buscar apenas versões estáveis como estabilidade mínima, que vão restringir para apenas as versão estáveis, ou seja, versões taggeadas.

Se você quiser reescrever essa opção, simplesmente altere a configuração `minimum-stability` dentro do seu arquivo `composer.json`.

```
1 "require": {  
2     "xmen/gambit": "dev-master"  
3 },  
4 "minimum-stability": "dev"
```

Existem outros valores disponíveis para a configuração de estabilidade mínima, mas explicar essas envolve um aprofundamento muito grande em tags de versões estáveis. Eu não quero deixar esse capítulo muito complicado entrando neste assunto. Eu posso voltar neste tópico depois, mas por enquanto eu sugiro olhar [A documentação do Composer para versões de pacotes](#)⁹ para procurar informações adicionais sobre este tópico.

As vezes, você pode ter algumas dependências que apenas são necessárias durante o desenvolvimento de sua aplicação. Essas dependências podem não ser obrigatórios para o dia-a-dia da sua aplicação no ambiente de produção.

O Composer possui uma alternativa para esta situação graças a seção `require-dev`. Vamos imaginar por um momento que sua aplicação necessita do [framework de testes Codeception](#)¹⁰ para fornecer testes de aceitação.

```
1 "require": {  
2     "xmen/gambit": "dev-master"  
3 },  
4 "require-dev": {  
5     "codeception/codeception": "1.6.0.3"  
6 }
```

O pacote `codeception/codeception` vai ser instalado apenas se nós usarmos o `--dev` com o Composer. Nós vamos ver mais sobre este tópico na seção de instalação e utilização.

Como você pode ver acima, a seção `require-dev` utiliza exatamente o mesmo formato que a seção `require`. Na verdade, existem outras seções que utilizam o mesmo formato. Vamos dar uma olhada rapidamente no que temos disponível.

```
1 "conflict": {  
2     "marvel/spiderman": "1.0.0"  
3 }
```

A seção de conflitos contém uma lista de pacotes que não vão funcionar como deveriam juntamente o nosso pacote.

⁹<http://getcomposer.org/doc/01-basic-usage.md#package-versions>

¹⁰<http://codeception.com/>

```
1  "replace": {  
2      "xmen/gambit": "1.0.0"  
3  }
```

A seção `replace` informa que este pacote pode ser usado como substituto para outro pacote. Isto é útil para pacotes que foram criados através de forks de outros pacotes, mas oferecem a mesma funcionalidade.

```
1  "provide": {  
2      "xmen/gambit": "1.0.0"  
3  }
```

Esta seção indica pacotes que são fornecidos dentro do código do seu pacote. Se o pacote do Gambit fosse incluso dentro do nosso pacote principal, então seria um pouco de retrabalho instalar novamente. Utilize esta seção para que o Composer saiba quais pacotes estão embutidos dentro do nosso pacote principal. Lembre-se, você não precisa listar suas dependências aqui. Qualquer coisa listada no `require` não conta.

```
1  "suggest": {  
2      "xmen/gambit": "1.0.0"  
3  }
```

Seu pacote pode ter um número de pacotes extras que aumentam suas funcionalidades, mas não são obrigatórios. Por que não adicioná-los na seção `suggest`? O Composer vai mencionar qualquer pacote que esteja nesta seção como uma sugestão de instalação quando rodar o comando de instalação.

Bem, isto é tudo que eu tenho sobre dependências. Vamos dar uma olhada no próximo pedaço da mágica do Composer. Autoloading!

Autoloading

Nós já temos o conhecimento para que o Composer gerencie nossas dependências, mas como nós usamos elas? Nós poderíamos utilizar a função `require()` para nós mesmos buscarmos os arquivos com PHP, mas para isto nós precisamos saber exatamente onde os arquivos estão.

E ninguém tem tempo para isso. O Composer vai cuidar disto para nós. Se nós falarmos para o Composer onde nossas classes estão, e qual o método que pode ser utilizado para carregá-las, então elas terão o seu próprio método de carregamento, que pode ser usado pela nossa aplicação para carregá-las.

Ações falam mais que palavras, então vamos dar uma olhada em um exemplo.

```
1 "autoload": {  
2  
3 }
```

Esta é a seção na qual todas as configurações de autoload vão ser declaradas. Simples, certo? Ótimo! Nada de teatro de meias para você.

Vamos dar uma olhada no mecanismo mais simples de autoload, o método `files`.

```
1 "autoload": {  
2     "files": [  
3         "path/to/my/firstfile.php",  
4         "path/to/my/secondfile.php"  
5     ]  
6 }
```

O mecanismo de carregamento `files` deve conter um array de arquivos que serão carregados quando o autoloader do Composer for carregado dentro de nossa aplicação. Os diretórios dos arquivos são relativos com o diretório raiz do projeto. Este método de carregamento é efetivo, mas não é muito conveniente. Você não vai querer adicionar cada arquivo manualmente em um grande projeto. Vamos dar uma olhada em alguns métodos melhores para carregar uma grande quantidade de arquivos.

```
1 "autoload": {  
2     "classmap": [  
3         "src/Models",  
4         "src/Controllers"  
5     ]  
6 }
```

O `classmap` é outro mecanismo de carregamento que aceita um array. Porém desta vez, o array consiste em diretórios que são relativos a raiz do projeto.

Ao gerar o código de autoload, o Composer vai iterar pelos diretórios procurando arquivos que contenham classes PHP. Estes arquivos vão ser adicionados em uma coleção que mapeia o caminho de um arquivo, juntamente com o nome da classe. Quando a sua aplicação está usando o autoload e tenta instanciar uma classe que não existe, o Composer vai intervir e carregar a classe utilizando as informações armazenadas na coleção.

Porém, existe um lado negativo de utilizar esse mecanismo. Você vai precisar utilizar o comando `composer dump-autoload` para recriar o mapa de classes toda a vez que você adicionar um novo arquivo. Felizmente existe um outro mecanismo de carregamento. E o melhor de tudo, ele é inteligente o suficiente para não precisar de um mapa de classes. Vamos aprender sobre carregamento de classes pela PSR-0.

O carregamento de classes PSR-0 oferece uma maneira simples e clara de mapear classes pertencentes a namespaces no PHP, para seus respectivos arquivos.

Sabendo disso, se você adicionar uma declaração de namespace em um arquivo que contém uma classe, assim:

```
1  <?php
2
3  namespace Xmen;
4
5  class Wolverine
6  {
7      // ...
8  }
```

Então a classe se transforma em `Xmen\Wolverine`, e para o PHP ela é uma classe totalmente diferente da classe `Wolverine`.

Utilizando o autoloader da PSR-0, a classe `Xmen\Wolverine` precisa estar localizada no arquivo `Xmen\Wolverine.php`.

Viu só como o namespace combina com a estrutura de diretórios que a classe está localizada? A classe `Wolverine` que pertence ao namespace `Xmen` está localizada dentro do diretório `Xmen`.

Você também deve ter em mente que o nome do arquivo precisa ser igual ao nome da classe, incluindo os caracteres maiúsculos. O nome do arquivo igual ao nome da classe é essencial para que o autoloader PSR-0 funcione corretamente.

Namespaces podem possuir muitos níveis, por exemplo, considere a seguinte classe.

```
1  <?php
2
3  namespace Super\Happy\Fun;
4
5  class Time
6  {
7      // ...
8  }
```

A classe `Time` está localizada no namespace `Super\Happy\Fun`. Então o PHP vai reconhecer a classe como `Super\Happy\Fun\Time` e não `Time`.

Esta classe vai estar localizada no seguinte diretório.

1 Super/Happy/Fun/Time.php

Novamente, perceba como a estrutura de diretórios é igual ao namespace. Também, perceba que o nome o arquivo é exatamente igual ao nome da classe.

Isto é tudo que existe sobre o autoloader PSR-0. É muito simples! Vamos dar uma olhada em como podemos utilizar isto juntamente com o Composer para simplificar o carregamento das nossas classes.

```
1 "autoload": {  
2     "psr-0": {  
3         "Super\\Happy\\Fun\\Time": "src/"  
4     }  
5 }
```

Agora, nosso bloco de carregamento psr-0 é um objeto ao invés de um array. Isso porque precisamos tanto da chave quanto do valor.

A chave para cada valor do objeto representa o namespace. Não se preocupe com as duas barras invertidas. Elas são usadas porque uma barra invertida é usada para representar um caractere escapado dentro do JSON. Tenha isto em mente quando estiver mapeando namespaces dentro de um arquivo JSON!

O segundo valor é o diretório no qual o namespace está localizado. Eu descobri que você não precisa da barra no final, mas muitos exemplos gostam de utilizar para reforçar que é um diretório.

A próxima parte é muito importante, e é um daqueles momentos ‘Ah, entendi’ para um monte de pessoas. Por favor, leia atentamente.

O segundo parâmetro não é o diretório no qual as classes para estes namespaces estão localizadas. Ao invés disso, é o diretório no qual **começa** o namespace para o mapeamento de diretórios. Vamos dar uma olhada no exemplo anterior para ilustrar isto.

Lembra da classe Super Happy Fun Time? Vamos dar uma outra olhada nela.

```
1 <?php  
2  
3 namespace Super\Happy\Fun;  
4  
5 class Time  
6 {  
7     // ...  
8 }
```

Bem, nós sabemos que essa classe pode estar localizada no arquivo Super/Happy/Fun/Time.php. Com isso em mente, considere a seguinte snippet de autoloader.


```
1 "autoload": {  
2     "psr-0": {  
3         "Super\\Happy\\Fun\\Time": "src/"  
4     }  
5 }
```

Você pode esperar que o Composer procure a classe em `src/Time.php`. Isto vai estar **incorreto**, e a classe não vai ser encontrada.

Ao invés disto, a estrutura de diretórios deve estar no seguinte formato.

```
1 src/Super/Happy/Fun/Time.php
```

Isto é algo que pega muitas pessoas em suas primeiras vezes usando Composer. Eu não posso enfatizar o suficiente o quanto isto é importante de se lembrar.

Se nós rodarmos uma instalação do Composer agora, e depois adicionar uma nova classe `Life.php` no mesmo namespace, nós não vamos precisar gerar novamente o autoloader. O Composer sabe exatamente onde as classes deste namespace estão, e como carregá-las. Ótimo!

Você deve estar se perguntando por que eu botei meus arquivos do namespace em um diretório chamado `src`? Isto é uma convenção quando se está escrevendo bibliotecas baseadas no Composer. Na verdade, a estrutura a seguir é muito comum para pacotes do Composer.

```
1 src/                (Classes.)  
2 tests/              (Unit/Acceptance tests.)  
3 docs/               (Documentation.)  
4 composer.json
```

Sinta-se à vontade para seguir este padrão, ou qualquer outro que te faça feliz. O Laravel fornece suas próprias localizações para classes que estão descritas nos próximos capítulos.

Agora que você aprendeu como definir seus mecanismos de carregamento, é hora de vermos como instalar e utilizar o Composer, para que você possa começar a tirar vantagem do seu autoloader.

Instalação

Você pode estar se perguntando por que eu escolhi cobrir a instalação e utilização no final deste capítulo? Eu senti que tendo um bom conhecimento de suas configurações iria ajudar a entender o que o Composer está fazendo por trás dos panos.

Os seguintes métodos de instalação são especificamente para ambientes de desenvolvimento baseados em Unix, como Linux ou Mac OSX. Eu espero que o Taylor possa editar este capítulo e disponibilizar informações para instalação do Composer em um ambiente Windows, visto que eu evito este particular sistema operacional como uma praga.

Composer é uma aplicação baseada em PHP, então você deve ter o cliente CLI do PHP instalado para poder usar. Você pode checar isto executando o seguinte comando.

```
1 php -v
```

Se o PHP estiver instalado corretamente, você vai ver algo parecido com..

```
1 $ php -v
2 PHP 5.4.4 (cli) (built: Jul  4 2012 17:28:56)
3 Copyright (c) 1997-2012 The PHP Group
4 Zend Engine v2.4.0, Copyright (c) 1998-2012 Zend Technologies
5     with XCache v2.0.0, Copyright (c) 2005-2012, by m0o
```

Se o estado da saída disser que você está usando qualquer versão menor que a 5.3.2, então você não vai poder usar o Composer até que você atualize sua versão do PHP. Na verdade, se você estiver usando qualquer versão menor que a 5.3.7 você não poderá utilizar nem o Laravel.

Você pode utilizar CURL para fazer o download do Composer. O Mac OSX vem com isto por padrão. Muitas das distribuições de Linux possuem CURL nos seus repositórios de softwares, se já não tiverem instaladas por padrão. Vamos utilizar o CURL para fazer o download do arquivo executável do Composer.

```
1 curl -sS https://getcomposer.org/installer | php
```

Usuários experientes de Linux podem estar preocupados que o CURL está canalizando o script de instalação com PHP. Isto é uma preocupação muito comum, mas a instalação do Composer vem sendo usada por milhares de desenvolvedores e está bem provado que é segura. Não deixe isto não te impedir utilizar este belo software!

Assumindo que o processo de instalação foi concluído com sucesso (ele vai te dizer), você agora possui um arquivo chamado `composer.phar` no diretório da sua aplicação. Este arquivo pode ser usado para executar o Composer, por exemplo...

```
1 php composer.phar
```

..vai te mostrar uma lista de comandos que estão disponíveis para você.

Agora, você pode continuar a usar o Composer assim, mas eu sugiro que você instale globalmente. Desta maneira ele pode ser usado em todos os seus projetos que são baseados em Composer, e você vai ter um comando menor para executar.

Para instalar o Composer globalmente, simplesmente mova-o para uma localização que esteja dentro da sua variável de ambiente PATH. Você pode ver estas localizações utilizando o seguinte comando.

```
1 echo $PATH
```

Porém, o diretório `/usr/local/bin` é um lugar aceitável para a maioria dos sistemas. Quando nós movemos o arquivo, nós vamos precisar renomea-lo para `composer`, para que seja mais fácil de utilizarmos. Este é o comando que precisamos:

```
1 sudo mv composer.phar /usr/local/bin/composer
```

Tendo instalado o Composer globalmente, nós podemos utilizar o seguinte comando para obter a mesma lista de comandos do Composer. Este comando também é executável de qualquer localização do sistema.

```
1 composer
```

Muito mais limpo, não é? Vamos começar a usar isto.

Utilização

Vamos assumir que nós criamos o seguinte arquivo `composer.json` para o nosso pacote.

```
1 {
2     "name":            "marvel/xmen",
3     "description":     "Mutants saving the world for people who hate them."
4     "keywords":       ["mutant", "superhero", "bald", "guy"],
5     "homepage":        "http://marvel.com/xmen",
6     "time":            "1963-09-01",
7     "license":         "MIT",
8     "authors": [
9         {
10            "name":      "Stan Lee",
11            "email":     "stan@marvel.com",
12            "homepage":  "http://marvel.com",
13            "role":      "Genius"
14        }
15    ],
16    "require": {
17        "xmen/wolverine": "1.0.0",
18        "xmen/cyclops":   "1.0.1",
19        "xmen/storm":     "1.2.0",
20        "xmen/gambit":    "1.0.0"
21    },
22    "autoload": {
23        "classmap": [
24            "src/Xmen"
25        ]
26    }
27 }
```

Vamos utilizar o comando `install` para instalar todas as dependências do nosso pacote, e configurar o nosso autoloader.

```
1 composer install
```

A saída que nós vamos ter do Composer vai se parecer com:

```
1 Loading composer repositories with package information
2 Installing dependencies
3
4 - Installing tools/claws (1.1.0)
5   Cloning bc0e1f0cc285127a38c232132121a2fd53e94
6
7 - Installing tools/yellow-mask (1.1.0)
8   Cloning bc0e1f0cc285127a38c6c12312325dba2fd53e95
9
10 - Installing power/regeneration (1.0.0)
11   Cloning bc0e1f0cc2851213313128ea88bc5dba2fd53e94
12
13 - Installing xmen/wolverine (1.0.0)
14   Cloning bc0e1f0cc285127a38c6c8ea88bc523523523535
15
16 - Installing xmen/cyclops (1.0.1)
17   Cloning bc0e1f0cc2851272343248ea88bc5dba2fd54353
18
19 - Installing xmen/storm (1.2.0)
20   Cloning bc0e1f0cc285127a38c6c8ea88bc5dba2fd53343
21
22 - Installing xmen/gambit (1.0.0)
23   Cloning bc0e1f0cc285127a38c6c8ea88bc5dba2fd56642
24
25 Writing lock file
26 Generating autoload files
```

Lembre-se que estes pacotes são fakes e apenas usados para os nossos exemplos. Fazer o download deles não vai funcionar! Mas eles são mais divertidos, visto que eles são os X-Men! Yay!

Mas por que existem sete pacotes instalados quando eu apenas listei quatro? Bem, você está esquecendo que o Composer gerencia as dependências das dependências também. Os três pacotes extras são dependências do pacote `xmen/wolverine`.

Eu imagino que você provavelmente está se perguntando onde esses pacotes foram instalados? O Composer cria um diretório chamado `vendor` na raiz do nosso projeto, que contém os arquivos dos pacotes.

O pacote `xmen/wolverine` pode ser encontrado em `vendor/xmen/wolverine`, onde você vai encontrar seus arquivos fontes juntamente com o seu `composer.json`.

O Composer também guarda alguns de seus próprios arquivos relacionados ao sistema de autoloading no diretório `vendor/composer`. Não se preocupe. Você nunca vai precisar editar esse arquivo diretamente.

Mas como nós tiramos proveito das incríveis habilidades de autoloading? Bem, a resposta para isto é ainda mais simples do que configurar o próprio autoloading. Simplesmente use o `require()` ou o `include()` do PHP para incluir o arquivo `vendor/autoload.php` dentro da sua aplicação. Por exemplo:

```
1 <?php
2
3 require 'vendor/autoload.php';
4
5 // Your awesome application bootstrap here!
```

Ótimo! Agora nós podemos instanciar a classe pertencente a qualquer uma de nossas dependências, por exemplo..

```
1 <?php
2
3 $gambit = new \Xmen\Gambit;
```

O Composer vai fazer toda a mágica e carregar as definições de classe para você. Quão fantástico é isso? Sem mais sujar os nossos arquivos com milhares de declarações de `include()`.

Se você adicionar um arquivo em um diretório que está em um diretório carregado pelo `classmap`, você vai precisar rodar um comando do Composer para poder adicioná-lo no autoloading.

```
1 composer dump-autoload
```

O comando acima vai reconstruir todos os mapeamentos e criar um novo `autoload.php` para você.

E se nós quisermos adicionar outra dependência para o nosso projeto? Vamos adicionar `xmen/beast` para o nosso arquivo `composer.json`.

```

1  {
2      "name":          "marvel/xmen",
3      "description":   "Mutants saving the world for people who hate them.",
4      "keywords":      ["mutant", "superhero", "bald", "guy"],
5      "homepage":      "http://marvel.com/xmen",
6      "time":          "1963-09-01",
7      "license":       "MIT",
8      "authors": [
9          {
10             "name":      "Stan Lee",
11             "email":      "stan@marvel.com",
12             "homepage":   "http://marvel.com",
13             "role":       "Genius"
14         }
15     ],
16     "require": {
17         "xmen/wolverine": "1.0.0",
18         "xmen/cyclops":   "1.0.1",
19         "xmen/storm":      "1.2.0",
20         "xmen/gambit":     "1.0.0",
21         "xmen/beast":      "1.0.0"
22     },
23     "autoload": {
24         "classmap": [
25             "src/Xmen"
26         ]
27     }
28 }

```

Agora nós precisamos rodar `composer install` novamente para que o Composer possa instalar o nosso pacote recém adicionado.

```

1  Loading composer repositories with package information
2  Installing dependencies
3
4  - Installing xmen/beast (1.1.0)
5      Cloning bc0e1f0c34343347a38c2321321321a2fd53e94
6
7  Writing lock file
8  Generating autoload files

```

Agora `xmen/beast` foi instalado e nós já podemos utilizá-lo. Demais!

Você pode ter reparado a seguinte linha na saída do comando `composer install`.

1 Writing lock file

Você também deve ter percebido que o Composer criou um arquivo chamado `composer.lock` na raiz da nossa aplicação. O que é isto, eu ouvi você chorar?

O arquivo `composer.lock` contém informações sobre o seu pacote desde a última vez que você usou o comando `composer install` ou `composer update`. Ele também contém uma lista das **exatas** versões de cada dependência que foi instalada.

Por que isto? É simples. Sempre que você usa o `composer install` e existe um arquivo `composer.lock` no diretório, O Composer vai utilizar as versões contidas dentro do arquivo ao invés de puxar versões novas de cada dependência.

Isso significa que se você versionar o seu arquivo `composer.lock` juntamente com o código da sua aplicação (e eu recomendo que você faça isso), quando você fizer o deploy da sua aplicação para o ambiente de produção, ela vai utilizar exatamente as mesmas versões que você usou e testou no seu ambiente de desenvolvimento. Isto significa que você pode ter certeza que o Composer não vai instalar versões das dependências que quebrem a sua aplicação.

Note que você nunca vai editar o arquivo `composer.lock` manualmente.

Já que nós estamos no tópico de versões das dependências, por que não ver sobre atualizá-las? Por exemplo, se nós tivermos o seguinte requerimento no nosso arquivo `composer.json`.

```
1 "xmen/gambit": "1.0.*"
```

Composer pode instalar a versão `1.0.0` para nós. Porém, e se o pacote foi atualizado para `1.0.1` alguns dias depois?

Bem, nós podemos utilizar o comando `composer update` para atualizar todas as nossas dependências para suas últimas versões. Vamos dar uma olhada na saída.

```
1 $ composer update
2 Loading composer repositories with package information
3 Updating dependencies (including require-dev)
4
5 - Installing xmen/gambit (1.0.1)
6   Cloning bc0e1f0cc285127a38c6c8ea88bc5dba2fd56642
7
8 Generating autoload files
```

Ótimo! O pacote `xmen/gambit` foi atualizado para sua última versão, e o nosso arquivo `composer.lock` foi atualizado.

Se nós quisermos atualizar alguma das dependências ao invés de todas elas, nós podemos especificar o nome do pacote utilizando o comando `update`. Por exemplo:

```
1 composer update xmen/gambit
```

Espera, o que esse (`including require-dev`) significa? Bem, você deve lembrar da seção `require-dev` no arquivo `composer.json` onde nós listamos nossas dependências de desenvolvimento? Bem, o Composer espera o comando `update` para apenas ser executado dentro de um ambiente seguro de desenvolvimento ou de testes. Por isto, ele assume que você quer suas dependências de desenvolvimento, e faz o download delas para você.

Se você não quiser instalar as dependências de desenvolvimento então você pode usar o seguinte comando.

```
1 composer update --no-dev
```

Também, se você quiser instalar dependências usando o comando `install`, simplesmente adicione o seguinte comando.

```
1 composer install --dev
```

A última coisa que você deve saber, é que você pode usar o comando `composer self-update` para atualizar o arquivo binário do Composer. Tenha certeza de utilizar `sudo` se você instalou ele globalmente.

```
1 sudo composer self-update
```

Bem, isto cobre tudo que vamos precisar saber sobre Composer para trabalhar com o Laravel. Foram muitas informações para absorver, e um longo capítulo para estes dedos cansados, mas eu espero que você tenha tirado alguma coisa disto.

Se você sentir que algum tópico em particular precisa de mais informações, não se esqueça de me avisar!

Arquitetura

Eu quero que o Code Bright tenha uma experiência completa de aprendizado, e por isto, eu decidi incluir este capítulo. Se você não agüenta mais de tanta vontade de começar a codificar, sinta-se à vontade para pular este capítulo. Porém, eu acho que vai ser muito útil para você aprender como o Laravel é estruturado.

No Laravel 3, o IoC container era um componente que confundia muitas pessoas. No Laravel 4, é a parte do framework que mantém todas as peças juntas. Eu nem consigo dizer o quão importante ele é. Na verdade, eu preciso que esta capítulo faça sentido. Vamos começar.

O Container

No Laravel 4, o container é criado durante o processo de inicialização. É um objeto chamado `$app`. Quando nós falamos sobre o container, o que vem à sua mente?

Vamos dar uma olhada na definição do dicionário. Por quê? Não tenho certeza... mas eu vejo muitos outros autores fazendo isso. Eu não quero que eles olhem torto para mim. Vamos lá.

Container. Um objeto usado para ou capaz de carregar, seja para o transporte ou armazenamento, como um cartão, um box, etc.

Eu acho que a descrição combina com o container do Laravel muito bem. É certamente utilizado para armazenar coisas. Vamos ser honestos... se ele não guardasse coisas, então 'Container' seria um nome terrível. Quanto ao transporte, certamente ele deixaria mais fácil para buscar outros componentes do framework quando nós precisássemos acessá-los.

Na verdade, não é um cartão ou uma caixa. É mais um sistema de armazenamento por chaves.

Uma vez que o objeto `$app` é criado, você pode acessá-lo de qualquer lugar da aplicação utilizando a função `app()`. Honestamente, você provavelmente não vai precisar. Isto por causa de outra funcionalidade que eu vou explicar mais tarde neste capítulo.

Vamos colocar uma pressão neste objeto '`$app`' e dar uma olhada nele. Nós podemos fazer isto no nosso arquivo `app/routes.php`.

```
1 <?php
2
3 // app/routes.php
4
5 var_dump($app);
6
7 die();
```

Note que nós não utilizamos a função auxiliar `app()` dentro do arquivo `app/routes.php`. O objeto `$app` está disponível no escopo global, e então nós podemos acessá-lo diretamente.

Wow, é um grande objeto! Eu acho que ele deve ser... é o coração do framework. Você vai perceber que ele é uma instância da classe `Illuminate\Foundation\Application`. Todos os componentes do Laravel 4 estão dentro do namespace `Illuminate`. Ele foi um nome utilizado durante o desenvolvimento inicial do framework, e nós não temos coragem de removê-lo. Pense no Laravel como se estivesse ‘iluminando’ o mundo do PHP!

Se nós dermos uma olhada no código fonte original da classe `Application`, nós vamos ver que ela estende o `Container`. Vá em frente, você pode encontrar o arquivo em:

<https://github.com/laravel/framework/blob/master/src/Illuminate/Foundation/Application.php>¹¹

A classe `Container` é onde a mágica acontece. Ela também implementa a interface `ArrayAccess`. Ela é uma interface especial que permite que os atributos do objeto possam ser acessados como um array, numa maneira muito similar com os objetos literais do JavaScript. Por exemplo, as duas maneiras de acessar os atributos do objeto seriam possíveis:

```
1 <?php
2
3 $object->attribute = 'foo';
4 $object['attribute'] = 'foo';
```

Durante a inicialização do framework, as classes provedoras de serviços (Service Providers) são executadas. Estas classes tem o propósito de registrar os componentes do framework dentro do container da aplicação.

E o que eu quero dizer com componente? Assim, o framework é construído por muitas partes. Por exemplo, nós temos uma camada de rotas, um sistema de validação, uma camada de autenticação e muitos outros pacotes que lidam com funções específicas. Nós chamados estes pacotes de componentes do framework, e eles se juntam dentro do container para criar um framework completo como o Laravel.

Se é a primeira vez que você está usando o Laravel, então o próximo exemplo não vai fazer muito sentido. Mas não se preocupe com isso! É apenas uma coisa para os usuários do Laravel 3 pensarem a respeito.

¹¹<https://github.com/laravel/framework/blob/master/src/Illuminate/Foundation/Application.php>

```
1 <?php
2
3 // app/routes.php
4
5 $app['router']->get('/', function()
6 {
7     return 'Obrigado por comprar o Code Bright!';
8 });
```

Aqui nós estamos acessando a camada rotas, que está dentro do container, para criar uma rota que responde à uma requisição HTTP do tipo 'GET'. Isto vai parecer familiar para os usuários de Laravel 3, embora a sintaxe pareça um pouco estranha.

Como você pode ver, nós estamos acessando o componente de rotas usando a sintaxe de array no nosso container. Esta é a mágica que a interface `ArrayAccess` nos permite. Se nós quisermos, nós também podemos acessar a camada de rotas assim:

```
1 <?php
2
3 // app/routes.php
4
5 $app->router->get('/', function()
6 {
7     return 'Sério, Obrigado por comprar o Code Bright!';
8 });
```

Acessar o componente como um atributo do nosso container funciona exatamente da mesma maneira que o método que nós usamos anteriormente.

Embora não seja muito bonito, não é? Assim, se você é novo no framework, você provavelmente já ouviu sobre a elegante sintaxe do Laravel. Se você usou Laravel 3 anteriormente, então você já vem apreciando isso.

E certamente que nós não vamos estragar nossa reputação, não é mesmo? Claro que não! Vamos dar uma olhada em algumas mágicas em ação.

Facades

Meu amigo Kenny Meyers fez um belo review sobre o livro original Code Happy no [The Nerdary](http://www.thenerdary.net/)¹². Eu adoraria retribuir o favor. Ele nos falou na última conferência do Laravel que ele tem dificuldade de pronunciar certas palavras, e eu aposto que esta é uma delas. Então para o Kenny, e para aqueles que não falam inglês como sua primeira língua, aqui está como se pronuncia Facade.

¹²<http://www.thenerdary.net/>

“fah-sahd”

No Laravel 3, a maioria dos componentes eram acessados utilizando métodos estáticos. Por exemplo, o exemplo de rotas do capítulo anterior vai ser parecido com isto:

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return 'Thanks Kenny, we love you!';
8 });
```

Parece ótimo... nós temos um nome descritivo para o componente, e ainda um verbo descrevendo a ação executada por esse componente.

Infelizmente, desenvolvedores mais experientes vão ficar receosos com o uso de um método estático. Métodos estáticos podem ser muito difíceis de se testar. Eu não quero complicar as coisas tão cedo no livro explicando o porque disto, então você vai ter que confiar em mim.

Isto criou um grande problema durante o desenvolvimento do Laravel 4. Nós amamos nossa sintaxe simples e clara, mas nós também queremos adotar as melhores práticas de codificação, e isso inclui escrever muitos, muitos testes para garantir que o nosso código é robusto.

Felizmente, o Taylor chegou com esta maravilhosa ideia de classes Facade, nomeada por causa do design pattern ‘Facade’. Utilizando Facades nós podemos ter o melhor dos dois mundos. Belos métodos estáticos, e ainda componentes instanciados com métodos públicos. Isto significa que nosso código é belo, e muito testado. Vamos ver como isso funciona.

No Laravel 4, existem algumas classes que são mapeadas para o namespace global. Estas classes são as nossas classes Facade, e elas estendem a classe `Illuminate\Support\Facades\Facade`. Esta classe é muito esperta. Eu vou explicar sua inteligência com alguns exemplos de código. Nós podemos utilizar o método estático de roteamento da mesma maneira que o bom e velho Laravel 3, assim:

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return 'Thanks for Facades, Taylor!';
8 });
```

Neste exemplo, o Facade é a classe 'Route'. Cada Facade é ligado à uma instância de um componente no container. Os métodos estáticos de um Facade são apenas atalhos, e quando eles são chamados, eles chamam o verdadeiro método público do objeto que eles representam dentro do container. Isto significa que o método `Route::get()` na verdade vai chamar o seguinte método:

```
1  <?php
2
3  // app/routes.php
4
5  $app['router']->get('/', function()
6  {
7      return 'Thanks for Facades, Taylor!';
8  });
```

Por que isso é importante? Bem, isso oferece uma grande flexibilidade.

Flexibilidade

O que flexibilidade significa? Eu não vou referenciar o dicionário desta vez. Eu sei que isso significa a habilidade de mudar. E esta já é uma explicação muito boa sobre os benefícios do container. Assim, nós podemos mudar, ou melhor, substituir os objetos e os componentes que já foram registrados dentro do container. E isto oferece uma flexibilidade muito grande.

Vamos imaginar por um momento que nós não sabemos como a camada de roteamento funciona (não se preocupe, você vai adorar isto, apenas imagina que você não sabe).

Já que todos somos excelentes desenvolvedores, nós vamos escrever nossa própria camada de roteamento, com uma classe chamada `SuperRouter`. Devido a grande flexibilidade do container, nós podemos substituir a camada de roteamento do framework pela nossa. Sinceramente, é mais fácil do que você imagina. Nós apenas atribuímos nossa classe para a chave 'router' já existente dentro do container. Vamos ver isso em ação. Ah, note que eu não recomendo fazer isto tão cedo, especialmente se você é um iniciante. Nós apenas faremos agora porque será um grande exemplo.

```
1  <?php
2
3  // app/routes.php
4
5  $app['router'] = new SuperRouter();
```

Agora, não apenas nós podemos usar a nossa classe de roteamento diretamente dentro do container, mas o nosso Facade `Route` vai continuar acessando este componente.

Na verdade, isso ainda fica mais impressionante. Quando você substitui os componentes do Laravel, o framework vai tentar utilizar o seu substituto para suas tarefas, como se o seu componente fosse o componente do próprio framework.

Permitir acessar o código fonte dos componentes do framework desta maneira oferece muito poder e flexibilidade para o usuário. Você está ficando mal acostumado, não é mesmo?

Robustez

Como eu mencionei algumas vezes anteriormente, o Laravel é feito de um conjunto de componentes individuais. Cada componente é responsável pelo seu próprio pedaço de funcionalidade. Isto significa que eles respeitam o princípio da responsabilidade única.

Na verdade, muitos dos componentes do Laravel podem ser utilizados sozinhos, completamente desacoplados do framework. Por isto, cópias dos componentes podem ser encontrados na [organização Illuminate no github](https://github.com/illuminate/illuminate)¹³. Este conjunto de componentes também está [disponível no on Packagist](https://packagist.org/)¹⁴ para que você possa aproveitar os recursos nos seus próprios projetos que utilizam Composer.

Mas por que este capítulo é chamado Robustez? Bem, cada componente usado pelo framework Laravem é muito bem testado. Todo o framework contém uma suíte de testes com mais de 900 testes e 1700 asserções.

Graças aos testes, nós podemos aceitar que contribuidores façam alterações que vão prover um futuro muito rico para o framework, sem nos preocuparmos com uma alteração quebrar alguma funcionalidade que já estava funcionando, ou remover uma funcionalidade já existente.

Bem, isto é o suficiente sobre arquitetura. Eu aposto que você está pronto para começar o desenvolvimento, certo? Vamos lá.

¹³<https://github.com/illuminate>

¹⁴<https://packagist.org/>

Começando

O Laravel é um framework para a linguagem de programação PHP. Enquanto o PHP é conhecido por ter uma sintaxe não muito desejável, é fácil de usar, fácil de implantar, e pode ser encontrada como a linguagem de muitos sites que você utiliza no seu dia-a-dia. O Laravel não só fornece atalhos úteis, ferramentas e componentes para ajudar você a atingir o sucesso com todos os seus projetos web, mas também visa corrigir algumas falhas do PHP.

O Laravel tem uma bela, semântica e criativa sintaxe que permite que ele se destaque entre muitos frameworks disponíveis para a linguagem. Ele faz com que o PHP seja muito agradável de usar, sem sacrificar o seu poder e sua eficiência. O Laravel é uma grande escolha, tanto para projetos amadores quanto para grandes soluções empresariais. E mesmo se você já é experiente na linguagem, ou se é um novato, o Code Bright vai ajudar a tornar suas ideias em aplicações web funcionais.

Vamos dar uma olhada nos requerimentos para o framework e este livro.

Requerimentos

- **Um Computador** Ler o livro é muito bom, mas você vai aprender mais se testar os exemplos de código que você vai encontrar neste livro.
- **Um Servidor Web** O Laravel precisa de um servidor web. Não importa qual você use, mas a maioria da comunidade utiliza ou Apache ou Nginx e fazer o mesmo vai tornar muito mais fácil procurar ajuda quando necessário.
- **PHP: Hypertext Preprocessor 5.3 ou maior** O Laravel é um framework PHP, e necessita da linguagem de programação PHP. Sério, você vai precisar dela. Como o Laravel utiliza funcionalidades modernas da linguagem, você vai precisar da versão 5.3.7 ou maior. Você pode descobrir sua versão do PHP na maioria dos servidores digitando `php -v` no seu console, ou utilizando a função `phpinfo()`.
- **Um Servidor de Banco de Dados** Mesmo que não seja uma exigência do framework, muitos dos exemplos neste livro interagem com um banco de dados. Por isto, eu recomendo configurar um servidor de banco de dados suportado pelo conector PDO. Eu recomendo você utilizar o gratuito e flexível MySQL da Su.. Oracle. Outros servidores populares também incluem SQL Server, Postgres e SQLite.
- **Um Editor de Texto** Você vai precisar de um para brincar com os exemplos encontrados dentro deste livro. Eu recomendo o Sublime Text 2, que enquanto não é gratuito, é muito prático. Porém existem milhares de editores e IDEs disponíveis, ache um que se encaixe com a sua maneira de trabalho.

Agora antes de nós começarmos a trabalhar nos nossos projetos com Laravel, nós precisamos baixar uma cópia do framework.

Instalação

Eu não sei realmente se instalação é o título certo para esta seção. Porém, eu não posso pensar em algo melhor.

O Laravel possui um repositório no Github que funciona como um ‘template’ para sua nova aplicação. Vamos copiar este repositório para nossa máquina.

Nós vamos utilizar o git para ‘clonar’ o repositório para uma pasta no nosso servidor web de desenvolvimento. Este é o comando.

```
1 git clone git@github.com:laravel/laravel.git my_project
```

Agora você vai ter um template da aplicação do Laravel dentro da pasta `my_project`. Você pode achar outras referências para este template como o ‘app package’. Este pacote contém toda sua aplicação e todo o diretório vai estar versionado.

Os usuários mais experientes do Laravel podem lembrar de um diretório chamado `laravel` que continha os arquivos que faziam o framework funcionar. Nós nos referimos para esses arquivos como os arquivos do ‘core’ do framework. Bem, este diretório não existe mais. Aproveitando o poder do Composer, os arquivos do ‘core’ do framework agora ficam em um pacote separado, que é uma dependência do nosso template.

Vamos dar uma olhada dentro do arquivo `composer.json` dentro da pasta `my_project` para ver o que tem de novo.

```
1 {
2     "require": {
3         "laravel/framework": "4.0.*"
4     },
5     "autoload": {
6         "classmap": [
7             "app/commands",
8             "app/controllers",
9             "app/models",
10            "app/database/migrations",
11            "app/database/seeds",
12            "app/tests/TestCase.php"
13        ]
14    },
15    "scripts": {
16        "post-update-cmd": "php artisan optimize"
17    },
18    "minimum-stability": "dev"
19 }
```


Note que as versões das dependências podem ter mudado desde a última atualização desta seção. O resultado porém, vai ser o mesmo.

O pacote da nossa aplicação depende apenas do pacote `laravel/framework`, que contém todos os componentes necessários para rodar nossa aplicação. Este arquivo `composer.json` é seu. É para a sua aplicação e você pode editá-lo como quiser. Porém, alguns padrões já foram fornecidos para você. Eu também não recomendo remover o pacote `laravel/framework`. Coisas muito ruins vão acontecer.

Agora nós temos apenas o template. Vamos rodar o comando `composer install` para instalar o core do framework.

```
1 Loading composer repositories with package information
2 Installing dependencies
3   - Installing doctrine/lexer (dev-master bc0e1f0)
4     Cloning bc0e1f0cc285127a38c6c8ea88bc5dba2fd53e94
5
6   - Installing doctrine/annotations (v1.1)
7     Loading from cache
8
9   ... Many more packages here. ...
10
11  - Installing ircmaxell/password-compat (1.0.x-dev v1.0.0)
12    Cloning v1.0.0
13
14  - Installing swiftmailer/swiftmailer (dev-master e77eb35)
15    Cloning e77eb358a1aa7157afb922f33e2afc22f6a7bef2
16
17  - Installing laravel/framework (dev-master 227f5b8)
18    Cloning 227f5b85cc2201b6330a8f7ea75f0093a311fe3b
19
20  predis/predis suggests installing ext-redis (Allows faster serialization and \
21  deserialization of the Redis protocol)
22  symfony/translation suggests installing symfony/config (2.2.*)
23  symfony/translation suggests installing symfony/yaml (2.2.*)
24  symfony/routing suggests installing symfony/config (2.2.*)
25  symfony/routing suggests installing symfony/yaml (2.2.*)
26  symfony/event-dispatcher suggests installing symfony/dependency-injection (2.2.*)
27  symfony/http-kernel suggests installing symfony/class-loader (2.2.*)
28  symfony/http-kernel suggests installing symfony/config (2.2.*)
29  symfony/http-kernel suggests installing symfony/dependency-injection (2.2.*)
30  symfony/debug suggests installing symfony/class-loader (~2.1)
31  monolog/monolog suggests installing mlehner/gelf-php (Allow sending log messages \
32  to a GrayLog2 server)
33  monolog/monolog suggests installing ext-amqp (Allow sending log messages to an AM\
```

```
34 QP server (1.0+ required))
35 monolog/monolog suggests installing ext-mongo (Allow sending log messages to a Mo\
36 ngoDB server)
37 monolog/monolog suggests installing doctrine/couchdb (Allow sending log messages \
38 to a CouchDB server)
39 monolog/monolog suggests installing raven/raven (Allow sending log messages to a \
40 Sentry server)
41 Writing lock file
42 Generating autoload files
```

Novamente, as versões dos pacotes podem ter alterado, mas o resultado vai ser o mesmo.

Esta é uma longa lista de pacotes! Para que são todos eles? Assim, o Laravel utiliza o poder do código aberto, e a riqueza de pacotes que existem com o Composer. Esses pacotes são dependências do próprio framework.

Você definitivamente deve ir atrás do Laravel e dar uma olhada nestes pacotes que estão listados [no site do Packagist¹⁵](http://packagist.org). Não tem porque reinventar a roda quando se trata de códigos já feitos.

Como você já leu o capítulo sobre Composer, você vai saber que os arquivos do ‘core’ do Laravel foram instalados dentro da pasta vendor. Você não quer versionar suas dependências juntamente com o seu código, então o Laravel disponibiliza um arquivo `.gitignore` que ignora a pasta vendor, juntamente com alguns outros arquivos.

Nós estamos praticamente prontos com a configuração do Laravel no nosso ambiente de desenvolvimento. Apenas resta um tópico. Como configurar o nosso servidor web.

Configuração do Servidor Web

Esta seção vai ser difícil de escrever, Porque assim, todo mundo possui uma configuração um pouco diferente, e existe um número grande de diferentes servidores web disponíveis.

Então iso é o que eu vou fazer. Eu vou cobrir o básico de onde um servidor web precisa ser apontado. Eu também vou mostrar alguns exemplos de configurações para os servidores mais utilizados. Eles vão, porém, ser mito genéricos, e talvez vão precisar de algumas alterações para se funcionarem em determinadas situações. Porém, você não pode dizer que eu não tentei!

Vamos dar uma olhada no que é o principal aqui. O Laravel possui um diretório chamado `public` que contém o seu código de inicialização. Esse código é usado para iniciar o framework, e recebe todas as requisições da aplicação.

O diretório `public` também contém todos os seus assets públicos, como JavaScript, CSS e imagens. Essencialmente, qualquer coisa que possa ser acessada via link direto deve ir dentro do diretório `public`.

¹⁵<http://packagist.org>

O que isso significa para a nossa configuração do servidor web? Bem, nossa primeira tarefa é ter certeza de que o servidor está apontando para o lugar certo. Nós precisamos editar sua configuração para que ele aponte para o diretório `public`, e não para a raiz do projeto.

A próxima tarefa vai ser fazer o servidor web manipular as URLs amigáveis.

Eu escolhi um nome de domínio muito bonito, eu não tenho uma URL amigável?

Infelizmente, não é assim que funciona. A vida não é só tortas e doces né? O processo de inicialização do Laravel acontece dentro de um arquivo chamado `index.php`, que está dentro do diretório `public`. Todas as requisições do framework vão para este arquivo. Isso significa que por padrão, a URL para a página ‘guestbook’ vai ficar assim:

1 <http://islifeallcakesandpies.com/index.php/guestbook>

Os usuários do nosso site não precisam saber que tudo é redirecionado para o arquivo `index.php`. E isto também não ajuda em relação a otimização para mecanismos de busca (SEO). Por isso o nosso servidor web deve estar configurado para remover o `index.php` da URL, deixando apenas a parte ‘amigável’ da URL. Normalmente isto é alcançado por uma configuração trabalhada com um expressão regular.

Vamos agora pegar um exemplo de configuração de um servidor web que nos permite chegar nos objetivos mencionados anteriormente. Novamente, eu devo lembrá-lo que estas configurações são usadas como um orientação. Para informações mais detalhadas sobre configurações de servidores web, eu recomendo visitar a documentação oficial do servidor escolhido.

Vamos começar com o Nginx.

Nginx

Nginx, pronunciado ‘engine’X’, é um grande servidor web que eu recentemente comecei a utilizar. Para mim a escolha foi simples. A performance é muito maior que o Apache, e não necessita de configurações com XML. Tudo fez sentido para mim.

Em uma distribuição Linux baseada em Debian, como o Ubuntu, você pode instalar Nginx e PHP com o seguinte comando.

1

```
sudo apt-get install nginx php5-fpm
```

O segundo pacote é o PHP-FPM, um módulo FastCGI que permite que o Nginx execute o código PHP.

No Mac estes pacotes estão disponíveis no [Macports](http://www.macports.org/)¹⁶.

¹⁶<http://www.macports.org/>

```
1 sudo port install php54-fpm nginx
```

Os arquivos de configurações do Nginx normalmente estão localizados em `/etc/nginx/sites-enabled`. Este é um template que você pode utilizar quando for configurar um novo site.

```
1 server {
2
3     # Port that the web server will listen on.
4     listen      80
5
6     # Host that will serve this project.
7     server_name  app.dev
8
9     # Useful logs for debug.
10    access_log    /path/to/access.log;
11    error_log     /path/to/error.log;
12    rewrite_log   on;
13
14    # The location of our projects public directory.
15    root          /path/to/our/public;
16
17    # Point index to the Laravel front controller.
18    index         index.php;
19
20    location / {
21
22        # URLs to attempt, including pretty ones.
23        try_files $uri $uri/ /index.php?$query_string;
24
25    }
26
27    # Remove trailing slash to please routing system.
28    if (!-d $request_filename) {
29        rewrite    ^/(.+)/$ /$1 permanent;
30    }
31
32    # PHP FPM configuration.
33    location ~* \.php$ {
34        fastcgi_pass            unix:/var/run/php5-fpm.sock;
35        fastcgi_index           index.php;
36        fastcgi_split_path_info ^(.+\.php)(.*)$;
37        include                 /etc/nginx/fastcgi_params;
```

```
38         fastcgi_param                SCRIPT_FILENAME $document_root$fastcgi\
39 i_script_name;
40     }
41
42     # We don't need .ht files with nginx.
43     location ~ /\.ht {
44         deny all;
45     }
46
47 }
```

Nem todos os servidores web serão os mesmos. Isto significa que uma configuração genérica que se encaixe em todas as situações seria uma tarefa impossível. Mesmo assim, é o suficiente para você poder começar.

Eu compartilhei as configurações padrões utilizadas neste capítulo no Github para que todo mundo possa contribuir com elas. Você pode encontrá-las no repositório [daylerees/laravel-website-configs](https://github.com/daylerees/laravel-website-configs)¹⁷.

Enquanto Nginx é uma grande escolha de servidor web, o Apache também é muito usado. Vamos dar uma olhada em como configurá-lo.

Apache

O servidor web Apache pode ser instalado nos sistemas Linux baseados em Debian com o seguinte comando.

```
1 sudo apt-get install apache2 php5
```

Aqui tem uma configuração de um VirtualHost do Apache que vai se encaixar na maioria das situações, sintá-se à vontade para contribuir com o [repositório no github](https://github.com/daylerees/laravel-website-configs)¹⁸ se qualquer alteração for necessária.

¹⁷[http://github.com/daylerees/laravel-website-configs](https://github.com/daylerees/laravel-website-configs)

¹⁸[http://github.com/daylerees/laravel-website-configs](https://github.com/daylerees/laravel-website-configs)

```
1  <VirtualHost *:80>
2
3      # Host that will serve this project.
4      ServerName      app.dev
5
6      # The location of our projects public directory.
7      DocumentRoot    /path/to/our/public
8
9      # Useful logs for debug.
10     CustomLog        /path/to/access.log common
11     ErrorLog         /path/to/error.log
12
13     # Rewrites for pretty URLs, better not to rely on .htaccess.
14     <Directory /path/to/our/public>
15         <IfModule mod_rewrite.c>
16             Options -MultiViews
17             RewriteEngine On
18             RewriteCond %{REQUEST_FILENAME} !-f
19             RewriteRule ^ index.php [L]
20         </IfModule>
21     </Directory>
22
23 </VirtualHost>
```

Estrutura do Projeto

Eu tenho um capítulo no livro anterior que cobria a estrutura dos diretórios do pacote, e onde os arquivos ficavam. Eu acho que foi um capítulo muito bom, e eu gostaria de fazer novamente juntamente com as alterações na estrutura dos diretórios que aconteceram desde o Laravel 3.

Esta seção vai partir do princípio que você já rodou `composer install` em uma nova instalação do Laravel 4.

Raiz do Projeto

Vamos começar dando uma olhada na estrutura do diretório raiz.

- app/
- bootstrap/
- vendor/
- public/

- .gitattributes
- .gitignore
- artisan
- composer.json
- composer.lock
- phpunit.xml
- server.php

Por que nós não passamos por todos esses itens da raiz? Parece uma boa ideia para mim!

app

Primeiramente nós temos o diretório `app`. Ele é usado para fornecer um ambiente inicial para todos os arquivos dos seus projetos. Isto inclui classes que contém funcionalidades da aplicação, arquivos de configuração e muito mais. A pasta `app` é muito importante, então ao invés de dar uma breve introdução em um simples parágrafo, nós vamos cobrir toda ela no final desta seção. Por enquanto, apenas saiba que é onde os arquivos do seu projeto vão estar.

bootstrap

- `autoload.php`
- `paths.php`
- `start.php`

O diretório `bootstrap` contém alguns arquivos que executam os processos de inicialização do framework. O arquivo `autoload.php` contém muitos destes processos, e só deve ser editado por usuários experientes do Laravel.

O arquivo `paths.php` constrói um array dos diretórios do sistema que são usados pelo framework. Se por alguma razão você decidir alterar a estrutura dos diretórios do framework, você provavelmente vai precisar alterar o conteúdo deste arquivo para que suas alterações tenham efeito.

O arquivo `start.php` contém mais processos de inicialização do framework. Eu não quero entrar muito em detalhes agora pois isso pode causar uma confusão desnecessária. Ao invés disso, você deve saber que os `environments` (ambientes) podem ser editados aqui. Se você não sabe para que os `environments` são usados, não se preocupe. Nós vamos passar por isto depois.

O conteúdo da pasta `bootstrap` só deve ser editado por usuários experientes do Laravel que precisam alterar a estrutura do framework. Se você é novo no Laravel, então simplesmente ignore este diretório por enquanto, mas não esqueça dele completamente! O Laravel necessita dele para funcionar.

vendor

A pasta `vendor` contém todos os pacotes do Composer que são usados pela sua aplicação. E claro, inclui o pacote do framework Laravel. Para mais informações sobre este diretório, por favor volte para o capítulo inicial que fala sobre Composer.

public

- `packages/`
- `.htaccess`
- `favicon.ico`
- `index.php`
- `robots.txt`

A pasta `public` deve ser o único diretório acessível via web de uma aplicação Laravel. É normalmente onde seus assets como CSS, JavaScripts e imagens vão estar. Vamos dar uma olhada no seu conteúdo.

Na pasta `packages` é onde vão ficar os assets que precisam ser instalados por pacotes de terceiros. Eles são guardados em um diretório separado para que eles não tenham nenhum tipo de conflito com os assets da sua própria aplicação.

O Laravel 4 vem com um arquivo padrão `.htaccess` para o servidor Apache. Ele contém algumas configurações padrões que vão funcionar para a maioria dos usuários do framework. Se você usou algum outro servidor, você pode ignorar ou deletar este arquivo.

Por padrão, servidores web vão tentar apontar para a raiz do diretório para tentar encontrar um arquivo `favicon.ico`. Este é um arquivo que controla a pequena imagem de 16 x 16px que é exibida nas abas dos navegadores. O problema é, quando este arquivo não existe o servidor web gosta de reclamar por isso. E isto causa entradas de log desnecessárias. Para contornar este problema, o Laravel fornece um `favicon.ico` padrão, em branco, que depois pode ser substituído caso você queira.

O arquivo `index.php` é o principal controlador do Laravel. É o primeiro arquivo que o servidor web utiliza quando uma requisição é recebida do navegador. Este é o arquivo que vai começar o processo de inicialização do framework. Não delete este arquivo, não é uma boa ideia!

O Laravel inclui um arquivo `robots.txt` padrão que permite todos os hosts por padrão. Isso é para sua conveniência, sintá-se livre para alterar este arquivo se necessário.

.gitattributes

O Laravel também fornece algumas configurações padrões para utilizar com o controlador de versão Git. Git é a escolha mais popular para versionamento. Se você não pretende versionar o seu código com Git sintá-se à vontade para remover estes arquivos.

.gitignore

O arquivo `.gitignore` contém alguns padrões para informar para o Git quais diretórios não devem ser versionados. Note que isto também inclui o diretório `vendor` e o diretório `app/storage`. A pasta `vendor` é incluída para evitar que os pacotes de terceiros sejam versionados.

O arquivo `composer.lock` também está incluído, porém você talvez queira removê-lo, e versionar o seu arquivo `lock` para permitir que o seu ambiente de produção instale exatamente as mesmas dependências que o seu ambiente de desenvolvimento. Você vai encontrar mais informações sobre este assunto no capítulo inicial sobre Composer.

artisan

O arquivo `artisan` é um executável que é usado para executar os comandos do Artisan pelo terminal. O Artisan possui muitos comandos úteis com atalhos ou funcionalidades adicionais para o framework. Seus comandos vão ser vistos em detalhes em um capítulo posterior.

composer.json e composer.lock

Os dois arquivos, `composer.json` e `composer.lock`, contém informações sobre os pacotes do Composer usados no projeto. Novamente, você pode ver mais sobre estes arquivos no capítulo inicial sobre Composer.

phpunit.xml

O arquivo `phpunit.xml` fornece uma configuração padrão para utilizar com o framework de testes unitários PHPUnit. Ele também carrega as dependências do Composer e executa qualquer teste que esteja dentro da pasta `app/tests`. Informações sobre testes unitários com o Laravel vão ser vistos em um capítulo posterior, fique atento!

O diretório app

Aqui é onde sua aplicação vai tomar forma. É o diretório que você vai passar a maior parte do seu tempo. Por isto, por que não ficamos mais familiarizados com ele?

- `commands/`
- `config/`
- `controllers/`
- `database/`
- `lang/`
- `models/`
- `start/`
- `storage/`
- `tests/`
- `views/`
- `filters.php`
- `routes.php`

commands

O diretório dos comandos contém todos os seus comandos do Artisan que são usados na sua aplicação. O Artisan CLI não fornece nenhuma funcionalidade padrão para ajudar a construir o seu projeto, mas você talvez queira criar comandos personalizados para fazerem isto por você.

config

As configurações do framework e da sua aplicação são mantidos dentro desta pasta. As configurações do Laravel são um conjunto de arquivos PHP contendo informações chave-valor. O diretório

também contém subdiretórios que permitem diferentes configurações para diferentes environments (ambientes).

controllers

Como o nome sugere, este diretório vai conter os seus controllers. Controllers podem ser usados para a lógica da aplicação, e para juntar diferentes partes da sua aplicação. O diretório foi adicionado aos diretórios padrões que o autoload do `composer.json` carrega.

database

Você deve escolher utilizar um banco de dados como um método de armazenamento a longo prazo, e este diretório vai ser usado para guardar os seus arquivos para criação do seu banco de dados, e métodos para adicionar registros no banco de dados com exemplos. O banco de dados padrão SQLite também está localizado nesta pasta.

lang

A pasta `lang` contém os arquivos PHP com arrays de strings que podem ser utilizados para disponibilizar suporte a localização para sua aplicação. Subpastas nomeadas por região permitem que os arquivos possam existir para múltiplas línguas.

models

O diretório dos models contém os seus models. Surpreso? Os Models são usados para representar o seu modelo de negócio, ou fornecer interação com o banco de dados, Confuso? Não se preocupe. Não vamos cobrir os models detalhadamente em um capítulo posterior. Saiba que o model `User` já foi disponibilizado para você poder utilizar autenticação na sua aplicação mais facilmente. Como o diretório de controllers, este diretório também está nas configurações padrões de autoload do `composer.json`.

start

Como o diretório `bootstrap` mantém os processos de inicialização que pertencem ao framework. O diretório `start` contém processos de inicialização que pertencem à sua aplicação. Como sempre, alguns padrões já foram disponibilizados para você.

storage

Quando o Laravel precisa escrever alguma coisa em disco, isto é feito dentro do diretório `storage`. Por isto seu servidor web precisa de permissão de escrita neste diretório.

tests

O diretório `tests` vai conter todos os seus testes unitários e de aceitação da sua aplicação. Nas configurações padrões do PHPUnit disponibilizadas pelo Laravel, vão procurar por testes dentro desta pasta por padrão.

views

A pasta de views é usada para manter os templates visuais da sua aplicação. Uma view padrão é disponibilizada para exemplo.

filters.php

O arquivo `filters.php` é usado para manter os filtros de rotas para sua aplicação. Nós vamos aprender mais sobre filtros em um futuro capítulo.

routes.php

O arquivo `routes.php` contém todas as rotas da sua aplicação. Você não sabe o que são as rotas? Bem, então não vamos mais perder tempo. Vamos para o próximo capítulo!

Roteamento Básico

Você está em um caminho longo e direto de lama para o deserto de... Que país tem um deserto mesmo? Austrália! Austrália vai servir.

Não entre em pânico! Esta seção necessita de um pouco de imaginação, então me acompanhe. Eu suponho que nosso personagem principal deva ter um nome, porque o livro parece fazer isso. Hmm... vamos chamá-lo de Browsy Mc'Request.

O Browsy Mc'Request está caminhando sozinho na estrada, cheio de lama, em algum lugar do país dos Cangurus. Browsy não é um nativo nestas terras e está totalmente perdido e sem ideia de onde está. Ele, porém, sabe onde ele quer chegar. Um amigo seu lhe disse anteriormente sobre "Jason Lewis' Shelter for Moustached Marsupials". Ele sabe que novembro está se aproximando rapidamente e os estilistas de canguru vão precisar de toda ajuda que puderem ter.

Derrepente, Browsy se depara com o que parece uma encruzilhada mais adiante na estrada. Animado ao ver algo que não seja a já familiar linha reta de lama, ele corre até a encruzilhada com a esperança de que ele ache alguma direção.

Perto de chegar na encruzilhada, ele procura desesperadamente por alguma indicação de qual caminho possa o levar para o abrigo, mas não acha nada,

Felizmente para Browsy, neste exato momento, um experiente desenvolvedor web aparece. O forte e bonito desenvolvedor bate em um poste de informações, e some em um piscar de olhos.

Browsy é deixado perplexo tanto pela velocidade quanto pela leve beleza do desenvolvedor. Eu mencionei que ele era bonito? O desenvolvedor, não o Browsy. O Browsy parece aquele cara daquele livro que fica falando 'meu precioso' um monte. Pelo menos é assim que eu vejo.

Após recuperar o fôlego, Browsy começa a estudar o poste de informações.

À esquerda você vai encontrar "Bob's Bobcat Riding Stables", e a direita... ah! Agora sim. À direita você vai encontrar "Jason Lewis' Shelter for Moustached Marsupials".

Eventualmente, Browsy encontra o caminho para o Marsupial Shelter e vive uma vida longa e feliz aprendendo a preparar os diferentes tipos de bigodes que crescem nos cangurus.

A moral da história é que sem o (lindo) desenvolvedor mostrando o caminho, Browsy Mc'Request não teria encontrado o caminho para sua aplica... para o Marsupial Shelter.

Como na história, uma requisição vinda de um navegador não vai ser direcionada para a lógica de sua aplicação sem uma forma de roteamento.

Roteamento Básico

Vamos dar uma olhada na requisição que está sendo feita para o Laravel.

```
1 http://domain.com/my/page
```

Neste exemplo, nós estamos utilizando o protocolo ‘http’ (usado pela maioria dos navegadores), para acessar sua aplicação Laravel hospedada no domínio `domain.com`. A parte `my/page` da URL é o que nós vamos usar para rotear as requisições para a lógica apropriada.

Eu vou tomar a frente a jogá-lo para o final. Rotas são definidas dentro do arquivo `app/routes.php`, então vamos criar uma rota que vá escutar a requisição que mencionamos anteriormente.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('my/page', function() {
6     return 'Hello world!';
7 });
```

Agora digite `http://domain.com/my/page` no seu navegador, substituindo `domain.com` pelo endereço da sua aplicação Laravel. Provavelmente o endereço seja `localhost`.

Se tudo foi configurado corretamente, você vai ver as palavras ‘Hello world’ na bela fonte Times New Roman! Por que não damos uma olhada mais de perto na declaração da rota para ver como funciona.

Rotas são sempre declaradas usando a classe `Route`. Está é a parte que vai logo no início, antes dos `::`. A parte do `get` é o método da classe de roteamento que é usado para pegar as requisições feitas com o verbo HTTP ‘GET’ para uma determinada URL.

Todas as requisições feitas por um navegador possuem um verbo. Na maioria das vezes, este verbo vai ser o GET, que é usado para requisitar uma página web. Uma requisição GET é enviada toda vez que você digita um novo endereço no seu navegador.

Porém, ele não seja o único tipo de requisição. Também existe o POST, que é usado para fazer uma requisição e enviar dados juntamente com ela. Estas requisições são normalmente resultados do envio de formulários, onde os dados precisam ser enviados para o servidor web sem serem exibidos na URL.

Existem outros verbos HTTP disponíveis também. Aqui estão alguns dos métodos que a classe de roteamento disponibiliza para você.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get();
6 Route::post();
7 Route::put();
8 Route::delete();
9 Route::any();
```

Todos estes métodos aceitam alguns parâmetros, então sintá-se à vontade para utilizar o verbo HTTP correto para cada situação. Isto é conhecido como RESTful routing. Nós vamos entrar em mais detalhes sobre isto em um futuro tópico. Por enquanto, tudo que você precisa saber é que GET é utilizado para fazer requisições, e o POST é usado quando você precisa enviar dados para o servidor.

O método `Route::any()` é usado para pegar qualquer verbo HTTP. Porém, eu recomendo utilizar o verbo HTTP correto para cada situação se você está fazendo uma aplicação mais evidente.

Vamos voltar ao exemplo. Aqui está ele novamente para refrescar a sua memória.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('my/page', function() {
6     return 'Hello world!';
7 });
```

A próxima parte da snippet é o primeiro parâmetro para o método `get()` (ou qualquer outro verbo). Este parâmetro define a URI que você deseja que a URL mapeie. Neste caso, nós estamos mapeando a URI `my/page`.

O parâmetro final é usado nesta instância para fornecer a lógica da aplicação para manipular a requisição. Aqui nós estamos usando uma `Closure`, que também é conhecida como função anônima. Closures são funções simples, sem um nome e que podem ser atribuídas a variáveis, como outros tipos simples de valores.

Por exemplo, o exemplo anterior poderia ter sido escrito assim:

```
1 <?php
2
3 // app/routes.php
4
5 $logic = function() {
6     return 'Hello world!';
7 }
8
9 Route::get('my/page', $logic);
```

Aqui nós estamos armazenando a Closure dentro da variável `$logic`, e depois passando ela para o método `Route::get()`.

Neste caso, o Laravel vai executar a Closure apenas quando a requisição estiver usando o verbo HTTP GET e associe a URI `my/page`. Nestas condições, a declaração `return` vai ser executada e a string “Hello world!” vai ser exibida no navegador.

Você pode definir quantas rotas você quiser, por exemplo:

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('first/page', function() {
6     return 'First!';
7 });
8
9 Route::get('second/page', function() {
10    return 'Second!';
11 });
12
13 Route::get('third/page', function() {
14    return 'Potato!';
15 });
```

Tente navegar nestas URLs para ver como sua aplicação se comporta.

```
1 http://domain.com/first/page
2 http://domain.com/second/page
3 http://domain.com/third/page
```

Você vai provavelmente querer mapear a raiz de duas aplicações, por exemplo..

```
1 http://domain.com
```

Normalmente, isto vai ser usado para hospedar a página inicial da sua aplicação. Vamos criar uma rota para associar isto.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function() {
6     return 'In soviet Russia, function defines you.';
7 });
```

Hey, espere um pouco! Nós não temos uma barra na nossa URL?!

SE ACALME LEITOR! Você está ficando frenético.

Uma rota contendo uma barra vai associar a URL do site, contendo uma barra ou não. A rota acima vai responder para as duas seguintes URLs.

```
1 http://domain.com
2 http://domain.com/
```

URLs podem ter quantos segmentos (as partes entre as barras) você quiser. Você pode usar para montar a hierarquia do site.

Considere a seguinte estrutura:

```
1 /
2 /books
3     /fiction
4     /science
5     /romance
6 /magazines
7     /celebrity
8     /technology
```

OK, até agora um pequeno site, mas um belo exemplo de uma estrutura que você vai encontrar facilmente na web. Vamos recriar isto utilizando as rotas do Laravel.

Para deixar mais claro, a Closure correspondente de cada rota está vazia.


```
1  <?php
2
3  // app/routes.php
4
5  // home page
6  Route::get('/', function() {});
7
8
9  // routes for the books section
10 Route::get('/books', function() {});
11 Route::get('/books/fiction', function() {});
12 Route::get('/books/science', function() {});
13 Route::get('/books/romance', function() {});
14
15 // routes for the magazines section
16 Route::get('/magazines', function() {});
17 Route::get('/magazines/celebrity', function() {});
18 Route::get('/magazines/technology', function() {});
```

Com esta coleção de rotas, nós facilmente criamos a hierarquia de um site. Porém, você pode ter percebido um certo número de repetições. Vamos ver uma maneira de minimizar essas repetições, e assim, aderir ao princípio DRY (Don't Repeat Yourself).

Parâmetros de Rotas

Parâmetros de rotas podem ser usados para inserir placeholders (espaços reservados) nas suas definições de rotas. Isso vai criar um espaço em cada segmento que pode ser pego dentro da Closure e pode ser passado para a lógica da aplicação.

Isto pode parecer um pouco confuso, mas quando você vê funcionando tudo parece fazer mais sentido. Vamos lá...

```
1  <?php
2
3  // app/routes.php
4
5  // routes for the books section
6  Route::get('/books', function()
7  {
8      return 'Books index.';
9  });
10
```

```
11 Route::get('/books/{genre}', function($genre)
12 {
13     return "Books in the {$genre} category.";
14 });
```

Neste exemplo, nós eliminamos a necessidade de todas as rotas de gênero incluindo um placeholder na rota. O placeholder {genre} vai mapear qualquer coisa que seja digitado depois da URL /books/. E o Laravel vai passar o valor para o parâmetro \$genre da Closure, que vai nos permitir utilizar esta informação dentro de nossa lógica.

Por exemplo, se você visitar a seguinte URL:

```
1 http://domain.com/books/crime
```

Você vai receber a seguinte resposta:

```
1 Books in the crime category.
```

Nós também podemos remover o requerimento para o livro para a rota inicial dos livros usando um parâmetro opcional. Um parâmetro pode ser opcional ao adicionar um ponto de interrogação (?) no final do seu nome. Por exemplo:

```
1 <?php
2
3 // app/routes.php
4
5 // routes for the books section
6 Route::get('/books/{genre?}', function($genre = null)
7 {
8     if ($genre == null) return 'Books index.';
9     return "Books in the {$genre} category.";
10 });
```

Se um gênero não for fornecido com a URL, então o valor da variável \$genre vai ser nulo, e a mensagem Books index. vai aparecer.

Se nós não quisermos que o valor da rota seja nulo por padrão, nós podemos especificar uma alternativa utilizando uma atribuição. Por exemplo:

```
1 <?php
2
3 // app/routes.php
4
5 // routes for the books section
6 Route::get('/books/{genre?}', function($genre = 'Crime')
7 {
8     return "Books in the {$genre} category.";
9 });
```

Agora, se nós visitarmos a seguinte URL:

```
1 http://domain.com/books
```

Nós vamos receber esta resposta:

```
1 Books in the Crime category.
```

Eu espero que você já esteja começando a ver como as rotas são usadas para direcionar as requisições para o seu site, e como uma ‘cola de código’ que é usada para manter sua aplicação junta.

Ainda tem muita coisa sobre roteamento. Mas antes de voltarmos para isto, vamos cobrir mais do básico. No próximo capítulo nós vamos ver os tipos de respostas que o Laravel pode nos oferecer.


```
12         </head>
13         <body>
14             <p>This is the perfect response!</p>
15         </body>
16     </html>';
17 });
```

Incrível! Agora você começa a ver o poder e a facilidade do Laravel... apenas brincando. Nós não queremos exibir o HTML desta maneira. Incorporar lógica junto seria chato, e, mais importante, muito errado!

Por sorte, o Laravel possui um diferente número de objetos de resposta que retornam uma resposta significativa muito mais facilmente. Vamos dar uma olhada na resposta do objeto View, que é bem mais empolgante!

Views

As views são a parte visual da sua aplicação, Dentro do padrão Model View Controller elas são a parte das views. Por isso que elas são chamadas views. Não é ciência de foguetes. Ciência de foguetes vão ser vistas em um próximo capítulo.

A view é apenas um template que pode ser retornado pelo navegadores, embora é provável que suas views contenham HTML. As views são usadas com a extensão .php e normalmente estão dentro da pasta app/views. Isto significa que o código .php pode ser interpretado dentro de nossas views. Vamos criar uma simples view por enquanto.

```
1 <!-- app/views/simple.php -->
2
3 <!doctype html>
4 <html lang="en">
5 <head>
6     <meta charset="UTF-8">
7     <title>Views!</title>
8 </head>
9 <body>
10     <p>Oh yeah! VIEWS!</p>
11 </body>
12 </html>
```

Ótimo! Um pouco de HTML dentro de view app/views/simple.php. Agora vamos criar a view.

Nós já não fizemos isso?

Haha! Sim, você fez uma pequena view, mas eu não quis dizer ‘Vamos fazer um novo arquivo de view.’. Ao invés disto, vamos criar um novo objeto da View que utilize a view que nós acabamos de criar. Os arquivos representando as views são chamados de templates. Isso pode ajudar você a distinguir o objeto View dos templates HTML.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return View::make('simple');
8 });
```

Agora sim! Você estava falando do método `make()`.

De fato eu estava meu amigo! Utilizando o método `View::make()` nós podemos criar uma nova instância de um objeto de resposta View. O primeiro parâmetro que nós passamos para o método `make()` é o template que nós queremos usar. Você vai perceber que nós não passamos o caminho completo (`app/views/simple.php`). Isto é porque o Laravel é esperto. Ele vai por padrão assumir que suas views estão na pasta `app/views` e vai procurar pelo arquivo ali com a sua extensão apropriada.

Se você olhar atentamente a Closure você vai perceber e o objeto da View que nós criamos está sendo retornado. Isto é muito importante, visto que o Laravel vai servir o resultado da nossa Closure para o navegador.

Vá em frente e acesse a URI / da sua aplicação. Ótimo, este é o template que nós escrevemos!

Posteriormente neste livro, você vai aprender como fazer diferentes tipos de templates que funcionam com o objeto View para deixar sua vida mais fácil. Por enquanto, nós vamos continuar com os básicos para compreender os conceitos fundamentais do Laravel.

Dados nas Views

Poder exibir templates é ótimo. Realmente. Mas e se nós quisermos usar alguns dados vindos de nossas Closures? Em um capítulo anterior nós aprendemos como nós podemos utilizar parâmetros nas rotas. E se nós quisermos poder passar estes parâmetros para a View? Vamos dar uma olhada em como isto pode ser feito.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/{squirrel}', function($squirrel)
6  {
7      $data['squirrel'] = $squirrel;
8
9      return View::make('simple', $data);
10 });
```

Na rota acima, nós pegamos o parâmetro `$squirrel` e adicionamos ao array de dados da nossa view. O segundo parâmetro para o método `make()` recebe um array que é passado para o template. Eu normalmente chamo meu array de `$data` para indicar eu são os dados da minha view, mas você chamar como quiser.

Antes de nós começarmos a utilizar estes dados, vamos falar um pouco mais sobre o array de dados da view. Quando o array é passado para a view, as chaves do array são ‘extraídas’ dentro de variáveis que possuem o mesmo nome que a chave do array e o seu devido valor, É um pouco complicado de explicar sem um exemplo, então eu vou simplificar para você.

Aqui nós temos um array que var ser passado para o método `View::make()` com os dados da view.

```
1  <?php
2  array('name' => 'Taylor Otwell', 'status' => 'Code Guru')
```

No template, nós podemos acessar estes valores assim:

```
1  <?php echo $name;           // retorna 'Taylor Otwell' ?>
2
3  <?php echo $status;         // retorna 'Code Guru' ?>
```

A chave do array `name` vira uma variável `$name` dentro do nosso template. Você pode armazenar arrays multidimensionais de quantos níveis você precisar no seu array de dados. Sinta-se à vontade para experimentar!

Vamos utilizar a variável `$squirrel` no template que nós criamos.

```
1 <!-- app/views/simple.php -->
2
3 <!doctype html>
4 <html lang="en">
5 <head>
6     <meta charset="UTF-8">
7     <title>Squirrels</title>
8 </head>
9 <body>
10     <p>I wish I were a <?php echo $squirrel; ?> squirrel!</p>
11 </body>
12 </html>
```

Agora se nós visitarmos a URI /gray nós recebemos uma página começando com ‘I wish I were a gray squirrel!’.

Simples, não? Usando views, você não vai mais precisar retornar strings diretamente de suas Closures!

Anteriormente eu mencionei que existem diferentes tipos de respostas. Em algumas circunstâncias você talvez queira redirecionar o fluxo da sua aplicação para outra rota ou parte lógica de sua aplicação. Neste caso, o objeto de resposta `Redirect` vai ser útil. O Laravel também pensou nisso!

Redirecionamentos

Um redirecionamento é um tipo especial de objeto de resposta que redireciona o fluxo da sua aplicação para outra rota. Vamos criar algumas rotas de exemplo para que eu possa explicar com mais detalhes.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('first', function()
6 {
7     return 'First route.';
8 });
9
10 Route::get('second', function()
11 {
12     return 'Second route.';
13 });
```


Adicionando as rotas acima, você vai receber a mensagem 'First route.' quando visitar a URI `/first` e 'Second route.' quando visitar `/second`.

Excelente, isso é exatamente o que nós esperávamos que acontecesse. Agora vamos mudar completamente o fluxo da nossa aplicação adicionando um redirecionamento na primeira rota.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('first', function()
6  {
7      return Redirect::to('second');
8  });
9
10 Route::get('second', function()
11 {
12     return 'Second route.';
13 });
```

Na primeira rota nós estamos retornando o resultado do método `Redirect::to()` e passando a URI de localização desejada. Neste caso, nós estamos passando a URI da segunda rota como o local a ser redirecionado.

Agora se você visitar a URI `/first` você vai ser saudado com o texto 'Second route.'. Isto porque ao invés de receber o objeto retornado, o Laravel é redirecionado do fluxo da aplicação para outra rota. Neste caso o fluxo foi redirecionado para o fluxo da segunda rota.

Isto pode ser realmente útil quando alguma condição falhe e você precise redirecionar o usuário para uma localização mais útil. Este é um exemplo usado no sistema de autenticação (que nós vamos ver depois) que vai nos mostrar um outro caso de uso.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('books', function()
6  {
7      if (Auth::guest()) return Redirect::to('login');
8
9      // Show books to only logged in users.
10 });
```

Neste exemplo, se um usuário que não está logado no sistema visitar a URL `/books`, então ele é considerado um convidado e vai ser redirecionado para a página de login.

Posteriormente, você vai encontrar uma maneira melhor de bloquear o acesso quando nós entrarmos nos filtros de rotas, então não fique muito preso no exemplo acima. Ao invés disto, apenas tenha em mente que nós podemos redirecionar o usuário para um local desejado se necessário.

Respostas Personalizadas

Os dois objetos, `View` e `Redirect` herdam do objeto do Laravel `Response`. O objeto `Response` é uma instância da classe que pode ser entregue para o Laravel como resultado de uma closure ou uma ação de um controller para permitir que o framework sirva o tipo certo de resposta para o navegador nas diferentes situações possíveis.

Objetos de resposta normalmente contêm um corpo, um código de status, cabeçalhos HTTP e outras informações úteis. Por exemplo, o corpo da `View` seria o seu conteúdo HTML. O código de status de um redirecionamento seria `301`. O Laravel utiliza estas informações para montar resultados completos que podem ser enviados para o navegador.

Porém, nós não estamos limitados a usar a `View` e o `Redirect`. Nós também podemos criar os nossos próprios objetos de resposta para se encaixarem em nossas necessidades. Vamos dar uma olhada em como isto pode ser feito.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('custom/response', function()
6 {
7     return Response::make('Hello world!', 200);
8 });
```

No exemplo acima nós utilizamos a função `Response::make()` para criar um objeto de resposta. O primeiro parâmetro é o conteúdo da resposta e o segundo parâmetro é o código de status HTTP que vai ser enviado juntamente com a resposta.

Se você não viu códigos de status HTTP ainda, então pense neles como o status de notificação que o navegador recebe da sua página. Por exemplo, se tudo acontecer como o esperado, a resposta padrão deve conter um código de status `200`, que na linguagem dos servidores significa 'OK'. Um código `302` indica que um redirecionamento foi feito.

Na verdade, eu aposto que você já passou pela infame `404` página não encontrada. O `404` é o código recebido pelo navegador quando a URL requisitada não for encontrada.

Simplificando, a resposta acima vai exibir o conteúdo 'Hello world!' com o código de status de 200 para avisar o navegador que a requisição foi um sucesso.

Cabeçalhos HTTP são um conjunto de pares de chave-valor com dados representando informações úteis para o navegador que está recebendo a nossa resposta. Normalmente, eles são usados para indicar o formato do resultado ou por quanto tempo um conteúdo deve ficar no cache. Porém, nós podemos definir cabeçalhos customizados se nós quisermos. Vamos ver como nós podemos fazer isto.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('custom/response', function()
6  {
7      $response = Response::make('Hello world!', 200);
8      $response->headers->set('our key', 'our value');
9      return $response;
10 });
```

Nós criamos um simples objeto de resposta da mesma maneira que o exemplo anterior. Porém, desta vez, nós também setamos um cabeçalho customizado.

O conjunto de cabeçalhos HTTP pode ser acessados como a propriedade `headers` do objeto de resposta.

```
1  <?php
2
3  var_dump($response->headers);
```

Usando o método `set()` neste conjunto nós podemos adicionar nosso próprio cabeçalho para o conjunto, passando a chave como o primeiro parâmetro e valor do cabeçalho como o segundo.

Uma vez que o nosso cabeçalho foi adicionado nós simplesmente retornamos o objeto de resposta da mesma maneira que anteriormente. O navegador vai receber os cabeçalhos juntamente com a resposta e nós podemos utilizar esta informação caso for preciso.

Vamos pensar em um exemplo mais útil. Hmmmm... vamos imaginar que nós queremos que a nossa aplicação sirva respostas em markdown ao invés de HTML. Nós sabemos que um navegador não é capaz de formatar respostas em markdown, mas talvez nós tenhamos uma outra aplicação desktop que possa.

Para indicar que o conteúdo é markdown e não HTML, nós vamos modificar o cabeçalho `Content-Type`. O `Content-Type` é um cabeçalho comum utilizado pelos navegadores para eles distinguirem os diferentes formatos de conteúdo que são enviados para eles. Não se confunda! Vamos ver um exemplo.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('markdown/response', function()
6  {
7      $response = Response::make('***some bold text***', 200);
8      $response->headers->set('Content-Type', 'text/x-markdown');
9      return $response;
10 });
```

Setando o conteúdo da resposta para um simples markdown, neste caso **some bold text**, e o cabeçalho Content-Type para o mime type do Markdown, nós enviamos uma resposta que pode ser interpretada como Markdown.

Nossa aplicação desktop agora pode fazer um requisição para a URI /markdown/response, examinar o cabeçalho Content-Type, e por receber o valor text/x-markdown vai saber que precisa um processador de markdown para manipular o conteúdo.

Como somos amigos, eu vou compartilhar um segredo com você. Venha mais perto. Venha aqui. Vamos nos sentar. O objeto de resposta não precente ao Laravel.

TRAIÇÃO? QUE LOCURA É ESSA?

Não fique tão preocupado. Para não reinventar a roda, O Laravel utiliza alguns dos mais robustos componentes que pertencem ao projeto Symfony 2. O objeto de resposta do Laravel na verdade herda a maioria do seu conteúdo do objeto Response pertencente ao componente Symfony HTTPFoundation.

Isto significa que se nós dermos uma olhada na API do objeto de resposta do Symfony, veremos que temos um monte de métodos extras que não estão disponíveis na documentação do Laravel. Agora que eu revelei este segredo, nada vai impedir você de se tornar um mestre no Laravel.

A documentação da API do objeto Response do Symfony [pode ser encontrada aqui](http://api.symfony.com/2.2/Symfony/Component/HttpFoundation/Response.html)¹⁹. Se você olhar para a página você vai perceber que a classe tem um atributo chamado \$headers. Isso mesmo! Ele é o conjunto de cabeçalhos que nós estávamos usando um minuto atrás.

Como o objeto Response do Laravel herda seu conteúdo deste, sintá-se à vontade para utilizar qualquer um dos métodos disponíveis na documentação do Symfony. Por exemplo, vamos dar uma olhada no método `setTtl()`. O que a API diz?

¹⁹<http://api.symfony.com/2.2/Symfony/Component/HttpFoundation/Response.html>“OobjetoResponsedoSymfony”

public Response setTtl(**integer \$seconds**)

Seta o tempo de vida da resposta para caches compartilhados. Este método ajusta a diretiva Cache-Control/s-maxage directive.

Parameters:

integer \$seconds Number of seconds

Return Value:

Response

Certo, então este método seta o tempo de vida para caches compartilhados. Eu não sou nenhum expert neste tipo de coisa, mas um tempo de vida sugere por quanto tempo a informação é considerada útil antes de ser descartada. Neste caso o tempo de vida se refere ao cache do conteúdo.

Vamos alterar o valor por diversão. Tendo visto a documentação do método, nós vimos que ele aceita um valor inteiro representando o tempo de vida em segundos. Vamos dar esta resposta um tempo de vida de 60 segundos. Como algum vilão cruel do James Bond.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('our/response', function()
6 {
7     $response = Response::make('Bond, James Bond.', 200);
8     $response->setTtl(60);
9     return $response;
10 });
```

Agora quando nossa resposta é enviada, ela vai ter um tempo de vida de 60 segundos. Ao analisar o componente do Symfony, nós temos um leque de opções avançadas que podem ser usadas para modificar as respostas da nossa aplicação para se encaixar em nossas necessidades.

Não se sinta sobrecarregado com toda a complexidade contida dentro da base do objeto Response. Na maioria das vezes, você vai apenas precisar usar as classes View e Response do laravel para servir simples respostas HTTP. O exemplo acima serve como um bom ponto de começo para usuários mais avançados que estão procurando como personalizar suas aplicações para casos específicos.

Atalhos de Respostas

O Laravel é seu amigo. É verdade... não, na verdade não é. O Laravel é mais que um amigo. Ele te ama. Mesmo. Por isto, ele fornece alguns atalhos de respostas para deixar sua vida mais fácil. Vamos dar uma olhada no que temos disponível.

Respostas JSON

É comum dentro de uma aplicação nós termos alguns tipos de dados que nós queremos servir como JSON. Isso pode ser um simples objeto ou um array de valores.

O Laravel disponibiliza a função `Response::json()` que vai configurar a resposta com os detalhes que são específicos de respostas JSON. Por exemplo um cabeçalho HTTP Content-Type apropriado.

Nós podemos setar eles manualmente, mas por que nos dar ao trabalho quando o Laravel pode fazer isto para nós? Vamos dar uma olhada neste método em ação.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('markdown/response', function()
6 {
7     $data = array('iron', 'man', 'rocks');
8     return Response::json($data);
9 });
```

Ao passar o array para o método `Response::json()`, ele o converte para uma string JSON e seta como o conteúdo da nossa resposta. Os cabeçalhos apropriados também foram enviados para explicar que os dados disponibilizados são uma string JSON. O navegador vai receber o seguinte conteúdo.

```
1 ["iron","man","rocks"]
```

Que é compacto e correto. Você pode utilizar este atalho para criar APIs JSON que são claras e funcionais.

Respostas de Download

Servir arquivos diretamente necessita de cabeçalhos determinados. Felizmente, o Laravel toma conta disto para você utilizando o atalho `Response::download()`. Vamos ver em ação como funciona.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('file/download', function()
6 {
7     $file = 'path_to_my_file.pdf';
8     return Response::download($file);
9 });
```

Agora se nós navegarmos para a URI `/file/download` o navegador vai iniciar o download ao invés de exibir a resposta. O método `Response::download()` recebe o caminho para um arquivo que vai ser servido como resposta.

Você também pode prover um segundo e terceiro parâmetro opcionais para configurar um código de status HTTP e um array de cabeçalhos. Por exemplo:

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('file/download', function()
6 {
7     $file = 'path_to_my_file.pdf';
8     return Response::download($file, 418, array('iron', 'man'));
9 });
```

Aqui nós vamos servir o nosso arquivo com o código de status HTTP 418 (I'm a Teapot) e um cabeçalho com o valor `iron=man`.

Este capítulo foi longo e isto já era previsto, mas eu tenho certeza eu você vai ver que retornar as respostas apropriadas pode ser muito mais valioso do que retornar simples strings.

No próximo capítulo nós vamos dar uma olhada nos filtros de rotas, que vão nos permitir proteger as nossas rotas ou executar ações antes e depois que elas sejam executadas.

Filtros

Eu me lembro de alguns anos atrás quando Jesse O'brien estava planejando um evento privado onde ele e seus amigos poderiam olhar o time local de hóquei jogar sua última partida contra os pandas do Laravel.

Nós todos sabemos que o poderoso time Laravel Pandas nunca perderia para os London Knights, mas o Jesse não queria saber. Insistia que este poderia ser o começo de uma estrada para a glória dos Knights.

O evento foi planejado para acontecer no Hoser Hut no centro de Londres. Um lugar muito receptivo e amigável para alguém nascido na América do Norte. (Terra do Xarope.)

Infelizmente o Hoser Hut tem uma reputação por não ser tão receptivo com os visitantes de fora. É um fato conhecido que os Americanos são jogados para fora das janelas do Hoser Hut regularmente. Por isto, Jesse decidiu que ele precisava de algum filtro na porta para evitar que americanos desagradáveis entrassem. Claro que o bom e velho britânico Dayle Rees é sempre bem-vindo no Hoser Hut. Ele é bem-vindo em qualquer lugar.

Jesse empregou um segurança para ficar em frente ao Hoser Hut e pedir identificação para confirmar se o visitante era do Canadá ou não.

O que Jesse fez foi implementar um filtro. Aqueles que passassem pelos requerimentos do filtro poderiam entrar no quente e aconchegante Hoser Hut para olhar os Laravel Pandas destruírem os London Knights. Qualquer americano que tentasse entrar no bar não passaria nos critérios do filtro, e veriam o lado brilhante de uma bota.

Vamos deixar Jesse com o seu jogo e ver como nós podemos usar filtros para proteger nossas rotas.

Filtros Básicos

Filtros são um conjunto de regras que podem ser aplicadas em um rota. Elas podem ser aplicadas antes ou depois da lógica de uma rota ser executada. Porém, você vai achar os filtros usados antes da execução das rotas mais úteis. Utilizando filtros antes das rotas nós podemos alterar o fluxo da aplicação se um certo conjunto de regras ou critérios não forem atingidos. É uma grande maneira de proteger as nossas rotas.

Como sempre, um exemplo fala mais que mil palavras. Vamos dar uma olhada em um filtro, mas primeiro nós precisamos de uma outra coisa. Vamos ver:


```
1 <!-- app/views/birthday.php -->
2
3 <h1>Happy Birthday!</h1>
4 <p>Happy birthday to Dayle, hurray!</p>
```

Ótimo! Agora que nós temos uma view de feliz aniversário, nós podemos criar o nosso primeiro filtro. Vamos lá:

```
1 <?php
2
3 // app/filters.php
4
5 Route::filter('birthday', function()
6 {
7     if (date('d/m/y') == '12/12/84') {
8         return View::make('birthday');
9     }
10 });
```

Aqui nós temos o nosso primeiro filtro. O Laravel disponibiliza o arquivo `app/filters.php` como uma localização genérica para os nossos filtros, mas na verdade nós podemos botá-los onde nós quisermos.

Nós utilizamos o método `Route::filter()` para criar um novo filtro. O primeiro parâmetro é um nome amigável que nós vamos usar para referenciar o filtro em uma rota. Neste exemplo eu dei o nome de 'birthday' para o filtro. O segundo parâmetro é a função de retorno, que neste exemplo é uma Closure.

A função de retorno é uma função que é chamada apenas quando o filtro for executado. Se ela retornar um objeto de resposta, da mesma maneira que nós usamos dentro da lógica de nossa rota, então esta resposta que vai ser servida ao invés do resultado da lógica da rota. Se nenhuma resposta for retornada pela função de retorno do filtro, então as lógicas das rotas vão seguir seu fluxo normal.

Isto nos possibilita um grande poder, então vá em frente e pratique sua risada maléfica. Sério, isso é importante.

Muahahahah!

Nós podemos alterar o fluxo da aplicação ou executar uma ação e permitir que a lógica de uma rota continue sua execução. Por exemplo, nós podemos querer apenas exibir um certo conteúdo do nosso site, para um determinado usuário. Isto significaria retornar uma resposta de redirecionamento para outra página. Nós também poderíamos escrever um log toda vez que um filtro é executado, para ver quais páginas foram visitadas. Talvez eu esteja me adiantando muito, vamos dar uma outra olhada no nosso filtro de exemplo.

```
1  <?php
2
3  // app/filters.php
4
5  Route::filter('birthday', function()
6  {
7      if (date('d/m') == '12/12') {
8          return View::make('birthday');
9      }
10 });
```

Analisando a nossa Closure, nós podemos ver que nós temos uma condição, e uma resposta. No nosso filtro, se a data atual for igual a '12/12/84', que é claro que é a data que a pessoa mais importante do universo nasceu, a Closure vai retornar uma resposta. Se uma resposta for retornada da closure, então nós vamos ser direcionados para a view de feliz aniversário. Se não, a lógica da nossa rota será executada normalmente.

Claro, antes que o filtro se torne útil nós precisamos adicioná-lo em uma rota. Porém, antes de fazermos isto nós precisamos alterar um pouco a estrutura da rota. Você se lembra como eu te falei que os métodos de roteamento aceitam uma closure como o segundo parâmetro? Eu falei uma pequena mentira novamente. Desculpe.

Os métodos de roteamento também aceitam um array como o segundo parâmetro. Nós podemos usar este array para atribuir parâmetros adicionais na rota. Vamos dar uma olhada em como uma rota se parece com um array como o segundo parâmetro.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', array(function()
6  {
7      return View::make('hello');
8  }));
```

É muito similar. O que nós fizemos foi mover a Closure para um array. Funciona da mesma maneira que anteriormente. Na verdade, ao manter a Closure no array, nós podemos incluir outros valores. E é assim que nós vamos relacionar o nosso filtro. Vamos começar dando uma olhada no filtro executado antes da rota.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', array(
6      'before' => 'birthday',
7      function()
8      {
9          return View::make('hello');
10     }
11 ));
```

Como você pode ver, nós criamos outra opção dentro do array. O índice 'before' diz ao framework que nós queremos rodar o nosso filtro 'birthday' antes da lógica da rota ser executada. O valor 'birthday' é o nome que nós demos ao nosso filtro.

Vamos executar nossa rota visitando /. Agora, assumindo que hoje não é 12 de Dezembro você vai ver a página de recepção do Laravel. Isto porque a condição do filtro falhou, e nenhuma resposta foi retornada.

Certo, então vamos esperar até 12 de Dezembro para que nós possamos ver o que acontece quando as condições dos filtros passarem e uma resposta for retornada.

Brincadeira, vamos alterar o filtro para fazer nossa condição passar. Nós podemos alterar a condição para um valor booleano true.

```
1  <?php
2
3  // app/filters.php
4
5  Route::filter('birthday', function()
6  {
7      if (true) {
8          return View::make('birthday');
9      }
10 });
```

Agora sim, agora vamos visitar / para ver se alguma coisa mudou. Opa, é meu aniversário! Vamos todos cantar feliz aniversário para mim. Na verdade, vamos apenas esperar até Dezembro. Assim, nós podemos ver que o filtro de aniversário foi executado com sucesso, e a view de feliz aniversário foi retornada.

Nós podemos anexar um filtro para ser executado depois da rota, usando a opção 'after' no array das rotas. Assim, o seu filtro apenas será executado depois da lógica da rota. Vamos ver um exemplo:

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', array(
6      'after' => 'birthday',
7      function()
8      {
9          return View::make('hello');
10     }
11 ));
```

Porém, você deve lembrar que os filtros executados depois das rotas não podem substituir a resposta. Neste caso, o nosso filtro de aniversário é um pouco desnecessário. Porém, você pode salvar logs, ou executar alguma operação de limpeza. Apenas tenha em mente que ele está lá se você precisar!

Filtros Múltiplos

Uma coisa que você deve ter em mente é que você pode aplicar quantos filtros você quiser em uma rota. Vamos ver alguns exemplos disto em ação. Primeiro, vamos adicionar múltiplos filtros para serem executados antes da rota.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', array(
6      'before' => 'birthday|christmas',
7      function()
8      {
9          return View::make('hello');
10     }
11 ));
```

Aqui nós adicionamos os filtros 'birthday' e 'christmas' na rota. Eu vou deixar sua imaginação decidir o que o filtro 'christmas' faz, mas tenha certeza que seja algo mágico.

O caractere pipe | pode ser usado para separar a lista de filtros. Eles vão ser executados da esquerda pra direita, e se o primeiro retornar uma resposta, a requisição vai chegar ao seu fim, e o conteúdo da resposta vai ser entregue como o resultado.

Se você quiser, você também pode usar um array para passar múltiplos filtros. Isto pode parecer mais com a maneira PHP de fazer as coisas.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', array(
6     'before' => array('birthday', 'christmas'),
7     function()
8     {
9         return View::make('hello');
10    }
11 ));
```

Use o que melhor se encaixar no seu código. Pessoalmente, eu prefiro arrays. Se você quiser, você também pode usar os filtros ‘before’ e ‘after’ ao mesmo tempo, assim:

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', array(
6     'before'    => 'birthday',
7     'after'     => 'christmas',
8     function()
9     {
10         return View::make('hello');
11    }
12 ));
```

Naturalmente, o filtro ‘before’ vai rodar primeiro, depois a lógica da rota e finalmente o filtro ‘after’. Então, você acha que nós terminados com filtros? Não se deixe levar!

Parâmetros nos Filtros

Como as funções PHP, os filtros podem aceitar parâmetros. Isto é uma grande maneira para evitar repetições, e aumentar a flexibilidade. Vamos começar com um exemplo, como sempre.

```
1  <?php
2
3  // app/filters.php
4
5  // before
6
7  Route::filter('test', function($route, $request)
8  {
9
10 });
11
12 // after
13
14 Route::filter('test', function($route, $request, $response)
15 {
16
17 });
```

Espera, por que existem dois filtros?

Bem notado! Bem, eles realmente são o mesmo filtro, mas mesmo assim, sua questão é válida. O Laravel disponibiliza um diferente conjunto de parâmetros para os filtros ‘before’ e ‘after’. Você vai perceber que os dois filtros recebem as variáveis `$route` e `$request`. Você pode chamá-las como quiser, mas eu nomeei eles assim para facilitar o entendimento.

Se você utilizar o `var_dump()` no primeiro parâmetro, você vai ver que é uma instância de `Illuminate\Routing\Route`. Você deve lembrar que ‘Illuminate’ é o codinome dos componentes do Laravel 4. A classe ‘Route’ representa a rota usada pela camada de rotas. Esta instância é gigante, vá em frente e execute um `var_dump()` se você não acredita neste galês matreiro. Você pode analisar para ter informações mais detalhadas contidas dentro, ou até mesmo alterar alguns dos valores para manipular o framework. Porém, isto é um tópico mais avançado, e fora do escopo deste capítulo, então vamos dar uma olhada no próximo parâmetro ao invés disto.

Como você pode ter adivinhado, o próximo parâmetro é a objeto ‘Request’ atual. A instância do ‘IlluminateHttpRequest’ representa o estado da requisição que foi enviada para o servidor web. Ela contém informações sobre a URL, e dados passados com a requisição, junto com uma grande riqueza de informações adicionais.

O filtro ‘after’ recebe um parâmetro adicional, uma instância do objeto de resposta que é retornado pela rota que o filtro está ativo. Esta instância é também a resposta da requisição atual.

Certo, estes parâmetros que o Laravel nos passou podem ser úteis para usuários avançados do framework, mas não seria ótimo se nós pudéssemos passar os nossos próprios parâmetros para os nossos filtros? Vamos dar uma olhada em como nós podemos fazer isto.

Primeiro nós precisamos adicionar uma variável para a Closure do nosso filtro, e ela deve estar após dos parâmetros que o Laravel nos passa, assim:

```
1  <?php
2
3  // app/filters.php
4
5  Route::filter('birthday', function($route, $request, $date)
6  {
7      if (date('d/m') == $date) {
8          return View::make('birthday');
9      }
10 });
```

Nosso filtro de aniversário foi alterado para aceitar um parâmetro `$date`. Se a data for igual a data fornecida então o filtro 'birthday' será executado.

Agora tudo o que precisamos saber é como fornecer os parâmetros para o filtro. Vamos dar uma olhada.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', array(
6      'before' => 'birthday:12/12',
7      function()
8      {
9          return View::make('hello');
10     }
11 ));
```

O parâmetro que nós passamos para o nosso filtro vai depois dos dois pontos : quando atribuído à uma rota. Vá em frete e teste, altere a data para o dia atual e veja o filtro ser disparado.

Se nós quisermos fornecer parâmetros adicionais, então nós precisamos fornecer outras variáveis na Closure. Vai se parecer com algo assim.

```
1  <?php
2
3  // app/filters.php
4
5  Route::filter('birthday', function($route, $request, $first, $second, $third)
6  {
7      return "{$first} - {$second} - {$third}";
8  });
```

Nós podemos aceitar quantos parâmetros nós quisermos. Para fornecer múltiplos parâmetros nós precisamos adicionar os dois pontos : entre o nome do filtro e seus parâmetros. Os parâmetros devem ser separados por vírgula ,. Vamos ver um exemplo:

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', array(
6      'before' => 'birthday:foo,bar,baz',
7      function()
8      {
9          return View::make('hello');
10     }
11 ));
```

Os valores 'foo', 'bar', and 'baz' serão passados para os parâmetros que nós adicionamos ao filtro. É valido notar que assim como as funções, os parâmetros dos filtros podem ter valores padrões.

```
1  <?php
2
3  // app/filter.php
4
5  Route::filter('example', function($route, $request, $optional = 'Yep!')
6  {
7      return $optional;
8  });
```

Forneça parâmetros adicionais, ou não. Depende de você, é sua aplicação!

Sinta-se à vontade para usar quantos parâmetros você precisar para fazer seu filtro mais eficiente. Tire vantagem desta grande funcionalidade.

Classes de Filtros

Closures são ótimas. Elas são muito convenientes, e funcionam em muitos casos. Porém, elas ficam presas a lógica do que nós estamos escrevendo. Nós não podemos instanciá-las, e isto as torna difícil de testar.

Por isto, qualquer funcionalidade do Laravel que necessita que de uma Closure possui uma alternativa. Uma classe PHP. Vamos dar uma olhada em como nós podemos usar classes para representar nossos filtros.

Antes de criarmos a nossa classe, nós precisamos de algum lugar para ela. Vamos criar uma nova pasta dentro da pasta `/app` chamada `filters`, e atualizar o nosso `composer.json` para incluir a nova pasta.

```
1  "autoload": {  
2      "classmap": [  
3          "app/commands",  
4          "app/controllers",  
5          "app/models",  
6          "app/filters",  
7          "app/database/migrations",  
8          "app/database/seeds",  
9          "app/tests/TestCase.php"  
10     ]  
11 }
```

Agora vamos criar a nova classe para o nosso filtro de aniversário. Vamos lá:

```
1  <?php  
2  
3  // app/filters/Birthday.php  
4  
5  class BirthdayFilter  
6  {  
7      public function filter($route, $request, $date)  
8      {  
9          if (date('d/m') == $date) {  
10             return View::make('birthday');  
11          }  
12      }  
13  }
```

Eu chamei minha classe de `BirthdayFilter`. Você não precisa adicionar o sufixo 'Filter', mas eu gosto de fazer mesmo assim, depende de você. O que você precisa porém, é implementar o método `filter()`. Ele funciona da mesma maneira que a Closure. Na verdade, como ele funciona exatamente igual a Closure eu não preciso explicar novamente, e ao invés disto, vamos dar uma analisada em como nós podemos associar um filtro à nossa classe.

Primeiro nós precisamos criar um atalho para o filtro, e novamente nós vamos usar a função `Route::filter()`. Porém, desta vez nós vamos passar uma string ao invés de uma closure como o segundo parâmetro. Assim:

```
1 <?php
2
3 // app/routes.php
4
5 Route::filter('birthday', 'BirthdayFilter');
```

O segundo parâmetro para o método é uma string que identifica a classe de filtro que vai ser utilizada. Se a classe de filtro estiver dentro de um namespace, forneça o namespace junto.

Agora que o atalho do filtro foi criado, nós podemos adicioná-lo a rota da mesma maneira que nós fizemos antes.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', array(
6     'before' => 'birthday',
7     function()
8     {
9         return View::make('hello');
10    }
11 ));
```

Lembre-se que você vai precisar rodar `composer dump-autoload` para que o Laravel consiga encontrar a sua classe.

Se você pretende testar totalmente o seu código, então criar classes de filtros é a melhor maneira para isto. Nós vamos ver mais sobre testes em um capítulo posterior.

Filtros Globais

Se você der uma olhada no arquivo `/app/filters.php`, você vai perceber dois filtros estranhos. Eles são os filtros globais que são executados antes e depois de todas as requisições para sua aplicação.

```
1  <?php
2
3  // app/filters.php
4
5  App::before(function($request)
6  {
7      //
8  });
9
10
11 App::after(function($request, $response)
12 {
13     //
14 });
```

Eles funcionam da mesma maneira que os filtros normais, exceto que eles se aplicam a todas as rotas por padrão. Isto significa que não é necessário adicioná-los aos arrays before e after nas nossas rotas.

Filtros Padrões

No arquivo `app/filters.php` existem alguns filtros que já foram criados para você. Vamos dar uma olhada nos primeiros três.

```
1  <?php
2
3  // app/filters.php
4
5  Route::filter('auth', function()
6  {
7      if (Auth::guest()) return Redirect::guest('login');
8  });
9
10
11 Route::filter('auth.basic', function()
12 {
13     return Auth::basic();
14 });
15
16 Route::filter('guest', function()
17 {
```

```
18     if (Auth::check()) return Redirect::to('/');
19 });
```

Todos estes filtros tem relação com a camada de autenticação do Laravel. Eles podem ser usados para restringir usuários que estão, ou não, logados na aplicação de acessar uma rota.

Em um próximo capítulo nós vamos dar uma olhada bem de perto na camada de autenticação, e o conteúdo destes filtros vão fazer mais sentido. Por enquanto, apenas saiba que eles estão esperando por você.

O quarto filtro é o filtro de ‘cross site request forgery’, e é parecido com isto:

```
1  <?php
2
3  // app/filters.php
4
5  Route::filter('csrf', function()
6  {
7      if (Session::token() != Input::get('_token'))
8      {
9          throw new Illuminate\Session\TokenMismatchException;
10     }
11 });
```

Você pode adicioná-lo as suas rotas para proteger requisições postadas de fora de sua aplicação. Isto é uma medida de segurança muito útil que é usada primeiramente para proteger rotas que estão nas ações dos formulários ou envio de dados.

Sinta-se livre para tirar vantagem dos filtros que o Laravel fornece, eles foram colocados ali para poupar o seu tempo.

Filtros em Múltiplas Rotas

Não quer adicionar o filtro manualmente para todas as suas rotas? Eu não posso te culpar. Dedos cansados acontecem, eu estou escrevendo um livro, eu sei disto. Vamos tentar achar uma maneira de salvar os seus pequenos e coitados ossos da mão de algum esforço. Aqui está o ‘pattern filter’.

O ‘pattern filter’ vai permitir que você adicione um filtro do tipo ‘before’ para um número de rotas fornecendo um asterisco * nas suas rotas. Vamos ver em funcionamento:

```
1 <?php
2
3 // app/routes.php
4
5 Route::when('profile/*', 'birthday');
```

A função `Route::when()` acima, vai rodar o filtro 'birthday' em todas as URIs que começarem com `profile/`. O asterisco dentro do primeiro parâmetro vai funcionar como um coringa. Esta é uma boa maneira para adicionar um filtro 'before' para mais de uma rota ao mesmo tempo.

Controllers

Criando Controllers

No capítulo sobre Roteamento Básico nós vimos como criar a lógica das rotas da nossa aplicação com Closures. Closures são uma boa maneira de escrever nossa aplicação, e pessoalmente, eu acredito que elas funcionam muito bem com os exemplos deste livro. Porém, uma melhor escolha para desenvolver a lógica de sua aplicação são os Controllers.

O Controller é uma classe utilizada para armazenar a lógica de uma rota. Normalmente, o Controller vai conter diferentes métodos públicos, que são conhecidos como ações. Você pode pensar nas ações como uma alternativa para as Closures que nós utilizamos no capítulo anterior, e elas são muito similares em sua aparência e funcionalidade.

E não gosto de explicar o que é uma coisa, sem primeiro mostrar um exemplo. Então vamos dar uma olhada no que um controller se parece. Este aqui é um que eu fiz anteriormente! Cue Blue Peter theme. “Cue Blue Peter theme” apenas vai fazer sentido para os Britânicos. Esqueça, agora que já escrevi não tenho coragem para excluir! Então, controllers..

```
1  <?php
2
3  // app/controllers/ArticleController.php
4
5  class ArticleController extends BaseController
6  {
7      public function showIndex()
8      {
9          return View::make('index');
10     }
11
12     public function showSingle($articleId)
13     {
14         return View::make('single');
15     }
16 }
```

Aqui está o nosso controller! Bom e simples. Este exemplo se encaixa com a estrutura de um blog ou algum formulário de um CMS. Para um blog, o ideal é que se tenha uma página para listar todos os artigos, e uma outra página para exibir um artigo em detalhes. Estas duas atividades estão

relacionadas ao conceito de um artigo, o que significa que faz sentido agrupar sua lógica. Isto é porque a lógica está contida dentro de apenas uma classe chamada `ArticleController`.

Na verdade, você pode chamar o controller como você quiser. Contanto que o seu controller estenda a classe `BaseController` ou a classe `Controller`. Assim, o Laravel vai saber o que você está querendo fazer. Porém, adicionar o nome `Controller` no fim do nome de sua classe, por exemplo `ArticleController` é meio que um padrão utilizado pelos desenvolvedores. Se você está planejando trabalhar com outros desenvolvedores, padrões podem ser muito úteis.

O nosso controller foi criado dentro da pasta `app/controller`, que o Laravel já havia criado para nós. Este diretório está sendo carregado pelo Composer por padrão. Se o diretório `app/controllers` não se encaixar no seu fluxo de trabalho, você pode colocar os seus controllers onde você quiser. Porém, você vai precisar se certificar de que a classe esteja sendo carregada pelo autoloader do Composer. Para mais informações sobre este assunto, por favor volte para o capítulo inicial sobre Composer.

Os métodos do nosso Controller contém a nossa lógica. Você pode chamá-los como você quiser, mas pessoalmente eu gosto de usar a palavra 'show' como um prefixo se o resultado final exibe uma página. Estes métodos **precisam** ser públicos para o Laravel poder roteá-los. Você pode adicionar métodos adicionais na classe para uma maior abstração, mas eles não podem ser roteados. Na verdade, existe um lugar melhor para este tipo de código que nós vamos aprender no capítulo sobre Models.

Vamos dar uma olhada na nossa primeira ação, que neste exemplo vai ser utilizada para exibir a lista de artigos de um blog.

```
1 public function showIndex()
2 {
3     return View::make('index');
4 }
```

Hmm, não sabe o que parece muito familiar? Vamos comparar rapidamente com uma Closure que faz exatamente a mesma coisa.

```
1 Route::get('index', function()
2 {
3     return View::make('index');
4 });
```

Como você pode ver, a Closure é praticamente igual a ação do Controller. A única diferença é que a ação do controller possui um nome, e a closure é uma função anônima. Na verdade, uma ação de um controller pode conter qualquer código que uma Closure tenha. Isto significa que tudo que nós aprendemos até agora ainda é aplicável. Que pena, se fosse diferente eu poderia vender um outro livro sobre controllers!

Porém, existe uma pequena diferença entre os dois exemplos. No capítulo sobre Roteamento Básico nós aprendemos como rotear uma URI para uma closure que contenha nossa lógica. No exemplo acima, onde temos uma rota com a Closure, a URI `/index` seria roteada para a lógica da nossa Closure. Porém, a ação do nosso controller não possui nenhuma URI. Como o Laravel sabe como direcionar suas rotas para o nosso controller? Vamos dar uma olhada em como podemos rotear um controller e torcer para encontrarmos uma resposta para a nossa pergunta.

Roteamento de Controllers

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('index', 'ArticleController@showIndex');
```

Para fazer a conexão entre uma URI e o nosso Controller, nós precisamos criar uma nova rota dentro do arquivo `app/routes.php`. Nós vamos utilizar o mesmo método `Route::get()` que nós já utilizamos quando estávamos roteando Closures. Porém, o segundo parâmetro é completamente diferente. Desta vez nós temos uma string.

A string consiste em duas seções que são separadas por um sinal (`@`). Vamos dar uma outra olhada no Controller que nós criamos na última seção.

```
1 <?php
2
3 // app/controllers/ArticleController.php
4
5 class ArticleController extends BaseController
6 {
7     public function showIndex()
8     {
9         return View::make('index');
10    }
11
12    public function showSingle($articleId)
13    {
14        return View::make('single');
15    }
16 }
```

Nós podemos ver pelo exemplo que o nome da classe é `ArticleController` e o nome da ação que nós queremos rotear se chama `showIndex`. Vamos colocá-los juntos, com um (`@`) no meio.


```
1 ArticleController@showIndex
```

É realmente simples assim. Agora nós podemos utilizar qualquer método que nós vimos no capítulo de roteamento básico, e apontá-los para os nossos controllers. Por exemplo, a seguir temos uma ação de um controller que vai responder para uma requisição HTTP POST.

```
1 <?php
2
3 // app/routes.php
4
5 Route::post('article/new', 'ArticleController@newArticle');
```

Vocês são muito espertos, afinal, vocês compraram este livro, né? Então agora vocês podem ver que a rota acima vai responder para uma requisição do tipo POST na URI /article/new, e que vai ser tratada pela ação newArticle(), pertencente ao ArticleController.

Uma coisa útil de se saber é que você pode adicionar namespaces aos seus controllers e o Laravel não vai pestanejar. Apenas não se esqueça de incluir o namespace dentro da declaração de sua rota e tudo vai ser funcionar elegantemente! Vamos ver em ação.

```
1 <?php
2
3 // app/controllers/Article.php
4
5 namespace Blog\Controller;
6
7 use View;
8 use BaseController;
9
10 class Article extends BaseController
11 {
12     public function showIndex()
13     {
14         return View::make('index');
15     }
16 }
```

Acima nós temos um controller similar ao controller utilizado no outro exemplo. Porém desta vez, ele pertence ao namespace Blog\Controller. Como nosso controller está localizado dentro da seção Controller do namespace, eu não coloquei o nome Controller ao final do nome da classe. Isto é uma preferência pessoal, e deixo para você decidir se você quer manter o nome ou não.

Agora vamos ver como este controller dentro de um namespace pode ser roteado. Provavelmente você já sabe!

```
1 <?php
2
3 // app/routes.php
4
5 Route::post('index', 'Blog\Controller\Article@showIndex');
```

Funciona exatamente da mesma maneira que antes, apenas desta vez o nome do controller foi prefixado com o seu namespace. Os namespaces não precisam ser complicados! Você pode até armazenar o seu controller dentro em um subdiretório, ou em algum outro local que esteja sendo carregado pela PSR-0. O Laravel não se importa, contanto que o Composer saiba onde encontrar suas classes, ele pode usá-las.

Controllers RESTful

O Laravel oferece muitas soluções, nós já sabemos disto. E ele também oferece opções, e os Controllers RESTful são um bom exemplo disto.

Nós já sabemos que podemos definir o verbo da requisição HTTP que nós queremos utilizar com os métodos de roteamento. E isto é realmente muito útil quando estamos roteando Closures. Porém, quando você estiver roteando Controllers você talvez queira manter as definições dos verbos HTTP o mais próximo com a lógica de sua aplicação. A boa notícia é que o Laravel também oferece esta alternativa.

Vamos alterar o nosso Controller um pouco?

```
1 <?php
2
3 // app/controllers/Article.php
4
5 namespace Blog\Controller;
6
7 use View;
8 use BaseController;
9
10 class Article extends BaseController
11 {
12     public function getCreate()
13     {
14         return View::make('create');
15     }
16
17     public function postCreate()
```

```
18     {
19         // Handle the creation form.
20     }
21 }
```

Aqui nós temos o nosso `ArticleController` novamente. A intenção das ações são exibir um formulário para criação e a manipulação da criação de um novo artigo, respectivamente. Você vai perceber que os nomes das ações estão prefixadas com `get` e `post`. Estes são os nossos verbos HTTP.

Então, no exemplo acima você talvez pense que nós estamos representando o final das seguintes URLs:

```
1 GET /create
2 POST /create
```

Você também pode estar se perguntando como nós podemos rotear o nosso RESTful controller. Assim, utilizar os métodos que possuem nomes de verbos HTTP não fariam muito sentido aqui. Vamos dizer adeus para o roteamento de cara rota individualmente. Vamos dar uma olhada em um outro método de roteamento.

```
1 <?php
2
3 // app/routes.php
4
5 Route::controller('article', 'Blog\Controller\Article');
```

Este método vai rotear todas as ações representadas pelo nosso RESTful controller. Vamos dar uma olhada em como funciona este método.

O primeiro parâmetro é a base da URL. Normalmente o roteamento RESTful é utilizado para representar um objeto, então na maioria das circunstâncias a URL base vai ser o nome do objeto. Você pode pensar nisto como um prefixo para as ações que nós criamos dentro do nosso RESTful controller.

O segundo parâmetro você já está familiarizado. É o controller no qual nós pretendemos rotear. Novamente, O Laravel vai aceitar um controller pertencente a um namespace, então sintase à vontade para organizar os seus controllers conforme suas necessidades.

Como você pode ver, utilizar este método para rotear os controllers oferece uma vantagem em relação a forma original de roteamento. Com este método, nós apenas temos uma declaração de rotas para o controller, ao invés de rotear cada ação separadamente.

Blade

Neste capítulo, nós vamos aprender como dominar o Blade. Você vai precisar disto. Para buscar o seu lugar como um artesão PHP você vai precisar enfrentar e derrotar o High Lord Otwell em um combate.

É um ritual de passagem. Apenas então você poderá buscar o lugar merecido juntamente com o conselho do Laravel, e conseguir um lugar na mesa de bebidas do Phil Sturgeon's.

Uma vez por mês, os membros do conselho descem para o campo de batalha PHP e se sentam no topo dos nossos terríveis pandas do Laravel, para batalhar contra desenvolvedores de outros frameworks. Se você quiser se juntar conosco nesta batalha, e lutar pela honra do Laravel você **precisa** aprender a dominar o Blade.

Bem, precisa é uma palavra forte. Eu quis dizer, você também pode dominar os templates Blade. Não é tão extravagante, e você não vai ter os ferozes pandas envolvidos. É muito acessível também, e provavelmente mais adequado para desenvolvedores do que guerreiros.

Isso, vamos nos acalmar então, vamos dar uma olhada nos templates Blade.

Você deve estar se perguntando sobre o nome 'Blade'? Bem, eu também estava quando eu escrevi este capítulo, então eu decidi perguntar ao Taylor. Acontece na plataforma .NET existe uma ferramenta de template chamada 'Razor', de onde muito da sintaxe do blade foi derivada. Razor..blade..razorblade. É isto. Nada realmente divertido aqui, desculpe. :(

Na verdade esqueça o que eu acabei de te dizer, vamos reinventar esta história. Apenas entre nós. A linguagem de templates foi nomeada depois do alter ego 'Blade' do Taylor aparecer nos dias de caça aos vampiros. Esta é a história real.

Certo, vamos criar alguns templates.

Criando Templates

Eu sei, eu sei, eu já ensinei como criar views, certo? Elas são realmente úteis para separar os aspectos visuais da sua aplicação de sua lógica, mas, isto não significa que elas não possam ficar melhores.

O problema com os templates PHP padrões é que nós precisamos inserir aquelas tags horríveis do PHP dentro deles para utilizar dados que a parte lógica de nossa aplicação nos forneceu. As tags não parecem estar no lugar certo dentro de templates HTML. Elas parecem estar fora do lugar dentro dos nossos belos templates HTML. Isto me deixa tão irritado... que eu poderia... eu poderia. Não, deixe eu me acalmar um pouco.

Erm...

OK, minha raiva passou. Vamos criar um template Blade para tirar esta desagradável bagunça PHP do caminho. Para começar nós vamos precisar criar um novo arquivo. Os templates Blade ficam dentro da mesma pasta que as views padrões. A única diferença é que eles utilizam a extensão `.blade.php` ao invés de apenas `.php`.

Vamos criar um simples template.

```
1 <!-- app/views/example.blade.php -->
2
3 <h1>Dear Lord Otwell</h1>
4 <p>I hereby challenge you to a duel for the honour of Laravel.</p>
5
6 <?php echo $squirrel; ?>
```

Aqui nós temos o nosso template blade, que se parece muito simiar com o que nós já vimos, certo? Isto porque primeiro o blade compila o arquivo como PHP. Você vê o nosso `$squirrel`? Toda view deve ter um `'squirrel'`. Ok, isto não é verdade, mas isto nos mostra que o PHP pode ser interpretado como antes.

Nós podemos exibir este template utilizando a mesma sintaxe que nós usaríamos para uma view normal. Você pode ter achado que exibi-lo necessitaria passar `example.blade` para o método `View::make()`, mas você está errado.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('example', function()
6 {
7     return View::make('example');
8 });
```

Viu, o Laravel sabe o que é um template Blade, e como procurar por um. Por isto que o método `View::make()` não mudou em nada. Conveniente, não?!

De qualquer maneira, o Blade possui alguns truques. Vamos dar uma olhada no primeiro.

Saída PHP

Muitos dos nossos templates vão envolver escrever dados fornecidos pela parte lógica de nossa aplicação. Normalmente isto se pareceria com algo como isto:

```
1 <!-- app/views/example.blade.php -->
2
3 <p><?php echo $taylorTheVampireSlayer; ?></p>
```

Não é exatamente verboso, mas podemos melhorar um pouco. Vamos ver como nós podemos escrever valores utilizando os templates Blade.

```
1 <!-- app/views/example.blade.php -->
2
3 <p>{{ $taylorTheVampireSlayer }}</p>
```

Tudo que esteja entre {{ duas chaves }} é transformado em um ‘echo’ pelo Blade quando o template for processado. Isto é uma sintaxe muito mais limpa, e muito mais fácil de escrever.

Como as diretivas dos templates Blade são traduzidas diretamente para PHP, nós podemos utilizar qualquer código PHP dentro destas chaves, incluindo chamada de funções.

```
1 <!-- app/views/example.blade.php -->
2
3 <p>{{ date('d/m/y') }}</p>
```

Você não precisa nem fornecer o ponto e vírgula final. O Laravel faz isto por você.

As vezes você vai querer se proteger escapando um valor que você escreve. Você pode estar familiarizado com funções como `strip_tags()` e `htmlspecialchars()` para isto. Por que você faria isto? Vamos considerar a saída deste template.

```
1 <!-- app/views/example.blade.php -->
2
3 <p>{{ '<script>alert("CHUNKY BACON!");</script>' }}</p>
```

Que pedaço de código desagradável! Aconteceria que algum Javascript seria injetado dentro da página e o navegador iria exibir um alerta contendo o texto ‘CHUNKY BACON’. Maldito! Desagradáveis desenvolvedores Ruby estão sempre tentando quebrar os nossos sites.

Porém, nós temos o poder de proteger a saída de nossos templates. Se nós usarmos {{{ três chaves }}} ao invés de {{ duas }}, então a nossa saída vai ser escapada, os caracteres especiais serão convertidos para entidades HTML, e o Javascript vai ser exibido como texto dentro da página. Inofensivo!

Vamos ver em ação.

```
1 <!-- app/views/example.blade.php -->
2
3 <p>{{{ ' <script>alert("CHUNKY BACON!");</script>' }}}</p>
```

Tudo que nós alteramos foi o número de chaves em volta da snippet Javascript. Vamos ver o código fonte da página para ver como ficou.

```
1 <!-- app/views/example.blade.php -->
2
3 <p>&lt;script&gt;alert(&quot;CHUNKY BACON!&quot;);&lt;/script&gt;</p>
```

Como você pode ver, as tags HTML e alguns caracteres foram substituídos pela sua equivalente entidade HTML. Nosso site está salvo!

Continuando!

Estruturas de Controle

O PHP possui muitas estruturas de controle. If, while, for e foreach. Se você ainda não ouviu sobre elas este livro ainda não é para você!

Dentro dos templates você provavelmente já está familiarizado em utilizar a sintaxe alternativa para estruturas de controle usando dois pontos : . Sua declaração If irá se parecer com o exemplo abaixo:

```
1 <!-- app/views/example.blade.php -->
2
3 <? if ($something) : ?>
4     <p>Something is true!</p>
5 <? else : ?>
6     <p>Something is false!</p>
7 <? endif; ?>
```

Novamente, faz o trabalho, mas não é muito divertido de digitar. Elas vão te atrasar. Mas, felizmente para você, o Blade virá em seu socorro!

Aqui está como a snippet acima fica dentro de um template Blade.

```
1 <!-- app/views/example.blade.php -->
2
3 @if ($something)
4     <p>Something is true!</p>
5 @else
6     <p>Something is false!</p>
7 @endif
```

É muito mais limpo, certo? Vamos dar uma olhada no que nós removemos. Para começar, as tags de abertura <? e fechamento ?> se foram. Elas são provavelmente as mais complicadas de se digitar.

Nós também removemos os dois pontos : e o ponto e vírgula ; . Nós não precisamos deste desperdício de espaço em nossos templates!

Por último, nós adicionamos algo incomum a nossa sintaxe. Nós prefixamos a nossa estrutura de controle com um arroba @. Na verdade, todas as estruturas de controle e métodos de ajuda são prefixados com este símbolo, para que o compilador do template saiba como tratá-los.

Vamos adicionar um elseif à mistura para um outro exemplo.

```
1 <!-- app/views/example.blade.php -->
2
3 @if ($something == 'Red Panda')
4     <p>Something is red, white, and brown!</p>
5 @elseif ($something == 'Giant Panda')
6     <p>Something is black and white!</p>
7 @else
8     <p>Something could be a squirrel.</p>
9 @endif
```

Nós adicionamos mais uma declaração na mistura, seguindo as mesmas regras de remoção das tags PHP, dois pontos e ponto e vírgula, e adicionando o símbolo @. Tudo funciona perfeitamente.

Um desafio. Tente imaginar um loop foreach representado com a sintaxe blade. Feche os seus olhos, imagina. Foco... foco!

Se parece com isto?


```

1 <!-- app/views/example.blade.php -->
2
3      c~^p ,----- .
4  ,---'oo )          \
5  ( 0 0              )/
6  '=^='              /
7      \ , . /
8      \ \ |-----' | /
9      | |__|   |__|

```

Não? Que bom, porque isto é um hipopótamo. Porém, se ela parece com a estrutura abaixo você vai ganhar um biscoito.

```

1 <!-- app/views/example.blade.php -->
2
3 @foreach ($manyThings as $thing)
4     <p>{{ $thing }}</p>
5 @endforeach

```

Aproveite o seu biscoito! Como você pode ver, nós utilizamos a sintaxe Blade `{{ echo }}` para exibir o valor da saída do loop. Um loop for é assim como você imagina. Abaixo temos um exemplo para referência.

```

1 <!-- app/views/example.blade.php -->
2
3 @for ($i = 0; $i < 999; $i++)
4     <p>Even {{ $i }} red pandas, aren't enough!</p>
5 @endfor

```

Simples, e exatamente o que você estava esperando. O loop while segue as mesmas regras, mas eu vou mostrar um exemplo rápido para permitir que isto seja uma referência útil no futuro.

```

1 <!-- app/views/example.blade.php -->
2
3 @while (isPretty($kieraKnightly))
4     <p>This loop probably won't ever end.</p>
5 @endwhile

```

Certo, agora você dominou as estruturas de controle. Nada passa por você, certo parceiro? Nem mesmo a condição PHP `unless`.

Err Dayle, Ruby tem isto. Eu não sei se o PHP te...

OK, você me pegou. PHP não possui uma condição `unless`. Porém, Blade possui uma função que ajuda a permitir isto. Vamos dar uma olhada no exemplo abaixo.

```
1 <!-- app/views/example.blade.php -->
2
3 @unless (worldIsEnding())
4     <p>Keep smiling.</p>
5 @endunless
```

Unless é exatamente o oposto de uma declaração if. Uma declaração if checa se a condição é verdadeira, e depois executa a lógica. Porém, a declaração unless vai apenas executar a lógica se a condição for false. Você pode pensar nisto como uma estrutura de controle para pessimistas.

Templates

O Blade inclui alguns outros métodos de ajuda para deixar os seus templates mais fáceis de montar e manter. Ele não vai escrever as views para você, mas talvez nós possamos adicionar isto para a lista de coisas para o Laravel 5.

- php artisan project:complete
- Permitir compositores de views construírem cortes de cabelo.
- Ter views que se escrevem sozinhas.

Agora sim. Mas até estas funcionalidades serem implementadas, você vai precisar escrever seus próprios templates. Porém, isto não significa que nós tenhamos que colocar tudo em um arquivo.

Com PHP, você pode incluir (`include()`) um arquivo a partir do arquivo que você está executando seu conteúdo. Você também pode fazer isto com as views para quebrá-las em arquivos separados, para uma melhor organização.

Aqui nós temos um arquivo `header.blade.php` contendo o cabeçalho de nossa página, e possivelmente até outras páginas.

```
1 <!-- app/views/header.blade.php -->
2
3 <h1>When does the Narwhal bacon?</h1>
```

Aqui está o arquivo do template do rodapé.

```
1 <!-- app/views/footer.blade.php -->
2
3 <small>Information provided based on research as of 3rd May '13.</small>
```

E aqui temos o nosso template principal. O template que vai ser exibido pela nossa Closure ou pela ação do Controller em nossas rotas.

```
1 <!-- app/views/example.blade.php -->
2
3 <!doctype html>
4 <html lang="en">
5 <head>
6     <meta charset="UTF-8">
7     <title>Narwhals</title>
8 </head>
9 <body>
10     @include('header')
11     <p>Why, the Narhwal surely bacons at midnight, my good sir!</p>
12     @include('footer')
13 </body>
14 </html>
```

Como você pode ver, as funções de ajuda dentro do template `example.blade.php` estão puxando o conteúdo do nosso cabeçalho e do rodapé utilizando a função `include()`. O `include` recebe o nome da view como parâmetro, no mesmo formato reduzido que o método `View::make()`, que nós já utilizamos antes. Vamos dar uma olhada no resultado deste template.

```
1 <!doctype html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Narwhals</title>
6 </head>
7 <body>
8     <h1>When does the Narwhal bacon?</h1>
9     <p>Why, the Narhwal surely bacons at midnight, my good sir!</p>
10    <small>Information provided based on research as of 3rd May '13.</small>
11 </body>
12 </html>
```

Nossos templates foram... bem, incluídos na nossa página. Isto faz com que os nossos templates do cabeçalho e rodapé seja reutilizáveis e não se repitam (princípio DRY). Nós podemos incluí-los em outras páginas para evitar conteúdo repetido, e para fazer o conteúdo ser editado em um único local. Porém, existe uma maneira melhor ainda de fazer isto, então, continue lendo!

Herança de Templates

O Blade oferece uma maneira de montar templates que se beneficia da herança de templates. Muitas pessoas acham isto confuso, e mesmo assim uma funcionalidade muito bem-vinda. Eu vou tentar

simplificar ao máximo que eu puder e esperamos que logo você ache a arte de criar templates uma agradável experiência.

Primeiro vamos pensar em templates. Existem algumas partes de um site que não precisam mudar em todas as páginas. Estas seriam as tags que precisam estar em todas as páginas que nós vemos. Nós podemos criar um padrão se você quiser. Abaixo temos um exemplo:

```
1 <!doctype html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title></title>
6 </head>
7 <body>
8 </body>
9 </html>
```

Nós vamos utilizar este layout para todas as nossas páginas. Porque nós não falamos isto para o Laravel? Vamos dizer que este é um layout Blade. Para isto, nós apenas precisamos definir algumas áreas onde o conteúdo será inserido. No Laravel nós chamamos estas áreas de ‘seções’. E assim é como nós definimos elas:

```
1 <!-- app/views/layouts/base.blade.php -->
2
3 <!doctype html>
4 <html lang="en">
5 <head>
6     <meta charset="UTF-8">
7     <title></title>
8     @section('head')
9         <link rel="stylesheet" href="style.css" />
10    @show
11 </head>
12 <body>
13     @yield('body')
14 </body>
15 </html>
```

Aqui nós criamos um template com duas seções. Vamos dar uma olhada na seção dentro do body primeiro, esta é a mais fácil. Ela é assim:

```
1 @yield('body')
```

Esta declaração ao Blade para criar uma seção que nós poderemos preencher com conteúdo mais tarde. Nós apelidamos a seção de 'body' para poder referenciá-la mais tarde.

E a outra seção se parece com isto:

```
1 @section('head')
2     <link rel="stylesheet" href="style.css" />
3 @show
```

Ela é muito parecida com a seção 'yield', exceto que você pode fornecer algum conteúdo padrão nela. No exemplo acima, o conteúdo entre as tags @section e @show vai ser exibido, a não ser que um template filho escolha reescrevê-la.

O que eu quis dizer com template filho? Como sempre, vamos ver um exemplo disto.

```
1 <!-- app/views/home.blade.php -->
2
3 @extends('layouts.base')
4
5 @section('body')
6     <h1>Hurray! </h1>
7     <p>We have a template! </p>
8 @stop
```

Certo, vamos passar pelas declarações. Primeiro nós temos a função blade extends:

```
1 @extends('layouts.base')
```

Esta função diz ao blade qual layout nós estaremos utilizando para renderizar o nosso conteúdo. O nome que nós passamos para a função utiliza a mesma sintaxe que nós passamos para a função `View::make()`, então neste caso, nós estamos referenciando o arquivo `base.blade.php` dentro de pasta `layouts`, que fica dentro da pasta `app/views`. Lembre-se de utilizar um ponto (.) para separar os diretórios entre as views.

No Laravel 3 esta função era chamada de `layout()`, mas foi renomeada para trazer mais padronização com outras engines de template, como a Twig, so Symfony. Atenção, desenvolvedores do Laravel 3.

Agora que nós sabemos o layout nós estamos utilizando, é hora de preencher com conteúdo. Nós podemos usar a função `@section` do blade para inserir conteúdo nas seções do template pai. Podemos fazer isto da seguinte maneira:

```
1 @section('body')
2     <h1>Hurray!</h1>
3     <p>We have a template!</p>
4 @stop
```

Nós passamos para a função `@section` o apelido que nós demos a nossa seção dentro do template pai. Lembra? Nós apelidamos de `body`. Tudo que está contido entre `@section` e `@stop` vai ser inserido dentro do template pai, onde a declaração `@yield('body')` está.

Vamos criar uma rota para ver isto em ação. Para renderizar o template, nós apenas precisamos adicionar uma resposta `View::make()`. Assim:

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return View::make('home');
8 });
```

Agora se nós visitarmos / e olharmos o código fonte, nós veremos que a página se parece com isto:

```
1 <!doctype html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title></title>
6     <link rel="stylesheet" href="style.css" />
7 </head>
8 <body>
9     <h1>Hurray!</h1>
10    <p>We have a template!</p>
11 </body>
12 </html>
```

Ok, a formatação pode se parecer um pouco diferente, mas o conteúdo deve ser o mesmo. Nossa seção foi injetada dentro no nosso template pai. E como nós não reescrevemos o conteúdo da seção `head`, o valor padrão foi inserido.

Agora você vê, nós podemos ter quantos templates filhos nós quisermos herdando layouts de seus templates pais. Isto nos poupa o esforço de termos que repetir o código padrão de nosso site.

Vamos alterar um pouco o template filho para passar algum conteúdo para a seção `head`. Assim:

```
1 <!-- app/views/home.blade.php -->
2
3 @extends('layouts.base')
4
5 @section('head')
6     <link rel="stylesheet" href="another.css" />
7 @stop
8
9 @section('body')
10     <h1>Hurray!</h1>
11     <p>We have a template!</p>
12 @stop
```

Como você pode imaginar, a seção head foi injetada com o nosso CSS adicional, e o código fonte da nossa página vai se parecer com isto:

```
1 <!doctype html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title></title>
6     <link rel="stylesheet" href="another.css" />
7 </head>
8 <body>
9     <h1>Hurray!</h1>
10    <p>We have a template!</p>
11 </body>
12 </html>
```

Você se lembra como a seção ‘head’ tinha um conteúdo padrão entre as declarações @section e @show? Bem, você pode querer adicionar um conteúdo extra para esta seção, ao invés de substituí-lo. Para fazer isto, nós podemos utilizar a declaração @parent. Vamos modificar o template para usá-lo, como no exemplo abaixo:

```
1 <!-- app/views/home.blade.php -->
2
3 @extends('layouts.base')
4
5 @section('head')
6     @parent
7     <link rel="stylesheet" href="another.css" />
8 @stop
9
10 @section('body')
11     <h1>Hurray!</h1>
12     <p>We have a template!</p>
13 @stop
```

A declaração @parent diz ao blade para adicionar o conteúdo padrão no seu local. Esta declaração pode parecer um pouco confusa, mas na verdade é muito simples. Vamos dar uma olhada em como o código ficou para clarear as coisas.

```
1 <!doctype html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title></title>
6     <link rel="stylesheet" href="style.css" />
7     <link rel="stylesheet" href="another.css" />
8 </head>
9 <body>
10     <h1>Hurray!</h1>
11     <p>We have a template!</p>
12 </body>
13 </html>
```

Viu? Nossa declaração @parent foi substituída com o conteúdo padrão da seção pai. Nós podemos utilizar este método para adicionar novos itens no menu ou outros assets.

Nós podemos ter quantos níveis de herança nós quisermos com os templates Blade, o exemplo abaixo é totalmente aceitável.


```
1 <!-- app/views/first.blade.php -->
2
3 <p>First</p>
4 @yield('message')
5 @yield('final')
6
7 <!-- app/views/second.blade.php -->
8 @extends('first')
9
10 @section('message')
11     <p>Second</p>
12     @yield('message')
13 @stop
14
15 <!-- app/views/third.blade.php -->
16 @extends('second')
17
18 @section('message')
19     @parent
20     <p>Third</p>
21     @yield('message')
22 @stop
23
24 <!-- app/views/fourth.blade.php -->
25 @extends('third')
26
27 @section('message')
28     @parent
29     <p>Fourth</p>
30 @stop
31
32 @section('final')
33     <p>Fifth</p>
34 @stop
```

Bem maluco, certo? Tente seguir a herança para ver como o resultado foi construído. Talvez seja mais fácil começar a partir do template filho, ao invés dos templates pais. Se nós renderizarmos a view 'fourth', este será o resultado.

```
1 <p>First</p>
2 <p>Second</p>
3 <p>Third</p>
4 <p>Fourth</p>
5 <p>Fifth</p>
```

Para simplificar.

Fourth estende Third que estende Second que estende First que é o template base.

Você pode ter notado que a seção ‘final’ do template base possui conteúdo passado pelo template ‘fourth’. Isto significa que você pode disponibilizar conteúdo para uma seção de qualquer uma das camadas. Como você pode ver, o Blade é **muito** flexível.

Comentários

Como você provavelmente já sabe, o HTML possui seu próprio método para inclusão de comentários. Eles se parecem com isto.

```
1 <!-- This is a lovely HTML comment. -->
```

Ele é um belo comentário e parece OK, mas infelizmente ele é exibido com o código fonte da página. Nós não queremos que as pessoas leiam as informações que são colocadas lá apenas para os desenvolvedores.

Ao contrário dos comentários HTML, os comentários PHP são removidos quando a página é processada. Isto significa que eles não vão ser exibidos quando nós olharmos o código fonte. Nós podemos incluir comentários PHP em nossos templates assim.

```
1 <?php // This is a secret PHP comment. ?>
```

Claro, agora nosso comentário está escondido. Mas ele é um comentário feio, certo? Não temos espaço para coisas feitas dentro dos templates Blade. Vamos utilizar um comentário do Blade ao invés disto, visto que eles são compilados diretamente como comentários PHP.

```
1 {{-- This is a pretty, and secret Blade comment. --}}
```

Utilize os comentários do Blade que você quiser colocar anotações em suas views que apenas desenvolvedores devem ver.

Roteamento avançado

Oh, então você voltou para mais disto. Apenas o capítulo de roteamento básico não foi bom o suficiente para você? Um pouco gananciosos não? Bem, não tema, pois eu tenho uma sobremesa para você.

No capítulo sobre filtros você aprendeu como nós podemos passar um array como o segundo parâmetro para os métodos de roteamento para incluir mais informações nas definições das rotas. Assim:

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', array(
6      'before' => 'sexyfilter',
7      function() {
8          return View::make('hello');
9      }
10 ));
```

Neste exemplo, nós estamos usando a sintaxe com array para incluir informações sobre quais filtros nós queremos aplicar para a rota. Mas não termina por aqui, você pode fazer mais com este array. Vamos dar uma olhada no que temos disponível.

Rotas Nomeadas

URIs são finas e elegantes. Elas ajudam na estrutura do site, mas quando você tem um site mais complexo elas podem se tornar um pouco extensas e complexas. Você não quer lembrar de todas as URIs do seu site, isto se torna chato rapidamente.

Felizmente, Laravel oferece a opção de rotas nomeadas para nos aliviar um pouco desta chatisse. Nós podemos dar nomes as nossas rotas, e se é assim que nós podemos fazer isto:

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/my/long/calendar/route', array(
6      'as' => 'calendar',
7      function() {
8          return View::make('calendar');
9      }
10 ));
```

Utilizando o índice `as` no array da nossa rota, nós podemos dar um nome à ela. Tente manter ele simples, porém descritivo. O Laravel possui alguns métodos que ajudam a gerar links para os recursos de nossa aplicação, e muitos destes métodos possuem suporte para rotas nomeadas. Eu não vou mostrar todos aqui, existe um capítulo posterior que vai cobrir isto em detalhes, porém aqui temos um simples exemplo.

```
1  // app/views/example.blade.php
2
3  {{ route('calendar') }}
```

Esta simples função de ajuda retorna a URL da rota nomeada que você passou para ela. Neste caso, retornaria `http://localhost/my/long/calendar/route`. As chaves são simplesmente para escrever o retorno dentro de um template Blade. Você ainda lembra do Blade, certo? Eu espero que sim!

Então o quão útil é isto? Bem, como eu disse antes, você não precisa mais se lembrar de longas URLs. Embora, talvez você tenha um super cérebro. Lembrar URLs pode ser trivial para você. Bem, existe uma outra vantagem que eu gostaria de compartilhar.

Vamos imaginar por um segundo que você possui algumas views com links para uma certa rota. Se os links das rotas forem escritos manualmente, e você quiser alterar a URL da rota, você precisa alterar todas as suas views. Em uma grande aplicação isto pode ser uma grande perda de tempo, e vamos encarar, você é um desenvolvedor Laravel agora. Seu tempo vale muito dinheiro.

Se nós utilizarmos a função `route()`, e decidirmos trocar a URL, nós não precisamos alterar todos os links. Eles podem ser resolvidos pelos seus nomes. Eu sempre tento nomear minhas rotas quando eu posso, e isso me poupa muito tempo depois caso eu precise reestruturar minha aplicação.

Você se lembra do objeto de resposta `Redirect`? Bem, você pode utilizar o método `route` para redirecionar o usuário para uma rota nomeada. Por exemplo:

```
1 <?php
2
3 return new Redirect::route('calendar');
```

Também, se você quiser pegar o nome da rota atual, você pode usar o método `currentRouteName()`, que pertence à classe `Route`. Assim:

```
1 <?php
2
3 // app/routes.php
4
5 $current = Route::currentRouteName();
```

Lembre-se que todas estas funcionalidades avançadas estão disponíveis tanto para os Controllers, quanto para as Closures. Para rotear um Controller, nós simplesmente precisamos adicionar o índice `uses` no array da rota, junto com o par controller-ação.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/my/long/calendar/route', array(
6     'as' => 'calendar',
7     'uses' => 'CalendarController@showCalendar'
8 ));
```

Fácil, certo? Agora vamos ver como nós podemos fazer nossas rotas mais seguras.

Rotas Seguras

Você pode querer que suas rotas rodem em URLs HTTP seguras, para que as pessoas possam passar seus dados confidenciais. URLs HTTPS trabalham com o protocolo SSL ou TLS para aumentar a segurança. E assim é como nós podemos permitir que as rotas utilizem este protocolo.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('secret/content', array(
6      'https',
7      function () {
8          return 'Secret squirrel!';
9      }
10 ));
```

Adicionando o índice HTTPS ao nosso array, nossa rota vai responder para requisições feitas utilizando o protocolo HTTPS.

Restrições de Parâmetros

No capítulo de roteamento básico nós vimos como nós podemos utilizar parâmetros na estrutura da URL dentro da lógica de nossa aplicação. Para uma Closure, fica mais ou menos assim:

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('save/{princess}', function($princess)
6  {
7      return "Sorry, {$princess} is in another castle. :(";
8  });
```

Bem, eu nunca ouvi falar de uma princesa que se chame !1337f15h. Isto se parece muito mais com um jogador de conterstrike para mim. E nós não queremos que nossa rota responda para uma princesa falsa, então por que nós não tentamos validar o nosso parâmetro para ter certeza que ela tenha apenas letras?

Vamos ver um exemplo disto em ação.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('save/{princess}', function($princess)
6 {
7     return "Sorry, {$princess} is in another castle. :(";
8 }->where('princess', '[A-Za-z]+');
```

No exemplo acima, nós adicionamos um novo método `where()` no final da definição da nossa rota. O método ‘where’ aceita o nome do parâmetro da rota como o primeiro parâmetro, e uma expressão regular como o segundo.

Eu não vou cobrir expressões regulares em detalhes. Este tópico é extenso, mesmo, muito extenso. Pode ter um livro completo apenas sobre ele. Mas simplificando, a expressão regular acima garante que o nome da princesa contenha apenas letras maiúsculas e minúsculas, e precisa ter no mínimo uma letra.

Se o parâmetro não passar pela expressão regular que nós fornecemos, então a rota não será executada. E a classe de roteamento vai continuar tentando achar alguma rota que seja aceitável para os parâmetros passados.

Você pode adicionar quantas condições na sua rota você precisar. Dê uma olhada no exemplo abaixo:

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('save/{princess}/{unicorn}', function($princess, $unicorn)
6 {
7     return "{$princess} loves {$unicorn}";
8 }->where('princess', '[A-Za-z]+')
9   ->where('unicorn', '[0-9]+');
```

O parâmetro ‘unicorn’ foi validado para aceitar apenas um ou mais números, porque como nós sabemos, unicórnios sempre tem nomes com números. Como o meu bom amigo 3240012.

Grupos de Rotas

Lembra como nós podíamos passar filtros para nossas rotas no capítulo de filtros? Isto era realmente útil, certo? Porém, é uma vergonha ter que adicionar o mesmo filtro para muitas rotas.

Não seria ótimo se nós pudessemos encapsular nossas rotas, e aplicar um filtro em todas elas? Bem, você já deve ter adivinhado, mas aqui temos um exemplo de como podemos fazer isto:

```
1  <?php
2
3  // app/routes.php
4
5  Route::group(array('before' => 'onlybrogrammers'), function()
6  {
7
8      // First Route
9      Route::get('/first', function() {
10         return 'Dude!';
11     });
12
13     // Second Route
14     Route::get('/second', function() {
15         return 'Duuuuude!';
16     });
17
18     // Third Route
19     Route::get('/third', function() {
20         return 'Come at me bro.';
21     });
22
23 });
```

No exemplo acima nós utilizamos o método `group()` do objeto `Route`. O primeiro parâmetro é um array. Funciona como os outros que nós já utilizamos dentro dos métodos de roteamento. Aceita filtros, URLs seguras, e muitos outros tipos de filtros de rotas. O segundo parâmetro é uma Closure.

Quando você define rotas adicionais dentro desta Closure, as rotas vão herdar as propriedades deste grupo. As três rotas dentro do grupo acima estão protegidas pelo filtro `'onlybrogrammers'`.

Agora nós podemos utilizar os filtros que nós vimos no capítulo anterior também em grupos de rotas, mas nós também podemos utilizar algumas novas funcionalidades que são específicas apenas para grupos de rotas. Vamos dar uma olhada nelas.

Prefixo de Rotas

Se muitas rotas compartilharem a mesma estrutura de URL, você pode utilizar um prefixo nelas para evitar pequenas repetições.

Dê uma olhada no seguinte exemplo.


```
1  <?php
2
3  // app/routes.php
4
5  Route::group(array('prefix' => 'books'), function()
6  {
7
8      // First Route
9      Route::get('/first', function() {
10         return 'The Colour of Magic';
11     });
12
13     // Second Route
14     Route::get('/second', function() {
15         return 'Reaper Man';
16     });
17
18     // Third Route
19     Route::get('/third', function() {
20         return 'Lords and Ladies';
21     });
22
23 });
```

Utilizando o índice ‘prefix’ no array do nosso grupo de rotas, nós podemos especificar um prefixo para todas as URIs que forem definidas dentro do grupo. Por exemplo, as três rotas acima são acessíveis pelas seguintes URLs.

```
1  /books/first
2
3  /books/second
4
5  /books/third
```

Utilize os prefixos de rotas para evitar repetições dentro de suas rotas, e para agrupá-las com o propósito organizacional e/ou de estrutura.

Roteamento de Domínios

URIs não são a única maneira de diferenciar uma rota. O domínio também pode mudar. Por exemplo, as seguintes URLs podem referenciar diferentes recursos.

```
1 http://myapp.dev/my/route
2
3 http://another.myapp.dev/my/route
4
5 http://third.myapp.dev/my/route
```

Nos exemplos acima você pode ver que o subdomínio é diferente. Vamos ver como nós podemos utilizar o roteamento de domínios para exibir diferentes conteúdos para diferentes domínios.

Aqui temos um exemplo de roteamento de domínios:

```
1 <?php
2
3 // app/routes.php
4
5 Route::group(array('domain' => 'myapp.dev'), function()
6 {
7     Route::get('my/route', function() {
8         return 'Hello from myapp.dev!';
9     });
10 });
11
12 Route::group(array('domain' => 'another.myapp.dev'), function()
13 {
14     Route::get('my/route', function() {
15         return 'Hello from another.myapp.dev!';
16     });
17 });
18
19 Route::group(array('domain' => 'third.myapp.dev'), function()
20 {
21     Route::get('my/route', function() {
22         return 'Hello from third.myapp.dev!';
23     });
24 });
```

Adicionando o índice 'domain' para o array do grupo de rotas, nós podemos passar o nome do domínio, que *precisa* corresponder ao nome do domínio atual para qualquer uma das rotas dentro do grupo para serem executadas.

O domínio pode ser um subdomínio, ou um domínio completamente diferente. Contanto que o servidor web esteja configurado para redirecionar as requisições de cada domínio para o Laravel, então o Laravel poderá responder à elas.

Isto não é tudo sobre roteamento de domínios. Nós também podemos capturar porções de domínios e/ou subdomínios para utilizar como parâmetros, da mesma maneira que nós usamos anteriormente. Aqui temos um exemplo de como fazer isto.

```
1  <?php
2
3  // app/routes.php
4
5  Route::group(array('domain' => '{user}.myapp.dev'), function()
6  {
7      Route::get('profile/{page}', function($user, $page) {
8          // ...
9      });
10 });
```

Nós podemos passar um parâmetro dentro do índice ‘domain’ utilizando { chaves }, da mesma maneira que os outros parâmetros. O valor do parâmetro vai ser passado antes de qualquer parâmetro das rotas pertencentes ao grupo.

Por exemplo, se nós visitarmos a seguinte URL:

```
1  http://taylor.myapp.dev/profile/avatar
```

Então, o valor da variável \$user que é passado para a Closure será ‘taylor’, e o valor da variável \$page será ‘avatar’.

Utilizando esta combinação de subdomínios genéricos, e parâmetros nas rotas, você pode prefixar o domínio com o apelido de todos os usuários da aplicação.

Geração de URLs

Sua aplicação web gira em torno de URLs e rotas. Afinal, elas são o que direciona o usuário para suas páginas. No final, servir páginas é o que sua aplicação deve fazer.

Os seus usuários podem não estar interessados se você está apenas servindo uma página, e se você pretende levá-los para outras páginas do seu site ou aplicação. Mas você vai precisar usar uma das principais funcionalidades da web. Qual funcionalidade, você pergunta? Bem, hyperlinks.

Para construir hyperlinks nós precisamos construir URLs para nossa aplicação. Nós podemos usá-las manualmente, mas o Laravel pode nos poupar algum esforço disponibilizando algumas funções para nos ajudar a montar URLs. Vamos dar uma olhada nestas funções.

A URL Atual

Pegar a URL atual no Laravel é fácil. Simplesmente usar a função `URL::current()`. Vamos criar uma rota simples para testar.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/current/url', function()
6 {
7     return URL::current();
8 });
```

Agora se nós visitarmos a nossa url `/current/url`, nós vamos receber a seguinte resposta.

```
1 http://myapp.dev/current/url
```

Esta foi simples, não? Vamos dar uma olhada na função `URL::full()` agora, e você vai ver que ela retorna a URL atual.

Err. Nós não acabamos de fazer isto?

Bem, é um pouco diferente. Vamos tentar acessar esta nossa última rota novamente, mas desta vez nós vamos incluir alguns dados adicionais passados via GET.

```
1 http://myapp.dev/current/url?foo=bar
```

Você vai ver que o resultado da função `URL::current()` remove os dados adicionais por GET, assim:

```
1 http://myapp.dev/current/url
```

Por isto que o método `URL::full()` é um pouco diferente. Vamos modificar nossa rota existente para usá-la. Assim:

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/current/url', function()
6 {
7     return URL::full();
8 });
```

Agora vamos tentar acessar a URL `/current/url?foo=bar` novamente. Desta vez nós teremos o seguinte resultado:

```
1 http://myapp.dev/current/url?foo=bar
```

Como você pode ver, a função `URL::full()` também inclui os parâmetros adicionais passados via GET.

Esta próxima não é exatamente uma maneira de pegar uma URL, mas eu acho que certamente tem seu lugar neste tópico. É um método para pegar a URL anterior, recebido pelo cabeçalho 'referer'.

Neste próximo exemplo eu criei uma rota, que utiliza um redirecionamento para a segunda rota, que exibe a URL anterior.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('first', function()
6 {
7     // Redirect to the second route.
8     return Redirect::to('second');
9 });
10
```

```
11 Route::get('second', function()  
12 {  
13     return URL::previous();  
14 });
```

A nossa primeira rota redireciona para a segunda. A segunda rota vai exibir a URL anterior utilizando o método `URL::previous()`.

Vamos visitar a URI `/first` para ver o que acontece.

Você pode ver um erro do tipo ‘notice’ na rota de redirecionamento por um segundo, mas provavelmente você receberá a seguinte resposta:

```
1 http://demo.dev/first
```

Após o redirecionamento, a função `URL::previous` nos fornece a URL da requisição anterior, que neste caso é a URL da primeira rota. Simples assim!

Gerando URLs do Framework

Esta seção é sobre como gerar URLs que vão nos ajudar a navegar entre diferentes rotas e páginas do seu site ou aplicação.

Vamos começar gerando URLs para URIs especificar. Nós podemos fazer isto usando o método `URL::to()`. Assim:

```
1 <?php  
2  
3 // app/routes.php  
4  
5 Route::get('example', function()  
6 {  
7     return URL::to('another/route');  
8 });
```

A resposta que nós recebemos quando visitamos `/example` deve ser algo parecido com isto.

```
1 http://demo.dev/another/route
```

Como você pode ver, o Laravel montou a URL para a rota que nós pedimos. Você deve ter em mente que a URL `another/route` não existe, mas nós podemos criar um link para ela da mesma maneira. Tenha isto em mente quando estiver criando links para URIs.

Você pode especificar parâmetros adicionais para o método `URL::to()` no formato de um array. Estes parâmetros vão ser adicionados ao final da rota. Abaixo temos um exemplo:

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('example', function()
6 {
7     return URL::to('another/route', array('foo', 'bar'));
8 });
```

O resultado vai estar na seguinte forma.

```
1 http://myapp.dev/another/route/foo/bar
```

Se você quiser que suas URLs utilizem o protocolo HTTPS, então você terá duas opções. A primeira opção é passar `true` como o terceiro parâmetro para o método `URL::to()`, assim:

```
1 URL::to('another/route', array('foo', 'bar'), true);
```

Porém, se você não quiser fornecer parâmetros para sua URL, você vai precisar passar um array vazio como o segundo parâmetro. Ao invés disto, é melhor utilizarmos a descritiva função `URL::secure()`, assim:

```
1 URL::secure('another/route');
```

Novamente, você pode passar um array de parâmetros como o segundo parâmetro para a função `URL::secure()`, assim:

```
1 URL::secure('another/route', array('foo', 'bar'));
```

Vamos olhar o próximo método de geração. Você se lembra que nós descobrimos como dar nomes as nossas rotas dentro do capítulo de roteamento avançado? As rotas nomeadas são assim:

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('the/best/avenger', array('as' => 'ironman', function()
6  {
7      return 'Tony Stark';
8  }));
9
10 Route::get('example', function()
11 {
12     return URL::route('ironman');
13 });
```

Se nós visitarmos a rota /example, nós recebemos a seguinte resposta.

```
1  http://myapp.dev/the/best/avenger
```

O Laravel pegou nossa rota nomeada, e achou a URI associada. Se nós tivéssemos que alterar a URI, o conteúdo da rota `example` iria mudar também. Isto é muito útil para evitar a necessidade de alterar uma URI em muitas views.

Assim como o método `URL::to()`, o método `URL::route()` pode aceitar um array de parâmetros como o segundo parâmetro. E não só isto, mas também vai inserir os mesmos na ordem correta dentro da URI. Vamos dar uma olhada nisto em ação.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('the/{first}/avenger/{second}', array(
6      'as' => 'ironman',
7      function($first, $second) {
8          return "Tony Stark, the {$first} avenger {$second}.";
9      }
10 ));
11
12 Route::get('example', function()
13 {
14     return URL::route('ironman', array('best', 'ever'));
15 });
```

Se nós visitarmos a seguinte URL...


```
1 http://myapp.dev/example
```

...o Laravel vai preencher as variáveis na ordem correta, com estes parâmetros que nós passamos. A seguinte URL é exibida como resposta.

```
1 http://myapp.dev/the/best/avenger/ever
```

Existe um método final de roteamento deste tipo que você precisa saber, e é assim que nós podemos rotear ações de controllers. Na verdade, é muito simples, sendo que segue o mesmo padrão da função `URL::route()`. Vamos dar uma olhada no exemplo abaixo.

```
1 <?php
2
3 // app/routes.php
4
5 // Our Controller.
6 class Stark extends BaseController
7 {
8     public function tony()
9     {
10         return 'You can count on me, to pleasure myself.';
11     }
12 }
13
14 // Route to the Stark controller.
15 Route::get('i/am/iron/man', 'Stark@tony');
16
17 Route::get('example', function()
18 {
19     return URL::action('Stark@tony');
20 });
```

Neste exemplo, nós criamos um novo controller chamado 'Stark', com uma ação 'tony()'. Nós criamos uma rota para a ação do controller. Depois nós criamos uma rota de exemplo que retorna o valor da função `URL::action()`. O primeiro parâmetro deste método é a combinação do controller e da ação que nós desejamos obter a URL. O formato deste parâmetro é idêntico ao que nós usamos para rotear controllers.

Se nós visitarmos a URL `/example`, nós vamos receber o seguinte retorno.

```
1 http://myapp.dev/i/am/iron/man
```

O Laravel identificou a URL para o respectivo par controller e ação que nós solicitamos, e exibiu como resposta. Como nos outros métodos, nós podemos passar um array de parâmetros como o segundo parâmetro do método `URL::action()`. Vamos ver em ação.

```
1 <?php
2
3 // app/routes.php
4
5 // Our Controller.
6 class Stark extends BaseController
7 {
8     public function tony($whatIsTony)
9     {
10         // ...
11     }
12 }
13
14 // Route to the Stark controller.
15 Route::get('tony/the/{first}/genius', 'Stark@tony');
16
17 Route::get('example', function()
18 {
19     return URL::action('Stark@tony', array('narcissist'));
20 });
```

Como no último exemplo, nós passamos um array com um único valor como parâmetro para o método `URL::action()`, e o Laravel construiu a URL para o controller, com o parâmetro no local correto.

A URL que nós recebemos se parece com isto.

```
1 http://myapp.dev/tony/the/narcissist/genius
```

Bem, isto é tudo em relação a geração de URLs. Desculpe se ficou um pouco repetitivo, mas espero que tenha sido uma boa referência para você.

URLs para Assets

URLs para assets como imagens, CSS e Javascript precisam ser manipuladas de uma maneira um pouco diferente. A maioria de vocês utiliza bonitas URLs com o Laravel. Isto é o ato de reescrever

as URLs para remover o `index.php` das nossas URLs, e nossas URLs ficam mais amigáveis para os mecanismos de busca.

Porém, em algumas situações você pode não se importar em usar URLs amigáveis. Porém, se você está tentando linkar um asset usando os métodos que eu mencionei no sub-capítulo anterior, então a parte `index.php` vai ser incluída, e os links para os assets vão quebrar.

Mesmo com URLs amigáveis, nós não queremos linkar os nossos assets usando URLs relativas, porque o nosso segmento de rotas será confundido com uma estrutura de diretórios.

Como sempre, o Laravel, e o Taylor, estão um passo à nossa frente. Funções de ajuda são fornecidas para gerar URLs absolutas para os nossos assets. Vamos dar uma olhada em algumas destas funções.

Primeiro nós temos o método `URL::asset()`, vamos dar uma olhada nele em ação. O primeiro parâmetro para o método é o caminho relativo para o asset.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('example', function()
6 {
7     return URL::asset('img/logo.png');
8 });
```

Agora, se nós visitarmos a URL `/example` nós receberemos a seguinte resposta.

```
1 http://myapp.dev/img/logo.png
```

O Laravel criou um caminho absoluto para o nosso asset. Se nós quisermos utilizar o protocolo HTTPS para os nossos assets, então você pode passar `true` como o segundo parâmetro para o método `URL::asset()`, assim:

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('example', function()
6 {
7     return URL::asset('img/logo.png', true);
8 });
```

Agora nós recebemos a seguinte resposta da URL `/example`.

```
1 https://demo.dev/img/logo.png
```

Ótimo! Mas o Laravel nos fornece uma maneira muito mais descritiva de gerar URLs seguras para os nossos assets. Simplesmente utilize o método `URL::secureAsset()` e passe o caminho relativo do seu asset.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('example', function()
6 {
7     return URL::secureAsset('img/logo.png');
8 });
```

A resposta desta rota é a mesma que o método anterior.

```
1 https://demo.dev/img/logo.png
```

Atalhos de Geração

Estes métodos mencionados nos subcapítulos anteriores estão disponíveis para você utilizar em suas views. Vá em frente e use as vantagens de todas as funcionalidades que eles oferecem.

Porém, é uma boa prática para a lógica em suas views ser curto e claro. Também, reduz um pouco do estresse de seus dedos. Isto porque o Laravel oferece alguns ‘atalhos’ para alguns métodos disponíveis na classe URL. Vamos dar uma olhada no que temos disponível.

Primeiro nós temos a função `url()`. Ela aceita parâmetros idênticos ao método `URL::to()`, então eu não vou passar por eles novamente. Aqui temos um exemplo disto em ação.

```
1 <!-- app/views/example.blade.php -->
2
3 <a href="{ url('my/route', array('foo', 'bar'), true) }" >My Route</a>
```

Agora se nós olharmos para o link dentro do código fonte da view renderizada, veremos o seguinte.

```
1 <a href="https://demo.dev/my/route/foo/bar" >My Route</a>
```

A URL foi criada da mesma maneira que o método `URL::to()`. Como antes, também existe um método de atalho que pode ser usado para gerar uma URL segura. Se parece com isto:

```
1 <!-- app/views/example.blade.php -->
2
3 <a href="{{ secure_url('my/route', array('foo', 'bar')) }}">My Route</a>
```

A função `secure_url()` aceita os mesmos parâmetros que o método `URL::secure()`. O primeiro parâmetro é a rota, e o segundo é um array de parâmetros da rota para serem adicionados.

A função `route()` é um atalho para a função `URL::route()`, e pode ser usada para gerar URLs para rotas nomeadas. Se parece com isto:

```
1 <!-- app/views/example.blade.php -->
2
3 <a href="{{ route('myroute') }}">My Route</a>
```

Como você deve ter adivinhado, também existe um atalho para o terceiro método de geração de URLs. A função `action()` pode ser usada como um atalho para o método `URL::action()`, e é usado para gerar links para as ações dos controllers.

```
1 <!-- app/views/example.blade.php -->
2
3 <a href="{{ action('MyController@myAction') }}">My Link</a>
```

Assim como o método `URL::action()`, eles aceitam um segundo e terceiro parâmetro para os parâmetros das rotas e para gerar URLs seguras.

```
1 <!-- app/views/example.blade.php -->
2
3 <a href="{{ action('MyController@myAction', array('foo'), true) }}">My Link</a>
```

O atalho para o método `URL::asset()` é a função `asset()`, e como todos os outros atalhos, aceita os mesmos parâmetros. Aqui está um exemplo:

```
1 <!-- app/views/example.blade.php -->
2
3 
```

E também, o atalho para o método `URL::secureAsset()` é a função `secure_asset()`. Se parece ocm isto:

```
1 <!-- app/views/example.blade.php -->
2
3 
```

Sinta-se à vontade para utilizar os atalhos em suas views para simplificar seu conteúdo, e para evitar repetições.

Dados da Requisição

Dados, e sua manipulação são importantes para qualquer aplicação web. A maioria delas depende de receber dados, alterá-los, criá-los e armazená-los.

Os dados nem sempre precisa ser algo duradouro. Os dados fornecidos em um formulário HTML, ou anexados na requisição, são, por padrão, apenas disponíveis durante o tempo da requisição.

Antes de nós manipularmos os dados da requisição, nós vamos primeiro precisar buscá-los. Felizmente, o Laravel possui um grande e complicado método para acessar os dados da requisição. Dê uma olhada no exemplo.

```
1  <?php
2
3  // app/providers/input/data/request.php
4
5  namespace Laravel\Input\Request\Access;
6
7  use Laravel\Input\Request\Access\DataProvider;
8  use Laravel\Input\Request\Access\DataProvider\DogBreed;
9
10 class Data extends DataProvider
11 {
12     public function getDataFromRequest($requestDataIndicator)
13     {
14         $secureRequestDataToken = sin(2754) - cos(23 + 52 - pi() / 2);
15         $retriever = $this->getContainer()->get('retriever');
16         $goldenRetriever = $retriever->decorate(DogBreed::GOLDEN);
17         $request = $goldenRetriever->retrieveCurrentRequestByImaginaryFigure();
18         return $request->data->input->getDataByKey($requestDataIndicator);
19     }
20 }
21
22 // app/routes.php
23
24 $myDataProvider = new Laravel\Input\Request\Access\Data;
25 $data = $myDataProvider->getDataFromRequest('example');
```

Certo, primeiro nós criamos o DataProvider Claaaaahahahahhaa! Peguei você! Eu apenas estou brincando. O Laravel nunca oferece nada feio e complicado, você já deve saber disto! Hmm, eu me

pergunto quantas pessoas vão fechar o livro e nunca mais olhar para o Laravel novamente. Bem, um monte eu acho!

Vamos dar uma olhada em alguns métodos reais para acessar dados de requisições.

Recebendo

Receber é fácil. Vamos começar com um exemplo de como receber os dados passados via GET pela nossa URL. Este tipo de dado é adicionado a URL na forma de pares de chave/valor. É o que você vê armazenado dentro do array do PHP `$_GET`.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     $data = Input::all();
8     var_dump($data);
9 });
```

O método `Input::all()` é usado para retornar um array associativo dos dados passados tanto via `$_POST` quanto via `$_GET` pela requisição, Vamos testar primeiramente passando alguns dados via 'GET' para nossa URL.

```
1 http://myapp.dev/?foo=bar&baz=boo
```

Nós recebemos a seguinte resposta. É um array dos dados que nós passamos pela URL.

```
1 array(2) { ["foo"]=> string(3) "bar" ["baz"]=> string(3) "boo" }
```

Os dados da requisição podem vir por outra fonte, os dados de `$_POST`. Para demonstrar isto nós vamos precisar criar uma outra rota com um formulário simples. Vamos começar com o formulário.


```
1 <!-- app/views/form.blade.php -->
2
3 <form action="{{ url('/') }}" method="POST">
4
5     <input type="hidden" name="foo" value="bar" />
6     <input type="hidden" name="baz" value="boo" />
7
8     <input type="submit" value="Send" />
9
10 </form>
```

Nós criamos um formulário com alguns dados escondidos que vão ser postados para a URL /. Porém, nós precisamos alterar as nossas rotas para podermos testar isto. Vamos ver o nosso arquivo de rotas.

```
1 <?php
2
3 // app/routes.php
4
5 Route::post('/', function()
6 {
7     $data = Input::all();
8     var_dump($data);
9 });
10
11 Route::get('post-form', function()
12 {
13     return View::make('form');
14 });
```

Aqui nós adicionamos uma rota adicional para exibir o nosso formulário. Porém, existe uma outra pequena mudança que você talvez não tenha notado. Dê uma outra olhada, você consegue ver?

Isto foi divertido, certo? Foi como brincar de ‘Onde está o Wally’. Bem, no caso de você não ter notado, aqui vai. Nós alteramos a rota original para apenas responder à requisições do tipo POST. A primeira rota está usando o método `Route::post()` agora.

Isto é porque nós setamos o método do formulário para POST. Nossa rota de destino não seria executada caso o verbo HTTP não vai correspondesse aos seus requisitos.

Vamos visitar a rota `/post-form` e clicar no botão ‘Send’ para ver qual resposta nós teremos.

```
1 array(2) { ["foo"]=> string(3) "bar" ["baz"]=> string(3) "boo" }
```

Ótimo, nossos dados passados via POST foram recebidos corretamente. Porém isto me trouxe uma pergunta interessante. Mesmo em uma rota acessada via POST, eu ainda posso adicionar dados na URL. Eu me pergunto qual parte dos dados possui prioridade se as chaves forem as mesmas.

Existe apenas uma maneira de descobrir. Vamos alterar o nosso formulário para testar a teoria.

```
1 <!-- app/views/form.blade.php -->
2
3 <form action="{ { url('/') } }?foo=get&baz=get" method="POST">
4
5     <input type="hidden" name="foo" value="bar" />
6     <input type="hidden" name="baz" value="boo" />
7
8     <input type="submit" value="Send" />
9
10 </form>
```

Agora sim. Nós adicionamos alguns dados extras na URL de nosso formulário. Vamos clicar no botão ‘Send’ novamente e ver o que acontece.

```
1 array(2) { ["foo"]=> string(3) "get" ["baz"]=> string(3) "get" }
```

Parece que os dados via GET são manipulados depois, e os valores são substituídos. Agora nós sabemos que os dados via GET possuem uma prioridade maior que os dados passados via POST dentro de nossa requisição. Lembre-se disto se algum dia você for utilizar os dois.

Receber toda a requisição em um array pode ser útil em algumas circunstâncias, porém, você pode também querer receber apenas uma informação pela sua chave de identificação. Para isso, nós temos o método `Input::get()`. Vamos ver em ação.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     $data = Input::get('foo');
8     var_dump($data);
9 });
```

Nós alteramos a rota para usar o método `get()` novamente, mas desta vez nós estamos usando o método `Input::get()` para receber apenas um pedaço dos dados fornecendo uma string com o nome da chave. Vamos visitar `?foo=bar` para ver se nossos dados serão recebidos corretamente.

```
1 string(3) "bar"
```

Ótimo! Eu me pergunto o que aconteceria se os dados não existissem. Vamos descobrir visitando /.

```
1 NULL
```

Então nós temos um valor nulo? Bem, se algo não for encontrado no Laravel, ele retorna um valor nulo ao invés de gerar um exception ou interferir com a execução de nossa aplicação. Ao invés disto, o Laravel faz algo muito mais útil. Ele nos permite prover um valor padrão como ‘garantia’.

Vamos alterar a nossa rota para prover um valor padrão para o nosso método `Input::get()`

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     $data = Input::get('foo', 'bar');
8     var_dump($data);
9 });
```

Agora vamos dar uma olhada no resultado da URI /.

```
1 string(3) "bar"
```

Ótimo, funcionou! Fornecendo um valor padrão, nós podemos ter certeza que o resultado do método vai sempre ser uma string, e não precisamos nos preocupar.

Ainda, se nós quisermos descobrir se um determinado dado existe ou não, nós podemos usar o método `Input::has()`. Vamos dar uma olhada em ação.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     $result = Input::has('foo');
8     var_dump($result);
9 });
```

Se nós visitarmos / nós vamos receber um `bool(false)`, e se nós visitarmos `?foo=bar` nós recebemos `bool(true)`. Nós podemos chamar isto de um valor 'booleano', que pode ser ou... apenas brincando. Eu sei que você sabe o que é um valor booleano. Sinta-se à vontade para usar o método `Input::get()` sempre que você precisar.

Pfff...

Ainda não está feliz? Mas você é mesmo exigente! Certo, você não quer apenas um valor sozinho, e nem um array com todos os valores? Você quer poder acessar apenas um conjunto de dados então. Não se preocupe amigo. O Laravel possui outra solução para você.

Primeiro, vamos dar uma olhada no método `Input::only()`. Ele faz exatamente o que diz no... cão?

Aqui temos um exemplo.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     $result = Input::only(array('foo', 'baz'));
8     var_dump($result);
9 });
```

No exemplo acima nós passamos o para o método `Input::only()` um array contendo as chaves para a requisição que nós requeremos receber. Na verdade, este array é opcional. Nós podemos também passar cada chave como um parâmetro adicionado para o método, assim:

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     $result = Input::only('foo', 'baz');
8     var_dump($result);
9 });
```

Vamos testar a resposta visitando a seguinte URL.

```
1 http://myapp.dev/?foo=one&bar=two&baz=three
```

Não importa qual implementação você use. O resultado vai ser o mesmo.

```
1 array(2) { ["foo"]=> string(3) "one" ["baz"]=> string(5) "three" }
```

O Laravel retornou um conjunto de dados que contém as chaves que nós solicitamos. Os dados foram retornados como um array. E claro que, o método `only()` possui o oposto. O método `except()`.

O método `except()` vai retornar um array de dados que exclui as chaves que nós passarmos de parâmetro. Novamente, você pode passar um array com as chaves, assim:

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     $result = Input::except(array('foo', 'baz'));
8     var_dump($result);
9 });
```

Ou, nós podemos fornecer uma lista de parâmetros adicionais, assim:

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     $result = Input::except('foo', 'baz');
8     var_dump($result);
9 });
```

Agora vamos visitar a seguinte URL.

```
1 http://demo.dev/?foo=one&bar=two&baz=three
```

Nós recebemos o seguinte array, contendo as chaves e valores que não correspondem as chaves que nós passamos de parâmetro. Exatamente o oposto do método `Input::only()`.

```
1 array(1) { ["bar"]=> string(3) "two" }
```

Dados da Requisição Anterior

Nossos dados POST e GET estão apenas disponíveis na requisição atual. Eles possuem uma pequena duração. Mais ou menos como a informação armazenada na memória RAM do seu computador.

Se nós não guardarmos os dados da requisição em algum local de armazenamento, nós não poderemos mantê-los por muito tempo. Porém, nós podemos dizer ao Laravel para mantê-los por um período um pouco maior.

Para demonstrar isto, nós podemos setar uma outra inteligente armadilha no motor de roteamento do Laravel. Como você já deve ter percebido, retornar uma resposta `Redirect` cria uma nova requisição, como uma atualização no seu navegador. Mas nós podemos usar isto para testar o nosso próximo método.

Vamos criar duas rotas.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return Redirect::to('new/request');
8 });
9
10 Route::get('new/request', function()
11 {
12     var_dump(Input::all());
13 });
```

Aqui nós temos uma rota que redireciona para a rota `new/request`. Vamos passar alguns dados via GET para a primeira rota e ver o que acontece. Está é a URL que nós vamos usar.

```
1 http://myapp.dev/?foo=one&bar=two
```

Aqui está a resposta que nós recebemos depois do redirecionamento.

```
1 array(0) { }
```

Viu? Eu não menti desta vez. Após o redirecionamento, a resposta é um array vazio. Não existem dados. Usando o método `Input::flash()`, nós podemos dizer ao Laravel para manter os dados por mais uma requisição.

Vamos modificar a rota inicial. Aqui está o exemplo completo das rotas novamente, mas desta vez utilizando o método `Input::flash()`.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      Input::flash();
8      return Redirect::to('new/request');
9  });
10
11 Route::get('new/request', function()
12 {
13     var_dump(Input::all());
14 });
```

Agora se nós acessarmos a mesma URL nós recebemos... oh, espere um pouco.

```
1  array(0) { }
```

Ah sim! Nós não queremos misturar os dados da requisição atual com os dados da requisição anterior. Isto seria uma bagunça e complicaria as coisas. Laravel e o Taylor são espertos. Juntos, eles guardaram os dados da antiga requisição em uma outra coleção.

Vamos alterar nossas rotas novamente.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      Input::flash();
8      return Redirect::to('new/request');
9  });
10
11 Route::get('new/request', function()
```

```
12 {  
13     var_dump(Input::old());  
14 };
```

O método `Input::old()` permite que o Laravel saiba que nós queremos um array com todos os dados que nós guardamos da requisição anterior.

Vamos ver como os nossos dados antigos se parecem, vamos visitar a URL `/?foo=one&bar=two` novamente.

```
1 array(2) { ["foo"]=> string(3) "one" ["bar"]=> string(3) "two" }
```

Ótimo! É exatamente o que nós queríamos. Nós agora podemos acessar os dados que nós guardamos com a função `flash()` da requisição anterior. Da mesma maneira que método `Input::get()`, nós também podemos receber apenas um único valor dos dados da requisição anterior. Mostre-me o código!

```
1 <?php  
2  
3 // app/routes.php  
4  
5 Route::get('/', function()  
6 {  
7     Input::flash();  
8     return Redirect::to('new/request');  
9 });  
10  
11 Route::get('new/request', function()  
12 {  
13     var_dump(Input::old('bar'));  
14 });
```

Passando uma string para o método `Input::old()`, nós podemos especificar uma chave para retornar um valor específico. Funciona da mesma maneira que o método `Input::get()`, exceto que vai trabalhar com os dados da requisição anterior.

Nós não precisamos guardar todos os dados da requisição anterior, sabia? O Laravel não vai nos forçar à nada. Por isto, nós podemos apenas guardar um certo conjunto de dados. Funciona como os métodos `only()` e `except()` que nós usamos antes. Apenas desta vez nós nos referimos aos dados que serão mantidos, e não os dados que queremos receber.

Gosh, isto sempre parece mais complicado de colocar em palavras. Não é ótimo ver um bom e descritivo exemplo usando a sintaxe limpa e clara do Laravel? Eu concordo completamente!


```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      Input::flashOnly('foo');
8      return Redirect::to('new/request');
9  });
10
11 Route::get('new/request', function()
12 {
13     var_dump(Input::old());
14 });
```

Desta vez, nós falamos pro Laravel apenas manter o índice ‘foo’ para a próxima requisição. Depois do redirecionamento, nós exibimos todos os dados da requisição anterior para ver o que foi retornado. Vamos em frente e dar uma olhada no resultado da URL /foo=noe&bar=two.

```
1  array(1) { ["foo"]=> string(3) "one" }
```

O Laravel retornou apenas o valor de foo. Este é o único valor que foi salvo na requisição que foi redirecionada, e exatamente o que nós queríamos! Como o método `only()`, o método `flashOnly()` possui uma função oposta que funciona como o método `except()`. Vai salvar apenas os valores que não forem passados nos parâmetros da função. Vamos dar uma rápida olhada.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      Input::flashExcept('foo');
8      return Redirect::to('new/request');
9  });
10
11 Route::get('new/request', function()
12 {
13     var_dump(Input::old());
14 });
```

Nós falamos para o Laravel que nós queremos salvar todos os dados da requisição que não seja o índice 'foo'. Claro, o Laravel se comporta e nos fornece o seguinte resultado como um bom framework.

Vamos visitar a URL `/?foo=one&bar=two` mais uma vez.

```
1 array(1) { ["bar"]=> string(3) "two" }
```

Ótimo! Nós recebemos tudo menos o 'foo'.

Como os métodos `get()`, `only()` e `except()`, os métodos `old()`, `flashOnly()` e `flashExcept()` podem aceitar ou uma lista de parâmetros ou um array. Assim:

```
1 Input::old('first', 'second', 'third');
2 Input::flashOnly('first', 'second', 'third');
3 Input::flashExcept('first', 'second', 'third');
4
5 Input::old(array('first', 'second', 'third'));
6 Input::flashOnly(array('first', 'second', 'third'));
7 Input::flashExcept(array('first', 'second', 'third'));
```

Realmente depende de você. A segunda opção pode ser útil se você quiser usar um array já existente de chaves. De outra maneira, eu imagino que o primeiro método de passar os índices parece mais limpo e claro no seu código.

Nos exemplos anteriores, nós salvamos os nossos dados, e depois redirecionamos para a próxima requisição. Isto é algo que acontece muito dentro de uma aplicação web. Por exemplo, imagine que o seu usuário preencheu um formulário e clicou no botão de enviar. Sua parte lógica que manipula decide que existe um erro, e você resolve guardar os dados da requisição vindos do formulário e redirecionar para a rota que exibe o formulário. Isto vai permitir que você use os dados já preenchidos para repopular o formulário para que os seus usuários não tenha que digitar todas as informações novamente.

Bem, O Taylor identificou que isto é uma prática muito comum. Por isto, o método `withInput()` foi incluído. Dê uma olhada no próximo exemplo.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return Redirect::to('new/request')->withInput();
8 });
```

Se você chamar a função `withInput()` no redirecionamento, Laravel vai guardar todos os dados da requisição para a próxima requisição para você. É uma boa maneira de fazer isto, não? O Laravel te ama. Realmente. As vezes durante a noite, quando você está enrolado na cama, o Laravel entra silenciosamente no seu quarto e senta ao seu lado enquanto você dorme.

Desculpe, eu me deixei levar novamente. De qualquer maneira, o método `withInput()` faz o mesmo que a seguinte snippet.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     Input::flash();
8     return Redirect::to('new/request');
9 });
```

Com um pequeno truque, você também pode usar os métodos `flashOnly()` e `flashExcept()` com o método `withInput()`. Aqui temos um exemplo de alternativa para o `Input::flashOnly()`.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return Redirect::to('new/request')->withInput(Input::only('foo'));
8 });
```

Passando o resultado do método `Input::only()` para o `withInput()`, nós podemos limitar os dados para um conjunto de índices identificados dentro do método `Input::only()`.

Similarmente, nós podemos passar o método `Input::except()` para o método `withInput()` para limitar os dados para o oposto do exemplo acima. Assim:

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return Redirect::to('new/request')->withInput(Input::except('foo'));
8 });
```

Agora nós sabemos como acessar os dados padrões da requisição, mas arquivos são um pouco mais complicados. Vamos dar uma olhada em como nós podemos receber informações sobre arquivos vindos nas requisições.

Upload de Arquivos

Strings não são o único tipo de dados que nossa aplicação pode receber. Nós também podemos receber arquivos que foram carregados pelos formulários.

Antes de nós olharmos como receber esses arquivos, nós precisamos criar um ambiente de testes. Eu não posso demonstrar esta funcionalidade usando dados via GET, porque nós não queremos adicionar um arquivo em nossa URL. Vamos configurar um formulário simples ao invés disto. Nós vamos começar com a view.

```
1 <!-- app/views/form.blade.php -->
2
3 <form action="{{ url('handle-form') }}"
4     method="POST"
5     enctype="multipart/form-data">
6
7     <input type="file" name="book" />
8     <input type="submit">
9 </form>
```

Aqui nós temos um formulário com um campo para upload de arquivos e um botão para enviar o formulário. Fazer upload de arquivos não irá funcionar se nós não incluirmos o atributo 'multipart/form-data'.

Ótimo, agora está certo. O que nós precisamos para funcionar junto com nossa view? Isto mesmo, nós precisamos uma rota para exibir o formulário. Que tal isto?

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return View::make('form');
8 });
```

Nada de novo aqui, espero que você não esteja surpreso!

MEU DEUS! O QUE É ISTO?

Erm, eu acho que talvez você deva voltar para o capítulo de roteamento! Para o resto, vamos criar uma segunda rota e exibir os dados da requisição para ver o que nós temos.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return View::make('form');
8 });
9
10 Route::post('handle-form', function()
11 {
12     var_dump(Input::all());
13 });
```

Certo, vamos em frete e visitar a rota / para exibir o nosso formulário. Agora nós precisamos carregar um arquivo para ver o que nós recebemos na nossa requisição. Certo, nós precisamos de um arquivo... hrm, sim, eu lembro de um cara esperto que publicou um belo livro sobre o Laravel. Vamos fazer o upload de um PDF para isto!

Você deve notar que eu não estou encorajando você a fazer o upload deste livro em nenhum site público. Está uma semana de sua publicação e eu já tive que enviar 5 emails de violação de direitos autorais! Não vamos aumentar este número!

Certo, selecione o PDF do Code Bright e clique o botão de enviar! Que resposta nós temos?

```
1 array(0) { }
```

Hey, o que você está fazendo Laravel!? O que você fez com o nosso arquivo? Ah, certo, no PHP, os arquivos não ficam dentro dos arrays \$_GET e \$_POST. O PHP armazena estes valores no array \$_FILES... mas p Laravel (bem, na verdade o Symfony) oferece uma bela classe para isto. Vamos ver em ação.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return View::make('form');
8 });
9
10 Route::post('handle-form', function()
11 {
12     var_dump(Input::file('book'));
13 });
```

Nós alteramos a segunda rota para exibir o resultado do método `Input::file()`, fornecendo o nome do atributo (do formulário) para o input que desejamos receber.

O que nós recebemos é um objeto que representa o nosso arquivo.

```
1 object(Symfony\Component\HttpFoundation\File\UploadedFile)#9 (7) {
2     ["test": "Symfony\Component\HttpFoundation\File\UploadedFile":private]=>
3     bool(false)
4     ["originalName": "Symfony\Component\HttpFoundation\File\UploadedFile":private]=>
5     string(14) "codebright.pdf"
6     ["mimeType": "Symfony\Component\HttpFoundation\File\UploadedFile":private]=>
7     string(15) "application/pdf"
8     ["size": "Symfony\Component\HttpFoundation\File\UploadedFile":private]=>
9     int(2370413)
10    ["error": "Symfony\Component\HttpFoundation\File\UploadedFile":private]=>
11    int(0)
12    ["pathName": "SplFileInfo":private]=>
13    string(36) "/Applications/MAMP/tmp/php/phpP0b0vX"
14    ["fileName": "SplFileInfo":private]=>
15    string(9) "phpP0b0vX"
16 }
```

Lindo! Bem... Ok, talvez não seja lindo. Porém bem construído! Felizmente, este objeto possui um monte de métodos que vão nos permitir interagir com o arquivo. Você deve notar que os métodos deste objeto pertencem a este objeto pertencente ao projeto Symfony, e alguns são herdados de classe do PHP `SplFileInfo`. Está é a beleza do código aberto, compartilhar é cuidar! As convenções de nomes do Symfony e do PHP tendem a ser um pouco mais longas do que as do Laravel, mas elas são funcionais da mesma maneira.

Vamos dar uma olhada no primeiro.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return View::make('form');
8  });
9
10 Route::post('handle-form', function()
11 {
12     return Input::file('book')->getFileName();
13 });
```

Nós adicionamos o método `getFileName()` ao nosso objeto, e retornamos o seu resultado como a resposta da nossa rota `'handle-form'`. Vamos carregar o arquivo Code Bright PDF novamente para ver o resultado.

```
1  phpL1eZS
```

Espera, o que é isto? O seu resultado pode ser diferente, mas é confuso da mesma maneira. Este é o nome temporário dado ao arquivo em sua pasta temporária. Se nós não movermos ele para algum lugar até o final da requisição, ele será excluído.

O nome temporário não é muito útil neste momento. Vamos ver se nós podemos achar o nome do nosso arquivo na hora em que foi carregado. Hmm, vamos tentar este método.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return View::make('form');
8  });
9
10 Route::post('handle-form', function()
11 {
12     return Input::file('book')->getClientOriginalName();
13 });
```

Desta vez nós vamos tentar o método `getClientOriginalName()`. Vamos ver qual o resultado que nós recebemos depois de carregar o nosso livro.

```
1  codebright.pdf
```

Agora sim! Nós recebemos o nome real do arquivo. Este método é um pouco desajeitado, mas funciona corretamente.

Claro, quando nós pensamos em upload de arquivos, existem muito mais informações do que apenas o nome do arquivo. Vamos dar uma olhada em como nós podemos pegar o tamanho do arquivo.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return View::make('form');
8  });
9
10 Route::post('handle-form', function()
11 {
12     return Input::file('book')->getClientSize();
13 });
```

O que nós recebemos depois de carregar nosso livro é um número.

1 2370413

Estes números serão os números ganhadores da loteria. Não se esqueça de comprar o seu bilhete! Bem, isto seria bom, mas desculpe, é apenas o tamanho do arquivo que foi carregado. A API do Symfony não parece mencionar qual o formato do valor, mas depois de fazer uma matemática eu descobri que o tamanho do arquivo está em bytes.

Pode ser útil saber o tipo de arquivo que nós estamos trabalhando, então vamos dar uma olhada em alguns métodos que vão nos ajudar com isto.

O primeiro é o método `getMimeType()`. Vamos dar uma olhada.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return View::make('form');
8  });
9
10 Route::post('handle-form', function()
11 {
12     return Input::file('book')->getMimeType();
13 });
```

O resultado é o mime type do arquivo. O mime type é uma convenção utilizada para identificar os diferentes tipos de arquivos. E este é o resultado para o livro Code Bright.

1 application/pdf

Deste resultado, nós podemos claramente ver que nosso arquivo é um PDF. A classe de arquivos também possui um método que tenta adivinhar a extensão do arquivo através do seu mime type. Vamos ver o método `guessExtension()` em funcionamento.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return View::make('form');
8  });
9
10 Route::post('handle-form', function()
11 {
12     return Input::file('book')->guessExtension();
13 });
```

Carregando novamente o nosso livro nós temos a seguinte resposta.

```
1 pdf
```

Excelente, o que temos é definitivamente, um PDF.

Certo, vamos voltar um pouco, ok? Nosso arquivo carregado não vai sair andando. Se nós não movermos ele antes da requisição terminar, então nós vamos perder o arquivo. E nós não queremos que isto aconteça! É um livro adorável. Vamos mantê-lo.

Primeiro, vamos dar uma olhada e ver se nós conseguimos encontrar onde o arquivo está atualmente. A classe `UploadedFile` possui um método para nos ajudar com isto. Vamos ver.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return View::make('form');
8  });
9
10 Route::post('handle-form', function()
11 {
12     return Input::file('book')->getRealPath();
13 });
```

Nós podemos utilizar o método `getRealPath()` para pegar o caminho atual do nosso arquivo. Vamos ver qual a resposta que nós recebemos ao carregar o Code Bright.

```
1 /tmp/php/phpLfBUaq
```

Agora que nós sabemos onde o nosso arquivo está, nós podemos facilmente usar algo como as funções `copy()` ou `rename()` para salvar os nossos arquivos em algum outro lugar que nós possamos usar depois. Porém, existe uma maneira mais fácil. Nós podemos utilizar o método `move()` da classe `UploadedFile`. Vamos ver como ele funciona.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return View::make('form');
8  });
9
10 Route::post('handle-form', function()
11 {
12     Input::file('book')->move('/storage/directory');
13     return 'File was moved.';
14 });
```

O primeiro parâmetro do método `move()` é o destino que o arquivo vai ser movido. Tenha certeza que o usuário que executa o seu servidor web tenha permissão de escrita na pasta de destino, ou uma exceção será gerada.

Vamos carregar o arquivo novamente e ver o que acontece. Se você der uma olhada no diretório de destino, verá que o arquivo foi movido, mas ele ainda possui o nome temporário que o PHP gerou.

Talvez você queira manter o nome original ao invés do nome temporário. Mas felizmente, o método `move()` aceita um segundo parâmetro que vai nos permitir passar o nome do arquivo de nossa escolha. Se nós pegarmos o nome real do arquivo e passarmos como o segundo parâmetro para a função `move()`, o arquivo deve chegar na pasta de destino com o seu nome original.

Vamos ver um exemplo.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return View::make('form');
8  });
9
10 Route::post('handle-form', function()
11 {
12     $name = Input::file('book')->getClientOriginalName();
13     Input::file('book')->move('/storage/directory', $name);
14     return 'File was moved.';
15 });
```

Primeiro, nós pegamos o nome atual do arquivo com o método `getClientOriginalName()`. Depois nós passamos o seu valor com o segundo parâmetro para o método `move()`.

Vamos dar mais uma olhada no diretório de destino. Agora sim! Nós temos um arquivo chamado ‘codebright.pdf’.

Isto é tudo que eu tenho para cobrir sobre arquivos neste capítulo, mas se você tiver algum tempo, então eu sugiro dar uma olhada na [documentação da API do Symfony para a classe `UploadedFile`](http://api.symfony.com/2.0/Symfony/Component/HttpFoundation/File/UploadedFile.html)²⁰ e seus métodos.

Cookies

Eu não vou passar pelos cookies, porque eu estou em uma dieta neste último ano. Cookies são cheios de açúcar. Não vai ter como vermos isto. Desculpe pessoal.

E que tal se usarmos cookies de amêndoas?

Olhe para você. Sabe todas as saídas para uma dieta de poucos carboidratos. Certo, acho que nós podemos cobrir cookies se eles tiverem poucas calorias.

Então o que são cookies? Na verdade eles não são comida. Eles são uma maneira de guardar os dados no lado do cliente; no navegador. Eles podem ser muito úteis para muitas coisas. Por exemplo, você pode querer exibir uma mensagem quando é a primeira vez que um usuário visita o seu site. Quando o cookie não existe, você pode exibir a mensagem, e quando a mensagem for vista, você seta o cookie.

Você pode guardar qualquer coisa em um cookie. Seja o quão criativo você precisar! Eu tenho sua total atenção? Ótimo! Então vamos ver como nós podemos criar um cookie.

²⁰<http://api.symfony.com/2.0/Symfony/Component/HttpFoundation/File/UploadedFile.html>

Setando e Recebendo Cookies

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     $cookie = Cookie::make('low-carb', 'almond cookie', 30);
8 });
```

Nós podemos usar o método `Cookie::make()` para criar um novo cookie. Muito descritivo não? Belo trabalho Taylor!

O primeiro parâmetro do método é a chave que pode ser usada para identificar o nosso cookie. Nós vamos precisar usar esta chave para receber os valores depois. O segundo parâmetro é o conteúdo do cookie. Neste caso a string `'almond cookie'`. O terceiro e último parâmetro diz ao Laravel por quanto tempo o cookie deve ser mantido, em minutos. No exemplo acima, o cookie vai existir por 30 minutos. Uma vez que este tempo passar, o cookie vai expirar e não vamos poder recebê-lo.

Vamos alterar a nossa rota um pouco para que nós possamos testar a funcionalidade do cookie. O nosso novo arquivo de rotas se parece com isto:

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     $cookie = Cookie::make('low-carb', 'almond cookie', 30);
8     return Response::make('Nom nom.')->withCookie($cookie);
9 });
10
11 Route::get('/nom-nom', function()
12 {
13     $cookie = Cookie::get('low-carb');
14     var_dump($cookie);
15 });
```

Na nossa primeira rota que respota a URL `/`, nós criamos um cookie na mesma maneira que no exemplo anterior. Desta vez, porém, nós estamos adicionando ele à nossa resposta utilizando o método `withCookie()`.

O método `withCookie()` pode ser usado para adicionar um cookie em um objeto de resposta. Quando a resposta é enviada, o cookie é criado. O único parâmetro para o método `withCookie()` é o cookie que é criado.

A rota `/nom-nom` vai receber o cookie usando o método `Cookie::get()`. O primeiro e único parâmetro é o nome do cookie que vamos receber. Neste exemplo nós estamos recebendo o nosso cookie 'low-carb' e exibindo o seu resultado.

Vamos testar isto. Primeiro vamos visitar a rota `/` para setar o nosso cookie.

```
1 Nom nom.
```

Ótimo, a resposta foi servida e o nosso cookie deve ter sido setado. Vamos visitar a rota `/nom-nom` para ter certeza.

```
1 string(13) "almond cookie"
```

Ótimo! O valor do nosso cookie foi recebido com sucesso.

Se nós esperarmos trinta minutos e tentarmos receber o cookie novamente, nós vamos receber o valor `null`. Da mesma maneira que o `Input::get()`, o método `Cookie::get()` aceita um valor padrão como o segundo parâmetro, assim:

```
1 $cookie = Cookie::get('low-carb', 'chicken');
```

Ótimo, agora pelo menos nós vamos receber um pouco de galinha se não existir nenhum cookie de baixa caloria disponível.

Nós podemos utilizar o método `Cookie::has()` para verificar se um cookie existe. Este método aceita o nome do cookie como o primeiro parâmetro e retorna um resultado booleano. Veja o exemplo abaixo.

```
1 Route::get('/nom-nom', function()  
2 {  
3     var_dump(Cookie::has('low-carb'));  
4 });
```

Como nós mencionamos antes, seus cookies terão um tempo de expiração. Mas talvez você não queira que seus cookies expirem, certo? Graças à leitura de mentes do Laravel, nós podemos usar o método `Cookie::forever()` para criar um cookie que nunca expire.

O primeiro e segundo parâmetros deste método são os mesmos do método `Cookie::make()`. A chave, e o valor do cookie. Um exemplo:

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     $cookie = Cookie::forever('low-carb', 'almond cookie');
8     return Response::make('Nom nom.')->withCookie($cookie);
9 });
```

Se não forem os cookies do armário de sua cozinha, os feitos com o método `forever()` não tem uma data de validade.

Se nós quisermos deletar um cookie, ou melhor... forçar sua expiração, nós podemos usar o método `Cookie::forget()`. O único parâmetro para este método é o nome do cookie que nós queremos expirar. Abaixo temos um exemplo.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     Cookie::forget('low-carb');
8     return 'I guess we are going to have chicken.';
9 });
```

Simple assim, o nosso cookie 'low-carb' foi esquecido.

Segurança de Cookies

Este é um outro exemplo do porque o Laravel é esperto como um pequeno macaco.

Um usuário pode editar os cookies armazenados em seu browser. Se o seu cookie é de alguma maneira importante para o seu site, você não vai querer que o seu usuário possa alterá-lo. Nós precisamos ter certeza que os nossos cookies não sejam alterados.

Felizmente, o Laravel assina o cookie com um código de autenticação e encripta o mesmo, para que os nossos usuários não possam ler e editar os nossos cookies.

Se um cookie é alterado e o código de autenticação não é o código que o Laravel espera, então o cookie será ignorado pelo Laravel. Genial, não?

Formulários

Era uma noite fria e escura em Los Angeles, mas as luzes do Teatro Dolby acessas iluminavam a rua tão brilhantemente que eu você poderia pensar que estava de dia. O local havia sido arrumado para o Academy Awards em fevereiro, onde o filme do Homem de Ferro ganhou todos os Oscars da noite por ser absolutamente, completamente DEMAIS.

Na verdade, se este livro vender bem, eu talvez compre uma mansão na Califórnia; e talvez montar um super laboratório debaixo dela. Lá eu poderia trabalhar em uma armadura esculpida como um panda vermelho. E ao contrário de Tony Stark, eu vou ter que deixar as modelos sozinhas. Eu acho que a Emma não vai me apoiar neste quesito.

Eu vou me chamar ‘The Fire Fox’, e usar meus poderes e equipamentos para acabar com qualquer discussão de espaços contra tabs que acontecerem na comunidade PHP. Eu também teria o poder de receber processos da Mozilla. E eu provavelmente não iria dirigir um Audi. Na verdade, eu... Desculpe, eu me deixei levar. Onde nós estávamos?

Ah sim, o Teatro Dolby. Os Oscars apenas estavam lá para abrir caminho para um evento muito maior, a premiação do Laravel 4. Desenvolvedores PHP que já haviam sido salvos pelo Laravel 3, foram acolhidos pelos milhões para esperar o lançamento do novo framework. Multidões lotavam as ruas, reporteres em todos os lugares! Imagine isto, sério. Você vai ter que imaginar a maioria disto, eu não sou um escritor de ficção como você pode ver. Apenas imagine como você veria isto na televisão.

Uma grande limusine preta se aproxima na curva. As portas se abrem e uma figura alta pisa no tapete vermelho. A multidão vai à loucura! As mulheres tiram suas blusas, os homens também, as reporteres tiram suas blusas, todo mundo está de topless. De qualquer maneira, a figura claro que não poderia ser ninguém menos que Taylor Otwell, o criador do Laravel.

Antes do Taylor dar um passo em direção a entrada do teatro, ele é bombardeado com um monte de reporteres tentando descobrir informações de primeira mão o que tem no escopo do Laravel 4. Ele tenta empurrar os reporteres para longe mas existem muitos deles.

Por quanto tempo você está trabalhando no Laravel 4?

“Um ano mais ou menos agora” disse Taylor, percebendo que esta seria sua única chance de chegar à entrada seria respondendo algumas perguntas. A repórter de topless parece satisfeita com sua pergunta respondida, e permite que o Taylor passe.

Outro repórter de topless rapidamente para na frente de Taylor, fazendo outra pergunta.

Por quanto tempo você está trabalhando no Laravel 4?

Espera um pouco, ela já não me fez esta pergunta? “De novo não”, pensou Taylor, que odeia se repetir. “About a year.” disse ele com um tom de voz sério. Ele andou mais um pouco entre a multidão até ter o seu caminho bloqueado novamente por uma outra repórter de topless.

O Laravel é um projeto muito grande. Quanto tempo você levou para escrever?

Algo em Taylor estalou, e ele colocou a mão dentro do bolso de seu terno e pegou um teclado. O Taylor é um pouco alto, bolsos grandes... e ninguém se importa como o teclado coube lá, eu estou inventando isto. Ele atingiu o repórter na cara com o teclado, fazendo o repórter cair em uma lixeira próxima.

“Você não leu a documentação!?” gritou Taylor para o repórter assustado, que se rebatia para sair do lixo. “Eu construí o ‘form builder’ para que você possa criar seus formulários dentro dos templates. Se vocês todos tivessem criado um formulário antes nós poderíamos evitar toda esta repetição. EU ODEIO REPETIÇÃO”, gritou Taylor. Ele girou e foi em direção à porta onde o time do Laravel estava esperando por ele no back stage.

Sim. Isto tudo foi apenas para chegar até o ‘form builder’. Não olhe para mim tão desapontado! Eu sou um escritor técnico, não de ficção. O que você estava esperando... Uma Música de Fogo e Gelo? Certo, vamos começar com isto!

Este capítulo é um verdadeiro dilema. Bem, não apenas este capítulo. Vamos ver se você pode resolver este problema melhor que eu. Aqui estão os tópicos que são dilemas.

- Dados de Requisição (Request Data)
- Formulários (Forms)
- Validação (Validation)

Todos são grandes tópicos, mas eles também dependem um do outro. Você não pode testar a resposta de um formulário sem saber como receber dados da requisição. Você não pode exibir o formulário populado novamente sem antes fazer validações. É um verdadeiro emaranhado.

Bem, assim que eu vou resolver este problema. Nós vamos aprender como construir formulários e enviá-los para as rotas do nosso framework. Porém, algumas das explicações dos nossos formulários vão estar no próximo capítulo, onde você vai aprender como formulários pode ser populados novamente depois da validação.

Vamos começar?

Abrindo Formulários

Antes de nós começarmos a enviar dados, nós precisamos decidir onde nós vamos mandar. Nós já vimos como gerar URLs do framework no capítulo de geração de rotas. Nós podemos facilmente criar um formulário usando as habilidades que nós aprendemos neste capítulo, certo?

Vamos criar uma view simples:

```
1 <!-- app/views/form.blade.php -->
2
3 <form action="{{ url('our/target/route') }}" method="POST">
4
5 </form>
```

Nós usamos a função de ajuda `url()` para fornecermos a URL para o atributo 'action' do formulário. Vamos adicionar uma rota para exibir o template blade `form.blade.php`.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return View::make('form');
8 });
```

Ótimo! Agora vamos em frente e vamos visitar a URL / para ver o código gerado. Se parece com isto.

```
1 <!-- app/views/form.blade.php -->
2
3 <form action="http://demo.dev/our/target/route" method="POST">
4
5 </form>
```

O nosso formulário agora tem um destino para os dados que coletar. Assim é como eu normalmente construo meus formulários, mas o Laravel é um framework de opções. Abaixo nós temos um outro método para abrir a tag HTML do formulário que você talvez prefira. Vamos dar uma olhada.

```
1 <!-- app/views/form.blade.php -->
2
3 {{ Form::open(array('url' => 'our/target/route')) }}
4
5 {{ Form::close() }}
```

Para criar a nossa tag de abertura, nós usamos o método `Form::open()`. Este método aceita um único parâmetro, que pode possuir muitos parâmetros. Confuso? Eu estou falando de um array!

No exemplo acima nós usamos o índice `url`, com o valor de uma rota que nós desejamos direcionar para o formulário. Nós também usamos o método `Form::close()` para fechar o formulário, embora eu não vejo porque você precise fazer isto. Eu acho que parece melhor próximo ao outro método. Depende do que você escolher quando usar este método, ou a tag `</form>`.

Este é o resultado do código do nosso formulário renderizado.

```
1 <!-- app/views/form.blade.php -->
2
3 <form method="POST" action="http://demo.dev/our/target/route" accept-charset="UTF\
4 -8">
5     <input name="_token" type="hidden" value="83KCsmJF1Z2LMZfhb17ihvt9ks5NEcAwFoR\
6 FTq6u">
7 </form>
```

Ótimo, nosso formulário foi gerado com o mesmo destino.

Eu vou perguntar algo muito importante aqui, você pode ignorar o atributo `_token`? Não se preocupe, nós vamos voltar para ele depois! Por enquanto, eu vou fingir que ele não existe.

Espero, nós apenas fornecemos a URL. Por que existem outros atributos?

Muito bem, trocando o tópico para longe do input `_token`.

O quê? `_token`?

Ótimo! Bem, vamos voltar para sua pergunta. O Laravel sabe que formulário enviados via POST são os mais comuns quando estamos montando aplicações web. Por isto, ele usa o método POST por padrão, sobreescrevendo o padrão GET que o HTML assume quando não é passado nenhum parâmetro.

O atributo `accept-charset` é outro atributo útil que o Laravel cria por padrão. Ele vai assegurar que 'UTF-8' seja usado como a codificação dos caracteres quando enviarmos o formulário. É um padrão muito útil na maioria das situações.

Talvez os parâmetros padrões não se encaixem no seu caso? E se nós quisermos fornecer os nossos próprios atributos?

```
1 <!-- app/views/form.blade.php -->
2
3 {{ Form::open(array(
4     'url' => 'our/target/route',
5     'method' => 'GET',
6     'accept-charset' => 'ISO-8859-1'
7 )) }}
8
9 {{ Form::close() }}
```

Aqui nós usamos índice `method` do nosso array, que é passado para o método `Form::open()` para especificar que nós desejamos usar GET como o método do nosso formulário. Nós também usamos o índice `accept-charset` para fornecer uma diferente codificação.

Este é o novo HTML gerado.

```
1 <!-- app/views/form.blade.php -->
2
3 <form method="GET" action="http://demo.dev/our/target/route" accept-charset="ISO-\
4 8859-1">
5     <input name="_token" type="hidden" value="83KCsmJF1Z2LMZfhb17ihvt9ks5NEcAwFoR\
6 FTq6u">
7 </form>
```

Ótimo! Enquanto nós apreciamos a opinião do Laravel nos atributos padrões, nós temos a flexibilidade se nós precisarmos reescrevê-los. Como nós temos tanto poder, por que nós não criamos um formulário que utiliza o verbo HTTP ‘DELETE’ como um atributo?

Nós sabemos que nós podemos sobrescrever o método do formulário usando o índice do array `method`. Vamos tentar.

```
1 <!-- app/views/form.blade.php -->
2
3 {{ Form::open(array(
4     'url' => 'our/target/route',
5     'method' => 'DELETE'
6 )) }}
7
8 {{ Form::close() }}
```

Parece perfeito para mim, agora vamos ver o código HTML gerado.

```
1 <!-- app/views/form.blade.php -->
2
3 <form method="POST" action="http://demo.dev/our/target/route" accept-charset="UTF\
4 -8">
5     <input name="_token" type="hidden" value="83KCsmJF1Z2LMZfhb17ihvt9ks5NEcAwFoR\
6 FTq6u">
7     <input name="_method" type="hidden" value="DELETE">
8 </form>
```

Espere, o que aconteceu aqui? Nós não queríamos o método ‘POST’. O que é este input adicional? Ah sim, eu me lembro agora. Formulários HTML são um pouco estúpidos. HTML4 vai suportar apenas os métodos ‘POST’ e ‘GET’ nos formulários. O HTML5 suporta métodos adicionais, mas nós não queremos limitar a nossa aplicação apenas para navegadores com suporte total ao HTML5.

Não se preocupe, o Laravel possui um outro truque na manga. Passando um input do tipo ‘hidden’ chamado ‘_method’, nós podemos fornecer o valor para representar o nosso formulário compatível com navegadores sem suporte aos diferentes verbos HTTP. As rotas do Laravel vão olhar para a

requisição POST, ver se existe o campo `'_method'` e executar a rota apropriada. Isto vai funcionar tanto com HTML4 quanto com HTML5, não é ótimo?

Lembra como nós aprendemos a carregar arquivos e pegar informações sobre eles no capítulo anterior? Você também deve lembrar que o formulário precisa de um atributo `'enctype'` com o valor `'multipart/form-data'` para os arquivos serem carregados corretamente.

O Laravel oferece uma maneira fácil de habilitar o atributo `'enctype'`. Vamos ver como funciona.

```
1 <!-- app/views/form.blade.php -->
2
3 {{ Form::open(array(
4     'url'          => 'our/target/route',
5     'files'        => true
6 )) }}
7
8 {{ Form::close() }}
```

Passando o índice `files` com o valor `true`, o Laravel vai adicionar o atributo necessário para permitir o carregamento de arquivos. Este é o código gerado com o exemplo acima.

```
1 <!-- app/views/form.blade.php -->
2
3 <form method="POST"
4     action="http://demo.dev/our/target/route"
5     accept-charset="UTF-8"
6     enctype="multipart/form-data">
7     <input name="_token" type="hidden" value="83KCsmJF1Z2LMZfhb17ihvt9ks5NEcAwFoR\
8 FTq6u">
9 </form>
```

Eu formatei a resposta para ficar um pouco mais legível, mas eu não alterei o conteúdo. Como você pode ver, o Laravel forneceu o tipo de codificação (`enctype`) correto para nós. Obrigado novamente Laravel!

Nos capítulos sobre geração de URL e roteamento, você viu que URLs para rotas nomeadas e ações dos controllers podem ser geradas utilizando métodos especiais. Vamos ver como nós podemos direcionar estas rotas e ações usando índices especiais no método `Form::open()`.

Primeiro, vamos dar uma olhada em como direcionar para rotas nomeadas. Aqui está um exemplo.

```
1 <!-- app/views/form.blade.php -->
2
3 {{ Form::open(array(
4     'route' => 'my_route'
5 )) }}
6
7 {{ Form::close() }}
```

Ao invés de usar o índice `url`, nós usamos o índice `route` e fornecemos o nome da rota como o valor. Simples assim! Na verdade, direcionar para ações dos controllers é simples assim também. Vamos dar uma olhada em outro exemplo.

```
1 <!-- app/views/form.blade.php -->
2
3 {{ Form::open(array(
4     'action' => 'MyController@myAction'
5 )) }}
6
7 {{ Form::close() }}
```

Para direcionar o nosso formulário para uma ação de um controller, ao invés dos índices `url` e `route`, nós usamos o índice `action` e passamos o par controller-ação como o seu valor.

Bem, agora que nós já sabemos como abrir formulários, eu acho que já está na hora de começarmos a adicionar alguns campos. Vamos lá!

Campos de Formulário

Formulários não são muito úteis se eles não tiverem como coletar dados. Vamos olhar alguns dos campos que nós podemos gerar usando a biblioteca ‘form builder’ do Laravel.

Labels dos Campos

Espera, existe mais uma coisa que nós precisamos além dos próprios campos. Nós precisamos de labels, para que as pessoas saibam qual valor elas precisam digitar. Labels podem ser gerados usando o método `Form::label()`. Vamos dar uma olhada no exemplo.

```
1 <!-- app/views/form.blade.php -->
2
3 {{ Form::open(array('url' => 'my/route')) }}
4     {{ Form::label('first_name', 'First Name') }}
5 {{ Form::close() }}
```

O primeiro parâmetro do método `Form::label()` precisa ser igual ao atributo `name` do campo que nós estamos descrevendo. O segundo parâmetro, é o texto do label. Vamos dar uma olhada no código gerado.

```
1 <!-- app/views/form.blade.php -->
2
3 <form method="POST"
4     action="http://demo.dev/my/route"
5     accept-charset="UTF-8">
6     <input name="_token" type="hidden" value="83KCsmJF1Z2LMZfhb17ihvt9ks5NEcAwFoR\
7 FTq6u">
8     <label for="first_name">First Name</label>
9 </form>
```

Ótimo! Nós devemos usar labels sempre que nós precisarmos descrever os nossos campos. Eu vou dar o exemplo incluindo ele com outros exemplos dentro deste capítulo.

É importante ter em mente que nós podemos incluir outros atributos dentro do nosso elemento label gerado passando um terceiro parâmetro para o método `label()`. O terceiro parâmetro é um array de com pares de atributos e seus valores. Abaixo temos um exemplo.

```
1 <!-- app/views/form.blade.php -->
2
3 {{ Form::open(array('url' => 'my/route')) }}
4     {{ Form::label('first_name', 'First Name',
5         array('id' => 'first_name')) }}
6 {{ Form::close() }}
```

No exemplo acima, nós adicionamos o atributo `'id'` com o valor `'first_name'` para o elemento label. Este é o código gerado.

```
1 <!-- app/views/form.blade.php -->
2
3 <form method="POST"
4     action="http://demo.dev/my/route"
5     accept-charset="UTF-8">
6     <input name="_token" type="hidden" value="83KCsmJF1Z2LMZfhb17ihvt9ks5NEcAwFoR\
7 FTq6u">
8     <label for="first_name" id="first_name">First Name</label>
9 </form>
```

Brilhante! Bem, agora que nós sabemos como criar labels para os nossos campos, está na hora de começar a criar alguns campos.

Campos de Texto

Campos de texto podem ser usados para coletar valores de texto. Vamos dar uma olhada no método gerador para este tipo de campo.

```
1 <!-- app/views/form.blade.php -->
2
3 {{ Form::open(array('url' => 'my/route')) }}
4     {{ Form::label('first_name', 'First Name') }}
5     {{ Form::text('first_name', 'Taylor Otwell') }}
6 {{ Form::close() }}
```

O primeiro parâmetro para o método `Form::text()` é o atributo `name` do elemento. É ele que nós vamos usar para identificar o valor do campo ao receber a requisição.

O segundo parâmetro é opcional. É um valor padrão para o elemento `input`. Vamos ver o código gerador por este método.

```
1 <!-- app/views/form.blade.php -->
2
3 <form method="POST"
4     action="http://demo.dev/my/route"
5     accept-charset="UTF-8">
6     <input name="_token" type="hidden" value="83KCsmJF1Z2LMZfhb17ihvt9ks5NEcAwFoR\
7 FTq6u">
8     <label for="first_name">First Name</label>
9     <input name="first_name" type="text" value="Taylor Otwell" id="first_name">
10 </form>
```

O nosso elemento `input` foi criado, junto com o atributo `'name'` e seu valor padrão. Assim como o método `Form::label()`, o nosso método `Form::text()` também aceita um terceiro parâmetro (opcional) com um array de pares com atributo e valor, assim:


```
1 <!-- app/views/form.blade.php -->
2
3 {{ Form::open(array('url' => 'my/route')) }}
4     {{ Form::label('first_name', 'First Name') }}
5     {{ Form::text('first_name', 'Taylor Otwell',
6                 array('id' => 'first_name')) }}
7 {{ Form::close() }}
```

A verdade é que, todos os métodos de criação de campos vão aceitar um array opcional de atributos como o último parâmetro. Todos funcionam da mesma maneira, então eu não vou descrever esta funcionalidade em todas as seções. Apenas tenha isto em mente!

Campos Textarea

O campo textarea funcionam de uma maneira muito similar ao campo de texto, exceto que eles permitem textos multilinhas. Vamos ver como nós podemos gerá-los.

```
1 <!-- app/views/form.blade.php -->
2
3 {{ Form::open(array('url' => 'my/route')) }}
4     {{ Form::label('description', 'Description') }}
5     {{ Form::textarea('description', 'Best field ever!') }}
6 {{ Form::close() }}
```

O primeiro parâmetro é o atributo name do elemento, e o segundo parâmetro novamente é o seu valor padrão. Aqui está o formulário que aparece no código HTML gerado.

```
1 <!-- app/views/form.blade.php -->
2
3 <form method="POST"
4     action="http://demo.dev/my/route"
5     accept-charset="UTF-8">
6     <input name="_token" type="hidden" value="83KCsmJF1Z2LMZfhb17ihvt9ks5NEcAwFoR\
7 FTq6u">
8     <label for="description">Description</label>
9     <textarea name="description"
10         cols="50"
11         rows="10"
12         id="description">Best field ever!</textarea>
13 </form>
```

Como você pode ver no código gerado, o Laravel forneceu alguns valores padrões, como o número de colunas (cols) e linhas (rows). Nós podemos facilmente sobrescrever estes valores adicionando novos valores para eles no último parâmetro. Eu espero que você não tenha esquecido sobre o array de atributos e como usá-lo!

Campos de Senha

Algumas coisas são secretas. Na verdade, você pode me dizer os seus segredos se quiser. Eu não sou o tipo de pessoa que publica eles como piadas em um livro para dar algumas risadas.

Porém, se nós quisermos que os nossos usuários possam digitar os seus mais profundos segredos em nossa aplicação com segurança, então nós precisamos ter certeza que os caracteres digitados não apareçam na tela. O input do tipo 'password' é perfeito para isto. E assim é como você pode gerar um deles.

```
1 <!-- app/views/form.blade.php -->
2
3 {{ Form::open(array('url' => 'my/route')) }}
4     {{ Form::label('secret', 'Super Secret') }}
5     {{ Form::password('secret') }}
6 {{ Form::close() }}
```

O primeiro parâmetro para o método `Form::password()` é o atributo `name`. Como sempre, existe um parâmetro opcional com um array de atributos.

Este é o código gerado do exemplo acima.

```
1 <!-- app/views/form.blade.php -->
2
3 <form method="POST"
4     action="http://demo.dev/my/route"
5     accept-charset="UTF-8">
6     <input name="_token" type="hidden" value="83KCsmJF1Z2LMZfhb17ihvt9ks5NEcAwFoR\
7 FTq6u">
8     <label for="secret">Super Secret</label>
9     <input name="secret" type="password" value="" id="secret">
10 </form>
```

Checkboxes

Checkboxes permitem que você possa capturar valores booleanos. Vamos dar uma olhada em como nós podemos gerar checkboxes.

```
1 <!-- app/views/form.blade.php -->
2
3 {{ Form::open(array('url' => 'my/route')) }}
4     {{ Form::label('pandas_are_cute', 'Are pandas cute?') }}
5     {{ Form::checkbox('pandas_are_cute', '1', true) }}
6 {{ Form::close() }}
```

O primeiro parâmetro para o método `Form::checkbox()` é o atributo `name`, e o segundo parâmetro é o seu valor. O terceiro (e opcional), é se o checkbox está ou não checado por padrão. O padrão é ele não estar marcado.

Este é o código gerado pelo exemplo acima.

```
1 <!-- app/views/form.blade.php -->
2
3 <form method="POST"
4     action="http://demo.dev/my/route"
5     accept-charset="UTF-8">
6     <input name="_token" type="hidden" value="83KCsmJF1Z2LMZfhb17ihvt9ks5NEcAwFoR\
7 FTq6u">
8     <label for="pandas_are_cute">Are pandas cute?</label>
9     <input checked="checked"
10         name="pandas_are_cute"
11         type="checkbox"
12         value="1"
13         id="pandas_are_cute">
14 </form>
```

Botões Radio

Botões radio possuem uma utilização parecida com o método para gerar checkboxes. A diferença é que você pode usar mais de um campo do tipo radio para fornecer um método de seleção dentro de um grupo de valores. Aqui está um exemplo.

```
1 <!-- app/views/form.blade.php -->
2
3 {{ Form::open(array('url' => 'my/route')) }}
4     {{ Form::label('panda_colour', 'Pandas are?') }}
5     {{ Form::radio('panda_colour', 'red', true) }} Red
6     {{ Form::radio('panda_colour', 'black') }} Black
7     {{ Form::radio('panda_colour', 'white') }} White
8 {{ Form::close() }}
```

No exemplo acima você vai perceber que nós temos quatro campos com o mesmo nome. Apenas um deles vai poder ser selecionado. Isto vai permitir que os usuários escolham a cor do panda.

Este é o código gerado quando a view é renderizada.

```
1 <!-- app/views/form.blade.php -->
2
3 <form method="POST"
4     action="http://demo.dev/my/route"
5     accept-charset="UTF-8">
6     <input name="_token" type="hidden" value="83KCsmJF1Z2LMZfhb17ihvt9ks5NEcAwFoR\
7 FTq6u">
8     <label for="panda_colour">Pandas are?</label>
9     <input checked="checked"
10         name="panda_colour"
11         type="radio"
12         value="red"
13         id="panda_colour"> Red
14     <input name="panda_colour"
15         type="radio"
16         value="black"
17         id="panda_colour"> Black
18     <input name="panda_colour"
19         type="radio"
20         value="white"
21         id="panda_colour"> White
22 </form>
```

Select Boxes

Select boxes, ou drop-downs, são uma outra maneira de fornecer ao usuário da sua aplicação uma escolha entre valores pré-definidos. Vamos ver como eles podem ser construídos.

```
1 <!-- app/views/form.blade.php -->
2
3 {{ Form::open(array('url' => 'my/route')) }}
4     {{ Form::label('panda_colour', 'Pandas are?') }}
5     {{ Form::select('panda_colour', array(
6         'red'      => 'Red',
7         'black'    => 'Black',
8         'white'    => 'White'
9     ), 'red') }}
10 {{ Form::close() }}
```

O método `Form::select()` aceita o atributo `name` como o primeiro parâmetro, e um array de pares chave-valor como o segundo parâmetro. A chave para a opção selecionada vai ser retornada dentro dos dados da requisição. O terceiro (e opcional) parâmetro, é a chave do valor que vai aparecer selecionada por padrão. No exemplo acima, a opção 'Red' vai vir selecionada quando a página carregar.

Este é o código gerado.

```
1 <!-- app/views/form.blade.php -->
2
3 <form method="POST"
4     action="http://demo.dev/my/route"
5     accept-charset="UTF-8">
6     <input name="_token" type="hidden" value="83KCsmJF1Z2LMZfhb17ihvt9ks5NEcAwFoR\
7 FTq6u">
8     <label for="panda_colour">Pandas are?</label>
9     <select id="panda_colour" name="panda_colour">
10         <option value="red" selected="selected">Red</option>
11         <option value="black">Black</option>
12         <option value="white">White</option>
13     </select>
14 </form>
```

Se nós quisermos, nós podemos organizar as opções do nosso select em categorias. Para habilitar esta opção, primeiro nós precisamos fornecer um array multidimensional como o segundo parâmetro. O primeiro nível do array vai ser a categoria, e o segundo nível será a lista dos valores, assim como antes.

Abaixo está um exemplo.

```

1 <!-- app/views/form.blade.php -->
2
3 {{ Form::open(array('url' => 'my/route')) }}
4     {{ Form::label('bear', 'Bears are?') }}
5     {{ Form::select('bear', array(
6         'Panda' => array(
7             'red'      => 'Red',
8             'black'    => 'Black',
9             'white'    => 'White'
10        ),
11        'Character' => array(
12            'pooh'      => 'Pooh',
13            'baloo'     => 'Baloo'
14        )
15    ), 'black') }}
16 {{ Form::close() }}

```

Nós adicionamos uma nova dimensão para o nosso array de valores. Os primeiros níveis do array estão divididos entre 'Panda' e 'Character'. Estes dois grupos são representados pelo elemento 'optgroup' dentro do select.

Aqui está o código gerado.

```

1 <!-- app/views/form.blade.php -->
2
3 <form method="POST"
4     action="http://demo.dev/my/route"
5     accept-charset="UTF-8">
6     <input name="_token" type="hidden" value="83KCsmJF1Z2LMZfhb17ihvt9ks5NEcAwFoR\
7 FTq6u">
8     <label for="bear">Bears are?</label>
9     <select id="bear" name="bear">
10         <optgroup label="Panda">
11             <option value="red">Red</option>
12             <option value="black" selected="selected">Black</option>
13             <option value="white">White</option>
14         </optgroup>
15         <optgroup label="Character">
16             <option value="pooh">Pooh</option>
17             <option value="baloo">Baloo</option>
18         </optgroup>
19     </select>
20 </form>

```

Campo de Email

O campo de email possui a mesma estrutura que o método `Form::text()`. A diferença é que o método `Form::email()` cria um novo tipo de campo, do HTML5, que vai validar o valor digitado pelo cliente para garantir que seja um endereço de email válido.

Abaixo temos um outro exemplo.

```
1 <!-- app/views/form.blade.php -->
2
3 {{ Form::open(array('url' => 'my/route')) }}
4     {{ Form::label('email', 'E-Mail Address') }}
5     {{ Form::email('email', 'me@daylerees.com') }}
6 {{ Form::close() }}
```

O primeiro parâmetro para o método `Form::email()` é o atributo `name` do nosso campo. O segundo parâmetro é o seu valor padrão. Como sempre, e eu espero que você não tenha esquecido, existe o terceiro o último, opcional, array de atributos adicionais.

Aqui é como o código fonte da página será renderizado com o exemplo acima.

```
1 <!-- app/views/form.blade.php -->
2
3 <form method="POST"
4     action="http://demo.dev/my/route"
5     accept-charset="UTF-8">
6     <input name="_token" type="hidden" value="83KCsmJF1Z2LMZfhb17ihvt9ks5NEcAwFoR\
7 FTq6u">
8     <label for="email">E-Mail Address</label>
9     <input name="email"
10         type="email"
11         value="me@daylerees.com"
12         id="email">
13 </form>
```

Campo de Carregamento de Arquivos

No capítulo anterior, nós aprendemos como manipular arquivos carregados. Agora vamos ver como nós podemos gerar um campo do tipo 'file'.

```
1 <!-- app/views/form.blade.php -->
2
3 {{ Form::open(array(
4     'url' => 'my/route',
5     'files' => true
6 )) }}
7     {{ Form::label('avatar', 'Avatar') }}
8     {{ Form::file('avatar') }}
9 {{ Form::close() }}
```

O primeiro parâmetro para o método `Form::file()` é o atributo `name` do campo, porém, para o carregamento do arquivo funcionar, nós precisamos ter certeza que o formulário foi aberto com a opção `files` para incluir o tipo de codificação `enctype` correto.

Este é o código gerado.

```
1 <!-- app/views/form.blade.php -->
2
3 <form method="POST"
4     action="http://demo.dev/my/route"
5     accept-charset="UTF-8"
6     enctype="multipart/form-data">
7     <input name="_token" type="hidden" value="83KCsmJF1Z2LMZfhb17ihvt9ks5NEcAwFoR\
8 FTq6u">
9     <label for="avatar">Avatar</label>
10    <input name="avatar" type="file" id="avatar">
11 </form>
```

Hidden Fields

As vezes nossos campos não precisam capturar dados. Nós podemos usar campos do tipo `hidden` para adicionar informações extras junto com o nosso formulário. Vamos ver como nós podemos gerar campos `hidden`.

```
1 <!-- app/views/form.blade.php -->
2
3 {{ Form::open(array('url' => 'my/route')) }}
4     {{ Form::hidden('panda', 'luishi') }}
5 {{ Form::close() }}
```

O primeiro parâmetro para o método `Form::hidden()` é o atributo `name`, e eu aposto que você não imaginou isto, certo? O segundo parâmetro claro que é o seu valor (`value`).

E assim é como o código gerado se parece.


```
1 <!-- app/views/form.blade.php -->
2
3 <form method="POST"
4     action="http://demo.dev/my/route"
5     accept-charset="UTF-8">
6     <input name="_token" type="hidden" value="83KCsmJF1Z2LMZfhb17ihvt9ks5NEcAwFoR\
7 FTq6u">
8     <input name="panda" type="hidden" value="luishi">
9 </form>
```

Botões de Formulário

Nossos formulários não vão funcionar se nós não pudermos enviar eles. Vamos dar uma olhada nos botões que nós temos disponíveis para isto.

Botão de Submit

Primeiro vamos ver o botão de submit. Nada como os clássicos! E é assim que nós o usamos.

```
1 <!-- app/views/form.blade.php -->
2
3 {{ Form::open(array('url' => 'my/route')) }}
4     {{ Form::submit('Save') }}
5 {{ Form::close() }}
```

O primeiro parâmetro para o método `Form::submit()` é o valor (value), que no caso de um botão deste tipo é o texto usado para identificá-lo. Como todos os outros métodos de criação de campos, os métodos de criação de botões vão aceitar um último opcional parâmetro para adicionar outros atributos.

Este é o HTML gerado do exemplo acima.

```
1 <!-- app/views/form.blade.php -->
2
3 <form method="POST"
4     action="http://demo.dev/my/route"
5     accept-charset="UTF-8">
6     <input name="_token" type="hidden" value="83KCsmJF1Z2LMZfhb17ihvt9ks5NEcAwFoR\
7 FTq6u">
8     <input type="submit" value="Save">
9 </form>
```

Ótimo! Agora nós podemos enviar os nossos formulários. Vamos dar uma olhada em outras alternativas para os botões de formulário.

Botões Normais

Quando nós precisarmos de um botão que não vai ser usado para enviar o nosso formulário, nós podemos usar o método `Form::button()`. Abaixo temos um exemplo.

```
1 <!-- app/views/form.blade.php -->
2
3 {{ Form::open(array('url' => 'my/route')) }}
4     {{ Form::button('Smile') }}
5 {{ Form::close() }}
```

Os parâmetros para o método `Form::button()` são exatamente os mesmos que o método `Form::submit()` mencionado anteriormente. Abaixo está o código gerado pelo exemplo.

```
1 <!-- app/views/form.blade.php -->
2
3 <form method="POST"
4     action="http://demo.dev/my/route"
5     accept-charset="UTF-8">
6     <input name="_token" type="hidden" value="83KCsmJF1Z2LMZfhb17ihvt9ks5NEcAwFoR\
7 FTq6u">
8     <button type="button">Smile</button>
9 </form>
```

Botões de Imagens

Ao invés de um botão padrão, você pode usar um botão de imagem para enviar o seu formulário. Aqui temos um exemplo.

```
1 <!-- app/views/form.blade.php -->
2
3 {{ Form::open(array('url' => 'my/route')) }}
4     {{ Form::image(asset('my/image.gif', 'submit')) }}
5 {{ Form::close() }}
```

O primeiro parâmetro para o método `Form::image()` é a URL da imagem que vai ser usada pelo botão. Eu usei a função `asset()` para gerar a URL. O segundo parâmetro é o valor (`value`) do botão. Este é o código gerado.

```
1 <!-- app/views/form.blade.php -->
2
3 <form method="POST"
4     action="http://demo.dev/my/route"
5     accept-charset="UTF-8">
6     <input name="_token" type="hidden" value="83KCsmJF1Z2LMZfhb17ihvt9ks5NEcAwFoR\
7 FTq6u">
8     <input src="https://demo.dev/my/image.gif" type="image">
9 </form>
```

Botão de Reset

O botão de reset pode ser usado para limpar o conteúdo do formulário. É usado pelos usuários quando eles erram alguma coisa. Aqui está um exemplo de como podemos gerar um.

```
1 <!-- app/views/form.blade.php -->
2
3 {{ Form::open(array('url' => 'my/route')) }}
4     {{ Form::reset('Clear') }}
5 {{ Form::close() }}
```

O primeiro parâmetro para o método `Form::reset()` é o texto que você quer que apareça no botão. Aqui está o código gerado pelo exemplo acima.

```
1 <!-- app/views/form.blade.php -->
2
3 <form method="POST"
4     action="http://demo.dev/my/route"
5     accept-charset="UTF-8">
6     <input name="_token" type="hidden" value="83KCsmJF1Z2LMZfhb17ihvt9ks5NEcAwFoR\
7 FTq6u">
8     <input type="reset" value="Clear">
9 </form>
```

Form Macros

Na seção anterior nós vimos os vários métodos criadores de campos de formulário. Eles podem ser usados para você poupar um bom tempo, mas talvez você tenha um tipo customizado de campo de formulário para sua aplicação.

Para nossa sorte, Taylor pensou nisto. O Laravel vem equipado com um método que vai permitir que você defina os seus próprios métodos geradores. Vamos ver como ele funciona.

E onde eu coloco os meus ‘form macros’? Vamos criar um arquivo chamado `app/macros.php`, então nós podemos incluí-lo dentro do nosso arquivo de rotas. Aqui está o seu conteúdo.

```
1  <?php
2
3  // app/macros.php
4
5  Form::macro('fullName', function()
6  {
7      return '<p>Full name: <input type="text" name="full_name"></p>';
8  });
```

Para definir um macro nós usamos o método `Form::macro()`. O primeiro parâmetro é o nome que vai ser usado pelo método que vai criar o nosso campo de formulário. Eu sugiro nomear no formato `camelCase`.

O segundo parâmetro do método é uma Closure (função anônima). O valor retornado desta closure é o código gerador para montar o nosso campo.

Vamos criar uma simples rota para exibir o resultado do nosso novo método. Aqui está:

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return Form::fullName();
8  });
```

Nossa rota está retornando o valor do método `fullName()` da classe `Form`. Este não é um método estático normal, mas sim um pouco de ‘mágica’ que o Laravel fornece para você poder chamar os seus próprios macros. O nome da função é o mesmo nome que você deu para o seu macro. Vamos dar uma olhada no que foi retornado da rota que nós criamos.

```
1  <p>Full name: <input type="text" name="full_name"></p>
```

Como você pode ver, o resultado da nossa Closure foi retornado.

E se nós quisermos passar parâmetros para os nossos macros? Claro, sem problemas! Vamos primeiro alterar o nosso macro.

```
1 <?php
2
3 // app/macros.php
4
5 Form::macro('fullName', function($name)
6 {
7     return '<p>Full name: <input type="text" name="'. $name. '"></p>';
8 });
```

Adicionando parâmetros dentro de nossa closure, nós podemos usar os valores dentro de nosso código renderizado. No exemplo acima, nós adicionamos a possibilidade de fornecer o atributo 'name' para o nosso novo campo.

Vamos dar uma olhada em como este novo parâmetro pode ser passado.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return Form::fullName('my_field');
8 });
```

Bem, está foi fácil, não? Nós apenas passamos o valor do nosso parâmetro para o método mágico que o Laravel nos fornece. Vamos dar uma olhada no resultado da nossa rota /.

```
1 <p>Full name: <input type="text" name="my_field"></p>
```

Ótimo, o novo valor do nome foi inserido dentro do nosso campo! Outra bela maneira para evitar repetições. Aproveite!

Segurança de Formulários

É ótimo quando nós recebemos dados dos nossos formulários. É o motivo deles existirem, certo? O problema é, se nossas rotas podem ser direcionadas para os nossos formulários, então o que impede uma outra fonte de postar dados perigosos em nosso site?

O que nós precisamos é uma maneira de ter certeza que os dados que estão sendo enviados pertencem ao nosso próprio site. E isso não é um problema. O Laravel pode cuidar disto para nós. Você deve provavelmente saber que o ataque em um formulário é conhecido como 'Cross Site Request Forgery', ou CSRF abreviado.

Agora, você se lembra o campo `_token` que eu disse para você não se preocupar? Bem, agora eu quero que você comece a se preocupar com ele.

ARGH. Nós todos vamos morrer!

Certo, muito bem. Deixe-me explicar o que é este token. É como uma senha secreta que só o Laravel sabe. Foi criado usando o valor 'secret' das configurações da nossa aplicação. Se nós falarmos para o Laravel verificar este valor dentro dos dados da requisição, nós podemos ter certeza de que os dados vieram de um formulário que pertence à nossa aplicação.

Nós podemos verificar o token nós mesmos comparando o seu valor com o resultado do método `Session::token()`, mas o Laravel nos oferece uma opção melhor. Você lembra dos filtros padrões disponibilizados pelo Laravel do capítulos sobre filtros? Bem, o Laravel possui o filtro `csrf`. Vamos adicioná-lo à nossa rota.

```
1 <?php
2
3 // app/routes.php
4
5 Route::post('/handle-form', array('before' => 'csrf', function()
6 {
7     // Handle our posted form data.
8 }));
```

Ao adicionar o filtro `csrf` à nossa rota que vai manipular os dados do nosso formulário, nós podemos ter certeza que o campo `_token` está presente e correto. Se o nosso token não estiver presente, ou foi manipulado, então a exception `Illuminate\Session\TokenMismatchException` vai acontecer, e a lógica da rota não será executada.

Em um capítulo posterior nós vamos aprender como adicionar manipuladores de erros para determinados tipos de exceções, com o objetivo de manipular estas circunstâncias apropriadamente.

Se você usou `Form::open()` para criar o formulário, então o token de segurança vai ser incluído por padrão. Porém, se você quiser adicionar o token em formulários digitados manualmente, então você simplesmente precisa adicionar o método gerador de token. Abaixo temos um exemplo.

```
1 <!-- app/views/form.blade.php -->
2
3 <form action="{{ url('handle-form') }}" method="POST">
4     {{ Form::token() }}
5 </form>
```

O método `Form::token()` vai inserir um campo do tipo 'hidden' dentro de nosso formulário. E este é o código que vai ser gerado.

```
1 <!-- app/views/form.blade.php -->
2
3 <form action="http://demo.dev/handle-form" method="POST">
4     <input name="_token" type="hidden" value="83KCsmJF1Z2LMZfhb17ihvt9ks5NEcAwFoR\
5 FTq6u">
6 </form>
```

Para resumir, a não ser que você queira que aplicações e usuários externos possam postar dados nos seus formulários, você deve usar o filtro `csrf` para proteger as rotas que manipulam os dados de formulários.

Validação

Alguns meses atrás, eu estava muito estressado. Trabalhando para o lançamento do Laravel 4, o que significa criar e desenvolver um novo site, adicionando todas as alterações na documentação, junto com o típico suporte que um desenvolvedor de framework assume. Foram dias difíceis. Com todo este trabalho, eu também tive a pressão de fazer com que o sucesso do Code Bright fosse tão grande quanto o Code Happy, e assim eu poderia trazer muitos novos desenvolvedores para a comunidade.

O que eu deveria ter feito é ter dado um pequeno descanso em um dos Phill's Parks. Os Phill's Parks são parques de férias localizados em todo o mundo, e são destinos de férias para desenvolvedores. Eles foram originalmente abertos pelo seu CEO Phill Sparks, por volta dos anos dois mil, e agora são o único lugar que um desenvolvedor pode ir para descansar, e sentir-se confortável em um traje de banho.

Os Phill's Parks oferecem uma grande variedade de atividades para os desenvolvedores que desejam desacelerar depois de um ano de trabalho duro. Estes eventos incluem competitivos treinamentos de FizzBuzz, debates de espaços contra tabs e concursos de pescoço-barba.

Se eu soubesse dos Phill's Parks, eu poderia ter me acalmado, e aproveitar as semanas antes do lançamento do Laravel 4. Ah bem, você pelo menos sabe sobre elas, certo? Como um sortudo leitor do Code Bright, você tem acesso a um cupom que dá 90% de desconto na entrada dos parques. Para aproveitar este desconto, simplesmente procure por Phill Sparks no IRC, Twitter, E-mail ou pessoalmente, e fale 'CODE_BRIGHT_TROLLS_YOU'. Se ele não responder logo, apenas continue enviando mais mensagens com isto. Cedo ou tarde ele vai te enviar um cupom com código.

Mas o que isto tem a ver com Validação?

Bem, quando eu comecei a escrever este capítulo, eu tinha um belo e magnífico plano em mente. Infelizmente, eu me distrai completamente pelos divertidos jogos oferecidos nos Phill's Parks e agora não faço mais ideia. Eu vou inventar alguma coisa, eu tenho feito isso bem até agora, certo?

Phill's Parks são um lugar muito especial para desenvolvedores, e apenas desenvolvedores. O problema com desenvolvedores, é que sem carinho e atenção, pode ser difícil deles terem a determinação como outros papéis dentro da indústria de desenvolvimento web, por exemplo, designers.

Nós não queremos designers entrando nos Phill's Parks e posando de desenvolvedores. Eles vão começar a julgar a escolha da decoração, e reclamar do uso da Comic Sans nos cardápios da cafeteria. Vão arruinar tudo! Não, nós não queremos eles nos parques. O que nós precisamos é uma maneira de validar os nossos visitantes, para garantir que eles são desenvolvedores.

O que nós validamos é um conjunto de atributos. Vamos pensar sobre alguns dos atributos de um visitante de um parque. Oh, oh, eu sei. Nós podemos criar uma lista! Listas são divertidas!

- Bebida favorita.
- Escolha do navegador web.
- Tipo de barba.
- Habilidade de falar com mulheres.

Excelente, nós temos um número de atributos que podem ser usados para descrever os visitantes do parque. Usando algumas validações e regras, nós podemos ter certeza que o nosso visitante é um desenvolvedor. Para o parque, nós vamos apenas ter certeza que os atributos de um visitante do parque sejam os seguintes.

- Bebida favorita: Cerveja.
- Escolha do navegador web: Google Chrome.
- Tipo de barba: Barba no pescoço.
- Habilidade de falar com mulheres: Nenhuma.

Aviso: Estes valores são apenas para diversão, por exemplo, eu conheço muitas mulheres desenvolvedoras que são fantásticas no que fazem, e eu tenho certeza que elas tem a habilidade de falar com outras mulheres. Também, eu estou brincando um pouco com o estilo de barba do Van Dyke, e não a tradicional barba no pescoço.

De qualquer maneira, nos assegurando que os atributos de um visitante do parque sejam os que nós desejamos, nós podemos deixá-lo entrar no parque. Porém, se um safado chegar na recepção com os seguintes atributos...

- Bebida favorita: Frappuccino do Starbucks.
- Escolha do navegador web: Safari Mobile.
- Tipo de barba: Nenhuma.
- Habilidade de falar com mulheres: Sim, fala com mulheres.

Então não, nós temos um designer, ou um desenvolvedor Ruby, e nós não podemos deixá-lo passar dos portões.

Mais uma vez, isto é um pouco divertido, não me mande um email com ódio, espere até eu matar o personagem principal em um próximo capítulo... oh, eu quis dizer, apenas esqueça isto.

O que nós fizemos é implementar validação para que nós possamos ter certeza que as pessoas que entrem no parque são desenvolvedores, e não impostores.

Os Phill's parks estão seguros agora, mas e suas aplicações? Vamos dar uma olhada em como nós podemos usar validações no Laravel para ter certeza que nós possamos ter exatamente o que nós esperamos.

Validações Simples

Não confie nos seus usuários. Se existe uma coisa que eu aprendi trabalhando na indústria de TI, é que se você tem um ponto fraco na sua aplicação, seus usuários vão achá-lo. Confie em mim quando eu digo que eles vão achar. Não dê esta oportunidade para eles. Use validações para ter certeza que você receba os dados corretos.

O que eu quero dizer com dados corretos? Vamos dar uma olhada no exemplo.

Vamos dizer que nós temos um formulário HTML que é usado para coletar informações de cadastro para nossa aplicação. Na verdade, é sempre bom ter uma pequena sessão de revisão. Vamos criar um formulário com os templates Blade, e o Form Builder.

Aqui está nossa view.

```
1 <!-- app/views/form.blade.php -->
2
3 <h1>Registration form for Phill's Parks</h1>
4
5 {{ Form::open(array('url' => 'registration')) }}
6
7     {{-- Username field. -----}}
8     {{ Form::label('username', 'Username') }}
9     {{ Form::text('username') }}
10
11     {{-- Email address field. -----}}
12     {{ Form::label('email', 'Email address') }}
13     {{ Form::email('email') }}
14
15     {{-- Password field. -----}}
16     {{ Form::label('password', 'Password') }}
17     {{ Form::password('password') }}
18
19     {{-- Password confirmation field. -----}}
20     {{ Form::label('password_confirmation', 'Password confirmation') }}
21     {{ Form::password('password_confirmation') }}
22
23     {{-- Form submit button. -----}}
24     {{ Form::submit('Register') }}
25
26 {{ Form::close() }}
```

Wow, o quão linda é esta view?

Está é uma linda view.

Excelente, eu estou feliz em ver que você ainda está mentalmente condicionado de ler o Code Happy.

Você pode ver que o nosso formulário de cadastro está apontando para a rota `/registration`, e vai por padrão ter o método `POST`.

Nós temos um campo de texto para o nome do usuário, um campo de e-mail para o e-mail do usuário, e dois campos de senha para a senha e a confirmação da senha. No final do formulário nós temos um botão do tipo `'submit'` para enviar o formulário.

Você pode ter notado os comentários do blade que eu adicionei no código. Este é um hábito que eu tenho, e eu acho que fica mais fácil de achar o campo se eu tiver que alterar o formulário. Sinta-se à vontade para fazer o mesmo, mas se você não quiser, não se preocupe! Programe da sua maneira!

Certo, agora nós vamos precisar de algumas rotas para exibir e manipular este formulário. Vamos em frente e adicioná-las no nosso arquivo `routes.php`.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return View::make('form');
8  });
9
10 Route::post('/registration', function()
11 {
12     $data = Input::all();
13
14     // handle the form...
15 });
```

Nós temos uma rota `GET /` que vai ser usada para exibir o formulário, e uma rota `POST /registration` para tratar os dados quando o formulário for enviado.

Na rota para manipulação do formulário, nós pegamos todos os dados do formulário, mas nós não podemos usá-los ainda. Por quê? Ok, eu vou te mostrar.

Vá em frente e abra a rota `/` no seu navegador e você deve ver o formulário. Não preencha nada, apenas aperte no botão `'Register'`. A tela vai ficar branca quando a nossa segunda rota for chamada, e o nosso formulário postou informações em branco. Se nós usarmos a informação em branco, isto pode causar problemas sérios para nossa aplicação, ou até mesmo alguma vulnerabilidade de segurança. Estes dados são ruins.

Nós podemos evitar isto implementando validações para garantir que nossos dados são bons. Antes de nós realizarmos a validação, nós primeiros precisamos criar uma lista de regras de validação. Nós podemos fazer isto na forma de um array. Você está pronto? Ótimo, vamos dar uma olhada.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return View::make('form');
8  });
9
10 Route::post('/registration', function()
11 {
12     $data = Input::all();
13
14     $rules = array(
15         'username' => 'alpha_num'
16     );
17 });
```

Certo, vamos começar aos poucos. O conjunto de regras de validação tem a forma de um array. A índice do array representa o campo que está sendo validado. O valor do array vai consistir em uma ou mais regras que serão usadas para validar o campo. Nós vamos começar olhando uma regra de validação em um único campo.

```
1 array(
2     'username' => 'alpha_num'
3 )
```

No exemplo acima, nós desejamos validar o campo 'username' como um valor 'alpha_num' (Alfanumérico). A regra 'alpha_num' pode ser usada para garantir que um valor consiste em apenas caracteres alfanuméricos.

Vamos criar um objeto de validação para ter certeza que está nossa única regra está funcionando corretamente. Para fazer validação dentro do Laravel, primeiro nós precisamos criar uma instância do objeto `Validation`. Aqui vamos nós.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return View::make('form');
8  });
9
10 Route::post('/registration', function()
11 {
12     // Fetch all request data.
13     $data = Input::all();
14
15     // Build the validation constraint set.
16     $rules = array(
17         'username' => 'alpha_num'
18     );
19
20     // Create a new validator instance.
21     $validator = Validator::make($data, $rules);
22 });
```

Nós podemos usar o método `Validator::make()` para criar uma nova instância do nosso validador. O primeiro parâmetro para o método `make()` é um array com os dados que serão validados. Neste exemplo nós queremos validar os dados vindos da requisição, mas nós poderíamos facilmente validar um outro array de dados. O segundo parâmetro para o método é um conjunto de regras que serão usadas para validar os dados.

Agora que a nossa instância do objeto de validação foi criada, nós podemos usá-lo para verificar se os dados vindos do nosso formulário estão de acordo com o conjunto de regras que nós passamos. Vamos ver um exemplo.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return View::make('form');
8  });
9
10 Route::post('/registration', function()
```

```
11 {
12     // Fetch all request data.
13     $data = Input::all();
14
15     // Build the validation constraint set.
16     $rules = array(
17         'username' => 'alpha_num'
18     );
19
20     // Create a new validator instance.
21     $validator = Validator::make($data, $rules);
22
23     if ($validator->passes()) {
24         // Normally we would do something with the data.
25         return 'Data was saved.';
26     }
27
28     return Redirect::to('/');
29 };
```

Para testar o resultado da validação nós podemos chamar o método `passes()` da instância do validador. Este método vai retornar um valor booleano como resposta para dizer se a validação passou ou não. Um retorno `true` indica que os dados estão de acordo com o conjunto de regras. Um retorno `false` indica que os dados não estão de acordo com as regras.

Nós usamos uma declaração `if` no exemplo acima para ver se nós salvamos os dados, ou redirecionamos o usuário para o formulário novamente. Vamos testar visitando a URI `/`.

Primeiro digite o valor `'johnzoidberg'` no campo `username` e envie o formulário. Nós sabemos que o `username` `'johnzoidberg'` é uma string alfanumérica, então a validação vai passar. Nós veremos a seguinte resposta.

```
1 Data was saved.
```

Ótimo, exatamente o que nós esperávamos. Agora vamos inverter o resultado da validação. Vá em frete e acesse a URI `/` novamente. Desta vez, digite o valor `'!!!'`. Nós sabemos que um ponto de exclamação não é alfanumérico, então a validação deve falhar. Envie o formulário, e nós devemos ser redirecionados novamente para o formulário.

Eu amo o método `passes()`, é muito útil. O único problema é que nós precisamos ser um pouco otimistas para usá-lo. O mundo não é um lugar perfeito, vamos ser honestos com nós mesmos. Nós não somos Robert Dowey Jr, Vamos nos permitir ser um pouco pessimistas. Excelente, espere... satisfatório. Vamos tentar usar o método `fails()` ao invés disto.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return View::make('form');
8  });
9
10 Route::post('/registration', function()
11 {
12     // Fetch all request data.
13     $data = Input::all();
14
15     // Build the validation constraint set.
16     $rules = array(
17         'username' => 'alpha_num'
18     );
19
20     // Create a new validator instance.
21     $validator = Validator::make($data, $rules);
22
23     if ($validator->fails()) {
24         return Redirect::to('/');
25     }
26
27     // Normally we would do something with the data.
28     return 'Data was saved.';
29 });
```

O método `fails()` retorna um booleano oposto ao método `passes()`. Maravilhosamente pessimista! Sinta-se à vontade para usar o que melhor se encaixar com o seu estilo de programação. Se você quiser, você pode escrever sobre os seus sentimentos utilizando comentários PHP.

Algumas regras de validações aceitam parâmetros. Vamos trocar a regra `alpha_num` pela regra `min`. Aqui está o código fonte.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return View::make('form');
8  });
9
10 Route::post('/registration', function()
11 {
12     // Fetch all request data.
13     $data = Input::all();
14
15     // Build the validation constraint set.
16     $rules = array(
17         'username' => 'min:3'
18     );
19
20     // Create a new validator instance.
21     $validator = Validator::make($data, $rules);
22
23     if ($validator->passes()) {
24         // Normally we would do something with the data.
25         return 'Data was saved.';
26     }
27
28     return Redirect::to('/');
29 });
```

A validação `min` garante que um valor seja maior que o parâmetro passado. O parâmetro é passado depois dos dois pontos `:`. Esta regra de validação é um pouco especial, e vai se diferentemente dependendo dos dados que foram fornecidos.

Por exemplo, em uma string, o nosso parâmetro `'3'` vai garantir que o valor tenha pelo menos três caracteres. Em um valor numérico, vai garantir que o valor é matematicamente maior que o nosso parâmetro. Finalmente, em arquivos, a regra `min` vai garantir que o tamanho do arquivo carregado em bytes é maior que o parâmetro passado.

Vamos testar a nossa nova regra de validação. Primeiro visite a página `/` para digitar o valor `'Jo'` no campo `username`. Se você enviar o formulário, você será redirecionado novamente para o formulário. Isto porque o nosso valor não é grande o suficiente.

Isto foi o que ela f...

Não você não falou isto. Este é um livro sério, não vamos estragá-lo com piadas do tipo ‘ela falou’. Certo, nós já usamos duas regras de validação, mas e se nós quisermos usá-las juntas? Não é um problema. O Laravel vai nos permitir utilizar quantas regras de validação nós quisermos em nossos campos. Vamos ver como isto pode ser feito.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return View::make('form');
8  });
9
10 Route::post('/registration', function()
11 {
12     // Fetch all request data.
13     $data = Input::all();
14
15     // Build the validation constraint set.
16     $rules = array(
17         'username' => 'alpha_num|min:3'
18     );
19
20     // Create a new validator instance.
21     $validator = Validator::make($data, $rules);
22
23     if ($validator->passes()) {
24         // Normally we would do something with the data.
25         return 'Data was saved.';
26     }
27
28     return Redirect::to('/');
29 });
```

Como você pode ver, nós podemos passar quantas regras de validação nós quisermos dentro do valor do nosso array de regras, separando elas com um pipe |. Este capítulo deveria ter vindo muito mais cedo, porém, eu uso um Linux no trabalho, e um Mac em casa, e demorou uns trinta segundos para achar a tecla pipe.

De qualquer maneira, existe uma outra maneira de fornecer múltiplas regras de validação. Se você não é um avô de sessenta anos então você talvez não goste de pipes. Se você quiser, você pode usar um array para especificar regras de validação adicionais.

Aqui temos um exemplo.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return View::make('form');
8  });
9
10 Route::post('/registration', function()
11 {
12     // Fetch all request data.
13     $data = Input::all();
14
15     // Build the validation constraint set.
16     $rules = array(
17         'username' => array('alpha_num', 'min:3')
18     );
19
20     // Create a new validator instance.
21     $validator = Validator::make($data, $rules);
22
23     if ($validator->passes()) {
24         // Normally we would do something with the data.
25         return 'Data was saved.';
26     }
27
28     return Redirect::to('/');
29 });
```

Laravel é um framework flexível, e oferece muitas opções para os usuários. Use o método que você preferir para setar múltiplas regras de validação. Eu vou usar o pipe, para simular um cavalheiro britânico.

Regras de Validação

Certo pessoal, prestem atenção. Existem um monte de regras de validação, então se nós vamos passar por isto eu vou precisar de toda sua atenção. Se você está lendo isto no seu Kindle deitado na cama, guarde o livro e vá dormir. Você vai precisar estar mais acordado.

Aqui estão as regras de validação disponíveis em ordem alfabética.

accepted

Esta regra pode ser usada para garantir que uma confirmação positiva foi fornecida. Os seguintes valores vão passar: 'yes', 'no' ou um número um (1). O seu propósito é para quando você quer garantir que o usuário concordou com alguma coisa, por exemplo, o checkbox de termos e condições.

```
1 array(  
2     'field' => 'accepted'  
3 );
```

active_url

A validação 'active_url' vai garantir que o valor é uma URL válida. Para fazer isto, ela usa a função do próprio PHP `checkdnsrr()`, que não apenas verifica a estrutura da URL, mas também confirma que a URL está disponível dentro dos registros de DNS.

```
1 array(  
2     'field' => 'active_url'  
3 );
```

after

A regra de validação 'after' aceita um único parâmetro, uma string de tempo. Esta regra vai garantir que o campo contém uma data que seja maior do que o parâmetro passado. O Laravel vai usar o método PHP `strtotime()` para converter os dois valores para um timestamp para comparação.

```
1 array(  
2     'field' => 'after:12/12/13'  
3 );
```

alpha

A validação 'alpha' garante que o valor fornecido consiste somente em caracteres alfabéticos.

```
1 array(  
2     'field' => 'alpha'  
3 );
```

alpha_dash

A regra 'alpha_dash' vai garantir que o valor fornecido consiste em caracteres alfabéticos, e também traços - e underlines _. Esta regra de validação é muito útil para validar URLs como 'slugs'.

```
1 array(  
2     'field' => 'alpha_dash'  
3 );
```

alpha_num

A regra 'alpha_num' vai garantir que o valor fornecido consiste em caracteres alfabéticos e numéricos. Eu gosto de usar esta regra de validação para validar nomes de usuário.

```
1 array(  
2     'field' => 'alpha_num'  
3 );
```

before

A regra 'before' aceita um único parâmetro. O valor em questão deve ser uma data inferior ao parâmetro quando os dois valores forem convertidos para timestamps usando o método do PHP strtotime(). É exatamente o oposto da regra after.

```
1 array(  
2     'field' => 'before:12/12/13'  
3 );
```

between

A regra 'between' aceita dois parâmetros. O valor que está sendo validado deve ter o tamanho que esteja entre estes dois parâmetros. O tipo de comparação depende do tipo dos dados que estão sendo comparados. Por exemplo, em um campo do tipo numérico, a comparação vai ser matemática. Em uma string, a comparação vai ser baseada no tamanho de caracteres da string. Em um arquivo, a comparação vai ser baseada no tamanho do arquivo em bytes.

```
1 array(  
2     'field' => 'between:5,7'  
3 );
```

confirmed

A regra de validação 'confirmed' pode ser usada para garantir que um campo existe, e que exista também um outro campo com o mesmo nome, porém com `_confirmation` no final do nome. O valor sendo validado deve ter o mesmo valor do outro campo. Um uso para esta regra são as confirmações de senha, para garantir que o usuário não digitou nada errado nos dois campos. O seguinte exemplo vai garantir que o campo 'field' seja igual ao campo 'field_confirmation'.

```
1 array(  
2     'field' => 'confirm'  
3 );
```

date

A regra de validação 'date' vai garantir que o valor é uma data válida. Isto vai ser confirmado ao rodar a função do PHP `strtotime()`.

```
1 array(  
2     'field' => 'date'  
3 );
```

date_format

A regra 'date_format' vai garantir que o nosso valor é uma string de data que tenha o mesmo formato que nós passamos por parâmetro. Para aprender como construir uma string com o formato de data, dê uma olhada na documentação do PHP para o método `date()`.

```
1 array(  
2     'field' => 'date_format:d/m/y'  
3 );
```

different

A regra de validação 'different' vai garantir que o valor sendo validado é diferente ao valor do campo passado por parâmetro.

```
1 array(  
2     'field' => 'different:another_field'  
3 );
```

email

A regra de validação 'email' vai garantir que o valor que está sendo validado é um e-mail válido. Esta regra é muito útil ao montar formulários de cadastro.

```
1 array(  
2     'field' => 'email'  
3 );
```

exists

A regra de validação 'exists' vai checar se o valor está presente dentro da tabela do banco de dados identificada pelos parâmetros passados. A coluna que vai ser buscada, vai ser a mesma do campo que está sendo validado. Como alternativa, você pode fornecer um segundo parâmetro opcional para especificar o nome da coluna.

Esta regra pode ser muito útil em formulários de cadastro para verificar se um nome de usuário já está sendo utilizado por outro usuário.

```
1 array(  
2     'field' => 'exists:users,username'  
3 );
```

Parâmetros adicionais que forem passados para a regra serão adicionados como condições where. Assim:

```
1 array(  
2     'field' => 'exists:users,username,role,admin'  
3 );
```

O exemplo acima vai verificar se o valor existe dentro da coluna 'username' da tabela 'users'. A coluna 'role' precisar ter o valor de 'admin'.

image

A regra de validação 'image' vai garantir que o arquivo que está sendo carregado é uma imagem válida. Por exemplo, a extensão do arquivo precisa ser uma das seguintes: .bmp, .gif, .jpeg or .png.

```
1 array(  
2     'field' => 'image'  
3 );
```

in

A regra de validação 'in' vai garantir que o valor do campo seja um dos parâmetros passados.

```
1 array(  
2     'field' => 'in:red,brown,white'  
3 );
```

integer

Esta é fácil! A regra de validação 'integer' vai garantir que o valor do campo é um inteiro.

```
1 array(  
2     'field' => 'integer'  
3 );
```

ip

A regra de validação 'ip' vai garantir que o valor do campo contém um formato válido de endereço de IP.

```
1 array(  
2     'field' => 'ip'  
3 );
```

max

A regra de validação 'max' é exatamente o oposto da regra 'min'. Ela vai garantir que o tamanho do campo que está sendo validado seja menos ou igual do parâmetro passado. Se o campo é uma string, o parâmetro vai se referenciar ao tamanho de caracteres da string. Para valores numéricos, será feita uma comparação matemática. Para um arquivo a comparação será feita com o tamanho do arquivo em bytes.

```
1 array(  
2     'field' => 'max:3'  
3 );
```

mimes

A validação 'mimes' garante que o valor fornecido seja o nome de um mime Francês. Apenas brincando. Esta regra vai checar o mime type do arquivo carregado para garantir que tenha um dos parâmetros fornecidos.

```
1 array(  
2     'field' => 'mimes:pdf,doc,docx'  
3 );
```

min

A regra de validação 'min' é exatamente o oposto da regra 'max'. Pode ser usada para garantir que um valor seja maior ou igual do que o parâmetro passado. Se o campo for uma string, será comparado o número de caracteres da string. Para valores numéricos, a comparação será matemática. Para arquivos, será feita uma comparação com o tamanho do arquivo em bytes.

```
1 array(  
2     'field' => 'min:5'  
3 );
```

not_in

Como o nome sugere, esta regra de validação é exatamente o oposto da regra 'in'. Vai garantir que o valor do campo não esteja dentro da lista de parâmetros fornecida.

```
1 array(  
2     'field' => 'not_in:blue,green,pink'  
3 );
```

numeric

A regra 'numeric' vai verificar se o campo que está sendo validado contém um valor numérico.


```
1 array(  
2     'field' => 'numeric'  
3 );
```

regex

A regra de validação 'regex' é a mais flexível das regras disponíveis no componente de validação do Laravel. Com esta regra nós podemos passar uma expressão regular personalizada como parâmetro que o campo que está sendo validado precisa estar de acordo. Este livro não vai cobrir expressões regulares em detalhes, visto que é um tópico muito grande que merece um livro apenas sobre ele.

Você deve saber que porque o caractere pipe | pode ser usado em uma expressão regular, você deve utilizar arrays ao invés de pipes para adicionar mais de uma regra de validação ao usar a regra 'regex'.

```
1 array(  
2     'field' => 'regex:[a-z] '  
3 );
```

required

A regra de validação 'required' pode ser usada para garantir que o campo exista dentro do array de dados sendo validados.

```
1 array(  
2     'field' => 'required'  
3 );
```

required_if

A regra de validação 'required_if' garante que o campo exista, se o valor do campo definido como primeiro parâmetro da regra seja igual do segundo parâmetro passado.

```
1 array(  
2     'field' => 'required_if:username,zoidberg'  
3 );
```

required_with

A regra 'required_with' é usada para garantir que o valor está presente, apenas se um ou mais campos definidos por parâmetro também estejam presentes.

```
1 array(  
2     'field' => 'required_with:age,height'  
3 );
```

required_without

A regra 'required_without' é exatamente o oposto da regra 'required_with'. Ela pode ser usada para garantir que o campo que está sendo validado esteja presente, apenas quando os campos definidos nos parâmetros não estejam presentes.

```
1 array(  
2     'field' => 'required_without:age,height'  
3 );
```

same

A regra de validação 'same' é exatamente o oposto da regra 'different'. Ela é usada para garantir que o valor do campo sendo validado seja igual do valor do campo definido nos parâmetros.

```
1 array(  
2     'field' => 'same:age'  
3 );
```

size

A regra 'size' pode ser usada para garantir que o campo que está sendo validado tenha um certo tamanho. Se o campo for uma string, a validação será feita com o número de caracteres da string. Para valores numéricos, a comparação será matematicamente. Para um arquivo, a comparação será feita com o tamanho do arquivo em bytes.

```
1 array(  
2     'field' => 'size:8'  
3 );
```

unique

A regra ‘unique’ garante que o valor do campo sendo validado não existe na tabela do banco de dados definida pelos parâmetros. Por padrão, a regra vai usar o nome do campo como a coluna da tabela para procurar o valor, porém, você pode usar uma outra coluna, passando o seu nome como o segundo parâmetro. Esta validação é muito útil para formulários de cadastro.

```
1 array(  
2     'field' => 'unique:users,username'  
3 );
```

Você pode fornecer parâmetros opcionais com uma lista de IDs que devem ser ignorados pela regra na hora da comparação.

```
1 array(  
2     'field' => 'unique:users,username,4,3,2,1'  
3 );
```

url

A validação ‘url’ pode ser usada para garantir que o campo contenha uma URL válida. Ao contrário da regra ‘active_url’, a regra ‘url’ apenas verifica o formato da string, e não os registros DNS.

```
1 array(  
2     'field' => 'url'  
3 );
```

Bem, isso é tudo sobre as regras. Não foi tão ruim certo? Mas nós ainda não acabamos. Vamos dar uma olhada nas mensagens de erro.

Mensagens de Erro

Na primeira seção, nós aprendemos como nós podemos executar validações, e como redirecionar novamente quando acontecer uma falha. Porém, isto não oferece ao usuário uma maneira muito construtiva de feedback.

Felizmente, o Laravel coleta um número de mensagens de erro descrevendo porque a tentativa de validação falhou. Vamos dar uma olhada em como nós podemos acessar estas informações.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return View::make('form');
8  });
9
10 Route::post('/registration', function()
11 {
12     // Fetch all request data.
13     $data = Input::all();
14
15     // Build the validation constraint set.
16     $rules = array(
17         'username' => 'alpha_num'
18     );
19
20     // Create a new validator instance.
21     $validator = Validator::make($data, $rules);
22
23     if ($validator->passes()) {
24         // Normally we would do something with the data.
25         return 'Data was saved.';
26     }
27
28     // Collect the validation error messages object.
29     $errors = $validator->messages();
30
31     return Redirect::to('/');
32 });
```

No exemplo acima, você vai ver que nós podemos acessar as mensagens de erro de validação usando

o método `messages()` da instância do nosso validador. Agora, visto que nós estamos redirecionando para a nossa rota do formulário, como nós acessamos as mensagens de erro dentro dos nossos formulários?

Bem, eu acho que você pode usar o método `withErrors()` para isto.

Eu não acredito, você continua mentindo para mim!

Ah é? Então veja isto.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return View::make('form');
8  });
9
10 Route::post('/registration', function()
11 {
12     // Fetch all request data.
13     $data = Input::all();
14
15     // Build the validation constraint set.
16     $rules = array(
17         'username' => 'alpha_num'
18     );
19
20     // Create a new validator instance.
21     $validator = Validator::make($data, $rules);
22
23     if ($validator->passes()) {
24         // Normally we would do something with the data.
25         return 'Data was saved.';
26     }
27
28     return Redirect::to('/')->withErrors($validator);
29 });
```

Perceba que nós passamos a instância do validador para o método `withErrors()`. Este método coloca os erros do formulário na sessão. Antes de continuarmos, vamos adicionar mais algumas regras de validação ao nosso exemplo.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return View::make('form');
8  });
9
10 Route::post('/registration', function()
11 {
12     // Fetch all request data.
13     $data = Input::all();
14
15     // Build the validation constraint set.
16     $rules = array(
17         'username' => 'required|alpha_num|min:3|max:32',
18         'email'     => 'required|email',
19         'password'  => 'required|confirm|min:3'
20     );
21
22     // Create a new validator instance.
23     $validator = Validator::make($data, $rules);
24
25     if ($validator->passes()) {
26         // Normally we would do something with the data.
27         return 'Data was saved.';
28     }
29
30     return Redirect::to('/')->withErrors($validator);
31 });
```

Agora vamos ver como nós podemos acessar as nossas mensagens de erro na nossa view.

```
1 <!-- app/views/form.blade.php -->
2
3 <h1>Registration form for Phill's Parks</h1>
4
5 {{ Form::open(array('url' => 'registration')) }}
6
7     <ul class="errors">
8         @foreach($errors->all() as $message)
9             <li>{{ $message }}</li>
10        @endforeach
11    </ul>
12
13    {{-- Username field. -----}}
14    {{ Form::label('username', 'Username') }}
15    {{ Form::text('username') }}
16
17    {{-- Email address field. -----}}
18    {{ Form::label('email', 'Email address') }}
19    {{ Form::email('email') }}
20
21    {{-- Password field. -----}}
22    {{ Form::label('password', 'Password') }}
23    {{ Form::password('password') }}
24
25    {{-- Password confirmation field. -----}}
26    {{ Form::label('password_confirmation', 'Password confirmation') }}
27    {{ Form::password('password_confirmation') }}
28
29    {{-- Form submit button. -----}}
30    {{ Form::submit('Register') }}
31
32 {{ Form::close() }}
```

Quando nossa view é carregada, a variável `$errors` é adicionada aos dados da view. Ela está sempre lá, e sempre é uma instância do container de mensagens, então você não precisa se preocupar em verificar sua existência ou o seu conteúdo. Se nós usarmos o método `withErrors()` para salvar as mensagens de erro na sessão na requisição anterior, o Laravel vai automaticamente adicioná-las ao objeto de erros. Que conveniente!

Nós podemos acessar um array com todas as mensagens usando o método `all()` no objeto `$errors`. Na view acima, nós percorremos todo o array de mensagens escrevendo cada uma delas dentro de um elemento de lista.

Certo, chega de enrolação. Vamos fazer uma tentativa. Vá em frente e visite a URL /. Submeta o

formulário sem digitar nenhuma informação para ver o que acontece.

Nós vamos ser redirecionados de volta para o formulário. Desta vez, porém, uma lista de mensagens de erros será exibida.

- The username field is required.
- The email field is required.
- The password field is required.

Ótimo! Agora os usuários da nossa aplicação sabem quais os erros de validação que aconteceram. Porém, é mais conveniente para os nossos usuários se as mensagens de erros aparecerem perto dos campos que elas descrevem. Vamos alterar nossa view um pouco.

```
1 <!-- app/views/form.blade.php -->
2
3 <h1>Registration form for Phill's Parks</h1>
4
5 {{ Form::open(array('url' => 'registration')) }}
6
7     {{-- Username field. -----}}
8     <ul class="errors">
9         @foreach($errors->get('username') as $message)
10             <li>{{ $message }}</li>
11         @endforeach
12     </ul>
13     {{ Form::label('username', 'Username') }}
14     {{ Form::text('username') }}
15
16     {{-- Email address field. -----}}
17     <ul class="errors">
18         @foreach($errors->get('email') as $message)
19             <li>{{ $message }}</li>
20         @endforeach
21     </ul>
22     {{ Form::label('email', 'Email address') }}
23     {{ Form::email('email') }}
24
25     {{-- Password field. -----}}
26     <ul class="errors">
27         @foreach($errors->get('password') as $message)
28             <li>{{ $message }}</li>
29         @endforeach
```



```

30     </ul>
31     {{ Form::label('password', 'Password') }}
32     {{ Form::password('password') }}
33
34     {{-- Password confirmation field. -----}}
35     {{ Form::label('password_confirmation', 'Password confirmation') }}
36     {{ Form::password('password_confirmation') }}
37
38     {{-- Form submit button. -----}}
39     {{ Form::submit('Register') }}
40
41     {{ Form::close() }}

```

Nós podemos usar o método `get()` nos erros de validação para receber um array de erros de um único campo. Simplesmente passe o nome do campo como o primeiro parâmetro do método `get()`.

Vamos reenviar o formulário. Desta vez, coloque um ponto de exclamação ! dentro do campo 'username'. Vamos dar uma olhada nos erros que aparecem acima do campo 'username'.

- The username may only contain letters and numbers.
- The username must be at least 3 characters.

Assim é melhor. Bem.. é um pouco melhor. A maioria dos formulários exibem apenas um erro de validação por campo, assim não sobrecarregamos o usuário da aplicação.

Vamos dar uma olhada em como podemos fazer isto com o objeto de validação do Laravel.

```

1  <!-- app/views/form.blade.php -->
2
3  <h1>Registration form for Phill's Parks</h1>
4
5  {{ Form::open(array('url' => 'registration')) }}
6
7      {{-- Username field. -----}}
8      <span class="error">{{ $errors->first('username') }}</span>
9      {{ Form::label('username', 'Username') }}
10     {{ Form::text('username') }}
11
12     {{-- Email address field. -----}}
13     <span class="error">{{ $errors->first('email') }}</span>
14     {{ Form::label('email', 'Email address') }}
15     {{ Form::email('email') }}
16

```

```

17      {{-- Password field. -----}}
18      <span class="error">{{ $errors->first('password') }}</span>
19      {{ Form::label('password', 'Password') }}
20      {{ Form::password('password') }}
21
22      {{-- Password confirmation field. -----}}
23      {{ Form::label('password_confirmation', 'Password confirmation') }}
24      {{ Form::password('password_confirmation') }}
25
26      {{-- Form submit button. -----}}
27      {{ Form::submit('Register') }}
28
29  {{ Form::close() }}
```

Usando o método `first()` no objeto de erros de validação e passando o nome do campo como parâmetro, nós podemos pegar a primeira mensagem de erro deste campo.

Novamente, envie o formulário com apenas um ponto de exclamação ! dentro do campo username. Desta vez nós recebemos apenas uma mensagem de erro para o primeiro campo.

- The username may only contain letters and numbers.

Por padrão, as instâncias dos objetos de validação retornam um array vazio no `null` se a mensagem não existe. Isto significa que você pode usar os métodos sem precisar checar se existem mensagens. Porém, se por alguma razão você quiser checar se existem ou não mensagens de erro em um campo, você pode usar o método `has()`.

```

1  @if($errors->has('email'))
2      <p>Yey, an error!</p>
3  @endif
```

No exemplo anterior, nos métodos `all()` e `first()` você percebeu que nós colocamos as mensagens dentro de elementos HTML. Porém, se um dos nossos métodos retornar `null`, o HTML ainda vai ser exibido.

Nós podemos evitar uma tag HTML vazia aparecendo no nosso código fonte passando os elementos HTML no formato de uma string como um parâmetro adicional para os métodos `all()` e `first()`. Para exemplo, aqui temos o método `all()` adicionando um elemento de lista ('li') como o parâmetro adicional.

```
1 <!-- app/views/form.blade.php -->
2
3 <h1>Registration form for Phill's Parks</h1>
4
5 {{ Form::open(array('url' => 'registration')) }}
6
7     <ul class="errors">
8         @foreach($errors->all('<li>:message</li>') as $message)
9             {{ $message }}
10        @endforeach
11    </ul>
12
13    {{-- Username field. -----}}
14    {{ Form::label('username', 'Username') }}
15    {{ Form::text('username') }}
16
17    {{-- Email address field. -----}}
18    {{ Form::label('email', 'Email address') }}
19    {{ Form::email('email') }}
20
21    {{-- Password field. -----}}
22    {{ Form::label('password', 'Password') }}
23    {{ Form::password('password') }}
24
25    {{-- Password confirmation field. -----}}
26    {{ Form::label('password_confirmation', 'Password confirmation') }}
27    {{ Form::password('password_confirmation') }}
28
29    {{-- Form submit button. -----}}
30    {{ Form::submit('Register') }}
31
32 {{ Form::close() }}
```

A parte `:message` do novo parâmetro para o método `all()` vai ser substituída pela mensagem de erro quando o array for construído.

O método `first()` possui um parâmetro adicional similar.

```
1 <!-- app/views/form.blade.php -->
2
3 <h1>Registration form for Phill's Parks</h1>
4
5 {{ Form::open(array('url' => 'registration')) }}
6
7     {{-- Username field. -----}}
8     {{ $errors->first('username', '<span class="error">:message</span>') }}
9     {{ Form::label('username', 'Username') }}
10    {{ Form::text('username') }}
11
12    {{-- Email address field. -----}}
13    {{ $errors->first('email', '<span class="error">:message</span>') }}
14    {{ Form::label('email', 'Email address') }}
15    {{ Form::email('email') }}
16
17    {{-- Password field. -----}}
18    {{ $errors->first('password', '<span class="error">:message</span>') }}
19    {{ Form::label('password', 'Password') }}
20    {{ Form::password('password') }}
21
22    {{-- Password confirmation field. -----}}
23    {{ Form::label('password_confirmation', 'Password confirmation') }}
24    {{ Form::password('password_confirmation') }}
25
26    {{-- Form submit button. -----}}
27    {{ Form::submit('Register') }}
28
29 {{ Form::close() }}
```

Regras de Validação Personalizadas

Eu vejo, você não está feliz com as tudo que o Laravel te oferece? Você deseja ter seus próprios métodos de validação? Muito bem, hora de trazer artilharia pesada. O Laravel é flexível o suficiente para deixar você criar suas próprias regras.

Vamos dar uma olhada em como isto pode ser feito.

```
1  <?php
2
3  // app/routes.php
4
5  Validator::extend('awesome', function($field, $value, $params)
6  {
7      return $value == 'awesome';
8  });
9
10 Route::get('/', function()
11 {
12     return View::make('form');
13 });
14
15 Route::post('/registration', function()
16 {
17     // Fetch all request data.
18     $data = Input::all();
19
20     // Build the validation constraint set.
21     $rules = array(
22         'username' => 'awesome',
23     );
24
25     // Create a new validator instance.
26     $validator = Validator::make($data, $rules);
27
28     if ($validator->passes()) {
29         // Normally we would do something with the data.
30         return 'Data was saved.';
31     }
32
33     return Redirect::to('/')->withErrors($validator);
34 });
```

Não existe um lugar padrão para suas regras de validação customizadas serem definidas, então eu adicionei no arquivos routes.php para simplificar o exemplo. Você pode criar um arquivo validators.php e criar suas validações personalizadas neste arquivo se você quiser.

Nós adicionamos nossa regra de validação awesome para o campo username. Vamos dar uma olhada mais de perto em como está regra de validação foi criada.

```
1 <?php
2
3 // app/routes.php
4
5 Validator::extend('awesome', function($field, $value, $params)
6 {
7     return $value == 'awesome';
8 });
```

Para criar a validação customizada nós usamos o método `Validator::extend()`. O primeiro parâmetro para o método é o nome do método de validação. Este é o nome que você vai adicionar ao campo. O segundo parâmetro para o método é uma closure. Ela deve retornar o booleano `true` se a validação passar. Se o booleano `false` for retornado, então a tentativa de validação vai falhar.

Vou explicar os parâmetros que são passados para a closure. O primeiro parâmetro é uma string contendo o nome do campo que está sendo validado. No exemplo acima, o primeiro parâmetro vai conter a string `'username'`.

O segundo parâmetro para a closure contém o valor do campo.

O terceiro parâmetro contém um array de qualquer parâmetros que foram passados para a regra de validação. Use eles para customizar suas regras de validações como precisar.

Se você preferir definir as suas regras de validação customizadas dentro de uma classe, ao invés de uma closure, você não vai poder. Pare de querer coisas.

Não espere, eu estou apenas brincando. O Laravel pode fazer isto. Vamos criar uma classe que vai fazer isto.

```
1 <?php
2
3 // app/validators/CustomValidation.php
4
5 class CustomValidation
6 {
7     public function awesome($field, $value, $params)
8     {
9         return $value == 'awesome';
10    }
11 }
```

Como você pode ver, nossa classe de validação pode possuir quantos métodos nós quisermos, contanto que tenham o mesmo tipo de retorno que a nossa closure. Isto significa que o nosso validador pode ter quantas regras de validações nós quisermos.

Novamente, não existe um lugar perfeito para estas classes, então você vai ter que definir sua estrutura do projeto. Eu gosto de colocar minhas validações dentro de uma pasta `app/validators`, e adicioná-la ao ‘classmap’ do Composer.

Bem, já cobrimos tido.

Espere, a classe de validação não contém o nome da validação.

Ah sim, eu quase esqueci. Bem notado leitor observador! Assim, para a regra de validação ser descoberta, nós precisamos usar o método `Validator::extend()` novamente. Vamos dar uma olhada.

```
1 <?php
2
3 // app/routes.php
4
5 Validator::extend('awesome', 'CustomValidation@awesome');
```

Desta vez nós passamos uma string como o segundo parâmetro para o método `Validator::extend()`. Da mesma maneira de quando roteamos um controller, a string consiste no nome da classe da regra de validação, e a ação representando a regra separada pelo símbolo @.

Em um capítulo posterior nós vamos aprender como estender a classe de validação, como uma alternativa mais avançada de criar regras de validação personalizadas.

Mensagens de Validação Customizadas

O Laravel disponibiliza mensagens de validação customizadas para todas as mensagens de validação padrões do framework, mas e se você não gostar delas, ou quiser escrever sua aplicação em regiões que não utilizam inglês como sua língua primária.

Bem, não entre em pânico! O Laravel deixa você sobrescrever as mensagens de validação padrões. Nós apenas precisamos montar um array adicional, e passar para o método `make()` do validador.

Hey, nós já não vimos o método `Validator::make()`?!

Isto é verdade, mas novamente eu menti para você.

Por que você fica me tormentando?

Eu não tenho certeza. Eu acho que já virou um hobby. De qualquer maneira, vamos ver um exemplo com o array de mensagens customizadas.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return View::make('form');
8  });
9
10 Route::post('/registration', function()
11 {
12     // Fetch all request data.
13     $data = Input::all();
14
15     // Build the validation constraint set.
16     $rules = array(
17         'username' => 'min:3',
18     );
19
20     // Build the custom messages array.
21     $messages = array(
22         'min' => 'Yo dawg, this field aint long enough.'
23     );
24
25     // Create a new validator instance.
26     $validator = Validator::make($data, $rules, $messages);
27
28     if ($validator->passes()) {
29         // Normally we would do something with the data.
30         return 'Data was saved.';
31     }
32
33     return Redirect::to('/')->withErrors($validator);
34 });
```

Este é um grande exemplo, vamos clicar no botão de foco.

Eu não consigo achar o botão de foco no meu teclado.

Bem, se você tem um Mac, está acima da tecla 'tab'. Se parece com uma tecla ±.

Você tem certeza?

Bem, você tem outras ideias do que este botão significa?

Oh, eu vejo o seu ponto.

Certo, vamos focar no exemplo.

```
1 <?php
2
3 // Build the custom messages array.
4 $messages = array(
5     'min' => 'Yo dawg, this field aint long enough.'
6 );
7
8 // Create a new validator instance.
9 $validator = Validator::make($data, $rules, $messages);
```

O array de mensagens é um opcional terceiro parâmetro para o método `Validator::make()`. Ele contém qualquer mensagens de validações customizadas que você queira passar, e também sobrescreve as mensagens padrões. O índice do array de mensagens é o nome da regra de validação, e o valor é a mensagem para exibir caso a regra falhe.

No exemplo acima nós sobrescrevemos a mensagem para a regra `min`

Nós também podemos usar o método para as mensagens das validações de regra customizadas. Nossas regras customizadas não terão uma mensagem de erro por padrão, então normalmente é uma boa ideia adicionar uma mensagem para elas.

```
1 <?php
2
3 // Build the custom messages array.
4 $messages = array(
5     'awesome' => 'Please enter a value that is awesome enough.'
6 );
7
8 // Create a new validator instance.
9 $validator = Validator::make($data, $rules, $messages);
```

Se nós quisermos uma mensagem de erro customizada para um campo específico, nós podemos fazer isto adicionando o nome do campo, um ponto separador `.` e o tipo da validação como o índice do array.

```
1 <?php
2
3 // Build the custom messages array.
4 $messages = array(
5     'username.min' => 'Hmm, that looks a little small.'
6 );
7
8 // Create a new validator instance.
9 $validator = Validator::make($data, $rules, $messages);
```

A mensagem no exemplo acima vai apenas ser exibida quando a regra de validação `min` falhar para o campo `'username'`. Todas as outras falhas da regra `min` vão usar a mensagem padrão.

Isto é tudo que eu tenho para oferecer sobre validação por enquanto. Em um próximo capítulo nós vamos aprender como estender a classe de validação do Laravel, e como disponibilizar mensagens de erros traduzíveis, mas por enquanto, vamos para o próximo capítulo sobre banco de dados.

Banco de Dados

Agora eu tenho que fazer uma confissão aqui. Eu não sou muito fã de bancos de dados. Eu fui mordido por um quando eu era criança, e uma vez mordido, duas vezes tímido. Ok, ok, eu estou brincando novamente. É porque os bancos de dados mataram meu irmão.

Novamente, estou apenas brincando. Eu não tenho nenhum irmão. Eu realmente não sei porque eu não gosto deles, acho que é porque eu gosto de coisas visuais, bonitas, divertidas. Bancos de dados são simplesmente grandes redes de dados, não são bonitos e não são divertidos. Felizmente, nos dias atuais somos mimados por adoráveis ORMs que nos permitem acessar nossos registros das bases de dados com objetos. Em um capítulo posterior você vai ver mais sobre o ORM do Laravel, chamado Eloquent. O Eloquent é adorável, e faz com que trabalhar com bases de dados seja uma agradável experiência, até mesmo para alguém que as odeia, como eu.

Bem, vamos colocar nossos ódios de lado um pouco e falar sobre o conceito de uma base de dados. Por que nós precisamos de uma? Bem, talvez nós não precisamos?

Sua aplicação precisa guardar dados que vão estar disponíveis em futuras requisições?

Eu apenas quero exibir páginas estáticas.

Bem, então você não precisa de um banco de dados, mas e se você precisar guardar dados para múltiplas requisições, e exibí-los ou usá-los nas rotas de sua aplicação? Bem, então você precisa de um método de armazenamento, e você vai gostar de ler os próximos capítulos.

Abstração

Então, quais bases de dados nós podemos usar com o Laravel 4? Bem, vamos ver se alguma delas é de sua preferência.

- [MySQL Community / Standard / Enterprise Server](#)²¹
- [SQLite](#)²²
- [PostgreSQL](#)²³
- [SQL Server](#)²⁴

²¹<http://www.mysql.com/products/>

²²<http://www.sqlite.org/>

²³<http://www.postgresql.org/>

²⁴<http://www.microsoft.com/en-us/sqlserver/default.aspx>

Como você pode ver, nós temos uma grande quantidade de escolhas para selecionar a nossa base de dados. Para este livro, eu vou utilizar o [MySQL Community Server Edition](https://www.mysql.com/products/community/)²⁵. É uma grande plataforma aberta, e uma das mais populares usadas para desenvolvimento.

Você não precisa se preocupar se for usar outra base de dados. O Laravel possui uma camada de abstração, onde desacopla os componentes de base de dados do framework das consultas executadas, fornecendo diferentes consultas para diferentes tipos de bancos de dados. Simplificando, você não precisa se preocupar com a sintaxe do SQL, deixe o Laravel tomar conta disto para você.

Outra vantagem de utilizar a camada de abstração de bases de dados do Laravel é segurança. Na maioria das situações, a não ser que você indique o contrário, você não precisa se preocupar com valores escapados que você envie para a base de dados pelo Laravel. O Laravel vai escapar estes valores para você, com o objetivo de prevenir várias formas de ataques de injeção de SQL.

Vamos colocar na balança a flexibilidade da camada de abstração do Laravel por um momento. Então você pode trocar de base de dados quando você quiser, sem se preocupar em alterar nenhum dos códigos que você escreveu, e você não precisa se preocupar com simples medidas de segurança? Pelo que me parece, uma grande parte do trabalho foi removida dos seus projetos. Escapar valores é clichê, nós não precisamos fazer isto. Deixe o Laravel tomar conta disto para nós.

Agora que nós sabemos que nós queremos utilizar uma base de dados, vamos aprender como nós podemos configurar o Laravel para usar uma. Não se preocupe, é um processo muito simples! Primeiro vamos dar uma olhada nas opções disponíveis.

Configuração

Toda a configuração dos bancos de dados do Laravel fica em um arquivo, localizado em `app/config/database.php`. Fácil de lembrar, certo? Vamos pegar um pedaço deste arquivo, e ver algumas das opções de configuração disponíveis.

```
1  /*
2  |-----
3  | PDO Fetch Style
4  |-----
5  |
6  | By default, database results will be returned as instances of the PHP
7  | stdClass object; however, you may desire to retrieve records in an
8  | array format for simplicity. Here you can tweak the fetch style.
9  |
10 /*
11
12 'fetch' => PDO::FETCH_CLASS,
```

²⁵<http://www.mysql.com/products/community/>

Quando linhas são retornadas de uma consulta executada pelo componente de banco de dados do Laravel, elas vão por padrão estar no formato do objeto PHP `stdClass`. Isto significa que você poderá acessar as colunas em uma forma similar a seguinte.

```
1 <?php
2
3 echo $book->name;
4 echo $book->author;
```

Porém, se você quiser alterar o formato no qual as linhas são retornadas, você simplesmente precisa alterar a opção `fetch` das configurações para algo mais adequado. Vamos alterar a opção para `PDO::FETCH_ASSOC` que vai utilizar um array para guardar os nossos dados. Agora nós podemos acessar as linhas desta maneira.

```
1 <?php
2
3 echo $book['name'];
4 echo $book['author'];
```

Para uma lista completa dos PDO fetch modes, dê uma olhada na [página de documentação das constantes do PDO](#)²⁶, e procure pelas constantes que comecem com `FETCH_`.

Agora vamos dar uma olhada no array de conexões. Esta é o seu estado inicial.

```
1 <?php
2
3 'connections' => array(
4
5     'sqlite' => array(
6         'driver' => 'sqlite',
7         'database' => __DIR__ . '/../database/production.sqlite',
8         'prefix' => '',
9     ),
10
11     'mysql' => array(
12         'driver' => 'mysql',
13         'host' => 'localhost',
14         'database' => 'database',
15         'username' => 'root',
16         'password' => '',
```

²⁶<http://www.php.net/manual/en/pdo.constants.php>

```
17         'charset'    => 'utf8',
18         'collation' => 'utf8_unicode_ci',
19         'prefix'     => '',
20     ),
21
22     'pgsql' => array(
23         'driver'    => 'pgsql',
24         'host'      => 'localhost',
25         'database'  => 'database',
26         'username'  => 'root',
27         'password'  => '',
28         'charset'   => 'utf8',
29         'prefix'    => '',
30         'schema'    => 'public',
31     ),
32
33     'sqlsrv' => array(
34         'driver'    => 'sqlsrv',
35         'host'      => 'localhost',
36         'database'  => 'database',
37         'username'  => 'root',
38         'password'  => '',
39         'prefix'    => '',
40     ),
41
42 ),
```

Wow, é uma grande lista de conexões! Ela deixa muito mais fácil de começar. Agora, no array acima, você pode pensar que nós temos um índice diferente para cada tipo de base de dados. Porém, se você olhar o array de opções, verá que existe um índice `driver` que pode ser usado para especificar o tipo de base de dados padrão. Isto significa que nós podemos facilmente ter um array com diferentes conexões MySQL, assim:

```
1 <?php
2
3 'connections' => array(
4
5     'mysql' => array(
6         'driver'    => 'mysql',
7         'host'      => 'localhost',
8         'database'  => 'database',
9         'username'  => 'root',
```

```
10     'password' => '',
11     'charset'  => 'utf8',
12     'collation' => 'utf8_unicode_ci',
13     'prefix'   => '',
14 ),
15
16 'mysql_2' => array(
17     'driver'     => 'mysql',
18     'host'       => 'localhost',
19     'database'   => 'database2',
20     'username'   => 'root',
21     'password'   => '',
22     'charset'    => 'utf8',
23     'collation'  => 'utf8_unicode_ci',
24     'prefix'     => '',
25 ),
26
27 'mysql_3' => array(
28     'driver'     => 'mysql',
29     'host'       => 'localhost',
30     'database'   => 'database3',
31     'username'   => 'root',
32     'password'   => '',
33     'charset'    => 'utf8',
34     'collation'  => 'utf8_unicode_ci',
35     'prefix'     => '',
36 ),
37
38 ),
```

Tendo um número diferente de conexões, nós podemos facilmente trocar de base de dados. Desta maneira sua aplicação não precisa ter uma única base de dados. Muito flexível, e eu acho que você vai concordar comigo nesta.

O primeiro índice do array de conexões é simplesmente o nome dado para a conexão, que nós podemos fornecer quando estivermos executando uma ação específica de uma conexão no nosso código. Você pode chamar suas bases de dados como você quiser! Agora vamos para as carnes e tomates. Nós vamos dar uma olhada em cada array de conexão. Abaixo está um exemplo novamente.

```
1  'my_connection' => array(  
2      'driver'      => 'mysql',  
3      'host'        => 'localhost',  
4      'database'    => 'database',  
5      'username'    => 'root',  
6      'password'    => '',  
7      'charset'     => 'utf8',  
8      'collation'   => 'utf8_unicode_ci',  
9      'prefix'      => '',  
10 ),
```

A opção driver pode ser usada para especificar o tipo de base de dados que nós pretendemos conectar.

```
1  'driver'      => 'mysql',
```

Estes são os possíveis valores.

- mysql - MySQL
- sqlite - SQLite
- pgsql - PostgreSQL
- sqlsrv - SQL Server

Depois nós temos o índice host, que pode ser usado para especificar a localização da máquina que hospeda o servidor de banco de dados.

```
1  'host'        => 'localhost',
```

Você pode fornecer um IP (168.122.122.5) ou um domínio (database.example.com). No ambiente de desenvolvimento local você provavelmente vai utilizar 127.0.0.1 ou localhost para se referenciar para a máquina local.



Bases de Dados SQLite

Bases de dados SQLite são armazenadas em um local no disco, e não possuem um índice 'host'. Por isto, você pode simplesmente omitir este índice para uma conexão SQLite.

O próximo índice do array de conexões é a opção database.

```
1  'database'    => 'database_name',
```

É uma string usada para identificar o nome da base de dados na qual a conexão deve apontar. No caso de uma base de dados SQLite, é usada para especificar o caminho usado para armazenar o banco de dados, por exemplo:


```
1 'database' => __DIR__ . '/path/to/database.sqlite',
```

Os índices `username` e `password` são usados para passar as credenciais de acesso para sua conexão de banco de dados.

```
1 'username' => 'dayle',
2 'password' => 'emma_w4tson_is_hot',
```



SQLite Databases

Novamente, bases de dados SQLite são um pouco diferentes. Elas não possuem credenciais de acesso. Você pode omitir estes índices do bloco de conexão SQLite.

O próximo índice de configuração é o `charset`, e pode ser usado para especificar o charset padrão da conexão da base de dados.

```
1 'charset' => 'utf8',
```



SQLite Databases

Você adivinhou! O SQLite não tem suporte para esta opção. Apenas deixe este índice de fora neste caso.

Você pode setar a `collation` da base de dados utilizando o índice `collation`.

```
1 'collation' => 'utf8_unicode_ci',
```



SQLite Databases

Novamente, SQLite escolheu ser um único floco de neve. Você não precisa fornecer nenhuma `collation` para ele.

Finalmente, nós temos a opção `prefix`, que pode ser usada para adicionar prefixos comuns para suas tabelas.

```
1 'prefix' => '',
```

Preparando

Se você quiser usar os exemplos dos próximos capítulos, você vai precisar configurar sua conexão com a base de dados. Vá em frente e instale uma plataforma de banco de dados.

Agora crie um array de conexões, e preencha os parâmetros obrigatórios. Você está quase lá. Nós simplesmente precisamos dizer ao Laravel qual conexão ele deve utilizar por padrão. Dê uma outra olhada no arquivo `app/config/database.php`.

```
1  /*
2  |-----
3  | Default Database Connection Name
4  |-----
5  |
6  | Here you may specify which of the database connections below you wish
7  | to use as your default connection for all database work. Of course
8  | you may use many connections at once using the Database library.
9  |
10 /*
11
12 'default' => 'mysql',
```

Dentro do índice do array `default`, nós devemos colocar o identificador da nova conexão que nós criamos. Desta maneira, nós não precisamos especificar a conexão todas as vezes que nós usamos o banco de dados.

Bem, eu sei que você está empolgado com bases de dados. Seu estranho! Não vamos mais perder mais tempo. Vire a página, e vamos dar uma olhada no Schema Builder.

Schema Builder

Certo, então você decidiu que quer guardar dados em um banco de dados. E o banco de dados não é apenas um dispositivo de armazenamento de chave-valor. Dentro dele, nossa estrutura pode consistir em diferentes tipos e pode ter relacionamentos. Relacionamentos lindos e sexy.

Para armazenar nossos dados estruturadamente, nós primeiro precisamos definir a estrutura. Este não é um livro sobre SQL, então eu espero que você entenda o conceito de uma tabela de banco de dados e suas colunas. Neste capítulo nós vamos dar uma olhada na classe `Schema`, que nós podemos usar para definir a estrutura de nossas tabelas. Nós não vamos armazenar nenhum dado neste capítulo, então tenha em mente que você verá a lógica para montar a estrutura, e não o conteúdo.

No próximo capítulo você vai aprender um local ideal para construir a estrutura da sua base de dados, mas eu gosto de descrever cada funcionalidade separadamente. Por enquanto, nós vamos escrever o nosso código para criar a estrutura das tabelas dentro de Closures, nas rotas. Bem, não vamos mais perder tempo, e dar uma olhada no Schema Builder.

Criando Tabelas

Para criar uma tabela nós precisamos utilizar o método `create()` da classe `Schema`. Abaixo temos um exemplo.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      Schema::create('users', function($table)
8      {
9          // Let's not get carried away.
10     });
11 });
```

O método `Schema::create()` aceita dois parâmetros. O primeiro é o nome da tabela que nós desejamos criar. Neste caso nós estamos criando uma tabela chamada 'users'. Se a tabela que nós estamos criando vai ser usada para representar algum tipo de objeto, nós devemos nomear a tabela em letras minúsculas e no plural do objeto. As colunas da base de dados normalmente são nomeadas

usando o padrão snake-casing, onde espaços são substituídos por underlines (_) e todos os caracteres estão em letras minúsculas, sem acentos.

O segundo parâmetro para o método é uma Closure, com um único parâmetro. No exemplo acima, eu chamei o parâmetro de `$table`, mas você pode chamá-lo como você quiser! O parâmetro `$table` vai ser usado para criar a estrutura de uma tabela.

Vamos adicionar uma chave primária auto incremental para nossa tabela, desta forma as linhas podem ser identificadas por um único índice.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      Schema::create('users', function($table)
8      {
9          $table->increments('id');
10     });
11 });
```

O método `increments()` está disponível no nosso objeto `$table` para criar uma nova coluna incremental. Uma coluna auto incremental vai automaticamente ser populada com um valor inteiro que incremente cada linha que for adicionada. Vai começar no número um. Esta coluna também vai ser a chave primária da tabela. O primeiro parâmetro para o método 'increments' é o nome da coluna que vai ser criada. Simples, certo?

Vamos continuar e adicionar mais colunas para nossa tabela.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      Schema::create('users', function($table)
8      {
9          $table->increments('id');
10         $table->string('username', 32);
11         $table->string('email', 320);
12         $table->string('password', 60);
13         $table->timestamps();
14     });
15 });
```

Ótimo! Agora nós temos um modelo para criar a estrutura da nossa tabela de usuários. Não se preocupe com cada uma das colunas agora, nós vamos passar uma por uma em detalhes na próxima seção. Primeiro vamos criar esta tabela visitando a URI / para executar nossa Closure.

Agora vamos dar uma olhada na base de dados que foi criada para nós. Eu não sei qual banco de dados você escolheu, mas eu vou utilizar MySQL para o livro, então eu vou visualizar a base de dados utilizando o MySQL na linha de comando. Sinta-se à vontade para utilizar o software que você se sentir mais confortável.

```

1  mysql> use myapp;
2  Database changed
3
4  mysql> describe users;
5  +-----+-----+-----+-----+
6  | Field      | Type                | Key | Extra          |
7  +-----+-----+-----+-----+
8  | id         | int(10) unsigned    | PRI | auto_increment |
9  | username   | varchar(32)         |     |                |
10 | email      | varchar(320)        |     |                |
11 | password   | varchar(60)         |     |                |
12 | created_at | timestamp           |     |                |
13 | updated_at | timestamp           |     |                |
14 +-----+-----+-----+-----+
15 6 rows in set (0.00 sec)

```

Bem, eu simplifiquei a descrição da tabela para caber no tamanho do livro, mas eu espero que você tenha entendido. Nossa estrutura da tabela ‘users’ foi criada usando o modelo que nós criamos com o nosso objeto \$table.

Você deve estar se perguntando quais métodos e colunas estão disponíveis para o objeto \$table? Vamos dar uma olhada!

Tipos de Coluna

Nós vamos examinar os métodos que estão disponíveis no objeto \$table. Eu não vou manter a Closure nestes exemplos para simplificar as coisas, então use sua imaginação. Vamos começar.

increments

O método increments vai adicionar uma chave primária, inteira e auto incremental para a tabela. Este é um método muito útil para construir a estrutura para os modelos do Eloquent ORM, o qual nós vamos aprender em um próximo capítulo.

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->increments('id');
6  });

```

O primeiro e único parâmetro para o método `increments()` é o nome da coluna que vai ser criada. Este é o resultado da estrutura da tabela.

```

1  +-----+-----+-----+-----+
2  | Field | Type           | Key | Extra           |
3  +-----+-----+-----+-----+
4  | id    | int(10) unsigned | PRI | auto_increment |
5  +-----+-----+-----+-----+

```

bigIncrements

Ah, então o método ‘increments’ não é grande o suficiente para você? Então o método `bigIncrements()` vai criar um inteiro grande, ao invés de um inteiro regular.

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->bigIncrements('id');
6  });

```

Assim como o método `increments()`, o método `bigIncrements()` vai aceitar um único parâmetro, que deve ser uma string com o nome da coluna.

```

1  +-----+-----+-----+-----+
2  | Field | Type           | Key | Extra           |
3  +-----+-----+-----+-----+
4  | id    | bigint(20) unsigned | PRI | auto_increment |
5  +-----+-----+-----+-----+

```

string

O método `string()` pode ser usado para criar colunas do tipo `varchar`, que são úteis para armazenar pequenas strings.

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->string('nickname', 128);
6  });

```

O primeiro parâmetro para o método `string()` é o nome da coluna que vai ser criada, porém, existe um opcional segundo parâmetro para definir o tamanho de caracteres da string. O valor padrão é 255.

```

1  +-----+-----+
2  | Field   | Type         |
3  +-----+-----+
4  | nickname | varchar(255) |
5  +-----+-----+

```

text

O método `text()` pode ser usado para armazenar grandes valores de texto que não vão caber em uma coluna do tipo `varchar`. Por exemplo, este tipo de coluna pode ser usado para armazenar o conteúdo do post de um blog.

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->text('body');
6  });

```

O método `text()` aceita um único parâmetro. O nome da coluna que vai ser criada.

```

1  +-----+-----+
2  | Field | Type |
3  +-----+-----+
4  | body  | text |
5  +-----+-----+

```

integer

A coluna do tipo inteiro pode ser usada para armazenar valores inteiros. Surpreso? Bem, eu não consigo pensar em alguma maneira de deixar isto mais interessante! Eu acho que eu posso mencionar como os valores inteiros são úteis ao referenciar valores inteiros de outra tabela. Nós podemos usar este método para criar relacionamentos entre tabelas.

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->integer('shoe_size');
6  });

```

O primeiro parâmetro para o método `integer()` é o nome da coluna. O segundo parâmetro é um valor booleano que pode ser usado para definir se a coluna deve ser auto incremental. O terceiro parâmetro é usado para definir se o valor deve ser ‘unsigned’. Um valor do tipo ‘signed’ pode ser positivo ou negativo, porém, se você definir um inteiro como ‘unsigned’, ele pode apenas ser positivo. Inteiros do tipo ‘signed’ podem conter um alcance de valores entre $-2,147,483,648$ até $2,147,483,647$, e um valor do tipo ‘unsigned’ pode ter um valor de 0 até $4,294,967,295$.*

```

1  +-----+-----+
2  | Field      | Type      |
3  +-----+-----+
4  | shoe_size  | int(11)   |
5  +-----+-----+

```

bigInteger

Valores do tipo ‘bigInteger’ funcionam exatamente como os inteiros normais, apenas tem um alcance maior. Um inteiro grande do tipo ‘signed’ possui um alcance de $-9,223,372,036,854,775,808$ até $9,223,372,036,854,775,807$, e um valor do tipo ‘unsigned’ possui um alcance de 0 até $18,446,744,073,709,551,615$. Um inteiro deste tamanho é utilizado para armazenar o tamanho da minha cintura em polegadas.


```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->bigInteger('waist_size');
6  });

```

Os parâmetros para todos os métodos inteiros são exatamente os mesmos do método `integer()`, então eu não vou perder mais tempo repetindo! Se você já esqueceu, talvez você queira dar uma olhada no bloco ‘integer’ novamente?

```

1  +-----+-----+
2  | Field      | Type      |
3  +-----+-----+
4  | waist_size | bigint(20) |
5  +-----+-----+

```

mediumInteger

Esta coluna é um outro tipo de inteiro. Vamos ver se passamos por estes tipos de colunas um pouco mais rápido, não? Eu apenas vou especificar o valor do alcance da coluna de agora em diante. A coluna ‘signed’ alcança de -8388608 to 8388607. A coluna ‘unsigned’ alcança de 0 até 16777215.

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->mediumInteger('size');
6  });

```

Os parâmetros deste método são exatamente os mesmos do método `integer()`.

```

1  +-----+-----+
2  | Field | Type      |
3  +-----+-----+
4  | size  | mediumint(9) |
5  +-----+-----+

```

tinyInteger

Este é um outro tipo de coluna inteira. O tipo ‘signed’ é de -128 até 127, e o ‘unsigned’ de 0 até 255.

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->tinyInteger('size');
6  });

```

Os parâmetros do método são idênticos ao do método `integer()`.

```

1  +-----+-----+
2  | Field | Type      |
3  +-----+-----+
4  | size  | tinyint(1) |
5  +-----+-----+

```

smallInteger

Este é um outro tipo de coluna inteira. O tipo ‘signed’ vai de -32768 to 32767 enquanto o ‘unsigned’ vai de 0 até 65535.

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->smallInteger('size');
6  });

```

Os parâmetros do método são idênticos ao do método `integer()`.

```

1  +-----+-----+
2  | Field | Type      |
3  +-----+-----+
4  | size  | smallint(6) |
5  +-----+-----+

```

float

Colunas do tipo float são usadas para armazenar números de ponto flutuante. E assim é como elas podem ser definidas.

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->float('size');
6  });

```

O primeiro parâmetro é o nome usado para identificar a coluna. Os opcionais segundo e terceiro parâmetros podem ser usados para especificar o tamanho do valor, e o tamanho de casas decimais do valor. O padrão para estes valores são 8 e 2, respectivamente.

```

1  +-----+-----+
2  | Field | Type       |
3  +-----+-----+
4  | size  | float(8,2) |
5  +-----+-----+

```

decimal

O método `decimal()` é usado para armazenar... espere... valores decimais! É muito parecido com o método `float()`.

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->decimal('size');
6  });

```

O método aceita o nome da coluna como o primeiro parâmetro, e dois opcionais parâmetros para representar o tamanho do número e o número de casas decimais da coluna. Os padrões para estes parâmetros são, novamente, 8 e 2.

```

1  +-----+-----+
2  | Field | Type       |
3  +-----+-----+
4  | size  | decimal(8,2) |
5  +-----+-----+

```

boolean

Nem todos os valores consistem em grandes alcances de dígitos e caracteres. Alguns apenas possuem dois estados, true ou false, 1 ou 0. Colunas do tipo booleano podem ser usadas para representar estes valores.

```
1 <?php
2
3 Schema::create('example', function($table)
4 {
5     $table->boolean('hot');
6 });
```

O único parâmetro para o método booleano é o nome da coluna.

```
1 +-----+-----+
2 | Field | Type      |
3 +-----+-----+
4 | hot   | tinyint(1) |
5 +-----+-----+
```

O `tinyint` do exemplo acima não é um erro de digitação. Inteiros do tipo `tinyint` são usados para representar valores booleanos, como 1 e 0. Também vale mencionar que eu quase queimei minha cozinha agora enquanto eu estava distraído com esta seção. Eu acho que deve ser interessante saber sobre a vida perigosa de um escritor! Não? Certo, vamos continuar com as colunas.

enum

O tipo 'enum' vai armazenar strings que estão contidas dentro de uma lista de valores aceitos. Aqui temos um exemplo.

```
1 <?php
2
3 Schema::create('example', function($table)
4 {
5     $allow = array('Walt', 'Jesse', 'Saul');
6     $table->enum('who', $allow);
7 });
```

O primeiro e opcional parâmetro é o nome da coluna que vai ser criada. O segundo parâmetro é um array de valores que são permitidos para este tipo.

```

1  +-----+-----+-----+
2  | Field | Type                | Null |
3  +-----+-----+-----+
4  | who   | enum('Walt','Jesse','Saul') | NO    |
5  +-----+-----+-----+

```

date

Como o nome sugere, o método `date()` pode ser usado para criar colunas que armazenam datas.

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->date('when');
6  });

```

O primeiro e único parâmetro é usado para especificar o nome da coluna que vai ser criada.

```

1  +-----+-----+
2  | Field | Type |
3  +-----+-----+
4  | when  | date |
5  +-----+-----+

```

dateTime

O método `dateTime()` não vai apenas armazenar uma data, mas também o tempo. Sem brincadeira, ela realmente vai. Eu sei, eu sei, um monte destes métodos são similares, mas confie em mim, vai ser um grande capítulo de referência!

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->dateTime('when');
6  });

```

Novamente, o nome da coluna que vai ser criada é o único parâmetro.

```

1  +-----+-----+
2  | Field | Type   |
3  +-----+-----+
4  | when  | datetime |
5  +-----+-----+

```

time

Não quer incluir a data com os seus tempos? Ok! Apenas use o método `time()` então.

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->time('when');
6  });

```

Novamente, o primeiro e único parâmetro para o método `time()` é o nome da coluna que vai ser criada.

```

1  +-----+-----+
2  | Field | Type   |
3  +-----+-----+
4  | when  | time   |
5  +-----+-----+

```

timestamp

O método `timestamp()` pode ser usado para armazenar uma data e seu tempo no formato `TIMESTAMP`. Surpreso? Não? Bem... vamos dar uma olhada em como ele funciona.

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->timestamp('when');
6  });

```

O primeiro e único parâmetro é o nome da coluna que vai ser criada.

```

1  +-----+-----+-----+
2  | Field | Type      | Default          |
3  +-----+-----+-----+
4  | when  | timestamp | 0000-00-00 00:00:00 |
5  +-----+-----+-----+

```

binary

O método `binary()` é usado para criar colunas que vão armazenar dados binários. Estes tipos de colunas são muito úteis para armazenar arquivos binários como imagens.

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->binary('image');
6  });

```

O único parâmetro para este método é o nome da coluna que vai ser criada.

```

1  +-----+-----+
2  | Field | Type |
3  +-----+-----+
4  | image | blob |
5  +-----+-----+

```

Tipos de Colunas Especiais

O Laravel inclui alguns tipos de colunas especiais que possuem múltiplos usos. Vamos dar uma olhada neles. Primeiramente, o método `timestamps()`.

O método `timestamps()` pode ser usado para adicionar duas colunas do tipo 'TIMESTAMP' na tabela. As colunas `created_at` e `updated_at` podem ser usadas para indicar quando uma linha foi criada e atualizada. Em um próximo capítulo, nós vamos ver como o Eloquent ORM pode automaticamente atualizar estas colunas quando uma instância é criada ou atualizada. Vamos dar uma olhada em como podemos usar o método `timestamps()`.

```
1 <?php
2
3 Schema::create('example', function($table)
4 {
5     $table->timestamps();
6 });
```

O método `timestamps()` não aceita nenhum parâmetro. E esta é a estrutura da tabela que é criada.

```
1 +-----+-----+-----+
2 | Field      | Type      | Default          |
3 +-----+-----+-----+
4 | created_at | timestamp | 0000-00-00 00:00:00 |
5 | updated_at | timestamp | 0000-00-00 00:00:00 |
6 +-----+-----+-----+
```

Agora nós temos o método `softDeletes()`. Ocasionalmente você pode querer marcar uma linha da tabela como excluída, sem realmente excluir os dados da tabela. Isto é útil se você quiser restaurar os dados no futuro. Com o método `softDeletes()`, você pode criar uma coluna para indicar quando um registro foi excluído. A coluna que é criada se chama `deleted_at`, e vai ser do tipo 'TIMESTAMP'. Novamente, o Eloquent vai poder atualizar esta coluna, sem deletar o registro quando você utilizar o método 'delete'. Abaixo um exemplo de como nós podemos adicionar a coluna `deleted_at` em nossas tabelas.


```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->softDeletes();
6  });

```

O método `softDeletes()` não aceita nenhum parâmetro. A seguir temos um exemplo do resultado da tabela.

```

1  +-----+-----+-----+
2  | Field      | Type      | Null |
3  +-----+-----+-----+
4  | deleted_at | timestamp | YES  |
5  +-----+-----+-----+

```

Modificadores de Colunas

Os modificadores de coluna podem ser usados para adicionar novas colunas ou novas propriedades para as colunas que nós já criamos com o método `create()`. Por exemplo, antes nós utilizamos o método `increments` para um índice que era uma chave primária e auto incremental. Este é um atalho conveniente, mas vamos dar uma olhada em como nós podemos transformar uma outra coluna em uma chave primária utilizando os modificadores.

Primeiro, vamos criar uma nova coluna e declarar que ela deve apenas conter valores únicos.

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->string('username')->unique();
6  });

```

Ao chamar o método `unique()` após o método de criação da coluna, nós informamos para a base de dados que valores duplicados não serão permitidos nesta coluna. Nossa chave primária deve ser usada para identificar registros individualmente, então nós não queremos ter valores duplicados.

Vamos fazer a coluna ‘username’ a chave primária da tabela.

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->string('username')->unique();
6      $table->primary('username');
7  });

```

Nós podemos marcar qualquer coluna como chave primária usando o método `primary()`. O único parâmetro para este método é uma string representando o nome da coluna que queremos marcar como a chave primária. Vamos ver como ficou a tabela que nós acabamos de criar.

```

1  +-----+-----+-----+-----+-----+
2  | Field   | Type           | Null | Key | Default | Extra |
3  +-----+-----+-----+-----+-----+
4  | username | varchar(255)   | NO   | PRI | NULL    |       |
5  +-----+-----+-----+-----+-----+

```

Ótimo! Agora nós temos uma chave primária.

E também temos um pequeno truque. Os dois métodos, `primary()` e `unique()` podem ser chamados separadamente, ou chamados após um valor existente. Isto significa que o exemplo acima pode ser substituído por isto:

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->string('username')->unique()->primary();
6  });

```

O exemplo acima mostra como os modificadores de coluna podem ser chamados em uma definição de coluna já existente. Também, os modificadores de colunas podem ser usados separadamente, com a única diferença que precisamos passar o nome da coluna como parâmetro neste caso.

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->string('username');
6      $table->unique('username');
7      $table->primary('username');
8  });

```

Se você não está satisfeito com uma única chave primária para sua tabela, você pode usar múltiplas chaves, passando um array com os nomes das colunas para o método `primary()`. Vamos dar uma olhada.

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->integer('id');
6      $table->string('username');
7      $table->string('email');
8      $keys = array('id', 'username', 'email');
9      $table->primary($keys);
10 });

```

Agora nossas três colunas serão nossa chave primária composta, onde qualquer combinação entre os três valores precisa ser único. Vamos dar uma olhada na estrutura da tabela.

```

1  +-----+-----+-----+-----+-----+
2  | Field   | Type           | Null | Key | Default | Extra |
3  +-----+-----+-----+-----+-----+
4  | id      | int(11)        | NO   | PRI | NULL    |       |
5  | username | varchar(255)   | NO   | PRI | NULL    |       |
6  | email    | varchar(255)   | NO   | PRI | NULL    |       |
7  +-----+-----+-----+-----+-----+

```

Nós podemos aumentar a performance de nossas consultas marcando as colunas que nós costumamos buscar informações como índices. E nós podemos utilizar o método `index()` para isto. Ele pode ser chamado após a declaração de uma coluna, assim:

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->integer('age')->index();
6  });

```

Ou separadamente, assim:

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->integer('age');
6      $table->index('age');
7  });

```

De qualquer maneira, o resultado será o mesmo. A coluna será marcada como um índice.

```

1  +-----+-----+-----+-----+-----+-----+
2  | Field | Type      | Null | Key | Default | Extra |
3  +-----+-----+-----+-----+-----+-----+
4  | age   | int(11)   | NO   | MUL | NULL    |       |
5  +-----+-----+-----+-----+-----+-----+

```

Nós também podemos passar um array de colunas para o método `index()` para marcar múltiplas colunas como índices. Abaixo temos um exemplo.

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->integer('age');
6      $table->integer('weight');
7      $table->index(array('age', 'weight'));
8  });

```

Este é o resultado da estrutura da tabela.

```

1  +-----+-----+-----+-----+-----+-----+
2  | Field | Type      | Null | Key | Default | Extra |
3  +-----+-----+-----+-----+-----+-----+
4  | age   | int(11)   | NO   | MUL | NULL    |      |
5  | weight | int(11)   | NO   |     | NULL    |      |
6  +-----+-----+-----+-----+-----+-----+

```

As vezes nós queremos setar uma coluna para um estado onde ela pode ou não ter um valor nulo. Nós podemos setar a coluna para aceitar um valor nulo usando o método `nullable()`. Este método pode ser chamado após a declaração da coluna, assim:

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->string('name')->nullable();
6  });

```

E esta é a estrutura da tabela.

```

1  +-----+-----+-----+
2  | Field | Type           | Null |
3  +-----+-----+-----+
4  | name  | varchar(255)   | YES  |
5  +-----+-----+-----+

```

Como você pode ver, a coluna agora aceita um valor nulo. Se nós **não** quisermos que a coluna aceite um valor nulo, nós podemos passar um valor booleano `false` como o primeiro parâmetro para o método `nullable()`, assim:

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->string('name')->nullable(false);
6  });

```

Agora vamos dar uma outra olhada na estrutura da tabela.

```

1  +-----+-----+-----+
2  | Field | Type           | Null |
3  +-----+-----+-----+
4  | name  | varchar(255)    | NO   |
5  +-----+-----+-----+

```

Como você pode ver, a coluna ‘name’ não aceita mais valores nulos.

Se nós quisermos que nossas colunas tenham um valor padrão quando um novo registro for criado, nós podemos definir este valor chamando o método `default()` após a definição da coluna. Abaixo temos um exemplo.

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->string('name')->default('John Doe');
6  });

```

O primeiro e único parâmetro para o método `default()` é o valor padrão que nós desejamos que nossa coluna tenha. Vamos dar uma olhada na estrutura da tabela criada.

```

1  +-----+-----+-----+-----+-----+
2  | Field | Type           | Null | Key | Default |
3  +-----+-----+-----+-----+-----+
4  | name  | varchar(255)    | NO   |     | John Doe |
5  +-----+-----+-----+-----+-----+

```

Se nós não passarmos um valor para a coluna ‘name’ ao criar um novo registro, então o seu valor será ‘John Doe’.

Nós temos um último modificador de coluna para ver. Este não é muito necessário, mas é um belo atalho. Você se lembra de quando nós criamos colunas do tipo inteiro na seção anterior? Nós usamos um parâmetro booleano para especificar quando uma coluna deve ser do tipo ‘signed’ ou do tipo ‘unsigned’. Bem, nós podemos usar o método `unsigned()` em uma coluna inteira para especificar que ela não pode conter valores negativos. Abaixo temos um exemplo.

```
1 <?php
2
3 Schema::create('example', function($table)
4 {
5     $table->integer('age')->unsigned();
6 });
```

Este é a estrutura da tabela criada após usar o método `unsigned()`.

```
1 +-----+-----+
2 | Field | Type           |
3 +-----+-----+
4 | age   | int(10) unsigned |
5 +-----+-----+
```

A escolha de usar um valor booleano ou o método `unsigned()`, é totalmente sua.

Atualizando Tabelas

Uma vez que uma tabela é criada, não existe maneira de mudá-la.

Você tem certeza? Porque o cabeçalho diz...

Eu tenho certeza, não existe maneira mesmo.

Hmm, mas o cabeçalho não diz atualizando tabelas?

Você simplesmente não vai deixar isto pra lá, não é mesmo? Eu ia tirar uma soneca, mas você me convenceu. Você precisa saber como atualizar tabelas, então vamos começar.

Primeiro de tudo, nós podemos mudar o nome da tabela que nós já criamos anteriormente utilizando o método `Schema::rename()`. Vamos dar uma olhada no exemplo abaixo.

```
1  <?php
2
3  // Create the users table.
4  Schema::create('users', function($table)
5  {
6      $table->increments('id');
7  });
8
9  // Rename the users table to idiots.
10 Schema::rename('users', 'idiots');
```

O primeiro parâmetro do método `rename()` é o nome da tabela que nós desejamos renomear. O segundo parâmetro é o novo nome da tabela.

Se nós quisermos alterar as colunas de uma tabela existente, nós precisamos usar o método `Schema::table()`. Vamos dar uma olhada mais de perto.

```
1  <?php
2
3  Schema::table('example', function($table)
4  {
5      // Modify the $table...
6  });
```

O método `table()` é quase igual ao método `create()` que nós usamos anteriormente para criar a tabela. A única diferença é que ele utiliza uma tabela já existente que nós especificamos como o primeiro parâmetro do método. Novamente, o segundo parâmetro contém uma Closure, que recebe de parâmetro uma instância do Table Builder.

Nós podemos utilizar qualquer um dos métodos de criação que nós vimos nas seções anteriores para adicionar novas colunas em uma tabela existente. Abaixo temos um exemplo.

```
1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->increments('id');
6  });
7
8  Schema::table('example', function($table)
9  {
10     $table->string('name');
11 });
```


No exemplo acima, nós usamos o método `Schema::create()` para criar a tabela ‘example’ com uma chave primária. Depois nós usamos o método `Schema::table()` para adicionar uma coluna do tipo ‘string’ para a tabela existente.

E este é o resultado gerado:

```

1  +-----+-----+-----+-----+
2  | Field | Type           | Key | Extra           |
3  +-----+-----+-----+-----+
4  | id    | int(10) unsigned | PRI | auto_increment |
5  | name  | varchar(255)    |     |                 |
6  +-----+-----+-----+-----+

```

Você pode usar qualquer um dos métodos que nós já vimos nas seções anteriores para adicionar novas colunas na tabela ou modificar as existentes. Eu não vou cobrir cada um dos métodos de criação novamente, pois o modo de utilização é exatamente igual. Se você quiser dar uma rápida revisada, dê uma olhada novamente na seção ‘Tipos de Coluna’.

Se nós decidirmos que nós não queremos mais uma coluna em nossa tabela, nós podemos utilizar o método `dropColumn()` para removê-la. Vamos dar uma olhada neste método em ação.

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->increments('id');
6      $table->string('name');
7  });
8
9  Schema::table('example', function($table)
10 {
11     $table->dropColumn('name');
12 });

```

No exemplo acima, nós criamos a tabela ‘example’ com duas colunas. Depois nós usamos o método `dropColumn()` para remover a coluna ‘name’ da tabela. O método `dropColumn()` aceita um único parâmetro, que é o nome da coluna que nós desejamos remover.

Abaixo temos um exemplo de como nossa tabela ‘example’ vai se parecer depois do código acima ser executado.

```

1  +-----+-----+-----+-----+-----+
2  | Field | Type           | Null | Key | Extra           |
3  +-----+-----+-----+-----+-----+
4  | id    | int(10) unsigned | NO   | PRI | auto_increment |
5  +-----+-----+-----+-----+-----+

```

Como você pode ver, a coluna 'name' foi removida com sucesso.

Se nós desejarmos remover mais de uma coluna ao mesmo tempo, nós podemos passar um array com os nomes das colunas como o primeiro parâmetro do método `dropColumn()`...

```

1  <?php
2
3  Schema::table('example', function($table)
4  {
5      $table->dropColumn(array('name', 'age'));
6  });

```

... ou nós podemos simplesmente passar mais de um parâmetro para o método.

```

1  <?php
2
3  Schema::table('example', function($table)
4  {
5      $table->dropColumn('name', 'age');
6  });

```

Sinta-se à vontade para utilizar o método que melhor se encaixar no seu estilo de programação.

Mas nós não precisamos remover nossas colunas. Se nós quisermos, nós podemos simplesmente renomeá-las. Vamos dar uma olhada no exemplo abaixo.

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->string('name');
6  });
7
8  Schema::table('example', function($table)
9  {
10     $table->renameColumn('name', 'nickname');
11 });

```

O método `renameColumn()` é usado para alterar o nome de uma coluna. O primeiro parâmetro para o método é o nome da coluna que nós queremos renomear, e o segundo parâmetro é o novo nome da coluna. Esta é a estrutura da tabela gerada com o exemplo acima.

```

1  +-----+-----+
2  | Field   | Type           |
3  +-----+-----+
4  | nickname | varchar(255)   |
5  +-----+-----+
```

Agora, você se lembra das chaves primárias que nós construímos na seção anterior? O que acontece se nós não quisermos mais que estas colunas sejam chaves primárias? Sem problemas, vamos apenas removê-las. Abaixo temos um exemplo.

```

1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->string('name')->primary();
6  });
7
8  Schema::table('example', function($table)
9  {
10     $table->dropPrimary('name');
11 });
```

Usando o método `dropPrimary()`, nós passamos o nome da coluna como parâmetro. Esta coluna vai ter o seu atributo de chave primária removido. Abaixo temos a estrutura da tabela depois do código ser executado.

```

1  +-----+-----+-----+-----+-----+-----+
2  | Field | Type           | Null | Key | Default | Extra |
3  +-----+-----+-----+-----+-----+-----+
4  | name  | varchar(255)   | NO    |     | NULL    |       |
5  +-----+-----+-----+-----+-----+-----+
```

Como você pode ver, o nome da coluna não é mais uma chave primária. Para remover um número de chaves compostas de uma tabela, nós também podemos passar uma array de colunas como o primeiro parâmetro para o método `dropPrimary()`. Abaixo temos um exemplo.

```
1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->string('name');
6      $table->string('email');
7      $table->primary(array('name', 'email'));
8  });
9
10 Schema::table('example', function($table)
11 {
12     $table->dropPrimary(array('name', 'email'));
13 });
```

Nós podemos remover o atributo unique de uma coluna usando o método `dropUnique()`. Este método aceita um único parâmetro, que consiste no nome da tabela, nome da coluna, e a palavra 'unique' separados por underlines (_). Abaixo um exemplo de como podemos remover o atributo unique de uma coluna.

```
1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->string('name')->unique();
6  });
7
8  Schema::table('example', function($table)
9  {
10     $table->dropUnique('example_name_unique');
11 });
```

Novamente, nós podemos passar um array de nomes de colunas no mesmo formato para o método `dropUnique()` se nós quisermos. Abaixo um exemplo.

```
1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->string('name')->unique();
6      $table->string('email')->unique();
7  });
8
9  Schema::table('example', function($table)
10 {
11     $columns = array('example_name_unique', 'example_email_unique');
12     $table->dropUnique($columns);
13 });
```

E finalmente, nós podemos excluir o índice de uma tabela usando... espere... ok, você acertou. Nós podemos usar o método `dropIndex()`. Simplesmente passe o nome da coluna no mesmo formato que nós usamos com o método `dropUnique()`, que é o nome da tabela, nome da coluna, e a palavra 'index' separados por underlines (`_`). Por exemplo:

```
1  <?php
2
3  Schema::create('example', function($table)
4  {
5      $table->string('name')->index();
6  });
7
8  Schema::table('example', function($table)
9  {
10     $table->dropIndex('example_name_index');
11 });
```

Por alguma razão, eu não consegui passar um array de colunas para o método `dropIndex()`. Eu vou perguntar ao Taylor sobre isto e atualizar este capítulo com qualquer alteração. Por enquanto, vamos continuar.

Removendo Tabelas

Para remover uma tabela, simplesmente corte suas pernas.

Apenas brincando, nós podemos remover uma tabela usando o método `Schema::drop()`, vamos dar uma olhada neste método em ação.

```
1 <?php
2
3 Schema::create('example', function($table)
4 {
5     $table->string('name');
6 });
7
8 Schema::drop('example');
```

Para remover uma tabela, nós simplesmente passamos o nome da tabela como o primeiro parâmetro para o método `Schema::drop()`. Vamos tentar visualizar a tabela para ver se ela existe.

```
1 mysql> describe example;
2 ERROR 1146 (42S02): Table 'myapp.example' doesn't exist
```

Bem, acho que funcionou! Parece que a tabela se foi.

Se nós tentarmos excluir uma tabela que não existe, nós vamos receber um erro. Mas nós podemos evitar isto ao utilizar o método `dropIfExists()`. Como o nome sugere, ele vai apenas remover a tabela se ela existir. Abaixo temos um exemplo.

```
1 <?php
2
3 Schema::create('example', function($table)
4 {
5     $table->string('name');
6 });
7
8 Schema::dropIfExists('example');
```

Assim como o método `drop()`, o método `dropIfExists()` aceita um único parâmetro, que é o nome da tabela a ser removida.

Truques Finais

Truques? Talvez não. Porém, esta seção é usada para métodos que simplesmente não se encaixavam nas seções anteriores. Não vamos mais perder tempo e vamos olhar o primeiro método.

Nós podemos utilizar o método `Schema::connection()` para realizar nossas alterações em uma outra base de dados ou conexão. Vamos dar uma olhada no exemplo.

```
1  <?php
2
3  Schema::connection('mysql')->create('example', function($table)
4  {
5      $table->increments('id');
6  });
7
8  Schema::connection('mysql')->table('example', function($table)
9  {
10     $table->string('name');
11 });
```

O método `connection()` pode ser colocado antes de qualquer método usado na classe `Schema`. O primeiro parâmetro para o método é o nome da conexão de banco de dados que nós queremos executar as ações.

O método `connection()` pode ser muito útil se você precisar escrever uma aplicação que utiliza múltiplos bancos de dados.

Agora, nós vamos ver alguns métodos que podem ser usados para verificar a existência de colunas e tabelas. Vamos em frente e ver um exemplo.

```
1  <?php
2
3  if (Schema::hasTable('author')) {
4      Schema::create('books', function($table)
5      {
6          $table->increments('id');
7      });
8  }
```

Nós podemos usar o método `hasTable()` para verificar a existência de uma tabela. O primeiro parâmetro para o método é o nome da tabela que nós desejamos verificar. No exemplo acima, nós apenas vamos criar a tabela 'books' se a tabela 'authors' existir.

Como você já pode ter adivinhado, nós possuímos um método similar para verificar a existência de uma coluna. Vamos dar uma olhada em outro exemplo.

```
1 <?php
2
3 if (Schema::hasColumn('example', 'id')) {
4     Schema::table('example', function($table)
5     {
6         $table->string('name');
7     });
8 }
```

Nós podemos utilizar o método `Schema::hasColumn()` para verificar se uma tabela possui uma coluna. O primeiro parâmetro para o método é a tabela e o segundo parâmetro é o nome da coluna que nós desejamos verificar. No exemplo acima, a coluna 'name' vai ser adicionada na tabela 'example'. apenas se a coluna 'id' existir.

Se você é um gênio em bancos de dados, você talvez queira alterar o motor de armazenamento usado pela tabela. Abaixo temos um exemplo.

```
1 <?php
2
3 Schema::create('example', function($table)
4 {
5     $table->engine = 'InnoDB';
6     $table->increments('id');
7 });
```

Simplesmente altere o valor ao atributo `engine` na estrutura da tabela para o nome do motor de armazenamento que você deseja usar. Aqui temos alguns dos motores disponíveis para as bases de dados MySQL.

- MyISAM
- InnoDB
- IBMDM2I
- MERGE
- MEMORY
- EXAMPLE
- FEDERATED
- ARCHIVE
- CSV
- BLACKHOLE

Para mais informações sobre os motores de armazenamento, por favor consulte a [documentação do MySQL](#)²⁷ para o tópico.

Em bases de dados MySQL, você vai poder reordenar as colunas de uma tabela utilizando o método `after()`. Abaixo segue um exemplo.

```
1 <?php
2
3 Schema::create('example', function($table)
4 {
5     $table->string('name')->after('id');
6     $table->increments('id');
7 });
```

Simplesmente chame o método `after()` após a coluna que você deseja reposicionar. O único parâmetro é o nome da coluna que nós desejamos que a nossa nova coluna seja colocada após. Sinta-se à vontade para utilizar este método, embora eu recomende simplesmente criar as suas tabelas já na ordem pretendida, pois vai parecer muito mais claro.

Bem, isto é tudo que eu tenho sobre criação de bases dados. Por que nós não aprendemos mais sobre um local mais adequado para criar as nossas tabelas? Vamos seguir para o capítulo sobre migrações.

²⁷<http://dev.mysql.com/doc/refman/5.1/en/storage-engines.html>

Migrações

Nós temos um sistema impressionante aqui na Dayle Manor. Um sistema que nos permite que todas as tarefas diárias sejam completadas sem nenhuma bagunça pelo meu exército de pandas vermelhos mordomos. Deixe-me compartilhar com você. Aqui está uma pequena lista dos meus mordomos. Você pode ajudar se quiser.

- **9:00 AM** - Lavar e vestir o Dayle.
- **10:00 AM** - Cozinhar e grelhar carnes exóticas para o café da manhã.
- **12:00 PM** - (Almoço) Os pandas sobem uma árvore e descansam um pouco.
- **02:00 PM** - Polir a coleção de dispositivos da Apple.
- **04:00 PM** - Preparar o trono de escrita para o próximo capítulo do Code Bright.
- **09:00 PM** - Arrastar o Dayle, dormindo, do trono de escrita para a cama.

Esta é a lista dos meus pandas vermelhos. Eles tem um dia e tanto. Eu não sei o que eu faria sem eles. O problema é que a lista possui uma ordem bem específica. Nós não queremos que os pandas me levem para a cama antes da minha visita ao trono da escrita, senão vocês não terão um novo capítulo. Também, não existem motivos para executar as tarefas duas vezes. Os pandas precisam ter certeza que eles façam apenas uma vez, e na ordem correta.

Os pandas são tão espertos, que veio deles a solução para todo o problema. Eu dei a eles a lista original, escrita em um bloco de notas, e eles ficam bem animados quando ganham presentes. De qualquer maneira, eles decidiram que iriam escrever suas próprias listas, para dobrar a diversão, certo?

Os pandas decidiram escrever uma lista secundária. Quando eles completassem uma tarefa, que claro, é completada no mesma ordem da lista original, eles iriam escrever a hora e o nome da tarefa na segunda lista. Desta maneira, a mesma tarefa nunca se repetiria.

Não é uma má ideia. Eu tenho que admitir. Felizmente, alguns caras espertos inventaram uma ideia muito similar para bancos de dados. Vamos dar uma olhada nas migrações.

Conceito Básico

Ao construir um banco de dados, você pode criar sua estrutura na mão. Digite alguns códigos SQL para descrever suas colunas e pronto. Mas o que acontece se acidentalmente você excluir sua base de dados? E se você estiver trabalhando com um time? Nós não queremos ter que passar SQL dumps para o time para manter a base de dados sincronizada.

Aqui é onde as migrações entram. Migrações são alguns scripts PHP que são usados para mudar a maneira que nós estruturamos nossas bases de dados. As migrações tem hora marcada, então elas sempre são executadas na ordem correta.

O Laravel mantém um registro de quais migrações já foram executadas dentro de uma outra tabela na sua conexão padrão de base de dados. Desta maneira, apenas novas migrações são executadas.

Usando migrações, você e seu time vão sempre ter a mesma estrutura de dados, em um estado consistente e estável. Quer saber? Ações falam mais do que palavras. Vamos criar uma nova migração e começar o processo de aprendizagem.

Criando Migrações

Para criar uma migração nós precisamos usar o Artisan, uma ferramenta executada na linha de comando. Vá em frente e abra uma janela do terminal, e navegue até o diretório do seu projeto utilizando o software de linha de comando que você preferir. Nós aprendemos sobre o ‘Schema Builder’ no capítulo anterior, e eu disse que nós iríamos ver um lugar melhor para utilizá-lo. Claro que eu estava falando das migrações. Vamos recriar a estrutura que nós usamos para criar a tabela de usuários. Nós vamos começar usando o Artisan para criar a migração `create_users`.

```
1 $ php artisan migrate:make create_users
2 Created Migration: 2013_06_30_124846_create_users
3 Generating optimized class loader
4 Compiling common classes
```

Nós chamamos o comando do Artisan `migrate:make`, e passamos o nome da nossa migração. O Laravel gerou um template para uma nova migração dentro do diretório `app/database/migrations`. Este template vai ficar localizado em um arquivo nomeado com o parâmetro que você forneceu no comando `migrate:make`, juntamente com a data e hora de criação do arquivo. Neste caso, nosso template está localizado dentro do seguinte arquivo.

```
1 app/database/migrations/2013_06_30_124846_create_users.php
```

Vamos abrir o arquivo no nosso editor de texto e ver o que nós temos.

```
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4
5  class CreateUsers extends Migration {
6
7      /**
8       * Run the migrations.
9       *
10      * @return void
11      */
12     public function up()
13     {
14         //
15     }
16
17     /**
18      * Reverse the migrations.
19      *
20      * @return void
21      */
22     public function down()
23     {
24         //
25     }
26
27 }
```

Acima nós temos nossa classe de migração. É importante que você sempre utilize o comando do Artisan para criar migrações, pois você não vai querer escrever a hora de criação das suas migrações, e contar a história da estrutura de sua base de dados. Seja um bom leitor, utilize o comando.

Dentro de classe de migração nós vamos ter dois métodos públicos, `up()` e `down()`. Agora imagine uma linha entre estes dois métodos, ou digite um comentário caso você não tenha aprendido sobre imaginação com o nosso amigo Barney.

Assim, o lado oposto da linha é exatamente o oposto do que deve ocorrer no outro lado. Seja o que for que você fizer no método `up()`, você deve desfazer dentro do método `down()`. As migrações são bidirecionais. Nós podemos rodar uma migração para atualizar a estrutura ou o conteúdo de nossa base de dados, mas nós também podemos usar uma migração para voltar ao seu estado original.

Primeiro, vamos completar o método `up()`.

```
1  <?php
2
3  /**
4   * Run the migrations.
5   *
6   * @return void
7   */
8  public function up()
9  {
10     Schema::create('users', function($table)
11     {
12         $table->increments('id');
13         $table->string('name', 128);
14         $table->string('email');
15         $table->string('password', 60);
16         $table->timestamps();
17     });
18 }
```

Espero que não tenha nada de confuso dentro do método para você. Se você não entender o que está aqui, simplesmente dê uma outra olhada no capítulo ‘Schema Builder’.

Certo, nós sabemos que o que sobe, deve descer. Por isto, vamos completar o método `down()` e criar o inverso da estrutura que nós criamos no método `up()`.

Aqui vamos nós...

```
1  <?php
2
3  /**
4   * Reverse the migrations.
5   *
6   * @return void
7   */
8  public function down()
9  {
10     Schema::drop('users');
11 }
```

Certo, certo. Eu acho que não é exatamente o oposto. Eu estou achando que você quer remover todas as colunas individualmente e depois a tabela. Na verdade, as duas maneiras fazem a mesma coisa. A tabela `users` vai ser excluída. Então por que não fazer em apenas uma linha?

Antes de continuarmos para a próxima seção, vamos ver alguns truques que são relacionados a criar migrações. Usando as opções `--create` e `--table` no comando `migrate:make`, nós podemos automaticamente criar um template para a criação de uma nova tabela.

Nós simplesmente rodamos...

```
1 php artisan migrate:make create_users --create --table=users
```

E recebemos a seguinte migração.

```
1  <?php
2
3  use Illuminate\Database\Schema\Blueprint;
4  use Illuminate\Database\Migrations\Migration;
5
6  class CreateUsers extends Migration {
7
8      /**
9       * Run the migrations.
10      *
11      * @return void
12      */
13     public function up()
14     {
15         Schema::create('users', function(Blueprint $table)
16         {
17             $table->increments('id');
18             $table->timestamps();
19         });
20     }
21
22     /**
23      * Reverse the migrations.
24      *
25      * @return void
26      */
27     public function down()
28     {
29         Schema::drop('users');
30     }
31
32 }
```

Ótimo! O atalho nos salvou um pouco de tempo. Você vai perceber que além de adicionar os métodos `Schema::create()` e `Schema::drop()` para a nossa tabela, o Laravel também adicionou os métodos `increments()` e `timestamps()`. Isto torna mais fácil a criação de uma tabela compatível com o Eloquent ORM. Não se preocupe com o Eloquent por enquanto, nós vamos ver tudo sobre ele logo.

Um truque final para a criação de migrações, é como armazená-las em um local diferente do diretório padrão `app/database/migrations`. Nós podemos usar a opção `--path` para definir o novo local de nossa classe de migração.

```
1 $ php artisan migrate:make create_users --path=app/migs
2 Created Migration: 2013_06_30_155341_create_users
3 Generating optimized class loader
4 Compiling common classes
```

Agora nossa migração será criada dentro do diretório `app/migs`, relativo a raiz do nosso projeto. Porém, ao rodar suas migrações, o Artisan não vai procurar neste novo local por padrão, então tenha certeza de dizer para o Laravel onde procurar pelas migrações. Nós vamos ver mais sobre isto na próxima seção.

Rodando Migrações

Nós já fizemos todo este esforço para criar nossas migrações, seria uma pena não rodá-las, não acha? Vamos preparar a nossa base de dados para utilizar as migrações. Lembra como eu te falei que o Laravel utiliza uma tabela para marcar o status de suas migrações. Bem, primeiro nós precisamos criar esta tabela.

Você pode chamar a tabela de migrações como você quiser. A configuração do nome da tabela fica dentro do arquivo `app/config/database.php`.

```
1  /*
2  |-----
3  | Migration Repository Table
4  |-----
5  |
6  | This table keeps track of all the migrations that have already run for
7  | your application. Using this information, we can determine which of
8  | the migrations on disk have not actually be run in the databases.
9  |
10 /*
11
12 'migrations' => 'migrations',
```

Simplesmente altere o valor do índice `migrations` para o nome da tabela que você deseja usar. Um nome padrão sempre é fornecido pelo Laravel.

Nós podemos instalar a nossa tabela de migrações rodando um outro comando do Artisan. Vamos rodar o comando `install` agora.

```
1 $ php artisan migrate:install
2 Migration table created successfully.
```

Agora vamos examinar nossa base de dados, e olhar a tabela `migrations` para ver o que foi criado.

```
1 mysql> describe migrations;
2 +-----+-----+-----+-----+-----+
3 | Field      | Type          | Null | Key | Default | Extra |
4 +-----+-----+-----+-----+-----+
5 | migration  | varchar(255)  | NO   |     | NULL    |       |
6 | batch      | int(11)       | NO   |     | NULL    |       |
7 +-----+-----+-----+-----+-----+
8 2 rows in set (0.01 sec)
```

Uma nova tabela com dois campos foi criada. Não se preocupe com a implementação da tabela de migrações, apenas descanse tranquilo sabendo que ela foi criada, e o sistema de migração já está instalado.

Ok, eu menti para você novamente. Eu não sei como eu continuo fazendo isto. Talvez eu deva visitar um psiquiatra. De qualquer maneira, eu te disse que nós precisamos instalar a tabela de migrações, e eu menti.

Assim, o Laravel vai automaticamente criar a tabela para nós caso ela ainda não exista quando as migração forem executadas pela primeira vez. Isto vai instalar o sistema de migrações para você. Pelo menos você sabe sobre o comando `migrate:install` agora, certo? É quase como se eu tivesse planejado isto tudo...

Certo, vamos começar e rodar as nossas migrações pela primeira vez. Nós podemos utilizar o comando `migrate` para isto.

```
1 $ php artisan migrate
2 Migrated: 2013_06_30_124846_create_users
```

A saída do comando é uma lista das migrações que foram executadas. Vamos dar uma olhada na nossa base de dados para ver se a tabela `'users'` foi criada.


```

1  mysql> describe users;
2  +-----+-----+
3  | Field      | Type                |
4  +-----+-----+
5  | id         | int(10) unsigned   |
6  | name       | varchar(128)        |
7  | email      | varchar(255)        |
8  | password   | varchar(60)         |
9  | created_at | timestamp           |
10 | updated_at | timestamp           |
11 +-----+-----+
12 6 rows in set (0.01 sec)

```

Eu encurtei a tabela um pouco para ficar dentro do tamanho do livro, mas você pode ver que a tabela de usuários foi criada corretamente. Demais!

Agora vamos adicionar uma coluna ‘title’ para nossa tabela de usuários. Você talvez tenha tentado abrir a migração que nós já criamos e atualizar sua estrutura para incluir a nova coluna. Por favor, não faça isto.

Assim, se um dos seus colegas já está trabalhando no projeto, e já rodou a primeira migração, então ele não poderá receber nossa nova alteração, e nossas bases de dados não estarão em sincronia.

Ao invés disto, vamos criar uma nova migração para alterar nossa base de dados. Aqui vamos nós.

```

1  $ php artisan migrate:make add_title_to_users
2  Created Migration: 2013_06_30_151627_add_title_to_users
3  Generating optimized class loader
4  Compiling common classes

```

Você vai perceber que eu dei para a migração um nome descritivo, e você deve seguir este padrão. Vamos alterar a estrutura da nossa tabela de usuários dentro do método up() e adicionar a coluna ‘title’.

```

1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4
5  class AddTitleToUsers extends Migration {
6
7      /**
8       * Run the migrations.
9       *

```

```

10      * @return void
11      */
12      public function up()
13      {
14          Schema::table('users', function($table)
15          {
16              $table->string('title');
17          });
18      }
19
20      /**
21       * Reverse the migrations.
22       *
23       * @return void
24       */
25      public function down()
26      {
27          //
28      }
29
30  }

```

Ótimo, isto deve adicionar a coluna que nós queremos a nossa tabela ‘users’. Agora repita comigo.

Tudo que sobe, desce.

Você está certo, nós precisamos criar o método `down()` para esta classe de migração. Vamos alterar a tabela e remover a coluna ‘title’.

```

1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4
5  class AddTitleToUsers extends Migration {
6
7      /**
8       * Run the migrations.
9       *
10      * @return void
11      */
12      public function up()
13      {

```

```

14         Schema::table('users', function($table)
15         {
16             $table->string('title');
17         });
18     }
19
20     /**
21      * Reverse the migrations.
22      *
23      * @return void
24      */
25     public function down()
26     {
27         Schema::table('users', function($table)
28         {
29             $table->dropColumn('title');
30         });
31     }
32
33 }

```

Perfeito, agora o Laravel pode executar nossas migrações, e também reverter as alterações se precisar. Vamos executar as migrações novamente.

```

1 $ php artisan migrate
2     Migrated: 2013_06_30_151627_add_title_to_users

```

O Laravel sabe que nossa migração anterior já foi executada, e vai apenas executar nossa última migração. Vamos examinar a tabela novamente.

```

1 mysql> describe users;
2 +-----+-----+
3 | Field      | Type                |
4 +-----+-----+
5 | id         | int(10) unsigned   |
6 | name       | varchar(128)        |
7 | email      | varchar(255)        |
8 | password   | varchar(60)         |
9 | created_at | timestamp           |
10 | updated_at | timestamp           |
11 | title      | varchar(255)        |
12 +-----+-----+
13 7 rows in set (0.00 sec)

```

Como você pode ver, nossa nova coluna foi adicionada para a tabela de usuários. Se nossa migração for compartilhada com o resto do time, eles podem simplesmente rodar o comando `migrate` para terem suas bases de dados atualizadas com a nova estrutura.

Se por algum motivo nós precisarmos alterar algum dos nossos arquivos de migração, nós podemos usar o comando `migrate:refresh` para reverter todas as migrações, e depois rodá-los novamente. Vamos tentar fazer isto com a tabela de usuários.

```
1 $ php artisan migrate:refresh
2 Rolled back: 2013_06_30_151627_add_title_to_users
3 Rolled back: 2013_06_30_124846_create_users
4 Nothing to rollback.
5 Migrated: 2013_06_30_124846_create_users
6 Migrated: 2013_06_30_151627_add_title_to_users
```

Nossas migrações voltaram atrás utilizando os métodos `down()`, e depois foram executadas novamente na ordem correta com os seus respectivos métodos `up()`. Nossa base de dados está novamente em um estado perfeito.

Lembra como nós usamos a opção `--path` na seção anterior para criar nossas migrações em uma outra localização? Bem, eu prometi que eu iria te mostrar como executá-las. Eu posso ter mentido um pouco, mas eu nunca volto atrás em uma promessa. Vamos dar uma olhada em como nós podemos executar nossas migrações fora da localização padrão.

```
1 $ php artisan migrate --path=app/migs
2 Migrated: 2013_06_30_155341_create_users
```

Viu, fácil? Nós apenas usamos a opção `--path` novamente para especificar a localização onde nossas migrações estão armazenadas.

Eu disse que as migrações são bidirecionais, então isto deve significar que nós podemos revertê-las, certo? Vamos continuar.

Revertendo Migrações

Rolling, rolling, rolling on the riverrrrr...

Desculpe por isto, eu fiquei um pouco distraído. Vamos ver... ah sim! Reverter migrações. Nós sabemos que nós podemos utilizar o comando `migrate` para executar nossas migrações, mas como nós reverteremos elas?

Bem, vamos assumir que nós usamos o comando `migrate` para reestruturar nossa base de dados baseado em uma migração criada por um dos nossos colegas de trabalho. Infelizmente, a estrutura dos nossos colegas mudou e pode ter quebrado parte do seu código, deixando sua aplicação falha.

Nós precisamos reverter as alterações que o nosso colega de trabalho fez. Para isto, nós podemos utilizar o comando `rollback`. Vamos tentar.

```
1 $ php artisan migrate:rollback
2 Rolled back: 2013_06_30_151627_add_title_to_users
```

Quando nós utilizamos o comando `rollback`, o Laravel reverte apenas as migrações que foram executadas na última vez que nós rodamos o comando `migrate`. Como se a última vez que nós rodamos `migrate` nunca tivesse acontecido.

Se nós quisermos reverter **todas** as migrações, nós podemos utilizar o comando `reset`.

```
1 $ php artisan migrate:reset
2 Rolled back: 2013_06_30_151627_add_title_to_users
3 Rolled back: 2013_06_30_124846_create_users
4 Nothing to rollback.
```



Você deve ter em mente que o comando `reset` não vai excluir nossas classes de migração.

Truques de Migrações

Oh, você quer mais? Bem, não se preocupe, eu não vou segurar você. Vamos aprender algumas funcionalidades extras para o nosso sistema de migração.

Você se lembra do array de conexões que nós vimos no arquivo de configuração `app/config/database.php`? Nós podemos executar nossas migrações em uma outra conexão, passando a opção `--database` para qualquer comando de migração, assim:

```
1 $ php artisan migrate --database=mysql
2 Migrated: 2013_06_30_124846_create_users
3 Migrated: 2013_06_30_151627_add_title_to_users
```

Agora nossas migrações serão executadas na conexão que nós chamamos de `mysql` dentro do arquivo de configuração.

Hmm... ainda não está impressionado? Bem, eu tenho um outro truque para você. Eu acho que eu vou estragar você.. mas eu tenho que admitir. Você é um grande ouvinte.

Nós podemos executar as migrações sem alterar nossa base de dados, e nós podemos ver as consultas SQL de criação que são o resultado de nossas migrações. Desta maneira, nós podemos checar para ver o que nossa próxima migração vai fazer, sem arriscar danificar nossa base de dados. Isto é muito útil para depuração.

Para ver o resultado SQL do comando de migração, apenas adicione a opção `--pretend`. Abaixo temos um exemplo.

```
1 $ php artisan migrate --pretend
2 CreateUsers: create table `users` (`id` int unsigned not null
3 auto_increment primary key, `name` varchar(128) not null,
4 `email` varchar(255) not null, `password` varchar(60) not null,
5 `created_at` timestamp default 0 not null, `updated_at` timestamp
6 default 0 not null) default character set utf8 collate utf8_unicode_ci
7 AddTitleToUsers: alter table `users` add `title` varchar(255) not null
```

Agora nós podemos ver as consultas que vão ser executadas em nossa base de dados quando a opção `--pretend` não for passada. Belo truque, certo?

Sim, você me convenceu...

Eu te disse!

No próximo capítulo nós vamos ver o Eloquent ORM. O Eloquent é uma ótima maneira de representar nossos registros do banco de dados em objetos PHP, para que eles se encaixem em uma forma agradável de programação orientada a objetos.

Eloquent ORM

Nós aprendemos como configurar nossa base de dados e como nós podemos utilizar o Schema Builder para estruturar nossas tabelas dentro da nossa base de dados, mas agora está na hora de aprendermos como nós podemos armazenar informações na base de dados.

Alguns de vocês que talvez tenham visto os componentes de base de dados do Laravel, ou até mesmo aqueles que já utilizaram o Laravel 3 podem estar se perguntando porque eu escolhi começar com o ORM. Por que eu não comecei com declarações de SQL, e depois criação de consultas?

Bem, vamos dar um passo para trás e pensar porque nós estamos aqui. Você é um desenvolvedor, um desenvolvedor PHP! Como você está lendo este livro, eu espero que você seja um desenvolvedor PHP 5+, e já adotou a programação orientada a objetos.

Se nós estamos descrevendo as entidades da nossa aplicação como objetos, então faz sentido armazená-las e buscá-las como objetos.

Vamos imaginar que nós estamos escrevendo uma livraria online.

Aplicações orientadas a objetos nos ensinaram que nós precisamos identificar os objetos dentro de nossa aplicação. Bem, uma livraria não vai ter muito sucesso sem livros, certo? Portanto, nós vamos querer que o objeto de um livro represente os livros usados pela nossa aplicação. Normalmente, nós vamos nos referenciar a estes objetos como 'Models' (Modelos), sendo que eles representam parte do modelo de negócios da nossa aplicação. Abaixo temos um exemplo.

```
1  <?php
2
3  class Book
4  {
5      /**
6       * The name of our book.
7       *
8       * @var string
9       */
10     public $name;
11
12     /**
13      * A description for our book.
14      *
15      * @var string
16      */
17     public $description;
```

```
18 }
19
20 $book = new Book;
21 $book->name = 'The Colour of Magic';
22 $book->description = 'Rincewind and Twoflower in trouble!';
```

Ótimo!

Nós criamos um livro representando ‘The Colour of Magic’, do Terry Pratchett’s, um dos meus favoritos! Agora vamos armazenar este livro em nossa base de dados. Vamos assumir que nós já utilizamos o Schema Builder para criar a tabela ‘books’ com todas as colunas necessárias.

Primeiro nós precisamos montar um consulta SQL. Eu sei que você provavelmente montaria um consulta preparada, por razões de segurança, mas eu quero manter este exemplo simples. Isto deve servir...

```
1 <?php
2
3 $query = "
4     INSERT INTO
5         books
6     VALUES (
7         '{$book->name}',
8         '{$book->description}'
9     );
10 ";
```

Nós montamos um consulta SQL para inserir o objeto dentro da base de dados. Esta consulta pode ser executada utilizando qualquer conexão de banco de dados que você normalmente utilizaria.

É uma vergonha nós termos que montar uma string SQL apenas para inserir dados em nossa base de dados. Por que nos incomodar criando um objeto, se nós vamos transformá-lo em uma string para armazená-lo? E nós também temos que montar o objeto novamente quando buscarmos ele da base de dados. É uma perda de tempo...

Eu acho que nós deveríamos poder ‘jogar’ nossos objetos diretamente para a base de dados, sem precisar montar estas feiosas consultas SQL. Hmm, quem sabe algo assim?


```
1 <?php
2
3 $book = new Book;
4 $book->name = 'The Colour of Magic';
5 $book->description = 'Rincewind and Twoflower in trouble!';
6 $book->save();
```

O método `save()` vai manipular a consulta SQL para nós, persistindo o objeto para a base de dados. Isto seria ótimo! Alguém deveria montar esta minha ideia.

Isto já foi feito meu amigo.

O quê? Sério? Que pena... eu pensei que esta ideia me traria fama e fortuna. Bem, eu ahco que isto é uma boa coisa, na verdade.

Ah sim, eu me lembro agora. Esta funcionalidade é fornecida por mapeadores de objetos relacionais, ou simplesmente 'ORMs'. ORMs podem ser utilizados para nos permitir mapear os objetos da nossa aplicação para tabelas da base de dados, e as instâncias destes objetos como os registros das tabelas. Pense nos atributos destes objetos como as colunas da tabela.

O ORM vai buscar e persistir estes objetos para nós na base de dados, e nós não vamos precisar escrever nem uma linha de SQL. Isto é uma boa notícia, porque eu não suporto SQL! É feio e chato. Objetos são muito mais divertidos, certo?

Muitos ORMs também oferecem a possibilidade de manipular as relações entre múltiplos tipos de objetos. Por exemplo, livros e autores. Autores e editores, etc.

O Laravel vem com seu próprio componente de ORM chamado 'Eloquent'. O Eloquent (Eloquente) é muito verdadeiro em seu nome. Possui uma bela sintaxe, e faz com que a interação com a base de dados seja uma experiência agradável, ao invés de um trauma.

Quando eu penso em uma camada de armazenamento, a palavra CRUD vêm em minha mente. Não, eu não estou me referindo ao meu desgosto por SQL desta vez, mas sim as ações que podem ser executadas nesta camada.

- **C** - (Create) Criar novos registros.
- **R** - (Read) Ler registros existentes.
- **U** - (Update) Atualizar registros existentes.
- **D** - (Delete) Excluir registros existentes.

Vamos ver mais sobre o Eloquent executando estas ações em ordem. Nós vamos começar com a criação de instâncias do Eloquent.

Criando Novos Modelos

Antes de nós criarmos nosso primeiro modelo do Eloquent, nós precisamos de um tópico peculiar. Hmm... Eu acabo de comprar um novo jogo para computador, então vamos com vídeo games. Nós podemos criar alguns objetos para representar os vídeo games, mas primeiro, nós precisamos criar a estrutura da tabela.

Nós vamos criar uma nova migração para montar a estrutura da nossa tabela 'games'.

```
1  <?php
2
3  // app/database/migrations/2013_07_10_213946_create_games.php
4
5  use Illuminate\Database\Migrations\Migration;
6
7  class CreateGames extends Migration {
8
9      /**
10       * Run the migrations.
11       *
12       * @return void
13       */
14     public function up()
15     {
16         Schema::create('games', function($table)
17         {
18             $table->increments('id');
19             $table->string('name', 128);
20             $table->text('description');
21         });
22     }
23
24     /**
25      * Reverse the migrations.
26      *
27      * @return void
28      */
29     public function down()
30     {
31         Schema::drop('games');
32     }
33
34 }
```

Espero que o código acima não necessite de introduções. Se você achou algo confuso no exemplo, então dê uma nova olhada no capítulo sobre o Schema Builder.

Perceba que nós nomeamos a tabela como `games`. Isto porque nós pretendemos chamar o nosso modelo do Eloquent de `Game`. O Eloquent é esperto, e por padrão vai procurar pelo plural do nome do nosso modelo como a tabela utilizada para armazenar as instâncias de nossos objetos.

Os modelos do Eloquent possuem alguns requerimentos básicos. Um modelo deve ter uma coluna auto incremental chamada `id`. Esta é a chave primária que pode ser usada para identificar um registro na tabela. Você pode adicionar esta coluna à tabela facilmente utilizando o método `increments()`.

Vamos rodar a migração para atualizar a base de dados.

```
1 $ php artisan migrate
2 Migrated: 2013_07_10_213946_create_games
```

Agora nós podemos começar. Vamos criar um novo modelo do Eloquent para representar os nossos jogos.

```
1 <?php
2
3 // app/models/Game.php
4
5 class Game extends Eloquent
6 {
7
8 }
```

Aqui nós temos um modelo completo do Eloquent que pode ser usado para representar os nossos jogos. Surpreso? Sim, é um pouco pequeno, mas isto é uma coisa boa. Muitos outros ORMs vão necessitar que você monte um XML mapeando a estrutura da base de dados, ou que você crie anotações para cada uma das colunas da tabela em seus objetos. Nós não precisamos isto porque o Eloquent faz algumas razoáveis suposições.

Vamos criar um novo jogo.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $game = new Game;
8      $game->name = 'Assassins Creed';
9      $game->description = 'Assassins VS templars.';
10     $game->save();
11 });
```

Isto parece familiar! Não era assim que nós queríamos persistir os nossos objetos no início? É simples e claro. Nós criamos uma nova instância do nosso modelo 'Game' e setamos seus atributos públicos, que serão mapeados para as colunas da tabela com os valores que nós passamos. Quando nós estivermos prontos, simplesmente chamamos o método `save()` no objeto para persistir o novo registro na base de dados.

Vamos visitar a URI `/`. Nós estamos esperando que a resposta venha vazia, sendo que a consulta vai ser executada e não vai retornar nada para nossa rota. Porém, nós recebemos algo um pouco diferente.

Nós recebemos uma tela de erros. Uma bela tela de erros. Uma tela realmente bela de erros! O cara que fez este tema deve ter belas habilidades, certo? Hehe... De qualquer maneira, o que é este erro?

```
1  SQLSTATE[42S22]: Column not found: 1054 Unknown column 'updated_at' in 'field list'
2  (SQL: insert into `games` (`name`, `description`, `updated_at`, `created_at`) \
3  values (?, ?, ?, ?)) (Bindings: array ( 0 => 'Assassins Creed', 1 => 'Assassins V\
4  S templars.', 2 => '2013-07-14 16:30:55', 3 => '2013-07-14 16:30:55', ))
```

Quando o Eloquent criar nosso novo modelo, ele tenta popular as colunas `updated_at` e `created_at` da nossa tabela com a data e hora atual. Isto porque ele espera que nós utilizamos o método `->timestamps()` ao construir a estrutura da nossa tabela. Isto é padrão, visto que sempre é bom manter um histórico de quando o registro foi criado / atualizado. Porém, se você está usando uma tabela já existente, ou simplesmente não quer ter estas colunas em suas tabelas, você talvez vai querer desabilitar esta funcionalidade.

Para desabilitar estas atualizações nos modelos do Eloquent, simplesmente adicione um atributo público para seu modelo.

```
1 <?php
2
3 // app/models/Game.php
4
5 class Game extends Eloquent
6 {
7     public $timestamps = false;
8 }
```

O atributo público `$timestamps` é herdado da classe base do Eloquent. É um valor booleano que pode ser usado para habilitar ou desabilitar as atualizações de tempo automáticas. No exemplo acima, nós setamos este atributo para `false`, o que significa que o Eloquent vai saber que nós queremos desabilitar esta funcionalidade.

Vamos visitar a URL / novamente. Desta vez a página exibiu um resultado em branco. Não entre em pânico, isto é simplesmente porque nós não retornamos nenhuma resposta da nossa rota. Nós não recebemos nenhuma mensagem de erro, então a consulta SQL deve ter sido executada. Vamos verificar a tabela `games` para ver o resultado.

```
1 mysql> use myapp;
2 Database changed
3 mysql> select * from games;
4 +----+-----+-----+-----+
5 | id | name          | description          |
6 +----+-----+-----+-----+
7 |  1 | Assassins Creed | Assassins VS templars. |
8 +----+-----+-----+-----+
9 1 row in set (0.00 sec)
```

Nós podemos ver que nosso registro foi inserido corretamente. Ótimo! Nós inserimos um novo registro sem escrever nem uma linha de SQL. Este é o meu tipo de vitória.

Perceba que nós não especificamos o valor da coluna `id` em nosso objeto. A coluna `id` é automaticamente incrementada, então a camada do banco de dados vai manipular isto para nós. Normalmente é uma péssima ideia modificar a coluna `id` de um modelo do Eloquent. Tente evitar isto, a não ser que você realmente saiba o que está fazendo.

Nós utilizamos o atributo `$timestamps` para desabilitar as datas automáticas. Mas, vamos ver o que acontece quando nós habilitamos elas. Mas primeiro, nós precisamos alterar a estrutura da nossa base de dados. Não é uma boa ideia modificar a estrutura da nossa base de dados diretamente, ou alterar as migrações que já foram executadas. Isto porque nossa base de dados pode ficar ‘dessincronizada’ em relação aos nossos colegas de trabalho. Ao invés disto, vamos criar uma nova migração para que nossos colegas de trabalho também possam receber e executar nossas alterações.

```
1 $ php artisan migrate:make add_timestamps_to_games
2 Created Migration: 2013_07_14_165416_add_timestamps_to_games
```

Nossa migração foi criada. Perceba que nós demos um nome descritivo para nossa migração, representando nossas intenções com ela. Isto pode ser uma informação útil quando seus colegas de trabalho executarem a migração. Vamos utilizar o Schema Builder para adicionar os campos de datas para nossa tabela de jogos.

```
1  <?php
2
3  // app/database/migrations/2013_07_14_165416_add_timestamps_to_games.php
4
5  use Illuminate\Database\Migrations\Migration;
6
7  class AddTimestampsToGames extends Migration {
8
9      /**
10       * Run the migrations.
11       *
12       * @return void
13       */
14     public function up()
15     {
16         Schema::table('games', function($table)
17         {
18             $table->timestamps();
19         });
20     }
21
22     /**
23      * Reverse the migrations.
24      *
25      * @return void
26      */
27     public function down()
28     {
29         //
30     }
31
32 }
```

Nós utilizamos o método `Schema::table()` para alterar nossa tabela de 'games', e o método `timestamps()` para adicionar os campos de data. Agora senhoras e senhores, repitam comigo.

Tudo o que sobe, desce!

Você realmente aprende rápido! Ótimo trabalho. Vamos remover as colunas de data da tabela no método `down()`.

```
1  <?php
2
3  // app/database/migrations/2013_07_14_165416_add_timestamps_to_games.php
4
5  use Illuminate\Database\Migrations\Migration;
6
7  class AddTimestampsToGames extends Migration {
8
9      /**
10       * Run the migrations.
11       *
12       * @return void
13       */
14     public function up()
15     {
16         Schema::table('games', function($table)
17         {
18             $table->timestamps();
19         });
20     }
21
22     /**
23      * Reverse the migrations.
24      *
25      * @return void
26      */
27     public function down()
28     {
29         Schema::table('games', function($table)
30         {
31             $table->dropColumn('updated_at', 'created_at');
32         });
33     }
34
35 }
```

Eu utilizei o método `dropColumn()` do Schema Builder para remover as colunas `updated_at` e `created_at` da nossa tabela dentro do método `down()`. Eu acho que poderia existir um adorável

método `dropTimestamps()` para isto, mas aparentemente não existe. Sem problemas! É um projeto de código aberto, então eu vou enviar um ‘pull request’ mais tarde quando eu tiver um tempo livre. Vamos executar nossa nova migração para adicionar as novas colunas para nossa tabela de ‘games’.

```
1 $ php artisan migrate
2 Migrated: 2013_07_14_165416_add_timestamps_to_games
```

Agora nós podemos fazer uma escolha, nós podemos setar o atributo `$timestamps` dentro de nosso modelo para `true`, que habilitaria o preenchimento das datas automaticamente.

```
1 <?php
2
3 // app/models/Game.php
4
5 class Game extends Eloquent
6 {
7     public $timestamps = true;
8 }
```

Ou... nós podemos simplesmente removê-lo. Isto porque o valor padrão para o atributo `$timestamps` dentro da classe base do Eloquent é `true`. E seu valor é herdado dentro de nosso modelo.

```
1 <?php
2
3 // app/models/Game.php
4
5 class Game extends Eloquent
6 {
7
8 }
```

Ótimo, agora vamos executar nossa URI / novamente para inserir um novo registro. Vamos examinar a tabela ‘games’ dentro da base de dados para ver o resultado.


```

1 mysql> use myapp;
2 Database changed
3 mysql> select * from games;
4 +---+-----+-----+-----+-----+-----+-----+-----\
5 -----+
6 | id | name          | description          | created_at          | updated_a\
7 t      |
8 +---+-----+-----+-----+-----+-----+-----+-----\
9 -----+
10 | 1 | Assassins Creed | Assassins VS templars. | 2013-07-14 17:14:13 | 2013-07-1\
11 4 17:14:13 |
12 +---+-----+-----+-----+-----+-----+-----+-----\
13 -----+
14 1 row in set (0.00 sec)

```

Como você pode ver, as colunas `created_at` e `updated_at` foram populadas com a data atual para nós. Isto pode nos salvar muito tempo! Você pode querer lembrar os usuários de sua aplicação do seu aniversário de um ano utilizando sua aplicação, e para isso você pode comparar a data atual com a coluna `created_at`.

Antes de nós entrarmos na próxima seção, eu gostaria de compartilhar um pequeno truque com você. Se você não quiser que o nome de sua tabela esteja no plural do seu modelo, então você precisa dizer isto ao Eloquent. Apenas adicione o atributo público `$table` no seu modelo, e digite o nome da tabela como o seu valor. O Eloquent vai então utilizar o nome da tabela contido neste atributo para todas as consultas SQL que fizer.

```
1  <?php
2
3  // app/models/Game.php
4
5  class Game extends Eloquent
6  {
7      public $table = 'gamezilla_roar';
8  }
```

Se você escolher utilizar namespaces em seus modelos, você vai precisar utilizar o atributo `$table` para passar os nomes das tabelas. Isto porque o nome da tabela de um modelo, contido no namespace `MyApp\Models\Game` seria `my_app_models_games` para evitar conflitos com outros namespaces que também utilizem a base de dados. Perceba que o Eloquent é muito esperto, e vai transformar o namespace nomeado no padrão CamelCase para o padrão com underlines (`_`).

Nós já vimos como criar registros dentro de nossa base de dados utilizando o Eloquent. Agora, vamos ver como nós podemos ler estes registros.

Lendo Registros Existentes

O Eloquent oferece alguns métodos para buscar instâncias de nossos modelos. Nós vamos examinar todos eles em um futuro capítulo, mas por enquanto nós vamos utilizar o método `find()` para buscar uma única instância de um modelo da base de dados através da coluna `id`. Abaixo um exemplo.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $game = Game::find(1);
8      return $game->name;
9  });
```

Nós utilizamos o método estático `find()` do nosso modelo para buscar a instância do objeto `Game` que representa o registro de `id` número 1 em nossa base de dados. Nós podemos acessar os atributos públicos da nossa instância para exibir os valores das colunas, Vamos visitar a URL `/` para ver o resultado.

```
1  Assassins Creed
```

Ótimo, nosso registro já existente foi exibido com sucesso. O método estático `find()` é herdado da classe base do Eloquent, e não precisa ser criado dentro do seu modelo. Como eu disse antes, existem muitos outros métodos para buscar registros, que vão ser vistos em um futuro capítulo sobre consulta de modelos do Eloquent. Por enquanto, vamos ver como nós podemos atualizar registros já existentes.

Atualizando Modelos Já Existentes

Se você recentemente criou um novo modelo, então as chances de você ter atribuído ele à uma variável são grandes. Na seção anterior nós criamos uma nova instância do nosso modelo `Game`, atribuímos ela à variável `$game`, preenchemos suas colunas e utilizamos o método `save()` para salvar na base de dados.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $game = new Game;
8      $game->name = 'Assassins Creed';
9      $game->description = 'Assassins VS templars.';
10     $game->save();
11 });
```

Não é porque nós já salvamos nossa instância, que nós não podemos querer modificá-la. Nós podemos alterar seus valores e chamar o método `save()` novamente para atualizar um registro já existente. Na primeira vez que o método `save()` é usado em um novo objeto, ele vai criar um novo registro e atribuir o valor da coluna `id` em seus atributos. As próximas chamadas do método `save()` vão apenas atualizar os valores das colunas em nosso já existente registro.

Dê uma olhada no exemplo abaixo.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $game = new Game;
8      $game->name = 'Assassins Creed';
9      $game->description = 'Show them what for, Altair.';
10     $game->save();
11
12     $game->name = 'Assassins Creed 2';
13     $game->description = 'Requiescat in pace, Ezio.';
14     $game->save();
15
16     $game->name = 'Assassins Creed 3';
17     $game->description = 'Break some faces, Connor.';
18     $game->save();
19 });
```

Você poderia imaginar que o exemplo acima vai criar três novos registros na tabela `games`, mas você estaria enganado neste caso. Perceba que nós apenas criamos uma única instância da classe `Game`.


```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $game = Game::find(1);
8      $game->name = 'Assassins Creed 4';
9      $game->description = 'Shiver me timbers, Edward.';
10     $game->save();
11 });
```

Inspecionando a tabela de jogos, nós descobrimos que nosso registro anterior foi inserido com o `id` igual a 1. Utilizando o método estático `find()` em nosso modelo, com o `id` de número 1, nós podemos buscar a instância do objeto `Game` representando o registro de número um. Depois que a instância for retornada, nós podemos modificar os valores de suas colunas e utilizar o método `save()` da mesma maneira que nós fizemos anteriormente.

Abaixo é como nosso registro ficou depois de executar o código acima.

```

1 mysql> select * from games;
2 +---+-----+-----+-----+-----+-----+-----+-----\
3 -----+
4 | id | name                | description                | created_at            | upd\
5 ated_at              |
6 +---+-----+-----+-----+-----+-----+-----+-----\
7 -----+
8 |   1 | Assassins Creed 4    | Shiver me timbers, Edward. | 2013-07-14 17:38:50 | 201\
9 3-07-14 17:49:28 |
10 +---+-----+-----+-----+-----+-----+-----+-----\
11 -----+
12 1 row in set (0.00 sec)

```

Como você pode ver, nosso registro foi atualizado corretamente. Novamente, nós não escrevemos nem uma linha de SQL. Apenas um belo e eloquente PHP. Perceba também que a coluna `updated_at` foi preenchida com a data de atualização. Muito útil!

Excluindo Modelos Existentes

Excluir modelos do Eloquent é um processo muito simples. Primeiro, nós precisamos buscar a instância do modelo que nós desejamos excluir. Por exemplo, nós podemos utilizar o método `find()` que nós já vimos nas seções anteriores.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $game = Game::find(1);
8  });
```

Uma vez que nós tivermos nossa instância, nós podemos utilizar o método `delete()` para remover o registro que o nosso modelo representa na base de dados.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $game = Game::find(1);
8      $game->delete();
9  });
```

Nós também podemos excluir um registro diretamente passando o seu `id` para o método estático `destroy()`.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      Game::destroy(1);
8  });
```

Para excluir mais de um registro, você pode passar os `ids` que você quer excluir por parâmetro para o método `destroy()`...

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     Game::destroy(1, 2, 3);
8 });
```

... ou também um array de ids, assim:

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     Game::destroy(array(1, 2, 3));
8 });
```

Só depende de você!

O SQL oferece muitas diferentes e complexas maneiras de consultar registros específicos. Mas não se preocupe, o Eloquent também pode executar estas tarefas. No próximo capítulo nós vamos aprender sobre os diferentes métodos de consulta disponíveis no Eloquent ORM. Vá em frente, vire a página!

Consultas com o Eloquent

No capítulo anterior nós vimos como utilizar classes para representar nossas tabelas do banco de dados. Isto remove a necessidade de escrever declarações utilizando a linguagem de consulta estruturada (SQL - Structured Query Language, em inglês) e resulta em um código muito mais legível. Nós estamos escrevendo PHP, certo? Por que complicar as coisas adicionando outra linguagem?

Existem algumas partes boas no SQL. Por exemplo, o Q. Querying (Consultar). Com SQL, nós podemos utilizar complexas comparações entre os registros para buscarmos apenas os registros desejados. Replicar *toda* esta funcionalidade no Eloquent seria uma tarefa imensa, e felizmente, o Eloquent possui métodos para os tipos mais utilizados de consultas. Para todas as outras que o Eloquent não possui métodos, nós podemos utilizar consultas do tipo `raw`, onde nós podemos passar declarações de SQL. Porém, nós vamos dar uma olhada nisto depois. Vamos primeiro preparar a nossa base de dados.

Preparação

Logo nós vamos aprender como popular nossas tabelas com dados de exemplo utilizando uma técnica conhecida como `seeding` (semear), mas por enquanto nós vamos criar alguns registros dentro da base de dados utilizando o Eloquent. Nós não vamos utilizar nenhuma nova funcionalidade aqui, apenas o que nós já aprendemos nos capítulos anteriores.

Primeiro nós precisamos criar uma migração para montar a estrutura da nossa tabela. Nós vamos utilizar álbuns de música. Vamos criar a migração para montar a tabela `albums`.

```
1 $ php artisan migrate:make create_albums
2 Created Migration: 2013_07_21_103250_create_albums
```

Agora vamos estruturar nossa tabela nos métodos disponibilizados na migração.


```
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4
5  // app/database/migrations/2013_07_21_103250_create_albums.php
6
7  class CreateAlbums extends Migration {
8
9      /**
10       * Run the migrations.
11       *
12       * @return void
13       */
14     public function up()
15     {
16         Schema::create('albums', function($table)
17         {
18             $table->increments('id');
19             $table->string('title', 256);
20             $table->string('artist', 256);
21             $table->string('genre', 128);
22             $table->integer('year');
23         });
24     }
25
26     /**
27      * Reverse the migrations.
28      *
29      * @return void
30      */
31     public function down()
32     {
33         Schema::drop('albums');
34     }
35
36 }
```

No método `up()` da nossa migração, nós utilizamos a classe `Schema` para criar uma nova tabela chamada `albums`. A tabela vai conter colunas do tipo `varchar` para o nome do álbum, o nome do artista e o gênero da música. Nós também vamos ter um `id` auto incremental que é necessário para utilizar o Eloquent ORM, e finalmente um campo inteiro para armazenar o ano de lançamento do álbum.

No método `down()`, nós excluimos a tabela, voltando a base de dados para seu estado original.

Vamos rodar nossa migração e estruturar nossa base de dados.

```
1 $ php artisan migrate
2 Migration table created successfully.
3 Migrated: 2013_07_21_103250_create_albums
```

Nossa base de dados agora possui a estrutura para armazenar nossos álbuns. Agora nós precisamos criar um modelo do Eloquent para que ele possa interagir com nossa tabela recém criada utilizando um objeto PHP.

```
1 <?php
2
3 // app/models/Album.php
4
5 class Album extends Eloquent
6 {
7     public $timestamps = false;
8 }
```

Simples, limpo e adorável. Nós desabilitamos as atualizações automáticas de datas em nosso modelo `Album` para simplificar os exemplos nesta seção. Normalmente, eu gosto de adicionar atualizações automáticas de tempo em todos os meus modelos. Enquanto existe uma mínima perda de performance fazendo isto, e um pouco mais de espaço de armazenamento necessário, as datas de atualização e criação de seus registros podem ser muito úteis para análise dos seus dados no futuro.

Agora nós temos tudo o que nós precisamos para popular nossa base de dados com exemplos. Como eu mencionei anteriormente, o ideal seria nós estarmos utilizando o semeamento da base de dados para isto, mas por enquanto nós vamos apenas criar uma rota para isto. Como resultado, nós vamos ter um pouco de repetição, mas nós vamos deixar assim neste exemplo. Nós apenas vamos acessar esta rota uma vez.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/seed', function()
6 {
7     $album = new Album;
8     $album->title      = 'Some Mad Hope';
9     $album->artist     = 'Matt Nathanson';
10    $album->genre      = 'Acoustic Rock';
```

```
11     $album->year           = 2007;
12     $album->save();
13
14     $album = new Album;
15     $album->title           = 'Please';
16     $album->artist          = 'Matt Nathanson';
17     $album->genre           = 'Acoustic Rock';
18     $album->year           = 1993;
19     $album->save();
20
21     $album = new Album;
22     $album->title           = 'Leaving Through The Window';
23     $album->artist          = 'Something Corporate';
24     $album->genre           = 'Piano Rock';
25     $album->year           = 2002;
26     $album->save();
27
28     $album = new Album;
29     $album->title           = 'North';
30     $album->artist          = 'Something Corporate';
31     $album->genre           = 'Piano Rock';
32     $album->year           = 2002;
33     $album->save();
34
35     $album = new Album;
36     $album->title           = '...Anywhere But Here';
37     $album->artist          = 'The Ataris';
38     $album->genre           = 'Punk Rock';
39     $album->year           = 1997;
40     $album->save();
41
42     $album = new Album;
43     $album->title           = '...Is A Real Boy';
44     $album->artist          = 'Say Anything';
45     $album->genre           = 'Indie Rock';
46     $album->year           = 2006;
47     $album->save();
48 });
```

Estes são alguns dos meus favoritos. Eu espero que quem não seja fã de punk rock não se sinta ofendido pelo meu gosto musical, e continue lendo este capítulo.

Como você pode ver, para cada um dos registros nós criamos uma nova instância do modelo Album,

populamos seus campos, e salvamos nosso modelo na base de dados.

Vá em frente e acesse a URI `/seed` para popular a tabela `albums` com nossos dados de exemplo. Você deve ver uma página em branco, visto que nós não retornamos nenhuma resposta da nossa rota.

Agora que nossos dados foram gravados na base de dados, você talvez queira excluir a rota `/seed`. Nós não precisamos mais dela! Nosso ambiente já está preparado, vamos começar a aprender sobre consultas com o Eloquent.

Eloquent para String

Os objetos no PHP podem opcionalmente incluir o método `__toString()`. Você já pode ter visto ou utilizado este método. Ele foi adicionado no PHP 5.2, juntamente com algumas outras funções mágicas, que são prefixadas com dois underlines (`__`). Este método pode ser usado para controlar como um objeto deve ser representado quando for uma string.

Graças a este método, nossos modelos do Eloquent podem também ser expressados como strings. A classe base do Eloquent, que nós estendemos a partir dos nossos modelos, contém o método `__toString()`. Este método vai retornar uma string JSON que vai representar os valores do nosso modelo do Eloquent.

Isto pode parecer um pouco confuso, e já faz um tempo que não vemos um exemplo. Vamos primeiramente ver a maneira ‘normal’ de exibir valores contidos dentro de nossos modelos do Eloquent.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $album = Album::find(1);
8      return $album->title;
9  });
```

No exemplo acima nós utilizamos o método estático `find()` do nosso modelo `Album` e passamos o valor inteiro de 1 para buscar a instância representando o registro de id 1 em nossa base de dados. Depois nós retornamos o atributo `title` da instância do modelo para ser exibido como resposta para nossa view.

Se nós visitarmos a URI `/`, nós recebemos a seguinte resposta.

Some Mad Hope

O título do primeiro álbum inserido em nossa base de dados, como esperado. Agora vamos modificar a rota para retornar a instância do nosso modelo como resposta da nossa rota.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return Album::find(1);
8  });
```

Vamos visitar a URI / para verificar a resposta.

Isto parece JSON! Todas as strings de JSON criadas pelo framework possuem espaços extras e identificação removidos para poupar banda enquanto transfere os dados. Eu vou deixar os exemplos JSON identados e com espaços neste capítulo para facilitar a visualização.

Vamos ver a saída identada.

```
1  {
2      id: 1,
3      title: "Some Mad Hope",
4      artist: "Matt Nathanson",
5      genre: "Acoustic Rock",
6      year: 2007
7  }
```

O Laravel executou o método `__toString()`, herdado da classe base do Eloquent para representar seus valores como uma string JSON. Isto é muito útil ao criar RESTful API's para servir dados em JSON. Também é uma bela maneira de exibir o resultado de nossas consultas para o resto do capítulo.

Alguns dos métodos do Eloquent vão retornar mais de uma instância de nosso modelo como resultado, ao invés de uma única instância, como no exemplo acima. Vamos dar uma rápida olhada no método `all()`, que é utilizado para buscar todos os registros como instâncias do Eloquent.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $albums = Album::all();
8      foreach ($albums as $album) {
9          echo $album->title;
10     }
11 });
```

Nós utilizamos o método `all()` do nosso modelo `Album` para buscar um array de instâncias que representam os registros da nossa tabela `albums`. Então nós percorremos o array, e escrevemos o título de cada uma das instâncias do `Album`.

Este é o resultado que nós recebemos da URI `/`.

```
1 Some Mad HopePleaseLeaving Through The WindowNorth...Anywhere But Here...Is A Rea\  
2 1 Boy
```

Ótimo, estes são todos os títulos. Eles estão juntos porque nós não inserimos o elemento de quebra de linha `<br />` depois de cada título. Não se preocupe com isto, pelo menos nós conseguimos buscar todos eles.

Desculpe, mas eu menti novamente para você. Se você já utilizou o Laravel 3 anteriormente, o conceito de buscar um array de instâncias de modelos pode ser familiar para você. Porém, o Laravel 4 não retorna um array de instâncias, e sim uma coleção (`Collection`).

Eu não acredito em você. Se não retorna um array, como nós podemos percorrer seus resultados como se fosse um array?

Isto é simples. O objeto `Collection` implementa uma interface que permite que o objeto será iterado. E assim ele pode ser percorrido da mesma maneira que arrays PHP.

Hmm. Porém, eu não vou acreditar em um mentiroso tão facilmente.

Você quer mais provas? Vamos analisar o resultado do método `all()` com a função `var_dump` para ver com o que nós estamos trabalhando.

```
1 <?php  
2  
3 // app/routes.php  
4  
5 Route::get('/', function()  
6 {  
7     $albums = Album::all();  
8     var_dump($albums);  
9 });
```

Quando nós visitamos a URI `/` nós recebemos a seguinte resposta.

```

1 object(Illuminate\Database\Eloquent\Collection)[134]
2   protected 'items' =>
3     array (size=6)
4       0 =>
5         object(Album)[127]
6           public 'timestamps' => boolean false
7           protected 'connection' => null
8           protected 'table' => null
9           protected 'primaryKey' => stri
10
11     ... loads more information ...

```

Woah, você não estava mentindo desta vez!

Como você pode ver, o resultado de qualquer método que retorna múltiplas instâncias é representado como uma instância da classe `Illuminate\Database\Eloquent\Collection`. Você pode ver através da saída do `var_dump` que o objeto possui um array com as instâncias dos nossos modelos chamado `items`.

A vantagem de ter um objeto de coleção é que ele também possui alguns métodos úteis para transformar e buscar nossas instâncias. Em um próximo capítulo, nós vamos examinar estes métodos mais detalhadamente, mas por enquanto vale saber que o objeto `Collection` também inclui o método `__toString()`. Este método funciona de maneira muito similar ao `__toString()` dos nossos modelos, mas ao invés de criar um JSON que representa apenas um resultado, cria um JSON que representa todas as nossas instâncias, com um array multi dimensional.

Vamos retornar o objeto `Collection` que é o resultado do método `all()` como resposta da nossa rota. Assim:

```

1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return Album::all();
8 });

```

A resposta que nós recebemos após visitar a URI / é a seguinte.

```
1  [  
2    {  
3      id: 1,  
4      title: "Some Mad Hope",  
5      artist: "Matt Nathanson",  
6      genre: "Acoustic Rock",  
7      year: 2007  
8    },  
9    {  
10     id: 2,  
11     title: "Please",  
12     artist: "Matt Nathanson",  
13     genre: "Acoustic Rock",  
14     year: 1993  
15   },  
16   {  
17     id: 3,  
18     title: "Leaving Through The Window",  
19     artist: "Something Corporate",  
20     genre: "Piano Rock",  
21     year: 2002  
22   },  
23   {  
24     id: 4,  
25     title: "North",  
26     artist: "Something Corporate",  
27     genre: "Piano Rock",  
28     year: 2002  
29   },  
30   {  
31     id: 5,  
32     title: "...Anywhere But Here",  
33     artist: "The Ataris",  
34     genre: "Punk Rock",  
35     year: 1997  
36   },  
37   {  
38     id: 6,  
39     title: "...Is A Real Boy",  
40     artist: "Say Anything",  
41     genre: "Indie Rock",  
42     year: 2006
```



```
43     }  
44 ]
```

Nós recebemos uma string JSON contendo um array de objetos que representa os valores de cada um de nossos álbuns.

Mas por que aprender sobre a funcionalidade do método `__toString()` agora? Nós não queremos montar uma API JSON neste capítulo, queremos? Não, nós ainda não estamos prontos para isto.

Mas, eu posso utilizar JSON como saída para exibir os resultados das consultas que nós vamos estar executando pelo resto do capítulo. É muito mais fácil para leitura do que um monte de loops `foreach()`. E agora você sabe exatamente porque estes resultados estão sendo exibidos como JSON. Todo mundo ganha!

Agora que nós já temos nossa base de dados populada com dados, nós precisamos achar uma maneira de exibir os resultados das consultas. Vamos dar uma olhada na estrutura das consultas do Eloquent.

Estrutura das Consultas

As consultas do Eloquent são utilizadas para buscar resultados baseados em um conjunto de regras e critérios. Você não quer sempre buscar todos os seus registros. As vezes, você apenas quer buscar a discografia de um único artista. Nestas circunstâncias, nós vamos utilizar uma consulta, para solicitar apenas os registros que tenham a coluna `title` do artista que nós desejamos.

As consultas do Eloquent podem ser quebradas em três partes separadas.

- Modelos.
- Restrições de consulta.
- Métodos de consulta.

O modelo é a instância que nós desejamos executar a consulta. Todos os exemplos desta seção são baseados no modelo `Album`.

Restrições de consulta são regras que são utilizadas para que nossos registros correspondam a um subconjunto. Assim nós podemos retornar apenas os registros que nós queremos. A restrição mais comum é a cláusula `WHERE` do SQL.

E finalmente, nós temos os métodos de consulta. Eles são os métodos que são utilizados para executar a consulta e retornar o resultado.

Vamos dar uma olhada na estrutura de uma consulta do Eloquent em seu estado mais simples.

```
1 <?php
2
3 Model::fetch();
```

Todas as nossas consultas acontecem em algum dos nossos modelos. Os métodos de restrição são totalmente opcionais, e no exemplo acima, não estão presentes. Depois, nós temos o método `fetch`. Este método não existe, nós apenas estamos utilizando ele para demonstrar a estrutura de uma consulta. O primeiro método sempre chamado de forma estática, com dois pontos `::`.

As consultas do Eloquent podem não ter métodos de restrição, podem ter um método de restrição, ou podem ter muitos deles. Só depende de você. E assim é como uma consulta se parece com uma única restrição.

```
1 <?php
2
3 Model::constraint()
4     ->fetch();
```

Note como a restrição é chamada estaticamente, e nosso método de consulta foi chamado logo após nosso primeiro método. Nós podemos adicionar quantas restrições nós quisermos, por exemplo:

```
1 <?php
2
3 Model::constraint()
4     ->constraint()
5     ->constraint()
6     ->fetch();
```

Restrições são totalmente opcionais, mas todas as consultas precisam começar com um modelo, e terminar com um método de consulta. Abaixo um exemplo utilizando o método `Album`.

```
1 <?php
2
3 Album::all();
```

O `Album` é o nosso modelo, e `all` é um dos nossos métodos de consulta, que é utilizado para buscar todos os registros de uma tabela.

Métodos de consulta podem ser usados para retornar ou uma instância única de um modelo, ou uma coleção (`Collection`) de instâncias. Porém, como nós vimos anteriormente, os dois podem ser retornados no formato JSON como resposta de nossas rotas e controllers.

Nós sabemos que nossas restrições de consulta são opcionais, então vamos começar vendo a variedade de métodos de consulta que estão disponíveis.

Métodos de Consulta

Vamos começar com um dos métodos de consulta que você pode já viu nos capítulos anteriores. O método `find()`.

Find

O método `find()` é utilizado para busca uma única instância de um modelo, através da coluna `id`. Se o primeiro parâmetro desta coluna for um inteiro, então uma única instância deste método será retornada. Aqui temos um exemplo.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return Album::find(1);
8  });
```

Nós queremos buscar o registro que tenha o `id` com o valor 1, então apenas uma única instância será retornada.

```
1  {
2      id: 1,
3      title: "Some Mad Hope",
4      artist: "Matt Nathanson",
5      genre: "Acoustic Rock",
6      year: 2007
7  }
```

Se nós passarmos um array de `ids`, nós vamos receber uma coleção de instâncias.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return Album::find(array(1, 3));
8 });
```

Este é o resultado. Uma coleção contendo as instâncias dos modelos que representam os registros com os ids 1 e 3.

```
1 [
2     {
3         id: 1,
4         title: "Some Mad Hope",
5         artist: "Matt Nathanson",
6         genre: "Acoustic Rock",
7         year: 2007
8     },
9     {
10        id: 3,
11        title: "Leaving Through The Window",
12        artist: "Something Corporate",
13        genre: "Piano Rock",
14        year: 2002
15    }
16 ]
```

All

O método `all()` é usado para retornar uma coleção que representa todos os registros da nossa tabela. Abaixo um exemplo de utilização do método `all()`.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return Album::all();
8  });
```

Nós recebemos uma coleção, contendo instâncias de todos os álbuns contidos dentro da nossa base de dados.

```
1  [
2      {
3          id: 1,
4          title: "Some Mad Hope",
5          artist: "Matt Nathanson",
6          genre: "Acoustic Rock",
7          year: 2007
8      },
9      {
10         id: 2,
11         title: "Please",
12         artist: "Matt Nathanson",
13         genre: "Acoustic Rock",
14         year: 1993
15     },
16     {
17         id: 3,
18         title: "Leaving Through The Window",
19         artist: "Something Corporate",
20         genre: "Piano Rock",
21         year: 2002
22     },
23     {
24         id: 4,
25         title: "North",
26         artist: "Something Corporate",
27         genre: "Piano Rock",
28         year: 2002
29     },
30     {
```

```
31         id: 5,  
32         title: "...Anywhere But Here",  
33         artist: "The Ataris",  
34         genre: "Punk Rock",  
35         year: 1997  
36     },  
37     {  
38         id: 6,  
39         title: "...Is A Real Boy",  
40         artist: "Say Anything",  
41         genre: "Indie Rock",  
42         year: 2006  
43     }  
44 ]
```

First

Em algumas circunstâncias onde uma coleção de instâncias normalmente seria retornada, o método `first()` pode ser utilizado para buscar apenas a *primeira* instância da coleção. É muito útil se você quiser que sua consulta retorne uma única instância, ao invés de uma coleção de instâncias. Se nós não usarmos nenhuma restrição, o método `first()` vai apenas retornar o primeiro registro de nossa base de dados.

Abaixo um exemplo.

```
1  <?php  
2  
3  // app/routes.php  
4  
5  Route::get('/', function()  
6  {  
7      return Album::first();  
8  });
```

Nós recebemos uma única instância, o primeiro álbum armazenado dentro de nossa base de dados.

```
1 {
2     id: 1,
3     title: "Some Mad Hope",
4     artist: "Matt Nathanson",
5     genre: "Acoustic Rock",
6     year: 2007
7 }
```

Update

Nós não precisamos apenas buscar as instâncias, nós também podemos alterá-las. Utilizando o método `update()`, nós podemos atualizar o valor dos registros que são o resultado de uma consulta do Eloquent. Simplesmente passe um array com chave-valor como o primeiro parâmetro para o método `update()` para alterar os valores das colunas de todos os registros contidos na coleção. A chave do array representa o nome da coluna que nós desejamos alterar, e o valor representa o novo valor da coluna.

O método `update()` é especial, e não pode ser utilizado sem nenhuma restrição. Então eu vou utilizar um simples `where()` como restrição neste exemplo. Se você não entender, não se preocupe. Nós vamos cobrir todas as restrições em detalhes dentro da próxima seção. Aqui temos um exemplo de como nós vamos modificar os registros da nossa tabela `albums`. (Não se preocupe, eu vou restaurar os registros para o próximo exemplo.).

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     Album::where('artist', '=', 'Matt Nathanson')
8         ->update(array('artist' => 'Dayle Rees'));
9
10     return Album::all();
11 });
```

Nós atualizamos a coluna `artist` de todos os registros que tinham o artista `Matt Nathanson`, alterando o valor para `Dayle Rees`. O método `update()` não retorna uma coleção. Então, para visualizar nossos registros alterados, nós utilizamos o método `all()`.

```
1  [  
2    {  
3      id: 1,  
4      title: "Some Mad Hope",  
5      artist: "Dayle Rees",  
6      genre: "Acoustic Rock",  
7      year: 2007  
8    },  
9    {  
10     id: 2,  
11     title: "Please",  
12     artist: "Dayle Rees",  
13     genre: "Acoustic Rock",  
14     year: 1993  
15   },  
16   {  
17     id: 3,  
18     title: "Leaving Through The Window",  
19     artist: "Something Corporate",  
20     genre: "Piano Rock",  
21     year: 2002  
22   },  
23   {  
24     id: 4,  
25     title: "North",  
26     artist: "Something Corporate",  
27     genre: "Piano Rock",  
28     year: 2002  
29   },  
30   {  
31     id: 5,  
32     title: "...Anywhere But Here",  
33     artist: "The Ataris",  
34     genre: "Punk Rock",  
35     year: 1997  
36   },  
37   {  
38     id: 6,  
39     title: "...Is A Real Boy",  
40     artist: "Say Anything",  
41     genre: "Indie Rock",  
42     year: 2006
```



```
43     }  
44 ]
```

Como você pode ver, agora eu sou um astro do rock. Demais!

Delete

Muito parecido com o método `update()`, o método `delete()` não vai retornar nenhuma instância. Ao invés disto, vai remover os registros que forem o resultado da consulta na base de dados.

```
1  <?php  
2  
3  // app/routes.php  
4  
5  Route::get('/', function()  
6  {  
7      Album::where('artist', '=', 'Matt Nathanson')  
8          ->delete();  
9  
10     return Album::all();  
11 });
```

Nós consultados por todos os álbuns que tenham a coluna `artist` igual à `Matt Nathanson`, e depois nós utilizamos o método `delete()` para excluir estes registros da base de dados.

```
1  [  
2      {  
3          id: 3,  
4          title: "Leaving Through The Window",  
5          artist: "Something Corporate",  
6          genre: "Piano Rock",  
7          year: 2002  
8      },  
9      {  
10         id: 4,  
11         title: "North",  
12         artist: "Something Corporate",  
13         genre: "Piano Rock",  
14         year: 2002  
15     },  
16     {
```

```
17         id: 5,  
18         title: "...Anywhere But Here",  
19         artist: "The Ataris",  
20         genre: "Punk Rock",  
21         year: 1997  
22     },  
23     {  
24         id: 6,  
25         title: "...Is A Real Boy",  
26         artist: "Say Anything",  
27         genre: "Indie Rock",  
28         year: 2006  
29     }  
30 ]
```

Os álbuns do Matt Nathanson foram removidos da nossa base de dados. O que é uma pena, porque ele faz músicas fantásticas.

Uma dica rápida. Se você pretende excluir todos os registros de uma tabela, você pode utilizar o método `truncate()`.

Abaixo um exemplo.

```
1  <?php  
2  
3  // app/routes.php  
4  
5  Route::get('/', function()  
6  {  
7      Album::truncate();  
8      return Album::all();  
9  });
```

Como você pode ver, nossos registros foram excluídos!

```
1  [ ]
```

Get

Get é o método mais importante entre os métodos de consulta. É utilizado para buscar os resultados da consulta. Por exemplo, se nós utilizarmos a restrição `where()` para limitar os resultados a um único artista, então não faria sentido utilizar o método `all()` para buscar todos os resultados. Neste caso, nós utilizamos o método `get()` para buscar uma coleção de instâncias que estejam de acordo com nossas restrições.

Confuso? Aqui está uma combinação dos métodos `where()` e `get()`.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return Album::where('artist', '=', 'Something Corporate')
8          ->get();
9  });
```

Nós recebemos uma coleção de instâncias que possuem a coluna `artist` com o valor `Something Corporate`.

```
1  [
2      {
3          id: 3,
4          title: "Leaving Through The Window",
5          artist: "Something Corporate",
6          genre: "Piano Rock",
7          year: 2002
8      },
9      {
10         id: 4,
11         title: "North",
12         artist: "Something Corporate",
13         genre: "Piano Rock",
14         year: 2002
15     }
16 ]
```

O método `get()` possui um parâmetro opcional. Você pode passar um array de colunas para ele, e o resultado vai apenas conter estas colunas. Abaixo um exemplo.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return Album::where('artist', '=', 'Something Corporate')
8         ->get(array('id', 'title'));
9 });
```

Nós passamos um array com os valores `id` e `title` para o método `get()`, e aqui está o resultado desta consulta.

```
1 [
2     {
3         id: 3,
4         title: "Leaving Through The Window"
5     },
6     {
7         id: 4,
8         title: "North"
9     }
10 ]
```

Como você pode ver, apenas as colunas que nós solicitamos foram representadas nos resultados.

Pluck

O método `pluck()` pode ser usado para buscar o valor de uma única coluna. Aqui está um exemplo.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return Album::pluck('artist');
8 });
```

O primeiro e único parâmetro, é o nome da coluna que nós desejamos buscar o valor. Se a consulta tiver múltiplos resultados, então apenas o valor do primeiro resultado vai ser retornado. Aqui está o resultado do exemplo acima.

1 Matt Nathanson

Lists

Enquanto o método `pluck()` apenas busca o valor de uma coluna específica, o método `lists()` retorna um array de resultados com a coluna especificada. Vamos deixar mais claro com um exemplo.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return Album::lists('artist');
8 });
```

Novamente, o método `lists()` aceita um único parâmetro. O nome da coluna que nós desejamos buscar os valores. Aqui está o resultado da nossa consulta.

```
1 [
2     "Matt Nathanson",
3     "Matt Nathanson",
4     "Something Corporate",
5     "Something Corporate",
6     "The Ataris",
7     "Say Anything"
8 ]
```

Como você pode ver, nós buscamos os valores da coluna `artist` de todos os nossos registros.

ToSql

Certo, este não é exatamente um método de consulta, mas é muito útil! Você pode utilizar o método `toSql()` em qualquer lugar onde você normalmente utilizaria um método de consulta. Ele vai retornar uma string contendo a SQL que vai ser executada.

Vamos ver um exemplo.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return Album::where('artist', '=', 'Something Corporate')
8         ->toSql();
9 });
```

Assim como o exemplo anterior, apenas que desta vez nós chamamos o método `toSql()` ao invés do método `get()`. Abaixo o resultado recebido.

```
1 select * from `albums` where `artist` = ?
```

Muito útil para debugar!

O que é o ponto de interrogação?

O Query Builder do Laravel utiliza declarações preparadas (prepared statements, em inglês). Isto significa que os pontos de interrogação serão substituídos pelos seus respectivos valores mais tarde. E com isto, os valores são escapados antes de serem substituídos, e assim evitamos qualquer tentativa de invasão por injeção de SQL (SQL Injection, em inglês).

Agora que nós descobrimos os métodos de consulta, é hora de aprender sobre como adicionar restrições em nossas consultas.

Restrições de Consulta

Todos os métodos de consulta da seção anterior são úteis para buscar coleções e instâncias da nossa base de dados. Porém, as vezes nós precisamos de resultados específicos. E são nestes casos que as restrições de consulta são úteis.

Na matemática, a aritmética básica nos permite buscar subconjuntos dentro de conjuntos maiores. Isto é essencialmente o que nós fazemos utilizando restrições de consulta. Porém, também existem métodos nesta seção relativos a ordenação dos resultados.

Vamos começar com o método que representa a restrição mais comum do SQL, a cláusula `WHERE`.

Where

O primeiro método de restrição que nós vamos examinar é o método `where()`. Se você já utilizou SQL no passado, então você provavelmente já utilizou a cláusula `WHERE` para restringir seus resultados.

Vamos ver um exemplo.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return Album::where('artist', '=', 'Matt Nathanson')
8         ->get();
9 });
```

Nós utilizamos o método `where()` para limitar os resultados aos álbuns que sejam do artista Matt Nathanson apenas.

O método `where()` aceita três parâmetros. O primeiro parâmetro é o nome da coluna que nós desejamos executar a comparação. Neste exemplo, nós desejamos executar a comparação na coluna `artist`. O segundo parâmetro é o operador utilizado para esta comparação. Neste exemplo nós queremos garantir que a coluna `artist` **seja igual** a um valor, por isto nós utilizamos o símbolo `=`.

Nós poderíamos ter utilizado qualquer outro símbolo de comparação suportado pelo SQL, como `<`, `>`, `=>`, `=<`, etc. Experimente diferentes tipos de operadores para obter diferentes resultados.

O terceiro parâmetro é o valor que nós queremos comparar. Em nosso exemplo, nós queremos garantir que apenas os registros que tenham a coluna `artist` igual a Matt Nathanson sejam retornadas, então neste caso `'Matt Nathanson'` é o valor.

Novamente, o método `where()` é apenas a restrição de consulta. Nós vamos utilizar o método `get()` para buscar a coleção de resultados. Vamos dar uma olhada na resposta da URI `/`.

```
1 [
2     {
3         id: 1,
4         title: "Some Mad Hope",
5         artist: "Matt Nathanson",
6         genre: "Acoustic Rock",
7         year: 2007
8     },
9     {
10        id: 2,
11        title: "Please",
12        artist: "Matt Nathanson",
13        genre: "Acoustic Rock",
14        year: 1993
15    }
16 ]
```

Excelente. Os dois álbuns da base de dados que tinham a coluna `artist` com o valor `Matt Nathanson` foram retornados. Isto seria muito útil se nós desejarmos criar seções em um site de música para exibir discografias de um único artista.

Vale lembrar que você pode chamar tanto o método `get()` quanto o método `first()`. Vamos alterar o exemplo para buscar apenas a primeira instância da nossa consulta.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return Album::where('artist', '=', 'Matt Nathanson')
8          ->first();
9  });
```

Agora a consulta vai apenas buscar uma única instância representando o primeiro registro da nossa consulta. Abaixo o resultado da URI `/`.

```
1  {
2      id: 1,
3      title: "Some Mad Hope",
4      artist: "Matt Nathanson",
5      genre: "Acoustic Rock",
6      year: 2007
7  }
```

Vamos tentar um outro operador com o método `where()`. Que tal o `LIKE`? O operador SQL `LIKE` é usado para comparar partes de uma string utilizando o símbolo `%`.

Aqui temos um exemplo.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return Album::where('title', 'LIKE', '...%')
8          ->get();
9  });
```


No exemplo acima, nós estamos buscando os registros em que a coluna `title` comece com reticências O sinal de porcentagem `%` vai dizer para o banco de dados que nós não nos importamos com o restante do valor, depois das reticências.

Vamos dar uma olhada no resultado da URI `/`.

```
1  [  
2    {  
3      id: 5,  
4      title: "...Anywhere But Here",  
5      artist: "The Ataris",  
6      genre: "Punk Rock",  
7      year: 1997  
8    },  
9    {  
10     id: 6,  
11     title: "...Is A Real Boy",  
12     artist: "Say Anything",  
13     genre: "Indie Rock",  
14     year: 2006  
15   }  
16 ]
```

Nós recebemos uma coleção de álbuns com os nomes ‘...Anywhere But Here’ e ‘...Is A Real Boy’. Os dois começam com reticências, e também são ótimos álbuns.

Nós não estamos limitados a utilizar apenas uma chamada `where()` em um consulta. Nós podemos utilizar múltiplas vezes o método `where()` para restringir nossa consulta. Abaixo um exemplo.

```
1  <?php  
2  
3  // app/routes.php  
4  
5  Route::get('/', function()  
6  {  
7      return Album::where('title', 'LIKE', '...%')  
8          ->where('artist', '=', 'Say Anything')  
9          ->get();  
10 });
```

No exemplo acima, nós queremos encontrar os registros que em que a coluna `title` comece com reticências e a coluna `artist` seja igual a ‘Say Anything’. Os registros precisam estar de acordo com as duas restrições para serem retornados.

Este é o resultado do exemplo acima.

```
1  [  
2    {  
3      id: 6,  
4      title: "...Is A Real Boy",  
5      artist: "Say Anything",  
6      genre: "Indie Rock",  
7      year: 2006  
8    }  
9  ]
```

Uma coleção com uma única instância, um álbum com o título que começa com três pontos e possui o artista ‘Say Anything’. Nós recebemos uma coleção (`Collection`) contendo o álbum ‘Say Anything’. Um dos meus favoritos.

orWhere

Porém, nem sempre nós precisamos que as duas (ou mais) restrições correspondam. As vezes, se apenas uma das condições corresponder já é o suficiente para nós. Em situações como esta, nós podemos utilizar o método `orWhere()`. Todas as restrições que esta seção vai cobrir possuem um método alternativo utilizando o prefixo `or`, que permite que apenas algumas das restrições sejam correspondidas. Por isto, eu não vou separar as seções com os métodos `or`.

Como de costume, abaixo um exemplo.

```
1  <?php  
2  
3  // app/routes.php  
4  
5  Route::get('/', function()  
6  {  
7      return Album::where('title', 'LIKE', '...%')  
8          ->orWhere('artist', '=', 'Something Corporate')  
9          ->get();  
10 });
```

A primeira restrição diz que o campo `title` do álbum *deve* começar com reticências. Depois nós incluímos o método `orWhere()` que diz que os nossos resultados também podem ter o campo `artist` com o valor `Something Corporate`.

Vamos dar uma olhada no resultado.

```
1  [  
2    {  
3      id: 3,  
4      title: "Leaving Through The Window",  
5      artist: "Something Corporate",  
6      genre: "Piano Rock",  
7      year: 2002  
8    },  
9    {  
10     id: 4,  
11     title: "North",  
12     artist: "Something Corporate",  
13     genre: "Piano Rock",  
14     year: 2002  
15   },  
16   {  
17     id: 5,  
18     title: "...Anywhere But Here",  
19     artist: "The Ataris",  
20     genre: "Punk Rock",  
21     year: 1997  
22   },  
23   {  
24     id: 6,  
25     title: "...Is A Real Boy",  
26     artist: "Say Anything",  
27     genre: "Indie Rock",  
28     year: 2006  
29   }  
30 ]
```

Nós recebemos uma coleção de resultados com as instâncias que possuem um título que comece com reticências **ou** o artista Something Corporate.

Você pode chamar quantos métodos `where()` e `orWhere()` você precisar para filtrar seus resultados.

WhereRaw

O método `whereRaw()` pode ser utilizado para passar uma string de SQL para executar a condição `WHERE`. Abaixo um exemplo.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return Album::whereRaw('artist = ? and title LIKE ?', array(
8          'Say Anything', '...%'
9      ))
10     ->get();
11 });
```

O método `whereRaw()` aceita uma string SQL como o primeiro parâmetro. Todos os pontos de interrogação `?` na string serão substituídos pelos elementos do array que é o segundo parâmetro, na ordem sequencial. Se você já utilizou declarações preparadas (prepared statements, em inglês) com SQL esta sintaxe vai ser familiar para você. Os valores passados vão ser escapados para evitar ataques de injeção de SQL (SQL injection).

Uma vez que o Query Builder executar as transformações necessárias, o SQL resultante vai ser mais ou menos assim:

```
1  artist = 'Say Anything' and title LIKE '...%'
```

O resultado da nossa consulta é o seguinte.

```
1  [
2      {
3          id: 6,
4          title: "...Is A Real Boy",
5          artist: "Say Anything",
6          genre: "Indie Rock",
7          year: 2006
8      }
9  ]
```

Você pode utilizar o método `whereRaw()` em circunstâncias onde você precisa utilizar um SQL mais complexo, que o método `where()` não suportaria. Assim como o método `where()`, o método `whereRaw()` pode ser chamado múltiplas vezes e com outros métodos de restrição também. Novamente, o método `orWhereRaw()` também está disponível para permitir que você utilize condições alternativas.

WhereBetween

O método `whereBetween()` é usado para verificar se o valor da coluna está entre os dois valores passados. É melhor descrevê-lo com um exemplo, na verdade, eu acho que tudo é. Estranho, certo? Talvez seja porque o código do Laravel fale por si só.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return Album::whereBetween('year', array('2000', '2010'))
8          ->get();
9  });
```

O primeiro parâmetro para o método `whereBetween()` é o nome da coluna que nós queremos comparar. O segundo parâmetro é um array de dois valores, o valor inicial e o valor limite. No exemplo acima, nós estamos verificando álbuns que tenham o ano (`year`) de publicação entre 2000 e 2012. Abaixo o resultado.

```
1  [
2      {
3          id: 1,
4          title: "Some Mad Hope",
5          artist: "Matt Nathanson",
6          genre: "Acoustic Rock",
7          year: 2007
8      },
9      {
10         id: 3,
11         title: "Leaving Through The Window",
12         artist: "Something Corporate",
13         genre: "Piano Rock",
14         year: 2002
15     },
16     {
17         id: 4,
18         title: "North",
19         artist: "Something Corporate",
20         genre: "Piano Rock",
21         year: 2002
22     }
23 ]
```

```
22     },
23     {
24         id: 6,
25         title: "...Is A Real Boy",
26         artist: "Say Anything",
27         genre: "Indie Rock",
28         year: 2006
29     }
30 ]
```

Recebemos o resultado esperado, com todos os álbuns dos anos 2000.

Como todos os outros método `where()`, você pode chamar este método quantas vezes quiser, e como sempre, temos o método alternativo `orWhereBetween()`.

WhereNested

O método `whereNested()` é uma maneira mais clara de aplicar múltiplas restrições para uma consulta. Simplesmente passe uma função anônima (closure) como o primeiro parâmetro para o método, e esta closure recebe um parâmetro que você pode chamar como quiser, mas eu normalmente chamo de `$query`.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return Album::whereNested(function($query)
8      {
9          $query->where('year', '>', 2000);
10         $query->where('year', '<', 2005);
11     })
12     ->get();
13 });
```

Dentro da closure você pode aplicar quantas restrições você desejar, utilizando o objeto `$query`. Todas as restrições aplicadas serão parte da sua consulta principal. Fica muito mais claro! E abaixo o resultado do exemplo acima.

```
1  [  
2    {  
3      id: 3,  
4      title: "Leaving Through The Window",  
5      artist: "Something Corporate",  
6      genre: "Piano Rock",  
7      year: 2002  
8    },  
9    {  
10     id: 4,  
11     title: "North",  
12     artist: "Something Corporate",  
13     genre: "Piano Rock",  
14     year: 2002  
15   }  
16 ]
```

Neste caso, não existe o método `orWhereNested()`. Mas eu vou contar um pequeno segredo... você também pode passar uma closure para o método `orWhere()`. Abaixo um exemplo.

```
1  <?php  
2  
3  // app/routes.php  
4  
5  Route::get('/', function()  
6  {  
7      return Album::whereNested(function($query)  
8      {  
9          $query->where('year', '>', 2000);  
10         $query->where('year', '<', 2005);  
11     })  
12     ->orWhere(function($query)  
13     {  
14         $query->where('year', '=', 1997);  
15     })  
16     ->get();  
17 });
```

Nós desejamos buscar os álbuns que tenham o ano de publicação entre 2000 e 2005, **ou** que tenham sido publicados em 1997. Este é o SQL gerado com o exemplo acima.

```
1 select * from `albums` where (`year` > ? and `year` < ?) or (`year` = ?)
```

E estes são os resultados da consulta acima.

```
1  [  
2    {  
3      id: 3,  
4      title: "Leaving Through The Window",  
5      artist: "Something Corporate",  
6      genre: "Piano Rock",  
7      year: 2002  
8    },  
9    {  
10     id: 4,  
11     title: "North",  
12     artist: "Something Corporate",  
13     genre: "Piano Rock",  
14     year: 2002  
15   },  
16   {  
17     id: 5,  
18     title: "...Anywhere But Here",  
19     artist: "The Ataris",  
20     genre: "Punk Rock",  
21     year: 1997  
22   }  
23 ]
```

WhereIn

O método `whereIn()` pode ser usado para verificar se o valor da coluna existe dentro de um conjunto de valores. É muito útil quando você já possui um array de possíveis valores na mão. Vamos dar uma olhada em como podemos utilizá-lo.


```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     $values = array('Something Corporate', 'The Ataris');
8     return Album::whereIn('artist', $values)->get();
9 });
```

O primeiro parâmetro para o método `whereIn()` é a coluna que nós desejamos comparar. O segundo parâmetro é o array com os valores em que a comparação será executada.

O resultado da SQL acima se parece com isto.

```
1 select * from `albums` where `artist` in (?, ?)
```

Abaixo a coleção de resultados que nós recebemos com a consulta acima.

```
1 [
2     {
3         id: 3,
4         title: "Leaving Through The Window",
5         artist: "Something Corporate",
6         genre: "Piano Rock",
7         year: 2002
8     },
9     {
10        id: 4,
11        title: "North",
12        artist: "Something Corporate",
13        genre: "Piano Rock",
14        year: 2002
15    },
16    {
17        id: 5,
18        title: "...Anywhere But Here",
19        artist: "The Ataris",
20        genre: "Punk Rock",
21        year: 1997
22    }
23 ]
```

O método `whereIn()` também possui o método alternativo `orWhereIn()` e pode ser chamado múltiplas vezes.

WhereNotIn

O método `whereNotIn()` é totalmente o oposto do método `whereIn()`. Desta vez, você passa uma lista de valores que **não** devem existir dentro dos seus registros.

Vamos dar uma olhada no exemplo.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $values = array('Something Corporate', 'The Ataris');
8      return Album::whereNotIn('artist', $values)->get();
9  });
```

Novamente, nós passamos a coluna que vai ser comparada como o primeiro parâmetro, e nosso array de valores como o segundo parâmetro.

Esta é a SQL resultante.

```
1  select * from `albums` where `artist` not in (?, ?)
```

E finalmente, aqui está o resultado da nossa consulta. Todos os álbuns que não são dos artistas que nós passamos no array.

```
1  [
2      {
3          id: 1,
4          title: "Some Mad Hope",
5          artist: "Matt Nathanson",
6          genre: "Acoustic Rock",
7          year: 2007
8      },
9      {
10         id: 2,
11         title: "Please",
12         artist: "Matt Nathanson",
13         genre: "Acoustic Rock",
14         year: 1993
15     },
16     {
```

```
17         id: 6,  
18         title: "...Is A Real Boy",  
19         artist: "Say Anything",  
20         genre: "Indie Rock",  
21         year: 2006  
22     }  
23 ]
```

Novamente, o método `orWhereNotIn()` também está disponível.

WhereNull

A restrição `whereNull()` pode ser usada quando você precisa buscar registros que tenham o valor `NULL`. Vamos ver um exemplo.

```
1 <?php  
2  
3 // app/routes.php  
4  
5 Route::get('/', function()  
6 {  
7     return Album::whereNull('artist')->get();  
8 });
```

O único parâmetro para o método `whereNull()` é o nome da coluna que você queira que tenha apenas valores nulos. Vamos dar uma olhada no SQL gerado para esta consulta.

```
1 select * from `albums` where `artist` is null
```

Agora vamos dar uma olhada no resultado desta consulta.

```
1 [ ]
```

Oh, claro, nós não temos nenhum valor nulo (`NULL`) em nossa base de dados! Eu não quero reescrever este capítulo inteiro novamente, então você vai ter que utilizar sua imaginação aqui. Se nós tivéssemos um álbum com a coluna `artist` e o valor `NULL`, então estes registros apareceriam nos resultados.

Sim, você acertou! O método `orWhereNull()` também está disponível.

WhereNotNull

O método `whereNotNull()` é o oposto do método `whereNull()`, então desta vez você deve ver alguns resultados. Ele vai retornar as colunas que não tenham valores nulos. Vamos dar uma olhada.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return Album::whereNotNull('artist')->get();
8  });
```

O primeiro e único parâmetro para o método é o nome da coluna. Aqui está o SQL gerado para esta consulta.

```
1  select * from `albums` where `artist` is not null
```

Este é o resultado da consulta acima.

```
1  [
2      {
3          id: 1,
4          title: "Some Mad Hope",
5          artist: "Matt Nathanson",
6          genre: "Acoustic Rock",
7          year: 2007
8      },
9      {
10         id: 2,
11         title: "Please",
12         artist: "Matt Nathanson",
13         genre: "Acoustic Rock",
14         year: 1993
15     },
16     ... 4 more ...
17 ]
```

Todos os álbuns da nossa base de dados. Isto por causa que nenhum dos nossos álbuns possui a coluna `artist` com o valor `NULL`.

Novamente, o método `orWhereNull()` também está disponível.

OrderBy

O método `orderBy()` pode ser usado para ordenar os resultados da nossa consulta pelo valor de uma coluna específica. Vamos ver um exemplo.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return Album::where('artist', '=', 'Matt Nathanson')
8          ->orderBy('year')
9          ->get();
10 });
```

O primeiro parâmetro para o método `orderBy()` é o nome da coluna que nós desejamos ordenar. Por padrão, a ordem será ascendente.

Esta é a SQL gerada.

```
1  select * from `albums` where `artist` = ? order by `year` asc
```

E este é o resultado da consulta.

```
1  [
2      {
3          id: 2,
4          title: "Please",
5          artist: "Matt Nathanson",
6          genre: "Acoustic Rock",
7          year: 1993
8      },
9      {
10         id: 1,
11         title: "Some Mad Hope",
12         artist: "Matt Nathanson",
13         genre: "Acoustic Rock",
14         year: 2007
15     }
16 ]
```

Ótimo, nossos álbuns foram retornados na ordem ascendente pelo ano de publicação. E se nós quisermos ordenar em ordem decrescente? Não se preocupe, o Laravel possui esta funcionalidade.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return Album::where('artist', '=', 'Matt Nathanson')
8          ->orderBy('year', 'desc')
9          ->get();
10 });
```

Nós adicionamos um segundo parâmetro para o método `orderBy()` com o valor `desc`. Isto diz ao Laravel que nós desejamos ordenar os resultados na ordem decrescente. Aqui está a SQL gerada.

```
1  select * from `albums` where `artist` = ? order by `year` desc
```

E os resultados da consulta.

```
1  [
2      {
3          id: 1,
4          title: "Some Mad Hope",
5          artist: "Matt Nathanson",
6          genre: "Acoustic Rock",
7          year: 2007
8      },
9      {
10         id: 2,
11         title: "Please",
12         artist: "Matt Nathanson",
13         genre: "Acoustic Rock",
14         year: 1993
15     }
16 ]
```

Ótimo! Nossos resultados agora estão na ordem decrescente.

Você pode utilizar a cláusula `orderBy()` com qualquer combinação de restrições já vistas dentro deste capítulo. Você também pode utilizar o método `orderBy()` mais de uma vez, para ordenações adicionais. A ordenação é feita na ordem em que os métodos são chamados.

Take

O método `take()` é usado para limitar os resultados. Aqui está um exemplo.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return Album::take(2)
8         ->get();
9 });
```

O primeiro parâmetro para o método `take()` é a quantidade de registros que você quer limitar. No exemplo acima, nós desejamos apenas dois registros.

Este é o código SQL gerado pela consulta.

```
1 select * from `albums` limit 2
```

E finalmente, o resultado que nós recebemos.

```
1 [
2     {
3         id: 1,
4         title: "Some Mad Hope",
5         artist: "Matt Nathanson",
6         genre: "Acoustic Rock",
7         year: 2007
8     },
9     {
10        id: 2,
11        title: "Please",
12        artist: "Matt Nathanson",
13        genre: "Acoustic Rock",
14        year: 1993
15    }
16 ]
```

Isto pode ser usado em combinação com qualquer uma das restrições de busca.

Skip

Ao utilizar o método `take()`, o método `skip()` pode ser usado para dizer a partir de qual resultado que os nossos registros devem ser limitados. Aqui está um exemplo.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return Album::take(2)
8          ->skip(2)
9          ->get();
10 });
```

O método `skip()` aceita um único parâmetro que é o número do registro que nossos resultados devem começar. No exemplo acima, os dois primeiros registros vão ser descartados dos resultados. Aqui está o SQL gerado.

```
1  select * from `albums` limit 2 offset 2
```

E este é o resultado que recebemos.

```
1  [
2      {
3          id: 3,
4          title: "Leaving Through The Window",
5          artist: "Something Corporate",
6          genre: "Piano Rock",
7          year: 2002
8      },
9      {
10         id: 4,
11         title: "North",
12         artist: "Something Corporate",
13         genre: "Piano Rock",
14         year: 2002
15     }
16 ]
```

Como você pode ver, o primeiro e o segundo registro foram pulados, começando pelo terceiro registro na base de dados.

Magic Where Queries

Certo, agora é hora para algo mágico! Agora, você já deve estar mais familiarizado com o método `where()`. O `where()` é responsável por restringir uma coluna a um certo valor dentro dos seus registros. Aqui está um pequeno exemplo para lembrá-lo.


```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return Album::where('artist', '=', 'Something Corporate')
8         ->get();
9 });
```

Nós pretendemos receber um conjunto de resultados em que suas colunas `artist` tenham o valor igual a `Something Corporate`. É uma maneira simples e clara de restringir os resultados. Será que pode ficar mais claro ainda? Bem, acontece que sim, pode! Nós podemos utilizar a sintaxe mágica do `where`.

Dê uma olhada no seguinte exemplo.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return Album::whereArtist('Something Corporate')->get();
8 });
```

Espera, o que é o método `whereArtist()`? Nós não aprendemos sobre isto na seção sobre restrições de consultas. Bem, este método é um pouco especial. Primeiro vamos visitar a URI / para ver o resultado.

```
1 [
2     {
3         id: 3,
4         title: "Leaving Through The Window",
5         artist: "Something Corporate",
6         genre: "Piano Rock",
7         year: 2002
8     },
9     {
10        id: 4,
11        title: "North",
12        artist: "Something Corporate",
```

```
13         genre: "Piano Rock",
14         year: 2002
15     }
16 ]
```

Funciona de uma maneira muito similar ao nosso método `where()` com um sinal de igual `=`. Certo, hora de explicar o que está acontecendo. O método `where` com o operador igual é a operação mais usada de todas, e por isto, o Taylor nos deu um conveniente atalho.

Você pode simplesmente adicionar o nome da coluna que você deseja filtrar ao método `where()`. Primeiro, você precisa que a primeira letra da coluna esteja com letra maiúscula. No nosso exemplo, nós utilizamos a coluna `artist`, então o nome do método ficou `whereArtist()`. Se o nome da sua coluna possui underlines (`_`), então você deve deixar cada palavra com letra maiúscula, por exemplo, a coluna `shoe_size` ficaria `whereShoeSize()`.

O único parâmetro para o método mágico `where()` é o valor esperado da coluna. Vamos ver um outro exemplo.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     return Album::whereTitle('North')->get();
8 });
```

Buscar todos os álbuns, com a coluna `title` igual a `North`. E aqui está o resultado desta consulta.

```
1 [
2     {
3         id: 4,
4         title: "North",
5         artist: "Something Corporate",
6         genre: "Piano Rock",
7         year: 2002
8     }
9 ]
```

Ótimo, aqui está o nosso álbum. Uma boa utilização para este método é quando você precisa buscar registros pelos valores de suas colunas.

Escopo de Consulta

Escopos de consulta (Query Scopes, em inglês) podem ser muito úteis se você se encontra repetindo as mesmas consultas muitas vezes. Vamos começar com um exemplo. Lembra aqueles álbuns que os nomes começam com reticências? ‘...Is A Real Boy’ e ‘...Anywhere But Here’. Vamos imaginar que buscar álbuns que comecem com reticências seja uma prática comum em nossa aplicação.

Vamos ver, nós podemos fazer a consulta pelos álbuns toda vez, assim.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return Album::where('title', 'LIKE', '...%')->get();
8  });
```

Porém, isto seria muito repetitivo, não seria? Nós não queremos nos repetir. Por que nós não utilizamos um escopo de consulta? Vamos começar. Primeiro, vamos ver o nosso modelo Album novamente. No momento ele está assim.

```
1  <?php
2
3  // app/models/Album.php
4
5  class Album extends Eloquent
6  {
7      public $timestamps = false;
8  }
```

Vamos adicionar um novo método ao nosso modelo. Agora nosso modelo está assim.

```
1  <?php
2
3  // app/models/Album.php
4
5  class Album extends Eloquent
6  {
7      public $timestamps = false;
8
9      public function scopeTriplePeriod($query)
10     {
11         return $query->where('title', 'LIKE', '...%');
12     }
13 }
```

Nós adicionamos o método `scopeTriplePeriod()` ao nosso modelo. É um método especial com uma função específica, ajudar a reutilizar consultas comuns em nossa aplicação. Todos os métodos de escopo começam com a palavra ‘scope’, e depois seu identificador. O método aceita um único parâmetro, o objeto `$query`. Este objeto pode ser utilizado para montar consultas da mesma maneira que nós já vimos nas seções anteriores. Em nosso exemplo, nós utilizamos a declaração `return` para retornar o valor do nosso método `where()`. O método `where()` possui a mesma forma que nós já vimos anteriormente.

Agora vamos voltar ao nosso arquivo de rotas. Vamos alterar nossa consulta existente. Este é o novo código.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return Album::triplePeriod()->get();
8  });
```

Nós alteramos nosso método `where()` para a chamada do escopo `triplePeriod()`. Depois nós simplesmente chamamos o método `get()` para buscar os registros na base de dados. Note que a parte `scope` do nome do método não é incluída, e não se esqueça que você deve remover esta parte na chamada dos seus métodos! Vamos ver o resultado.

```
1  [  
2    {  
3      id: 5,  
4      title: "...Anywhere But Here",  
5      artist: "The Ataris",  
6      genre: "Punk Rock",  
7      year: 1997  
8    },  
9    {  
10     id: 6,  
11     title: "...Is A Real Boy",  
12     artist: "Say Anything",  
13     genre: "Indie Rock",  
14     year: 2006  
15   }  
16 ]
```

Ótimo, este é o resultado que nós estávamos esperando. Utilize quantos escopos você precisar para evitar repetições.

Coleções do Eloquent

Eu adoro coleções. Eu tenho muitas. Quando criança eu colecionava brinquedos dos transformers. Agora adulto, eu coleciono vídeo games e mangas. Coleções para nerds são as melhores.

O Laravel também possui suas próprias coleções. Uma coleção de fãs, ansiosos para ajudar no desenvolvimento da comunidade. Uma coleção de ótimos desenvolvedores contribuindo para o framework. Uma coleção de histórias de onde o nome surgiu, e a maioria delas são falsas. E também possui as coleções do Eloquent.

A Classe de Coleções

As coleções do Eloquent são uma extensão da classe `Collection` do Laravel, com alguns métodos adicionais para manipular os resultados das consultas. A classe `Collection` é apenas uma ‘embalagem’ para um array de objetos, mas também possui alguns métodos interessantes para ajudá-lo a buscar os itens do array.

No Laravel 3, um array de instâncias era retornado de qualquer método de consulta que fosse utilizado para buscar resultados com mais de uma instância. Porém, no Laravel 4, você recebe uma `Collection` (Coleção) de instâncias. Mas não se preocupe, você ainda pode percorrer as instâncias de uma coleção com as funções de loop que o PHP oferece. A classe `Collection` herda algumas propriedades de um array, e também possui alguns métodos disponíveis.

Vamos dar uma olhada nos métodos disponíveis na classe `Collection`. Para o nosso exemplo, nós vamos utilizar a tabela `albums` do capítulo anterior, e assumir que a coleção é o resultado da chamada do método `Album::all()`, como no exemplo abaixo:

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     $collection = Album::all();
8 });
```

Métodos da Coleção

Vamos dar uma olhada nos métodos disponíveis na classe `Collection`. Alguns destes métodos são relacionados a inserir e buscar elementos pelas suas chaves. Porém, ao falar de uma coleção de

modelos do Eloquent, as chaves do array não são as mesmas que as chaves primárias da tabela, e é aqui que alguns dos métodos desta classe vão ser muito úteis para nós.

All

O método `all()` pode ser utilizado para buscar o array interno utilizado pelo objeto `Collection`. Isto significa que se você quiser que os seus resultados sejam idênticos aos retornados pelo Laravel 3, então apenas chame o método `all()` que você vai ter um array de instâncias.

Vamos utilizar a função `var_dump()` no resultado.

Aqui está o nosso código.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     $collection = Album::all();
8     var_dump($collection->all());
9 });
```

E aqui está o resultado.

```
1 array (size=6)
2     0 =>
3         object(Album)[127]
4             public 'timestamps' => boolean false
5             protected 'connection' => null
6             protected 'table' => null
7             protected 'primaryKey' => string 'id' (length=2)
8             protected 'perPage' => int 15
9             public 'incrementing' => boolean true
10            protected 'attributes' =>
11                array (size=5)
12                    'id' => int 1
13                    'title' => string 'Some Mad Hope' (length=13)
14                    'artist' => string 'Matt Nathanson' (length=14)
15                    'genre' => string 'Acoustic Rock' (length=13)
16                    'year' => int 2007
17            protected 'original' =>
18                array (size=5)
```

```

19         'id' => int 1
20         'title' => string 'Some Mad Hope' (length=13)
21         'artist' => string 'Matt Nathanson' (length=14)
22         'genre' => string 'Acoustic Rock' (length=13)
23         'year' => int 2007
24     protected 'relations' =>
25         array (size=0)
26         empty
27     protected 'hidden' =>
28         array (size=0)
29         empty
30     protected 'visible' =>
31         array (size=0)
32         empty
33     protected 'fillable' =>
34         array (size=0)
35         empty
36     protected 'guarded' =>
37         array (size=1)
38         0 => string '*' (length=1)
39     protected 'touches' =>
40         array (size=0)
41         empty
42     protected 'with' =>
43         array (size=0)
44         empty
45     public 'exists' => boolean true
46     protected 'softDelete' => boolean false
47 1 =>
48     object(Album)[128]
49     ... TONNES MORE INFO ...

```

Como você pode ver, nós temos um array de instâncias do nosso modelo Eloquent.

First

O método `first()` da coleção pode ser usado para buscar o primeiro elemento. Ele vai ser o primeiro elemento contido dentro do array interno da classe `Collection`.

Vamos testar.


```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     $collection = Album::all();
8     var_dump($collection->first());
9 });
```

Agora nós vamos visitar a URI / para ver o resultado. O que você espera que seja o resultado?

```
1 object(Album)[127]
2   public 'timestamps' => boolean false
3   protected 'connection' => null
4   protected 'table' => null
5   protected 'primaryKey' => string 'id' (length=2)
6   protected 'perPage' => int 15
7   public 'incrementing' => boolean true
8   protected 'attributes' =>
9     array (size=5)
10       'id' => int 1
11       'title' => string 'Some Mad Hope' (length=13)
12       'artist' => string 'Matt Nathanson' (length=14)
13       'genre' => string 'Acoustic Rock' (length=13)
14       'year' => int 2007
15   protected 'original' =>
16     array (size=5)
17       'id' => int 1
18       'title' => string 'Some Mad Hope' (length=13)
19       'artist' => string 'Matt Nathanson' (length=14)
20       'genre' => string 'Acoustic Rock' (length=13)
21       'year' => int 2007
22   protected 'relations' =>
23     array (size=0)
24       empty
25   protected 'hidden' =>
26     array (size=0)
27       empty
28   protected 'visible' =>
29     array (size=0)
30       empty
```

```
31     protected 'fillable' =>
32         array (size=0)
33             empty
34     protected 'guarded' =>
35         array (size=1)
36             0 => string '*' (length=1)
37     protected 'touches' =>
38         array (size=0)
39             empty
40     protected 'with' =>
41         array (size=0)
42             empty
43     public 'exists' => boolean true
44     protected 'softDelete' => boolean false
```

Certo! Uma única instância representando um dos nossos álbuns. O primeiro registro que nós inserimos na tabela. Note que é apenas o primeiro resultado porque nós utilizamos o método `all()` como parte de consulta. Se nós utilizássemos uma consulta diferente, então o array de resultados teria um ordem diferente e a chamada ao método `first()` teria um resultado diferente.

Last

Este deve ser meio óbvio. Se o método `first()` foi utilizado para retornar o primeiro modelo da nossa coleção, então o método `last()` deve retornar o último modelo da coleção.

Vamos provar nossa teoria.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $collection = Album::all();
8      var_dump($collection->last());
9  });
```

Aqui está o resultado da URI `/`.

```
1 object(Album)[138]
2   public 'timestamps' => boolean false
3   protected 'connection' => null
4   protected 'table' => null
5   protected 'primaryKey' => string 'id' (length=2)
6   protected 'perPage' => int 15
7   public 'incrementing' => boolean true
8   protected 'attributes' =>
9     array (size=5)
10      'id' => int 6
11      'title' => string '...Is A Real Boy' (length=16)
12      'artist' => string 'Say Anything' (length=12)
13      'genre' => string 'Indie Rock' (length=10)
14      'year' => int 2006
15   protected 'original' =>
16     array (size=5)
17      'id' => int 6
18      'title' => string '...Is A Real Boy' (length=16)
19      'artist' => string 'Say Anything' (length=12)
20      'genre' => string 'Indie Rock' (length=10)
21      'year' => int 2006
22   protected 'relations' =>
23     array (size=0)
24     empty
25   protected 'hidden' =>
26     array (size=0)
27     empty
28   protected 'visible' =>
29     array (size=0)
30     empty
31   protected 'fillable' =>
32     array (size=0)
33     empty
34   protected 'guarded' =>
35     array (size=1)
36     0 => string '*' (length=1)
37   protected 'touches' =>
38     array (size=0)
39     empty
40   protected 'with' =>
41     array (size=0)
42     empty
```

```

43     public 'exists' => boolean true
44     protected 'softDelete' => boolean false

```

Ótimo, este é o último álbum contido dentro do array interno da coleção. Também é o último registro da tabela, mas isto depende da ordenação que você utilizar em suas consultas.

Shift

O método `shift()` é similar ao método `first()`. Ele vai retornar o primeiro modelo contido dentro do array interno da coleção. Porém, ao contrário do método `first()`, o método `shift()` também vai remover o modelo do array. Vamos provar isto com um pequeno teste.

```

1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $collection = Album::all();
8      var_dump(count($collection));
9      var_dump($collection->shift());
10     var_dump(count($collection));
11 });

```

O nosso teste vai exibir um número de elementos dentro da coleção utilizando o método `count()` do PHP antes de nós utilizarmos o método `shift()`. Lembre que a coleção herda muitas propriedades de um array, e por isto que podemos utilizar o método `count()` diretamente nela.

Vamos dar uma olhada no resultado do nosso teste.

```

1  int 6
2  object(Album)[127]
3      public 'timestamps' => boolean false
4      protected 'connection' => null
5      protected 'table' => null
6      protected 'primaryKey' => string 'id' (length=2)
7      protected 'perPage' => int 15
8      public 'incrementing' => boolean true
9      protected 'attributes' =>
10         array (size=5)
11             'id' => int 1
12             'title' => string 'Some Mad Hope' (length=13)

```

```
13     'artist' => string 'Matt Nathanson' (length=14)
14     'genre' => string 'Acoustic Rock' (length=13)
15     'year' => int 2007
16     protected 'original' =>
17         array (size=5)
18             'id' => int 1
19             'title' => string 'Some Mad Hope' (length=13)
20             'artist' => string 'Matt Nathanson' (length=14)
21             'genre' => string 'Acoustic Rock' (length=13)
22             'year' => int 2007
23     protected 'relations' =>
24         array (size=0)
25             empty
26     protected 'hidden' =>
27         array (size=0)
28             empty
29     protected 'visible' =>
30         array (size=0)
31             empty
32     protected 'fillable' =>
33         array (size=0)
34             empty
35     protected 'guarded' =>
36         array (size=1)
37             0 => string '*' (length=1)
38     protected 'touches' =>
39         array (size=0)
40             empty
41     protected 'with' =>
42         array (size=0)
43             empty
44     public 'exists' => boolean true
45     protected 'softDelete' => boolean false
46     int 5
```

Como você pode ver, nós recebemos a nossa instância do Album, da mesma maneira de quando nós utilizamos o método `first()`. Porém, se você olhar para o segundo número inteiro, você vai perceber que o tamanho do nosso array diminuiu. Isto porque a instância não apenas foi retornada pelo método, como também foi removida do array.

Pop

Pop é um gênero de música que existe há décadas, e é normalmente utilizado para encorajar o consumo de álcool ou os sonhos selvagens de adolescentes.

Ah sim, também é um método da coleção de instâncias do Eloquent. Funciona de uma maneira similar ao método `shift()`, onde vai retornar o último modelo do array interno, e também removê-lo. Vamos provar isto com um teste.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $collection = Album::all();
8      var_dump(count($collection));
9      var_dump($collection->pop());
10     var_dump(count($collection));
11 });
```

Aqui está o resultado.

```
1  int 6
2  object(Album)[138]
3      public 'timestamps' => boolean false
4      protected 'connection' => null
5      protected 'table' => null
6      protected 'primaryKey' => string 'id' (length=2)
7      protected 'perPage' => int 15
8      public 'incrementing' => boolean true
9      protected 'attributes' =>
10         array (size=5)
11             'id' => int 6
12             'title' => string '...Is A Real Boy' (length=16)
13             'artist' => string 'Say Anything' (length=12)
14             'genre' => string 'Indie Rock' (length=10)
15             'year' => int 2006
16     protected 'original' =>
17         array (size=5)
18             'id' => int 6
19             'title' => string '...Is A Real Boy' (length=16)
20             'artist' => string 'Say Anything' (length=12)
```

```
21     'genre' => string 'Indie Rock' (length=10)
22     'year' => int 2006
23     protected 'relations' =>
24         array (size=0)
25             empty
26     protected 'hidden' =>
27         array (size=0)
28             empty
29     protected 'visible' =>
30         array (size=0)
31             empty
32     protected 'fillable' =>
33         array (size=0)
34             empty
35     protected 'guarded' =>
36         array (size=1)
37         0 => string '*' (length=1)
38     protected 'touches' =>
39         array (size=0)
40             empty
41     protected 'with' =>
42         array (size=0)
43             empty
44     public 'exists' => boolean true
45     protected 'softDelete' => boolean false
46     int 5
```

Nós recebemos o último elemento do array, e o resultado dos nossos métodos `count()` mostram que o tamanho do array diminuiu. Isto por causa que o valor final foi recebido e removido do array interno.

Each

Se você já utilizou a biblioteca Underscore, para Javascript ou PHP, você vai estar familiarizado com os próximos métodos. Ao invés de criar um loop `foreach()` para iterar os nossos resultados, nós podemos passar uma Closure para o método `each()`.

Isto é melhor descrito com um exemplo.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $collection = Album::all();
8      $collection->each(function($album)
9      {
10         var_dump($album->title);
11     });
12 });
```

Nossa Closure aceita um parâmetro que vai ser o objeto atual dentro da iteração. Esta Closure é passada para o método `each()`. Para cada iteração, nós vamos chamar a função `var_dump()` no título do `$album`.

Vamos examinar o resultado.

```
1  string 'Some Mad Hope' (length=13)
2  string 'Please' (length=6)
3  string 'Leaving Through The Window' (length=26)
4  string 'North' (length=5)
5  string '...Anywhere But Here' (length=20)
6  string '...Is A Real Boy' (length=16)
```

Brilhante! Aqui estão os nossos títulos dos álbuns, como esperado.

Map

O método `map()` funciona de uma maneira similar ao método `each()`. Porém, pode ser utilizado para alterar os elementos da coleção, e ao final é retornada uma nova coleção de resultados.

Vamos imaginar que nós queremos prefixar todos os títulos dos álbuns com `An ode to a fair panda:` . Nós podemos fazer isto com o método `map()`.


```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $collection = Album::all();
8
9      $new = $collection->map(function($album)
10     {
11         return 'An ode to a fair panda: '.$album->title;
12     });
13
14     var_dump($new);
15 });
```

Primeiro nós setamos o retorno do método `Collection::map()` em uma variável. Então, nós iteramos pela coleção de resultados, da mesma maneira que o método `each()`, mas desta vez nós retornamos cada valor que nós desejamos que esteja presente dentro da nossa nova coleção.

Aqui está o resultado.

```
1  object(Illuminate\Database\Eloquent\Collection)[117]
2      protected 'items' =>
3          array (size=6)
4              0 => string 'An ode to a fair panda: Some Mad Hope' (length=37)
5              1 => string 'An ode to a fair panda: Please' (length=30)
6              2 => string 'An ode to a fair panda: Leaving Through The Window' (length=50)
7              3 => string 'An ode to a fair panda: North' (length=29)
8              4 => string 'An ode to a fair panda: ...Anywhere But Here' (length=44)
9              5 => string 'An ode to a fair panda: ...Is A Real Boy' (length=40)
```

Agora nós temos uma coleção de strings que nós construímos utilizando nosso método iterativo `map()`.

Filter

O método `filter()` pode ser usado para reduzir o número de elementos contidos dentro da nossa coleção utilizando uma Closure. Se o resultado retornado da Closure for `true`, então o modelo que está sendo iterado vai estar na nova coleção. Se a Closure retornar `false` ou não retornar nada, o elemento não vai estar presente na nova coleção.

Isto é mais fácil de se entender com um exemplo.

```

1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $collection = Album::all();
8
9      $new = $collection->filter(function($album)
10     {
11         if ($album->artist == 'Something Corporate') {
12             return true;
13         }
14     });
15
16     var_dump($new);
17 });

```

Nós iteramos a coleção com o método `filter()` e com a nossa Closure. Para cada iteração, se o valor da coluna `artist` for igual a 'Say Anything', nós vamos retornar `true`. Isto indica que a instância que está sendo iterada deve estar presente na nossa nova coleção.

Aqui está o resultado.

```

1  object(Illuminate\Database\Eloquent\Collection)[117]
2      protected 'items' =>
3          array (size=2)
4              2 =>
5                  object(Album)[135]
6                      public 'timestamps' => boolean false
7                      protected 'connection' => null
8                      protected 'table' => null
9                      protected 'primaryKey' => string 'id' (length=2)
10                     protected 'perPage' => int 15
11                     public 'incrementing' => boolean true
12                     protected 'attributes' =>
13                         array (size=5)
14                             'id' => int 3
15                             'title' => string 'Leaving Through The Window' (length=26)
16                             'artist' => string 'Something Corporate' (length=19)
17                             'genre' => string 'Piano Rock' (length=10)
18                             'year' => int 2002
19                     protected 'original' =>

```

```

20         array (size=5)
21             'id' => int 3
22             'title' => string 'Leaving Through The Window' (length=26)
23             'artist' => string 'Something Corporate' (length=19)
24             'genre' => string 'Piano Rock' (length=10)
25             'year' => int 2002
26     3 =>
27     object(Album)[136]
28         public 'timestamps' => boolean false
29         protected 'connection' => null
30         protected 'table' => null
31         protected 'primaryKey' => string 'id' (length=2)
32         protected 'perPage' => int 15
33         public 'incrementing' => boolean true
34         protected 'attributes' =>
35             array (size=5)
36                 'id' => int 4
37                 'title' => string 'North' (length=5)
38                 'artist' => string 'Something Corporate' (length=19)
39                 'genre' => string 'Piano Rock' (length=10)
40                 'year' => int 2002
41         protected 'original' =>
42             array (size=5)
43                 'id' => int 4
44                 'title' => string 'North' (length=5)
45                 'artist' => string 'Something Corporate' (length=19)
46                 'genre' => string 'Piano Rock' (length=10)
47                 'year' => int 2002

```

Eu reduzi os resultados um pouco para poupar espaço, mas você pode claramente ver que nós temos dois álbuns do artista ‘Something Corporate’.

Sort

O método `sort()` pode ser utilizado para ordenar a coleção. Ele faz a nossa Closure ficar parecida com o método `uasort()` do PHP, que utiliza valores inteiros para representar uma comparação entre dois valores. Nossa Closure var receber dois parâmetros, vamos chamá-los de A e B. Quando ela receber, nós vamos aplicar nossas regras de ordenação para retornar um resultado inteiro a partir da nossa Closure.

Se $A > B$ então nós retornamos 1. Se $A < B$ então nós retornamos -1. Se $A = B$ então nós retornamos 0.

Abaixo um exemplo.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $collection = Album::all();
8
9      $collection->sort(function($a, $b)
10     {
11         $a = $a->year;
12         $b = $b->year;
13         if ($a === $b) {
14             return 0;
15         }
16         return ($a > $b) ? 1 : -1;
17     });
18
19     $collection->each(function($album)
20     {
21         var_dump($album->year);
22     });
23 });
```

Nós passamos uma Closure com dois parâmetros, que vão representar dois álbuns da nossa coleção. Primeiros nós atribuímos o ano (*year*) dos álbuns que estão sendo iterados para as variáveis *\$a* e *\$b*, para simplificar a comparação. Se *\$a* for igual a *\$b*, nós retornamos 0. Se *\$a* for maior que *\$b*, nós retornamos 1, se não, retornamos -1.

Este método é destrutivo, pois ele altera a coleção original. Nós iteramos pelos resultados fazendo um `var_dump()` no valor dos anos (*year*) para exibir a ordenação.

```
1  int 1993
2  int 1997
3  int 2002
4  int 2002
5  int 2006
6  int 2007
```

Como você pode ver, nossos álbuns agora estão ordenados pelo ano, em ordem ascendente. E aqui está uma lição de casa para você. Tente modificar o exemplo acima, alterando apenas um caracter para que os álbuns sejam ordenados em ordem decrescente. Eu prometo que isto pode ser feito!

Reverse

O método `reverse()` pode ser usado para reverter a ordem dos modelos contidos dentro do array interno da coleção. Precisa mesmo de um exemplo? Ok então, você é um bom leitor afinal de contas...

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $collection = Album::all();
8
9      $collection->each(function($album)
10     {
11         var_dump($album->title);
12     });
13
14     $reverse = $collection->reverse();
15
16     $reverse->each(function($album)
17     {
18         var_dump($album->title);
19     });
20 });
```

Primeiro nós iteramos por todos os álbuns exibindo seus títulos. Depois, nós revertemos a coleção, atribuindo a nova coleção para a variável `$reverse`. Então nós iteramos a coleção `$reverse` para verificar o que mudou.

E aqui está o resultado.

```
1  string 'Some Mad Hope' (length=13)
2  string 'Please' (length=6)
3  string 'Leaving Through The Window' (length=26)
4  string 'North' (length=5)
5  string '...Anywhere But Here' (length=20)
6  string '...Is A Real Boy' (length=16)
7
8
9  string '...Is A Real Boy' (length=16)
10 string '...Anywhere But Here' (length=20)
11 string 'North' (length=5)
```

```
12 string 'Leaving Through The Window' (length=26)
13 string 'Please' (length=6)
14 string 'Some Mad Hope' (length=13)
```

Ótimo! Nossa coleção foi revertida.

Merge

O método `merge()` pode ser usado para combinar coleções. O único parâmetro para este método é a coleção que deve ser combinada. Abaixo um exemplo.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $a = Album::where('artist', '=', 'Something Corporate')
8          ->get();
9      $b = Album::where('artist', '=', 'Matt Nathanson')
10         ->get();
11
12     $result = $a->merge($b);
13
14     $result->each(function($album)
15     {
16         echo $album->title.'<br />';
17     });
18 });
```

No exemplo acima, nós executamos duas consultas. A coleção `$a` é uma coleção dos álbuns do 'Something Corporate'. A coleção `$b` contém álbuns do 'Matt Nathanson'. Nós utilizamos o método `merge()` para combinar as coleções, e atribuímos o resultado para uma nova coleção nomeada `$result`. Então nós escrevemos os títulos dos álbuns dentro do loop `each()`.

E aqui estão os resultados.

```
1 Leaving Through The Window
2 North
3 Some Mad Hope
4 Please
```

Slice

O método `slice()` é equivalente a função PHP `slice()`. Pode ser utilizada para buscar um grupo específico de modelos utilizando a posição dos modelos. Confuso? Dê uma olhada nisto.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $collection = Album::all();
8
9      $sliced = $collection->slice(2, 4);
10
11     $sliced->each(function($album)
12     {
13         echo $album->title.'<br />';
14     });
15 });
```

Nós criamos uma nova coleção utilizando o método `slice()`. O primeiro parâmetro é a posição que a nova coleção deve começar. Neste caso, nós estamos dizendo para começar a partir do segundo elemento do array. O segundo (e opcional) parâmetro é o tamanho da coleção. Nós passamos para o método `slice()` que nós queremos quatro elementos, que venham após o segundo elemento do array.

Vamos dar uma olhada no resultado.

```
1  Leaving Through The Window
2  North
3  ...Anywhere But Here
4  ...Is A Real Boy
```

Você sabia que nós podemos passar valores negativos para o `slice()`? Por exemplo...

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $collection = Album::all();
8
9      $sliced = $collection->slice(-2, 4);
10
11     $sliced->each(function($album)
12     {
13         echo $album->title.'<br />';
14     });
15 });
```

Passando -2 como o primeiro parâmetro, nós estamos dizendo ao método para montar a coleção começando pelos dois últimos elementos da coleção. Aqui está o resultado.

```
1  ...Anywhere But Here
2  ...Is A Real Boy
```

Espera, nós não pedimos para buscar quatro modelos?

Sim, porém como nós solicitamos para começar a partir dos dois últimos modelos do array, apenas dois modelos estavam disponíveis. O método `slice()` não ‘da a volta’ na coleção para completar os resultados.

IsEmpty

O método `isEmpty()` pode ser utilizado para verificar se a coleção possui elementos. Eu aposto que você não imaginava isto! Ele não aceita nenhum parâmetro, e retorna um resultado booleano. Aqui está um exemplo.


```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      // This query will return items.
8      $a = Album::all();
9
10     // This query won't.
11     $b = Album::where('title', '=', 'foo')->get();
12
13     var_dump($a->isEmpty());
14     var_dump($b->isEmpty());
15 });
```

Nós sabemos que a primeira consulta vai retornar resultados, e a segunda não. Vamos verificar os resultados utilizando o método `isEmpty()` para ver o que ele nos retorna.

```
1  boolean false
2  boolean true
```

O primeiro `isEmpty()` retorna `false` porque a coleção possui elementos. A segunda coleção está vazia, e por isto o método `isEmpty()` retorna `true`.

ToArray

O método `toArray()` pode ser utilizado para retornar o array interno da coleção. Também, qualquer elemento dentro do array que pode ser transformado em um array, por exemplo os objetos do Eloquent, serão transformados no processo.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $collection = Album::all();
8      var_dump( $collection->toArray() );
9  });
```

Aqui está o resultado.

```
1 array (size=6)
2   0 =>
3     array (size=5)
4       'id' => int 1
5       'title' => string 'Some Mad Hope' (length=13)
6       'artist' => string 'Matt Nathanson' (length=14)
7       'genre' => string 'Acoustic Rock' (length=13)
8       'year' => int 2007
9   1 =>
10    array (size=5)
11      'id' => int 2
12      'title' => string 'Please' (length=6)
13      'artist' => string 'Matt Nathanson' (length=14)
14      'genre' => string 'Acoustic Rock' (length=13)
15      'year' => int 1993
16   2 =>
17    array (size=5)
18      'id' => int 3
19      'title' => string 'Leaving Through The Window' (length=26)
20      'artist' => string 'Something Corporate' (length=19)
21      'genre' => string 'Piano Rock' (length=10)
22      'year' => int 2002
23    ... and more ...
```

Como você pode ver, não apenas um array representando a coleção foi retornado, mas as instâncias dos modelos também foram transformadas em arrays.

Tojson

O método `toJson()` vai transformar a coleção em uma string JSON que pode ser utilizada para representar seus conteúdos. No capítulo anterior, nós vimos como retornar coleções diretamente das rotas e das ações de controllers para servir uma resposta JSON. O método `toString()` nos permite que a coleção seja transformada em uma string JSON, chamando o método `toJson()` internamente.

Vamos dar uma olhada no exemplo.

```

1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $collection = Album::all();
8      var_dump( $collection->toJson() );
9  });

```

E aqui está o resultado.

```

1  string ' [{"id":1,"title":"Some Mad Hope","artist":"Matt Nathanson","genre":"Acous\
2  tic Rock","year":2007},{ "id":2,"title":"Please","artist":"Matt Nathanson","genre"\
3  : "Acoustic Rock","year":1993},{ "id":3,"title":"Leaving Through The Window","artis\
4  t":"Something Corporate","genre":"Piano Rock","year":2002},{ "id":4,"title":"North\
5  ","artist":"Something Corporate","genre":"Piano Rock","year":2002},{ "id":5,"title\
6  ":"...Anywhere But Here","artist":"The Ataris","genre":"Punk Rock","year":1997},{ \
7  "id":6,"title":"...Is A Real Boy","artist":"Say Anything","genre":"Indie Rock","y\
8  ear":2006}] ' (length=570)

```

É a nossa coleção representada por uma string JSON.

Count

Em um exemplo anterior eu utilizei o método PHP `count()` para contar o número de modelos contidos dentro do array interno da coleção. Que esquecido que eu fui! Eu tinha me esquecido completamente do método `count()` da coleção. Na verdade, faz a mesma coisa. Vou te mostrar.

```

1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $collection = Album::all();
8      var_dump( $collection->count() );
9  });

```

E aqui está o resultado. Aposto que você não sabia!

```
1  int 6
```

Então se o resultado é o mesmo, qual nós devemos utilizar? O que melhor se encaixar com seu estilo. Eu vou deixar a decisão para você. Você já leu mais de 300 páginas deste livro, e já sabe muita coisa. Muito bem!

Melhores Práticas

Alguns dos métodos disponíveis na coleção são iguais aos métodos disponíveis no Query Builder. Por exemplo, nós podemos utilizar o método `first()` utilizando consultas do Eloquent para buscar um único modelo. Nós também podemos utilizar o método `first()` da coleção para buscar o primeiro elemento. Dê uma olhada no exemplo abaixo.

```
1  Album::all()->first();
2  Album::first();
```

Estas duas linhas acima retornam exatamente o mesmo resultado. O primeiro item da tabela de `albums` será retornado. Então qual nós devemos utilizar?

Como sempre, a resposta é ‘depende’, Sim, eu sei que ninguém gosta de ouvir esta resposta, mas as vezes é verdade. Vamos dar uma olhada em dois diferentes cenários.

No primeiro cenário, nós desejamos exibir o título do primeiro álbum armazenado dentro da base de dados. Claro, sem problemas. Nós podemos utilizar o método `first()` do Eloquent.

```
Album::first();
```

No segundo cenário, nós desejamos exibir todos os títulos dos álbuns, mas também exibir o título do primeiro álbum dentro de uma pequena caixa. Por exemplo, a seção ‘álbum do ano’. Bem, nós poderíamos fazer algo assim.

```
1  $allAlbums = Album::all();
2  $albumOfTheYear = Album::first();
```

Com certeza o exemplo acima faria o que nós precisamos, mas ele faria duas consultas no banco de dados. Aumentar o número de consultas ao banco de dados é uma maneira de diminuir a performance da sua aplicação. Pode não importar em nossos exemplos, mas dentro da aplicação de sua empresa qualquer segundo desperdiçado é dinheiro perdido.

Nós podemos aliviar um pouco do estresse do banco de dados, e reduzir o número de consultas passando uma das responsabilidades de consulta para a coleção.

```
1 $allAlbums = Album::all();  
2 $albumOfTheYear = $allAlbums->first();
```

Nós buscamos todos os álbuns, porém desta vez nós utilizamos o método `first()` na coleção para buscar o álbum do ano. Este exemplo executa apenas uma consulta no banco de dados.

Escolher entre executar uma consulta na coleção ou na base de dados é uma questão de escolha. Consultas no banco de dados são mais complexas e exigem um uso de memória / CPU do servidor maior do que quando utilizadas diretamente na coleção. Novamente, utilizar consultas diretamente nas coleções pode aumentar a performance de sua aplicação.

Utilize o seu julgamento para decidir quando utilizar qualquer um dos métodos das coleções. Eu confio em você!

Relacionamentos do Eloquent

Era uma noite fria e úmida na cidade de Eloquent. A chuva estava caindo na janela do escritório do Zack como lágrimas sem esperança, andando nas ruas da cidade. A cidade de Eloquent já foi um lugar pacífico, sem corrupção. A paz se transformou em guerras de gangues, contrabando e outros crimes. A polícia local foi comprada há muito tempo atrás, e agora fecha os olhos quando vê violência nas ruas.

Zack Kitzmiller já fez parte da polícia da cidade de Eloquent, mas como o último homem honesto naquela cidade corrupta, ele foi forçado a deixar o seu emprego e tentar limpar a cidade da sua maneira. Ele virou um investigador particular.

Ele sentou em sua velha cadeira, perto da janela de seu escritório no 14º andar com uma garrafa de whisky e um cigarro barato. O escritório estava acabado, e assim como Zack, não era limpo já fazia um bom tempo. Porém, ele era barato, e o dinheiro não estava exatamente batendo na sua porta.

Knock knock.

Ou estava? Uma loira linda e alta parou na porta. Zack olhou para o rosto da mulher angelical.

“Você é um investigador?” perguntou a misteriosa loira.

“Meu nome é Zack Kitzmiller, mas você pode me chamar como você quiser” disse Zack, com um sorriso em seu rosto coberto.

“Senhor Kitzmiller, meu nome é Pivoté Tableux, e eu estou procurando o homem que matou o meu marido. A culpa do seu assassinato **pertence ao** Enrico Barnez.”

“Enrico Barnez é o maior vendedor de drogas da cidade, ele mantém tudo muito bem escondido. Encontrá-lo não será uma tarefa fácil.” disse Zack. “Isto vai custar caro, você tem certeza que tem o dinheiro para isto?”

“Senhor Kitzmiller, dinheiro não é um problema. Eu **tenho muitos** fundos.” disse Pivoté.

“Isto é tudo que é preciso ouvir.” disse Zack com um sorriso frio.

Zack colocou seu chapéu e seu longo casaco, e se despediu da loira.

Zack atravessou a entrada principal das docas. Partes quebradas de navios e veículos irreconhecíveis estavam por toda a estrada.

“Você é Messy Stinkman?” disse Zack para um homem que trabalhava nas docas.

“Talvez eu seja, talvez não... depende de quem está perguntando.”

“Eu estou procurando pelo seu chefe. Enrico Barnez. Onde eu posso encontrá-lo?” disse Zack.

“Olhe, o chefe **possui muitos** amigos perigosos. Eu vou estar em perigo se eu te falar onde ele está. Está vendo este armazém? Ele **pertence ao** chefe. Isto é tudo que eu sei. Desculpe, mas não posso te ajudar mais que isso.”

Zack decidiu que ele não iria conseguir mais informações sobre o senhor Stinkman sem alguma inspiração. Ele pegou um cano de descarga e foi de encontro ao trabalhador. ***Zack Kitmiller é fodão.***

“Certo, eu falo. Você quer saber onde o Enrico está? Dê meia volta.” disse o trabalhador, com medo.

Zack se virou devagar, e deu de cara com Enrico Barnez. Enrico é um homem poderoso. Muito poderoso e muito baixinho.

“Você está procurando por mim?”

Zack sabia o quão perigoso era o homem que estava em sua frente. Ele decidiu não responder, e tirou uma arma de seu casaco, e apontou para a cabeça de Enrico.

Mas ele não foi rápido o suficiente, Enrico também puxou sua arma, e os dois estavam em um impasse. Os dois deram um longo suspiro. Uma arma foi disparada e um corpo caiu ao chão.

A vida não é só tortas e doces.

Introdução aos Relacionamentos

Nos capítulos anteriores nós vimos como representar os registros armazenados em nossa base de dados com objetos. As instâncias de classes representavam os registros. Isto significa que nós deixamos nossos objetos em sua forma mais simples. Book, Fruit, BoyBand, o que quer que seja que nós precisarmos.

Como estes objetos são simples, se nós quisermos armazenar dados relacionados a eles nós vamos precisar criar um novo objeto e criar uma relação entre eles. O que eu quero dizer com relação? Bem, não é o tipo com beijos e abraços. Este tipo de relação é melhor explicado com um exemplo.

Vamos pegar o nosso modelo ‘Book’ do capítulo anterior. Se nós pensarmos sobre livros por um momento, nós vamos chegar a conclusão de que eles foram escritos por alguém. Eu sei que você gosta de pensar que eu não existo, mas a verdade é que eu existo. Em algum lugar existe um britânico meio louco que é fanático pelo Laravel. E também existem outros autores.

Então nós sabemos que um livro **pertence a** um autor. Este é o nosso primeiro relacionamento. O autor possui uma conexão com o livro. Em um sentido, o autor identifica o livro. E ele não é necessariamente uma propriedade do livro, como o título por exemplo. O autor possui seus próprios atributos. Um nome, uma cidade, pizza favorita. Ele merece seu próprio modelo. O modelo ‘Author’.

Então como nós relacionamos estes dois modelos? Em bancos de dados relacionais, nós podemos utilizar chaves estrangeiras para identificar estas relações. Normalmente são colunas do tipo inteiro.

Vamos montar dois exemplos.

books

1	+-----+-----+-----+-----+
2	id (PK) name
3	+-----+-----+-----+-----+
4	1 Code Sexy
5	2 Code Dutch
6	3 Code Bright
7	+-----+-----+-----+-----+

Aqui está a nossa tabela 'books' com três livros dentro. Perceba que cada tabela possui uma chave primária. Ela pode ser usada para identificar individualmente cada um dos registros. Agora vamos dar uma olhada na segunda tabela.

authors

1	+-----+-----+-----+-----+
2	id (PK) name
3	+-----+-----+-----+-----+
4	1 Dayle Rees
5	2 Matthew Machuga
6	3 Shawn McCool
7	+-----+-----+-----+-----+

Aqui nós temos um outra tabela contendo os nomes dos três mais fantásticos desenvolvedores. Nós vamos chamá-la de 'authors'. Perceba como cada um dos registros possui uma chave primária para poderem ser identificados.

Certo, vamos criar uma relação entre as duas tabelas. Esta é uma relação entre um livro e um autor, nós vamos ignorar múltiplos autores por enquanto e assumir que o livro possui apenas um autor. Vamos adicionar uma nova coluna em nossa tabela.

books

1	+-----+-----+-----+			
2	id (PK)	name	author_id (FK)	
3	+-----+-----+-----+			
4	1	Code Sexy	2	
5	2	Code Dutch	3	
6	3	Code Bright	1	
7	+-----+-----+-----+			

Nós adicionamos nossa primeira chave estrangeira. O campo `author_id`. Novamente, ele é um campo inteiro, mas não é usado para identificar os registros desta tabela, mas sim para identificar registros relacionados pertencentes a outra tabela. Uma chave estrangeira normalmente é utilizada para referenciar uma chave primária de uma outra tabela, porém em algumas circunstâncias ela pode ser utilizada para referenciar outros campos também.

Você percebeu que a adição a chave estrangeira criou uma relação entre os livros e os autores? Nossa chave estrangeira `author_id` referencia a chave primária dentro da tabela de autores. Dê uma olhada na terceira coluna da tabela. Code Bright possui o `author_id` como 1. Agora olhe o registro com chave primária 1 dentro da tabela de autores, e você vai ver Dayle Rees. Isto significa que Code Bright foi escrito por Dayle Rees. Simples assim.

Porquê nós não colocamos uma chave estrangeira dentro da tabela de autores?

Bem, um autor pode ter muitos livros, não? Por exemplo, dê uma olhada nesta relação.

authors

1	+-----+-----+-----+			
2	id (PK)	name		
3	+-----+-----+-----+			
4	1	Dayle Rees		
5	2	Matthew Machuga		
6	3	Shawn McCool		
7	+-----+-----+-----+			

books

1	+-----+-----+-----+			
2	id (PK)	name	author_id (FK)	
3	+-----+-----+-----+			
4	1	Code Sexy	2	
5	2	Code Dutch	3	
6	3	Code Bright	1	
7	4	Code Happy	1	
8	+-----+-----+-----+			

Veja como o Dayle Rees, ou seja, eu, perceba como eu... ops, acho que estraguei esta frase. Perceba que existem dois livros que pertencem à mim. Os dois, 'Code Happy' e 'Code Bright' possuem o valor da coluna `author_id` como 1. Isto significa que eles foram escritos pelo Dayle Rees.

Agora vamos tentar expressar o exemplo acima com uma chave estrangeira na tabela de autores.

authors

1	+-----+-----+-----+			
2	id (PK)	name	book_one (FK)	book_two
3	+-----+-----+-----+			
4	1	Dayle Rees	3	4
5	2	Matthew Machuga	1	null
6	3	Shawn McCool	2	null
7	+-----+-----+-----+			

books

1	+-----+-----+-----+			
2	id (PK)	name	author_id (FK)	
3	+-----+-----+-----+			
4	1	Code Sexy	2	
5	2	Code Dutch	3	
6	3	Code Bright	1	
7	4	Code Happy	1	
8	+-----+-----+-----+			

Como você pode ver, nós adicionamos duas chaves estrangeiras na tabela de autores. Como alguns autores não vão ter dois livros, muitas colunas terão o valor `null`. E o que acontece se um autor tiver três livros? Nós não podemos ficar adicionando colunas, ou nossas tabelas vão começar a ficar bagunçadas!

Muito bem, vamos manter a chave estrangeiras na tabela de livros.

Então porquê nós não vemos o nome destes tipos de relacionamentos? Vai ser muito útil quando nós formos implementar estas relações com o Eloquent. Vamos lá.

Nós temos a coluna `id` na tabela de livros, que identifica o autor. Isto significa que ela pertence (**belongs to**, em inglês) ao autor. Este é o nome do nosso primeiro tipo de relacionamento.

Book **belongs_to** Author.

Relacionamentos também possuem variações inversas. Se um livro **pertence** a um autor, então isto significa que um autor possui muitos (**has many**, em inglês) livros. Acabamos de ver o nome de mais um tipo de relação.

Author **has_many** Book

Se o autor tivesse apenas um livro, e a tabela de livros tivesse uma chave primária identificando, nós usaríamos a relação **has_one** (possui um).

Não se esqueça destes três tipos de relações, mas vamos seguir em frente e ver um quarto tipo. Vamos precisar de um novo exemplo. Hmm... que tal um sistema de ‘favoritos’? Onde um usuário (User) pode votar em um livro (Book). Vamos tentar expressar isto utilizando as relações que nós já vimos anteriormente.

User **has_many** Book

Book **has_many** User

A relação *has many* (possui muitos) vai ser o tipo de relação que vai criar os favoritos. Nós não precisamos de uma entidade para representar os favoritos, visto que esta tabela não terá atributos. Quando os dois tipos de relação são **has_many**, (User para Book e Book para User, como vimos acima), então nós vamos precisar implementar um novo tipo de relação. Antes de tudo, ao invés de falar **has_many**, nós vamos falar **belongs_to_many** (pertence a muitos). Desta maneira nós não vamos confundir com os outros tipos de relação. Este novo tipo de relação forma uma relação **many_to_many** (de muitos para muitos), e necessita de uma nova tabela.

Vamos verificar a estrutura das tabelas.

users

1	+-----+-----+-----+-----+
2	id (PK) name
3	+-----+-----+-----+-----+
4	1 Dayle Rees
5	2 Matthew Machuga
6	3 Shawn McCool
7	+-----+-----+-----+-----+

books

```

1  +-----+-----+
2  | id (PK) | name          |
3  +-----+-----+
4  | 1       | Code Sexy     |
5  | 2       | Code Dutch    |
6  | 3       | Code Bright   |
7  | 4       | Code Happy    |
8  +-----+-----+

```

book_user

```

1  +-----+-----+
2  | id  | user_id | book_id |
3  +-----+-----+
4  | 1   | 1       | 2       |
5  | 2   | 1       | 3       |
6  | 3   | 2       | 2       |
7  | 4   | 3       | 2       |
8  +-----+-----+

```

Espere um pouco, o que é a terceira tabela?

Bem notado! Esta será a nossa tabela de junção, ou ‘pivot table’ (tabela de integração), ou tabela intermediária. Existem muitos nomes. A documentação do Laravel normalmente se referencia à estas tabelas como “pivot tables”, então nós também vamos utilizar isto. Sempre que você precisar de uma relação **many_to_many**, você vai precisar de uma ‘pivot table’. A ‘pivot table’ é a tabela na base de dados que junta as outras duas tabelas, utilizando chaves estrangeiras para referenciar os registros das outras tabelas.

Olhando para os dois primeiros registros da tabela `book_users`, nós podemos ver que o usuário ‘Dayle Rees’ favoritou os livros ‘Code Dutch’ e ‘Code Bright’. Nós também podemos ver que os usuários Matthew Machuga e Shawn McCool favoritaram o livro Code Dutch.

Também existe um outro tipo de relação, conhecido como **polymorphic** (polimórfico, em português). Por este ser muito complexo, nós vamos aprender mais sobre ele em um futuro capítulo que vai falar sobre táticas avançadas com o Eloquent. Nós ainda não vamos precisar dele.

Chega de aprender a teoria. É hora de aprender na prática! Agora que nós já sabemos quais os tipos de relações disponíveis, vamos ver como nós podemos implementá-las.

Implementando as Relações

Certo, vamos combinar. Primeiro nós vamos precisar criar algumas tabelas. Normalmente, eu criaria uma nova migração para cada tabela, mas para simplificar este exemplo, eu vou colocar tudo em uma migração.

Nós vamos criar um sistema utilizando artistas (Artists), álbuns (Albums) e ouvintes (Listeners). Os ouvintes são os usuários que escutam os álbuns. Vamos pensar nas relações deste sistema.

- Um artista possui muitos álbuns. (*Artist has_many Album*).
- Um álbum pertence a um artista. (*Album belongs_to Artist*).
- Um ouvinte pertence a muitos álbuns (*Listener belongs_to_many Album*).
- Um álbum pertence a muitos ouvintes (*Album belongs_to_many Listener*).

Nós temos uma relação simples entre um artista e muitos álbuns, e uma relação **many_to_many** (muitos para muitos) entre ouvintes e álbuns. Então vamos pensar na estrutura da nossa tabela. Se um álbum pertence (**belongs_to**) a um artista, então a chave estrangeira que vai identificar a relação vai estar na tabela de álbuns. A relação **many_to_many** vai precisar de uma 'pivot table' (tabela de integração).

Nós sabemos que o Eloquent precisa que o nome da tabela esteja no plural de seu nome (a não ser que nós especificarmos o nome da tabela na classe), mas como nós nomeamos a nossa tabela de integração? O padrão para a tabela de integração é o singular das duas tabelas que estão sendo integradas, separadas com um underline `_`. A ordem dos nomes deve ser alfabética. Isto significa que o nome da nossa nova tabela será `album_listener`.

Todas as chaves estrangeiras seguem uma convenção em seus nomes. O singular da tabela relacionada, com um `_id` no final. Nossa tabela de integração vai conter as colunas `album_id` e `listener_id`.

Vamos verificar a migração, e depois vamos examinar qualquer coisa nova em detalhes.

```
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4
5  // app/database/migrations/2013_08_26_130751_create_tables.php
6
7  class CreateTables extends Migration {
8
9      /**
10       * Run the migrations.
11       *
12       * @return void
13       */
14     public function up()
15     {
16         Schema::create('artists', function($table)
17         {
```

```
18         $table->increments('id');
19         $table->string('name', 64);
20         $table->timestamps();
21     });
22
23     Schema::create('albums', function($table)
24     {
25         $table->increments('id');
26         $table->string('name', 64);
27         $table->integer('artist_id');
28         $table->timestamps();
29     });
30
31     Schema::create('listeners', function($table)
32     {
33         $table->increments('id');
34         $table->string('name', 64);
35         $table->timestamps();
36     });
37
38     Schema::create('album_listener', function($table)
39     {
40         $table->integer('album_id');
41         $table->integer('listener_id');
42     });
43 }
44
45 /**
46  * Reverse the migrations.
47  *
48  * @return void
49  */
50 public function down()
51 {
52     Schema::drop('artists');
53     Schema::drop('albums');
54     Schema::drop('listeners');
55     Schema::drop('album_listener');
56 }
57
58 }
```

Note que no momento em que eu estou escrevendo este artigo, eu não consegui fazer as chaves

estrangeiras funcionarem. Eu recebo sempre este erro.

```
1 SQLSTATE[HY000]: General error: 1215 Cannot add foreign key constraint (SQL: alte\
2 r table `albums` add constraint albums_artist_id_foreign foreign key (`artist_id`\
3 ) references `artists` (`id`))
```

Provavelmente eu vou voltar para este capítulo para corrigir isto. Se você tiver alguma ideia do que está errado, por favor me envie um e-mail!

Além das tabelas de álbuns (Album), artistas (Artist) e ouvintes (Listener), nós temos a construção da tabela de integração. Nós simplesmente criamos a tabela `album_listener` com duas chaves estrangeiras. Nós não precisamos de uma chave primária ou timestamps (campos com as datas de criação e atualização), visto que esta tabela vai servir apenas de integração entre as outras.

Hora de criar os nossos modelos do Eloquent. Vamos começar com o Artist.

```
1 <?php
2
3 class Artist extends Eloquent
4 {
5     // Artist __has_many__ Album
6     public function albums()
7     {
8         return $this->hasMany('Album');
9     }
10 }
```

Aqui temos algo novo! Dentro do nosso modelo do Eloquent nós temos um método de relacionamento. Vamos examiná-lo.

```
1 public function albums()
```

O nome do método não precisa ter nenhum formato específico. Ele vai ser apenas como nós vamos nos referenciar quando estivermos utilizando as relações. Este método poderia facilmente ser chamado `relatedAlbums()`.

```
1 return $this->hasMany('Album');
```

Dentro do método de relacionamento nós retornamos o resultado do método `$this->hasMany()`. Este é um dos muitos métodos de relacionamento que são herdados da classe base do Eloquent. O primeiro parâmetro do método de relacionamento é o nome completo do modelo que vai ser relacionado. Se nós decidirmos adicionar um namespace ao nosso modelo `Album`, nós vamos precisar passar o namespace também.

Se a chave estrangeira da tabela de álbuns fosse nomeada de uma maneira fora do padrão, então nós podemos especificar o nome da coluna passando como o segundo opcional parâmetro deste método. Por exemplo:

```
1 return $this->hasMany('Album', 'the_related_artist');
```

Então o que exatamente vai ser retornado disto? Bem, não se preocupe com isto por enquanto. Vamos terminar de criar as definições de nossos modelos primeiro. Agora nós temos o modelo Album.

```
1 <?php
2
3 class Album extends Eloquent
4 {
5     // Album __belongs_to__ Artist
6     public function artist()
7     {
8         return $this->belongsTo('Artist');
9     }
10
11     // Album __belongs_to_many__ Listeners
12     public function listeners()
13     {
14         return $this->belongsToMany('Listener');
15     }
16 }
```

A tabela Album possui dois métodos de relacionamento. Novamente, os nomes dos nossos métodos públicos não importam. Vamos dar uma olhada no conteúdo do primeiro método.

```
1 return $this->belongsTo('Artist');
```

Visto que a chave estrangeira está presente nesta tabela (Album), nós utilizamos o método `$this->belongsTo()` para afirmar que o modelo Album é relacionado ao modelo Artist. O primeiro parâmetro, novamente, é o nome do modelo relacionado, e novamente você pode passar o opcional segundo parâmetro se sua coluna possuir um nome diferente do padrão.

O segundo método forma um dos lados da nossa relação **many_to_many** (de muitos para muitos). Vamos dar uma olhada.

```
1 return $this->belongsToMany('Listener');
```

O método `$this->belongsToMany()` informa o Eloquent que ele deve verificar a tabela de integração para buscar os modelos relacionados. O primeiro parâmetro é, novamente, o nome do modelo relacionado. Não se esqueça de incluir o namespace caso ele exista. Porém, desta vez, os próximos parâmetros são diferentes. Vamos montar um outro exemplo.


```
1 return $this->belongsToMany('Listener', 'my_pivot_table', 'first', 'second');
```

O segundo opcional parâmetro é o nome da tabela de integração entre os modelos. O terceiro e quarto parâmetros são usados para identificar o nome das chaves estrangeiras que estão na tabela de integração. Estes parâmetros também são opcionais.

Nós temos apenas mais um modelo restante. Vamos dar uma olhada no `Listener`.

```
1 <?php
2
3 class Listener extends Eloquent
4 {
5     // Listener __belongs_to_many__ Album
6     public function albums()
7     {
8         return $this->belongsToMany('Album');
9     }
10 }
```

O modelo `Listener` faz o inverso... mas é realmente o inverso? Este modelo faz o *outro* lado da relação **many_to_many**. Novamente, nós vamos utilizar o método `$this->belongsToMany()` para construir a relação.

Certo! Nossos modelos foram criados. Vamos criar algumas instâncias agora.

Relacionando e Buscando

Primeiro vamos criar um artista, um álbum e a associação entre os dois. Eu vou utilizar a rota `/` para isto pois acredito que seja uma boa maneira de exemplificar o código.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     $artist = new Artist;
8     $artist->name = 'Eve 6';
9     $artist->save();
10
11     $album = new Album;
12     $album->name = 'Horrorscope';
```

```
13     $album->artist()->associate($artist);
14     $album->save();
15
16     return View::make('hello');
17 });
```

Espere um pouco, o que é esta nova linha?

```
1 $album->artist()->associate($artist);
```

Vamos quebrar esta linha. Este é o primeiro método.

```
1 $album->artist();
```

Você se lembra deste método? Este é o método de relação que nós criamos no objeto Album. Então o que exatamente ele retorna? Este método retorna uma instância do Query Builder do Eloquent, assim como os métodos que nós utilizamos nos capítulos anteriores.

Porém, esta instância vai possuir alguns atributos setados para nós. Os resultados representado pelo Query Builder serão os artistas Artist relacionados com o Album.

É o mesmo que o seguinte.

```
1 Artist::where('album_id', '=', __OUR__CURRENT__ALBUM__);
```

Isto é bom, certo? Nós ainda podemos adicionar mais condições se nós quisermos. Por exemplo:

```
1 $album->artist()->where('genre', '=', 'rock')->take(2);
```

Não se esqueça que o Query Builder necessita de um método de consulta no final para retornar uma coleção de resultados. Assim:

```
1 $album->artist()->where('genre', '=', 'rock')->take(2)->get();
```

Isto também significa que nós podemos buscar uma coleção com todos os artistas relacionados executando o seguinte.

```
1 $album->artist()->get();
```

Ou, uma única instância utilizando o método `first()`.

```
1 $album->artist()->first();
```

Muito flexível! Certo, vamos dar uma olhada em nosso exemplo anterior.

```
1 $album->artist()->associate($artist);
```

Eu ainda não vi o método `associate`!

Não entre em pânico! Não é porque é novo que significa que seja ruim. Apenas aranhas são ruins. E o Kevin Bacon.

O método `associate` está disponível nas relações para criar a relação entre dois objetos. Ele vai atualizar a chave estrangeira de acordo com o modelo passado por parâmetro.

Isto significa que passando a instância do `$album` para o método `associate()`, nós vamos atualizar a coluna `artist_id` da nossa instância de `Album` com o ID do `Artist`. Se nós quisermos, nós também podemos passar a chave estrangeira diretamente.

Aqui está um exemplo.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/', function()
6 {
7     $artist = new Artist;
8     $artist->name = 'Eve 6';
9     $artist->save();
10
11     $album = new Album;
12     $album->name = 'Horrorscope';
13     $album->artist_id = $artist->id;
14     $album->save();
15
16     return View::make('hello');
17 });
```

As duas formas vão ter o mesmo resultado, e a relação será criada entre os objetos.

Não se esqueça de salvar o modelo que você quer relacionar, antes de passá-lo para o método `associate()`. Isto porque este modelo vai precisar de uma chave primária para criar a relação, e o outro modelo vai apenas receber a chave primária quando este for salvo.

Relacionar objetos que possuem relações de muitos para muitos funciona um pouco diferente. Vamos dar uma olhada.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $artist = new Artist;
8      $artist->name = 'Eve 6';
9      $artist->save();
10
11     $album = new Album;
12     $album->name = 'Horrorscope';
13     $album->artist()->associate($artist);
14     $album->save();
15
16     $listener = new Listener;
17     $listener->name = 'Naruto Uzumaki';
18     $listener->save();
19     $listener->albums()->save($album);
20
21     return View::make('hello');
22 });
```

Vamos focar nisto um pouco:

```
1  $listener = new Listener;
2  $listener->name = 'Naruto Uzumaki';
3  $listener->save();
4  $listener->albums()->save($album);
```

Depois de nós popularmos a instância do nosso modelo `Listener`, nós precisamos salvá-lo. Isto porque nós vamos precisar sua chave primária para criarmos um registro na tabela de integração.

Desta vez, ao invés de utilizar o método `associate()` em nosso relacionamento, nós utilizamos o método `save()`, passando o objeto que deve ser relacionado como o primeiro parâmetro. Desta vez, apenas a tabela de integração será atualizada para definir o relacionamento.

Se você gosta de associar o modelo por sua chave primária diretamente, você pode utilizar o método `attach()`, que aceita a chave primária como o primeiro parâmetro. Por exemplo:

```
1  $album->artist()->attach(2);
```

Para remover um relacionamento entre dois objetos, você pode utilizar o método `detach()`. Apenas passe a chave primária ou o objeto como o primeiro parâmetro.

Por exemplo:

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      $album = Album::find(5);
8      $listener = Listener::find(2);
9      $album->listeners()->detach($listener);
10
11      return View::make('hello');
12  });
```

Nós podemos remover todas as associações de uma objeto que possui uma relação **many to many** chamando o método `detach()` sem passar nenhum parâmetro.

Abaixo temos um exemplo.

```
1  $album->listeners()->detach();
```

Agora o álbum não vai possuir nenhum ouvinte relacionado.

O Eloquent é um grande e belo tópico cheio de muitas funcionalidades espetaculares, mas eu não quero encher você demais por enquanto. Isto vai apenas fazer você esquecer alguns dos conceitos mais básicos que nós acabamos de aprender.

Agora nós já temos todo o conhecimento necessário para começar a construir uma aplicação básica. Nós vamos construir um sistema para gerenciar os jogos de Playstation 3 que eu possuo. Eu odeio esquecer os jogos que eu tenho!

Se você ainda está com vontade de aprender mais sobre o Eloquent, não se preocupe. Nós vamos retornar para este maravilhoso ORM nos próximos capítulos.

Monte uma Aplicação 1: Coleção de Jogos de Playstation

Finalmente chegou a hora! Nós vamos montar uma aplicação completa utilizando o Laravel. Eu sei que você está tão empolgado quanto eu. Nossa primeira aplicação vai ser simples, um sistema para gerenciar meus jogos de Playstation. E eu tenho muitos!

Cada jogo vai possuir um nome, um distribuidor e se ele já foi completado ou não. A aplicação vai ser um CRUD simples (create/read/update/delete), mas é um grande ponto de partida para juntar todas as funcionalidades que nós aprendemos nos capítulos anteriores.

Assim que a aplicação estiver completa, nós vamos começar a ver algumas funcionalidades mais avançadas do framework, e como elas podem ser utilizadas para adicionar funcionalidades extras para nossas aplicações.

Agora vamos parar de perder tempo. Nós precisamos começar a planejar nossa aplicação.

Vamos Pensar Juntos

Nossa aplicação vai gerenciar vídeo games, então vamos verificar quais atributos que nós desejamos armazenar para cada jogo.

- Title (Título)
- Publisher (Distribuidor)
- Completed (Completo)

Agora nós precisamos pensar no tipo dos atributos de cada jogo. Isto vai nos ajudar a montar a estrutura da base de dados. Vamos verificar cada campo e atribuir um tipo e um tamanho para cada um deles.

- Title **string 128**
- Publisher **string 128**
- Completed **boolean**

Esta estrutura vai nos permitir descrever os jogos de uma maneira que faça sentido. Strings com 128 caracteres serão utilizadas para armazenar o título e a distribuidora do jogo, e um simples campo booleano para controlar se o jogo já foi completo ou não.

Agora vamos pensar nas funcionalidades que nossa aplicação vai ter. Vamos em frente e criar uma nova lista.

- Criar registros de vídeo game.
- Listar registros de vídeo game.
- Editar registros de vídeo game.
- Excluir registros de vídeo game.

Nós queremos poder criar um novo jogo e listar os jogos que nós já criamos. Nós também queremos poder atualizar e excluir os jogos já criamos. Por isto que nós descrevemos esta aplicação antes como uma aplicação CRUD. Isto vai ser fantástico! É um padrão que se aplica a muitas aplicações reais.

Agora, eu gosto de pensar sobre as páginas necessárias para executar estas ações. Vamos ver... a página de criação vai precisar um formulário para capturar as informações dos nossos novos jogos. A view da listagem vai exibir os jogos em alguma forma de tabulação.

A próxima parte é um pouco mais complicada. É uma boa prática, para melhorar a experiência do usuário, possuir um botão de 'editar' na listagem dos jogos. E nós também vamos precisar de uma página exibindo um formulário para editar o jogo. Muitos desenvolvedores utilizam a mesma página para criar e editar registros, mas para manter este exemplo simples eu vou separá-las em duas views diferentes.

Finalmente, o botão de excluir também vai ser exibido na tela de listagem dos jogos. Nós também vamos precisar de uma outra página para confirmar se o usuário realmente quer excluir o jogo.

Certo, vamos resumir estas páginas.

- Listagem dos jogos.
- Formulário para criação de um jogo.
- Formulário para edição de um jogo.
- Tela para confirmação de exclusão.

Ótimo! Eu acho que nós temos tudo que nós precisamos para começar.

Hora de Programar!

Todo o código deste projeto está disponível em um repositório no Github, no endereço [codebright/gamesapp](https://github.com/codebright/gamesapp)²⁸.

Primeiro eu vou copiar o Laravel para a pasta do meu projeto games. Como sempre, nós vamos precisar instalar todas as dependências com o Composer.

²⁸[http://github.com/codebright/gamesapp](https://github.com/codebright/gamesapp)

```
1  $ composer install
2  Loading composer repositories with package information
3  Installing dependencies (including require-dev)
4    - Installing doctrine/lexer (dev-master bc0e1f0)
5      Loading from cache
6
7    - Installing doctrine/annotations (v1.1.2)
8      Loading from cache
9
10   - Installing doctrine/collections (dev-master bcb5377)
11     Loading from cache
12
13   - Installing doctrine/cache (v1.1)
14     Loading from cache
15
16   - Installing doctrine/inflector (dev-master 8b4b3cc)
17     Loading from cache
18
19   ... lots more ...
20
21   - Installing laravel/framework (4.0.x-dev 733492c)
22     Downloading: 100%
23
24  monolog/monolog suggests installing mlehner/gelf-php (Allow sending log messages \
25  to a GrayLog2 server)
26  monolog/monolog suggests installing ext-amqp (Allow sending log messages to an AM\
27  QP server (1.0+ required))
28  monolog/monolog suggests installing ext-mongo (Allow sending log messages to a Mo\
29  ngoDB server)
30  monolog/monolog suggests installing doctrine/couchdb (Allow sending log messages \
31  to a CouchDB server)
32  monolog/monolog suggests installing raven/raven (Allow sending log messages to a \
33  Sentry server)
34  symfony/translation suggests installing symfony/config ()
35  symfony/translation suggests installing symfony/yaml ()
36  symfony/routing suggests installing symfony/config ()
37  symfony/routing suggests installing symfony/yaml ()
38  symfony/debug suggests installing symfony/class-loader ()
39  symfony/event-dispatcher suggests installing symfony/dependency-injection ()
40  symfony/http-kernel suggests installing symfony/class-loader ()
41  symfony/http-kernel suggests installing symfony/config ()
42  symfony/http-kernel suggests installing symfony/dependency-injection ()
```



```
43 predis/predis suggests installing ext-redis (Allows faster serialization and \
44 deserialization of the Redis protocol)
45 Writing lock file
46 Generating autoload files
47 Generating optimized class loader
```

Ótimo, agora nós podemos começar. Vamos começar pelo banco de dados.

Banco de Dados

Primeiro nós vamos criar uma base de dados em nosso ambiente de desenvolvimento. Eu vou criar uma base MySQL chamada games.

Nós vamos precisar alterar as configurações de conexão com o banco de dados para apontar para nossa nova base de dados. Aqui está a minha configuração.

```
1  // app/config/database.php
2
3  /*
4  |-----
5  | Default Database Connection Name
6  |-----
7  */
8
9  'default' => 'mysql',
10
11 /*
12 |-----
13 | Database Connections
14 |-----
15 */
16
17 'connections' => array(
18
19     'mysql' => array(
20         'driver'      => 'mysql',
21         'host'        => 'localhost',
22         'database'    => 'games',
23         'username'    => 'root',
24         'password'    => 'foobar',
25         'charset'     => 'utf8',
26         'collation'   => 'utf8_unicode_ci',
```

```
27         'prefix'      => '',
28     ),
29
30 ),
```

Agora vamos criar a estrutura da nossa base de dados utilizando o Schema Builder. Nós vamos criar uma migração utilizando o Artisan CLI (interface de linha de comando, em português).

```
1 $ php artisan migrate:make create_games
2 Created Migration: 2013_09_14_155847_create_games
3 Generating optimized class loader
```

Vamos completar os métodos `up()` e `down()` da nossa migração. Nós vamos utilizar as propriedades e os tipos que nós planejamos na seção anterior.

Aqui está o resultado final da migração.

```
1 <?php
2
3 use Illuminate\Database\Migrations\Migration;
4
5 // app/database/migrations/2013_09_14_155847_create_games.php
6
7 class CreateGames extends Migration {
8
9     /**
10      * Run the migrations.
11      *
12      * @return void
13      */
14     public function up()
15     {
16         Schema::create('games', function($table)
17         {
18             $table->increments('id');
19             $table->string('title', 128);
20             $table->string('publisher', 128);
21             $table->boolean('complete');
22             $table->timestamps();
23         });
24     }
25 }
```

```
26      /**
27       * Reverse the migrations.
28       *
29       * @return void
30       */
31      public function down()
32      {
33          Schema::drop('games');
34      }
35
36 }
```

No método `up()` nós criamos nossa tabela `games` com as propriedades mencionadas no capítulo de planejamento. Nós também adicionamos uma coluna `id` auto incremental, e as datas de criação e atualização (timestamps), pois nós queremos representar os nossos jogos com modelos do Eloquent.

No método `drop()` nós excluimos nossa tabela. É como se ela nunca tivesse existido!

Com a nossa migração finalizada, nós podemos executá-la rodando o comando de migração do Artisan.

```
1 $ php artisan migrate
2 Migration table created successfully.
3 Migrated: 2013_09_14_155847_create_games
```

Nossa base de dados agora está atualizada e a estrutura foi implementada. Por que nós não criamos um modelo do Eloquent para representar nossos jogos? Aqui está nossa nova classe.

```
1 <?php
2
3 // app/models/Game.php
4
5 class Game extends Eloquent
6 {
7
8 }
```

Bem, está foi fácil! Eu sempre acho a simplicidade do Eloquent realmente surpreendente. Vamos rir juntos por alguns momentos, e depois seguir para a próxima sessão.

Muahahahaahhaahahhahahahahaah.

Controller

Esta aplicação vai apenas possuir quatro páginas, e por isto um único controlador vai ser suficiente. Vamos criar nosso controlador agora. Nós também vamos criar algumas rotas e umas views vazias que nós vamos implementar depois. Nós vamos chamar o controlador de `GamesController`. Não se esqueça de excluir o controlador padrão `HomeController`, pois nós não vamos precisar dele!

```
1  <?php
2
3  // app/controllers/GamesController.php
4
5  class GamesController extends BaseController
6  {
7      public function index()
8      {
9          // Show a listing of games.
10         return View::make('index');
11     }
12
13     public function create()
14     {
15         // Show the create game form.
16         return View::make('create');
17     }
18
19     public function handleCreate()
20     {
21         // Handle create form submission.
22     }
23
24     public function edit(Game $game)
25     {
26         // Show the edit game form.
27         return View::make('edit');
28     }
29
30     public function handleEdit()
31     {
32         // Handle edit form submission.
33     }
34
35     public function delete()
```

```
36     {
37         // Show delete confirmation page.
38         return View::make('delete');
39     }
40
41     public function handleDelete()
42     {
43         // Handle the delete confirmation.
44     }
45 }
```

Nós implementamos algumas novas ações para nossa aplicação. Algumas são simplesmente utilizadas para exibir os formulários, e outras são utilizadas para manipular os envios. Note que os métodos `edit()` e `delete()` recebem o parâmetro definido (type-hinted) `$game`, que é uma instância do nosso modelo `Game`. Não se preocupe com isto agora, eu vou explicar depois quando nós formos criar as rotas.

Vamos criar as views que serão utilizadas pelo nosso controlador. Nós vamos adicionar o conteúdo delas depois. Nós também vamos excluir a view padrão `hello`.

- `app/views/index.blade.php`
- `app/views/create.blade.php`
- `app/views/edit.blade.php`
- `app/views/delete.blade.php`

Routes

Agora nós devemos criar nossas rotas para que as nossas ações possam ser executadas através de URLs. Aqui vamos nós...

```
1  <?php
2
3  // app/routes.php
4
5  /*
6   |-----
7   | Application Routes
8   |-----
9   |
10  | Here is where you can register all of the routes for an application.
11  | It's a breeze. Simply tell Laravel the URIs it should respond to
```

```
12 | and give it the Closure to execute when that URI is requested.
13 |
14 */
15
16 // Bind route parameters.
17 Route::model('game', 'Game');
18
19 // Show pages.
20 Route::get('/', 'GamesController@index');
21 Route::get('/create', 'GamesController@create');
22 Route::get('/edit/{game}', 'GamesController@edit');
23 Route::get('/delete/{game}', 'GamesController@delete');
24
25 // Handle form submissions.
26 Route::post('/create', 'GamesController@handleCreate');
27 Route::post('/edit', 'GamesController@handleEdit');
28 Route::post('/delete', 'GamesController@handleDelete');
```

Espera um pouco, o que é o método `Route::model()`? Bem, aqui está mais um truque que o Laravel oferece. O método `Route::model()` vai dizer ao Laravel que qualquer parâmetro de rota que tiver o mesmo nome do primeiro parâmetro passado para este método vai ser uma instância de um modelo do Eloquent definido no segundo parâmetro. No exemplo acima, qualquer parâmetro chamado `game`, como o `/edit/{game}` vai ser convertido para uma instância do modelo `Game`.

O Laravel vai verificar o número inteiro passado por parâmetro e executar uma consulta no modelo através de sua chave primária. Então, o Laravel vai passar a instância correspondente ao parâmetro para a ação do controller. Muito útil!

Você já deve estar familiarizado com o resto das rotas. Nós temos quatro rotas do tipo `GET`, utilizadas para exibir as páginas da aplicação, e três rotas do tipo `POST` para manipular os envios dos formulários.

Vamos garantir que tudo está funcionando como esperado. Nós vamos utilizar o comando `serve` do Artisan para iniciar um servidor web básico e testar nossa aplicação.

```
1 $ php artisan serve
2 Laravel development server started on http://localhost:8000
```

Nós podemos ver que o nosso servidor de desenvolvimento está rodando na porta 8000, então vamos visitar a URL `http://localhost:8000` para garantir que nós recebemos um template em branco. Funcionou corretamente para mim! Agora vamos partir para os templates.

Views

Para os nossos templates nós vamos utilizar o Twitter Bootstrap. Ele é um framework CSS que vai estilizar o HTML para nós. Eu acho isto muito bom para prototipar aplicações. Nós não vamos explicar em detalhes os templates porque este não é um livro sobre desenvolvimento front-end, mas nós passar pelos pontos mais importantes de cada template.

Primeiro de tudo, nós sabemos que os nossos templates vão possuir alguns pedaços de código em comum. Nós vamos adicionar este código em um template simples, como abaixo:

```
1 <!doctype html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Games Collection</title>
6 </head>
7 <body>
8     <!-- website content here -->
9 </body>
10 </html>
```

Nós também sabemos que nossa aplicação vai utilizar o mesmo arquivo CSS, então nós não precisamos adicioná-lo em cada página. Se nós fizéssemos isto, nós teríamos que alterar todos os arquivos caso nós mudássemos o arquivo CSS de lugar.

Ao invés disto, vamos resolver este problema utilizando a herança dos templates Blade. Nós vamos criar um template base chamado layout, que todas as nossas páginas vão estender.

Aqui está a nossa view layout.blade.php. Não vai ser um código muito bom para se visualizar em leitores eletrônicos, mas eu não posso comprimir o HTML. Desculpe por isto!

```
1 <!doctype html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Games Collection</title>
6     <link rel="stylesheet" href="{{ asset('css/bootstrap.min.css') }}" />
7 </head>
8 <body>
9     <div class="container">
10         <nav class="navbar navbar-default" role="navigation">
11             <div class="navbar-header">
12                 <a href="{{ action('GamesController@index') }}" class="navbar-bra\
```

```

13 <div>Games Collection</div>
14 </div>
15 </nav>
16 @yield('content')
17 </div>
18 </body>
19 </html>

```

Nós temos um HTML5 doctype com algumas linhas de código. Vamos dar uma olhada nelas.

```

1 <link rel="stylesheet" href="{{ asset('css/bootstrap.min.css') }}" />

```

Nós queremos utilizar um único arquivo global de CSS para estilizar todas as nossas views. Incluindo o CSS do Twitter Bootstrap em nosso layout, ele vai estar acessível dentro de todas as views que estenderem este layout. Nós utilizamos a função `asset()` para especificar um caminho para o nosso arquivo que seja relativo ao diretório público do projeto. Isto significa que o endereço utilizado pela nossa aplicação não vai interferir em relação aos arquivos públicos do nosso projeto.

A próxima linha interessante é a linha do menu. Vamos dar uma olhada mais de perto.

```

1 <a href="{{ action('GamesController@index') }}" class="navbar-brand">Games Collec\
2 tion</a>

```

Novamente, nós utilizamos uma função para evitar conflitos com o endereço do nosso projeto. Desta vez, nós utilizamos a função `action()` para gerar uma URL absoluta apontando para a rota apropriada. Utilizando a função `action()` junto com as `{{ chaves duplas }}`, nós criamos um link de retorno para a página inicial da nossa aplicação. O parâmetro para a função `action()` é o par controlador e ação, separados com o símbolo `@`, da mesma maneira que nós definimos ao definir nossas rotas.

Agora é hora da última linha do template, e a mais importante! Vamos dar uma olhada.

```

1 @yield('content')

```

O método `@yield()` do Blade nos permite criar seções onde nós podemos injetar conteúdos dentro de nossos layouts. Isto significa que os templates que estenderem nosso layout poderão injetar conteúdo dentro desta área. Nós utilizamos o nome `'content'` para identificar esta área.

Nós agora podemos começar a criar algumas views que vão estender este layout. Vamos começar com a view que será utilizada para listar todos os jogos. Antes de começarmos a editar o arquivo da view, vamos modificar a ação `index()` do nosso controlador para incluir um pouco de lógica.


```

1 public function index()
2 {
3     // Show a listing of games.
4     $games = Game::all();
5     return View::make('index', compact('games'));
6 }

```

Na nossa ação `index()` nós utilizamos o método `Game::all()` para buscar uma coleção de todos os jogos cadastrados no sistema. Depois, nós utilizamos o método `compact()` e `View::make()` para passar esta coleção para nossa view. Se você ainda não utilizou o método `compact()` ainda, ele simplesmente cria um array a partir das variáveis passadas para ele, e utiliza o nome das variáveis como a chave do array. É exatamente o oposto da função `extract()`. Eu gosto de utilizar a função `compact()` apenas quando eu tenho que passar um pequeno número de variáveis para minhas views.

Agora que nós já temos a nossa coleção de jogos, por que nós não exibimos elas? Aqui está o conteúdo da view `app/views/index.blade.php`.

```

1 @extends('layout')
2
3 @section('content')
4     <div class="page-header">
5         <h1>All Games <small>Gotta catch 'em all!</small></h1>
6     </div>
7
8     <div class="panel panel-default">
9         <div class="panel-body">
10             <a href="{{ action('GamesController@create', $game->id) }}" class="btn\
11 n btn-primary">Create Game</a>
12         </div>
13     </div>
14
15     @if ($games->isEmpty())
16         <p>There are no games! :( </p>
17     @else
18         <table class="table table-striped">
19             <thead>
20                 <tr>
21                     <th>Title</th>
22                     <th>Publisher</th>
23                     <th>Complete</th>
24                     <th>Actions</th>
25                 </tr>
26             </thead>

```

```

27         <tbody>
28             @foreach($games as $game)
29                 <tr>
30                     <td>{{ $game->title }}</td>
31                     <td>{{ $game->publisher }}</td>
32                     <td>{{ $game->complete ? 'Yes' : 'No' }}</td>
33                     <td>
34                         <a href="{{ action('GamesController@edit', $game->id) }}" \
35 class="btn btn-default">Edit</a>
36                         <a href="{{ action('GamesController@delete', $game->id) }} \
37 }}" class="btn btn-danger">Delete</a>
38                     </td>
39                 </tr>
40             @endforeach
41         </tbody>
42     </table>
43 @endif
44 @stop

```

O método `extends('layout')` do blade na primeira linha indica que nós desejamos que esta view estenda o layout que nós criamos anteriormente. O layout deve permitir que nossa view injete seu conteúdo dentro de suas seções.

Depois, nós temos a `@section`.

```

1 @section('content')
2     <!-- HTML here -->
3 @stop

```

O método `@section` do blade é utilizado para injetar o conteúdo dentro das regiões que nós definirmos em nossos layouts utilizando o método `@yield`. O único parâmetro para esta função é o nome da seção que nós desejamos injetar o conteúdo.

Nós criamos algum código HTML para estruturar a página, e a próxima linha que nos interessa é a linha que contém o `@if`. Primeiro nós verificamos se nossa coleção de jogos está vazia utilizando o método `isEmpty()`. Se nós não tivermos jogos, então nós vamos exibir um pequeno texto para avisar o usuário.

Se nós tivermos jogos cadastrados, nós vamos exibí-los. Nós criamos uma tabela, e dentro do corpo da tabela, fazemos um `foreach`. Ele vai iterar todos os nossos jogos e exibir os jogos.

```

1 @foreach($games as $game)
2 <tr>
3     <td>{{ $game->title }}</td>
4     <td>{{ $game->publisher }}</td>
5     <td>{{ $game->complete ? 'Yes' : 'No' }}</td>
6     <td>
7         <a href="{{ action('GamesController@edit', $game->id) }}" class="btn btn-\
8 default">Edit</a>
9         <a href="{{ action('GamesController@delete', $game->id) }}" class="btn bt\
10 n-danger">Delete</a>
11     </td>
12 </tr>
13 @endforeach

```

Para indicar se um jogo já foi completado ou não, nós utilizamos o operador ternário do PHP, combinado com as {{ chaves duplas }} para escrever o resultado. Vai ser 'Yes' ou 'No'.

```

1 {{ $game->complete ? 'Yes' : 'No' }}

```

Por último, nós temos dois botões que podem ser utilizados para editar ou excluir o jogo do sistema.

```

1 <a href="{{ action('GamesController@edit', $game->id) }}" class="btn btn-default"\
2 >Edit</a>
3 <a href="{{ action('GamesController@delete', $game->id) }}" class="btn btn-danger\
4 ">Delete</a>

```

Novamente, nós utilizamos a função action(), porém desta vez nós passamos um segundo parâmetro. Este é o id do jogo que nós desejamos excluir ou editar.

Agora vamos partir para a view que vai possuir o formulário para criar um novo jogo no sistema. Ela vai ficar em app/views/create.blade.php.

```

1 @extends('layout')
2
3 @section('content')
4     <div class="page-header">
5         <h1>Create Game <small>and someday finish it!</small></h1>
6     </div>
7
8     <form action="{{ action('GamesController@handleCreate') }}" method="post" rol\
9 e="form">
10     <div class="form-group">

```

```

11         <label for="title">Title</label>
12         <input type="text" class="form-control" name="title" />
13     </div>
14     <div class="form-group">
15         <label for="publisher">Publisher</label>
16         <input type="text" class="form-control" name="publisher" />
17     </div>
18     <div class="checkbox">
19         <label for="complete">
20             <input type="checkbox" name="complete" /> Complete?
21         </label>
22     </div>
23     <input type="submit" value="Create" class="btn btn-primary" />
24     <a href="{{ action('GamesController@index') }}" class="btn btn-link">Cancel\
25 el</a>
26 </form>
27 @stop

```

A página de criação estende o nosso layout base de uma maneira similar ao nosso `index.blade.php`.

```

1 <form action="{{ action('GamesController@handleCreate') }}" method="post" role="f\
2 orm">

```

Aqui nós utilizamos a função `action()` para adicionar o endereço para o qual nosso formulário deve submeter. Você pode estar se perguntando porque nós não utilizamos os métodos de abertura de formulários que nós aprendemos no capítulo sobre formulários. Fazendo isto, nós não precisaríamos escrever o atributo `method` do formulário. Porém, eu acho que utilizar as funções de rotas (como a `action()`) deixa as nossas views mais claras. Elas são mais HTML do que PHP, e as cores da sintaxe parecem funcionar melhor assim no meu editor. Mas isto é uma questão de preferência, então por favor, faça o seu próprio julgamento em relação a isto.

O resto do formulário é padrão. Nós temos alguns campos que representam as propriedades dos jogos. Nós temos um botão para enviar o formulário e um link para cancelar em caso do usuário mudar de ideia.

Agora nós temos a view de confirmação da exclusão. Nós queremos que o nosso usuário confirme que realmente quer excluir o registro da base de dados. Aqui está o conteúdo da view `app/views/delete.blade.php`.

```

1  @extends('layout')
2
3  @section('content')
4      <div class="page-header">
5          <h1>Delete {{ $game->title }} <small>Are you sure?</small></h1>
6      </div>
7      <form action="{{ action('GamesController@handleDelete') }}" method="post" rol\
8 e="form">
9          <input type="hidden" name="game" value="{{ $game->id }}" />
10         <input type="submit" class="btn btn-danger" value="Yes" />
11         <a href="{{ action('GamesController@index') }}" class="btn btn-default">N\
12 o way!</a>
13     </form>
14 @stop

```

Nós temos mais um formulário. Este aqui aponta para a rota `handleDelete` e contém um campo do tipo `hidden` que vai ser utilizado para identificar qual jogo nós queremos excluir.

O botão ‘Yes’ vai submeter o formulário e o jogo vai ser excluído. O botão ‘No’ vai simplesmente redirecionar novamente para a página inicial. Funciona como um mecanismo de cancelamento.

Então o que ainda falta? Certo! A view de edição. Uma vez que nós já criamos um jogo, ele deve poder ser editado. Então vamos criar uma view para isto. Aqui está nossa view `app/views/edit.blade.php`.

```

1  @extends('layout')
2
3  @section('content')
4      <div class="page-header">
5          <h1>Edit Game <small>Go on, mark it complete!</small></h1>
6      </div>
7
8      <form action="{{ action('GamesController@handleEdit') }}" method="post" role=\
9 "form">
10         <input type="hidden" name="id" value="{{ $game->id }}">
11
12         <div class="form-group">
13             <label for="title">Title</label>
14             <input type="text" class="form-control" name="title" value="{{ $game-\
15 >title }}" />
16         </div>
17         <div class="form-group">
18             <label for="publisher">Publisher</label>
19             <input type="text" class="form-control" name="publisher" value="{{ $g\

```

```

20 ame->publisher }}" />
21     </div>
22     <div class="checkbox">
23         <label for="complete">
24             <input type="checkbox" name="complete" {{ $game->complete ? 'checked' : '' }} /> Complete?
25         </label>
26     </div>
27     <input type="submit" value="Save" class="btn btn-primary" />
28     <a href="{{ action('GamesController@index') }}" class="btn btn-link">Cancel</a>
29 el</a>
30 </form>
31 @stop

```

Mais uma vez, nós injetamos um formulário dentro da seção ‘content’ do layout. Como o jogo já existe, nós já preenchemos os campos do formulário com os valores do jogo. Novamente nós utilizamos um operador ternário para atribuir o atributo ‘checked’ ao checkbox que diz se o jogo já foi completado ou não.

Excelente! Agora que nós já temos todas as nossas views e os nossos modelos prontos, é hora de juntar tudo isto criando os métodos das ações do nosso controlador.

Lógica da Aplicação

Vamos começar com a rota inicial. O que nós temos por enquanto?

```

1 public function index()
2 {
3     // Show a listing of games.
4     $games = Game::all();
5
6     return View::make('index', compact('games'));
7 }

```

Ah, isto já é tudo que nós precisamos, nossa coleção de jogos. Esta foi fácil, não? Nós não precisamos nos preocupar com a ação `create()` porque ela apenas serve para exibir nossa view. Vamos dar uma olhada na nossa ação `handleCreate()`.

```
1 public function handleCreate()  
2 {  
3     // Handle create form submission.  
4 }
```

Eu tenho quase certeza que este comentário não vai inserir o jogo para nós. Por que nós não fazemos isto? Vamos começar criando um novo objeto e populando seus atributos com os valores enviados pelo formulário para criar nosso novo jogo.

```
1 public function handleCreate()  
2 {  
3     // Handle create form submission.  
4     $game = new Game;  
5     $game->title = Input::get('title');  
6     $game->publisher = Input::get('publisher');  
7     $game->complete = Input::has('complete');  
8     $game->save();  
9 }
```

Nós criamos uma nova instância do modelo `Game`, setamos suas propriedades com os valores enviados pelo formulário através do método `Input::get()`, e finalmente, nós utilizamos o método `save()` para salvar nosso objeto na base de dados.

Nós precisamos retornar alguma resposta desta página para que não aconteça nenhum erro. O usuário deve ser direcionado para algum lugar que faça sentido. Nós podemos exibir uma página de sucesso, mas isto é apenas perda de tempo. O usuário vai precisar clicar novamente em um botão para ser direcionado para a página inicial. Por que nós simplesmente não redirecionamos o usuário para a página de listagem de jogos?

```
1 public function handleCreate()  
2 {  
3     // Handle create form submission.  
4     $game = new Game;  
5     $game->title = Input::get('title');  
6     $game->publisher = Input::get('publisher');  
7     $game->complete = Input::has('complete');  
8     $game->save();  
9  
10    return Redirect::action('GamesController@index');  
11 }
```

Desta vez, nós retornamos uma resposta de redirecionamento (`Redirect`). Agora nós temos nossa ação `edit()`. E aqui está como ela está no momento.

```
1 public function edit(Game $game)
2 {
3     // Show the edit game form.
4     return View::make('edit');
5 }
```

Nosso formulário de edição precisa poder repopular o formulário com os valores atuais do jogo, e para isto nós precisamos passar a instância do nosso `$game` para a view. Vamos fazer isto.

```
1 public function edit(Game $game)
2 {
3     // Show the edit game form.
4     return View::make('edit', compact('game'));
5 }
```

Nós utilizamos o método `compact()` novamente para criar o array de dados da view. Agora é hora de criarmos o método que vai manipular o envio do formulário de edição. A ação `handleEdit()` vai cuidar disto.

```
1 public function handleEdit()
2 {
3     // Handle edit form submission.
4 }
```

Está um pouco vazio, não? Vamos completá-lo! O formulário de edição fornece o ID do jogo como um campo do tipo `hidden`, então nós podemos usá-lo para buscar a instância do jogo e modificar o valor de suas colunas.

```
1 public function handleEdit()
2 {
3     // Handle edit form submission.
4     $game = Game::findOrFail(Input::get('id'));
5     $game->title      = Input::get('title');
6     $game->publisher  = Input::get('publisher');
7     $game->complete   = Input::has('complete');
8     $game->save();
9
10    return Redirect::action('GamesController@index');
11 }
```


Agora sim! Nós utilizamos o método `findOrFail()` da classe `Game`. Ele é muito similar ao `find()`, exceto que ele vai gerar um erro 404 se o registro não for encontrado na base de dados. Novamente, nós populamos nosso objeto `$game` com as informações enviadas através do formulário, salvamos na base de dados e redirecionamos para a página inicial.

Nossos jogos agora podem ser criados e editados. É hora que adicionarmos alguma funcionalidade para excluir o jogo. Vamos dar uma olhada na ação `handleDelete()` do `GameController`.

```
1 public function handleDelete()  
2 {  
3     // Handle the delete confirmation.  
4 }
```

Isto não vai excluir o jogo! Vamos fazer isto.

```
1 public function handleDelete()  
2 {  
3     // Handle the delete confirmation.  
4     $id = Input::get('game');  
5     $game = Game::findOrFail($id);  
6     $game->delete();  
7  
8     return Redirect::action('GamesController@index');  
9 }
```

Assim como a ação `handleEdit()`, nós buscamos uma instância do jogo utilizando o campo do tipo `hidden` enviado pelo nosso formulário de exclusão. Então, nós simplesmente excluimos o registro através do método `delete()`. Com o jogo excluído, nós podemos redirecionar o usuário para a página inicial.

Vamos dar uma olhada em nosso `GamesController` completo.

```
1 <?php  
2  
3 // app/controllers/GamesController.php  
4  
5 class GameController extends BaseController  
6 {  
7     public function index()  
8     {  
9         // Show a listing of games.  
10         $games = Game::all();  
11
```

```
12     return View::make('index', compact('games'));
13 }
14
15 public function create()
16 {
17     // Show the create game form.
18     return View::make('create');
19 }
20
21 public function handleCreate()
22 {
23     // Handle create form submission.
24     $game = new Game;
25     $game->title      = Input::get('title');
26     $game->publisher  = Input::get('publisher');
27     $game->complete   = Input::has('complete');
28     $game->save();
29
30     return Redirect::action('GamesController@index');
31 }
32
33 public function edit(Game $game)
34 {
35     // Show the edit game form.
36     return View::make('edit', compact('game'));
37 }
38
39 public function handleEdit()
40 {
41     // Handle edit form submission.
42     $game = Game::findOrFail(Input::get('id'));
43     $game->title      = Input::get('title');
44     $game->publisher  = Input::get('publisher');
45     $game->complete   = Input::has('complete');
46     $game->save();
47
48     return Redirect::action('GamesController@index');
49 }
50
51 public function delete(Game $game)
52 {
53     // Show delete confirmation page.
```

```
54         return View::make('delete', compact('game'));
55     }
56
57     public function handleDelete()
58     {
59         // Handle the delete confirmation.
60         $id = Input::get('game');
61         $game = Game::findOrFail($id);
62         $game->delete();
63
64         return Redirect::action('GamesController@index');
65     }
66 }
```

Toda a lógica da nossa aplicação coube em 66 linhas. Isto é lindo! É hora de testar nossa aplicação. Vamos lá.

Relaxe

Primeiro tenha certeza que sua aplicação está sendo servida. Se não estiver, você pode utilizar o comando `server` do Artisan.

```
1 php artisan serve
```

Agora visite a URL `http://localhost:8000` para ser contemplado com a página inicial que está cheia de jog... espere. Onde estão todos os jogos? Bem, não importa. Vamos criar nosso primeiro jogo.

Clique no botão ‘Create Game’ e a view de criação de jogos será apresentada. Eu vou digitar: ‘Beyond: Two Souls’ no box de título, e ‘Sony Computer Entertainment’ no campo de distribuidor. Eu ainda não completei o jogo, por isto vou apenas clicar no botão ‘Create’.

Agora o jogo ‘Beyond: Two Souls’ está sendo exibido em nossa página inicial. E ótimas notícias! Eu já completei o jogo. (Na verdade não, mas vamos utilizar um pouco de imaginação aqui.). É hora de editar nosso jogo. Clique no botão ‘Edit’. Agora você pode marcar o checkbox e clicar em ‘Save’ para atualizar o registro.

Me dói ter que fazer isto, mas por que nós não excluimos este jogo ? Aperte o botão ‘Delete’, e depois confirme a exclusão clicando em ‘Yes’. Ao que parece, todas as nossas funcionalidades estão funcionando como o esperado. Parabéns, você completou sua primeira aplicação com o Laravel!

Não se esqueça que todo o código deste projeto está no repositório [codebright/games-app](https://github.com/codebright/games-app)²⁹.

²⁹<http://github.com/codebright/gamesapp>

Lição de Casa

Todos juntos, é hora da lição de casa desta semana. Bem, eu honestamente raramente faço as seções ‘exercícios’ dos livros, então sinta-se à vontade para pular esta etapa se você quiser.

No momento, nossa aplicação aceita qualquer valor vindo dos formulários. Por exemplo, nós podemos criar um jogo com o título em branco. E isto pode quebrar nosso sistema. Por que nós não adicionamos uma validação em nossa aplicação utilizando o capítulo de validação como referência?

Em um futuro capítulo, nós vamos cobrir autenticação, que pode ser utilizada para proteger nossa aplicação de uso desautorizado. E por que não voltar para este capítulo e tentar implementar autenticação depois de ler este capítulo?

Sem tempo para o tema de casa? Não se preocupe! Vamos seguir para o próximo capítulo.

Autenticação

Eu sei que eu normalmente começo um capítulo com uma pequena história, mas eu estou meio sem tempo esta semana, então vamos fazer o seguinte. Se nós todos trabalharmos juntos e usarmos nossa imaginação, nós podemos fingir que eu escrevi uma história divertida sobre o Ian Landsman, dois leões marinhos treinados, uma garrafa de advocaat (uma bebida holandesa) e o sistema de metrô de Roma. É, isto deve servir.

E o que nós vamos ver neste capítulo? Nós todos possuímos segredos, certo? Segredos obscuros que você não quer que ninguém saiba? Não, somente eu? Bem, se por acaso você tiver alguns destes segredos, você vai precisar de uma maneira segura de protegê-los dos olhos espíões. Neste capítulo nós vamos ver como nós podemos utilizar o sistema de autenticação do Laravel para garantir que apenas os usuários autenticados possam acessar nosso conteúdo restrito.

Certo, vamos começar! Mas antes, nós vamos precisar de uma nova instalação do Laravel, com o banco de dados configurado. Você se lembra como configurar o banco de dados, certo? Caso não se lembre, por que não voltar para o capítulo de banco de dados novamente para dar mais uma olhada?

Antes de utilizar o sistema de autenticação, nós precisamos decidir onde nós vamos armazenar nossas sessões. Uma sessão é um registro que fica associado a um cookie de um navegador e vai permitir que o PHP e o Laravel saibam quem é você entre as requisições. Vamos dar uma olhada dentro do arquivo `app/config/session.php` para mais informações.

```
1  /*
2  |-----
3  | Default Session Driver
4  |-----
5  |
6  | This option controls the default session "driver" that will be used on
7  | requests. By default, we will use the lightweight native driver but
8  | you may specify any of the other wonderful drivers provided here.
9  |
10 | Supported: "native", "cookie", "database", "apc",
11 | "memcached", "redis", "array"
12 |
13 */
14
15 'driver' => 'database',
```

Como você pode ver, o Laravel nos oferece diversas maneiras para armazenar nossa sessão. Você pode utilizar o método que melhor se adaptar à sua aplicação, mas para este exemplo nós vamos utilizar a opção 'database' para armazenar nossas sessões dentro de uma tabela.

Existem algumas outras configurações neste arquivo, que incluem o tamanho da sessão, o nome do cookie da sessão, o nome da tabela das sessões e mais alguns outros. Nós vamos deixar todos estes itens com seus valores padrões neste exemplo, mas fique à vontade para pesquisar e ver quais configurações estão disponíveis.

Agora que nós decidimos que vamos utilizar o banco de dados como método de armazenamento das sessões, nós precisamos criar a tabela que vai armazenar as informações da sessão. Novamente, o Laravel já pensou nisso e criou um comando para isto. “Simplesmente utilize o comando `artisan session:table` para criar sua tabela de sessões.”, disse o Laravel.

“Obrigado, Laravel!”. Vamos testar.

```
1 $ php artisan session:table
2 Migration created successfully!
```

O comando `artisan session:table` criou uma migração que vai criar nossa tabela no banco de dados. Nós já aprendemos sobre migrações, então vamos rodar o comando `migrate` para executar as migrações.

```
1 $ php artisan migrate
2 Migrated: 2013_12_07_231153_create_session_table
```

Se nós examinarmos nosso banco de dados, veremos que nossa tabela de sessões (sessions) foi criada. Ótimo! Agora nós precisamos criar alguns usuários que serão utilizados para fazer login em nossa aplicação.

Por padrão, o Laravel contém o modelo `User`, que fica na pasta `app/models/User.php`. Vamos dar uma olhada nele.

```
1 <?php
2
3 use Illuminate\Auth\UserInterface;
4 use Illuminate\Auth\Reminders\RemindableInterface;
5
6 class User extends Eloquent implements UserInterface, RemindableInterface {
7
8     /**
9      * The database table used by the model.
10     *
11     * @var string
12     */
13     protected $table = 'users';
14 }
```

```
15     /**
16      * The attributes excluded from the model's JSON form.
17      *
18      * @var array
19      */
20     protected $hidden = array('password');
21
22     /**
23      * Get the unique identifier for the user.
24      *
25      * @return mixed
26      */
27     public function getAuthIdentifier()
28     {
29         return $this->getKey();
30     }
31
32     /**
33      * Get the password for the user.
34      *
35      * @return string
36      */
37     public function getAuthPassword()
38     {
39         return $this->password;
40     }
41
42     /**
43      * Get the e-mail address where password reminders are sent.
44      *
45      * @return string
46      */
47     public function getReminderEmail()
48     {
49         return $this->email;
50     }
51
52 }
```

Muitas coisas aqui. Vamos dar uma olhada mais detalhada em cada seção.

```
1 <?php
2
3 use Illuminate\Auth\UserInterface;
4 use Illuminate\Auth\Reminders\RemindableInterface;
5
6 class User extends Eloquent implements UserInterface, RemindableInterface {
```

Nosso modelo `User` implementa as interfaces `UserInterface` e `RemindableInterface`. Por que isto? Bem, a interface `UserInterface` serve para o Laravel saber que o nosso modelo contém todos os métodos necessários para autenticá-lo utilizando o sistema de autenticação do framework. A interface `RemindableInterface` permite que o Laravel saiba como obter o e-mail do usuário para que a funcionalidade de lembrete de senhas possa enviar os e-mails com lembretes de senha.

Nós já sabemos o que a propriedade `$table` faz, como vimos no capítulo do Eloquent ORM, mas o que é a próxima propriedade?

```
1 /**
2  * The attributes excluded from the model's JSON form.
3  *
4  * @var array
5  */
6 protected $hidden = array('password');
```

Os campos que estão contidos dentro do array `$hidden` vão ser excluídos de qualquer saída que seja gerada utilizando os métodos `toJson()` e `__toString()`. Nós não queremos que a senha do nosso usuário seja exibida nestes resultados. Você pode remover esta propriedade se quiser, eu vou deixá-la ali mesmo.

Vamos dar uma olhada nos métodos que estão definidos em nosso modelo com mais detalhes. Nós vamos começar com o método `getAuthIdentifier()`.

```
1 <?php
2
3 /**
4  * Get the unique identifier for the user.
5  *
6  * @return mixed
7  */
8 public function getAuthIdentifier()
9 {
10     return $this->getKey();
11 }
```


Este método nos permite especificar a propriedade única utilizada para identificar o nosso modelo. Por padrão, é retornada a chave primária utilizando o método `$this->getKey()`, que é herdada do nosso modelo base. Porém, nós podemos trocar esta propriedade para qualquer propriedade única que quisermos. Vamos deixar a padrão para este exemplo.

```
1  /**
2   * Get the password for the user.
3   *
4   * @return string
5   */
6  public function getAuthPassword()
7  {
8      return $this->password;
9  }
```

O método `getAuthPassword()` é utilizado para identificar qual propriedade é utilizada como a senha dos usuários em nosso modelo. O valor padrão é a coluna `password`, mas novamente, você pode alterá-la para atender suas necessidades.

Por último, nós temos o método `getReminderEmail()`.

```
1  /**
2   * Get the e-mail address where password reminders are sent.
3   *
4   * @return string
5   */
6  public function getReminderEmail()
7  {
8      return $this->email;
9  }
```

Este método é utilizado para identificar a propriedade do modelo que contém o e-mail do usuário. Este método vai ser utilizado quando nós utilizarmos o mecanismo de lembrete de senhas do Laravel.

Vamos criar a migração para o nosso modelo `User`. Você lembra de como utilizar o Schema Builder e o sistema de migração, certo? É algo mais ou menos assim...

```
1  $ php artisan migrate:make create_user_table
2  Created Migration: 2013_12_07_233727_create_user_table
3  Generating optimized class loader
```

Agora vamos completar a nossa migração que foi gerada. Nós vamos adicionar apenas as colunas `'username'`, `'password'` e `'email'`. O campo `'password'` vai possuir 60 caracteres, suportando o tamanho máximo do sistema de encriptação do Laravel. Veremos mais disto depois!

```
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4
5  class CreateUserTable extends Migration {
6
7      /**
8       * Run the migrations.
9       *
10      * @return void
11      */
12     public function up()
13     {
14         Schema::create('users', function($table)
15         {
16             $table->increments('id');
17             $table->string('username', 128)->unique();
18             $table->string('password', 60);
19             $table->string('email', 320)->unique();
20             $table->timestamps();
21         });
22     }
23
24     /**
25      * Reverse the migrations.
26      *
27      * @return void
28      */
29     public function down()
30     {
31         Schema::drop('users');
32     }
33
34 }
```

Eu também adicionei a restrição de campo único para as colunas 'username' e 'email', visto que nós não queremos ter estas colunas repetidas. Também, sempre é bom adicionar uma chave primária auto-incremental e as datas de criação e atualização na tabela. Vamos rodar nossas migrações novamente.

```
1 $ php artisan migrate
2 Migrated: 2013_12_07_233727_create_user_table
```

Agora nós precisamos criar uma maneira para adicionar usuários em nosso sistema. Vamos precisar de um formulário. Todos os desenvolvedores gostam de formulários, certo? Nós vamos pular a validação neste caso para não deixar este capítulo tão grande, mas se você quiser implementar validação por que não dar uma outra olhada no capítulo de validação? Considere isto seu dever de casa!

Vamos começar criando uma view para o nosso formulário de registro. Nós vamos mantê-lo simples. Eu faço um pouco de front-end também e sei que não faz sentido colocar os elementos do formulário dentro de uma tag `<p>`, porém, eu não quero adicionar CSS neste exemplo, então vamos utilizar desta maneira para melhor visualizar nossos elementos.

Abaixo está nossa view com o formulário que eu salvei em `app/views/create_user_form.blade.php`.

```
1 <html>
2   <head>
3     <title>Create User</title>
4   </head>
5   <body>
6     <form action="{{ url('user') }}" method="post">
7       <p><label for="username">Username:</label></p>
8       <p><input type="text" name="username" placeholder="Username" /></p>
9       <p><label for="email">Email:</label></p>
10      <p><input type="text" name="email" placeholder="Email" /></p>
11      <p><label for="password">Password:</label></p>
12      <p><input type="password" name="password" placeholder="Password" /></p>
13    </form>
14    <p><input type="submit" value="Create"></p>
15  </body>
16 </html>
```

Uma obra de arte, tenho certeza que você concorda! Eu gosto de escrever HTML, então raramente utilizo o componente Form Builder do Laravel. Se você quiser, vá em frente. Eu não vou impedir você. O Laravel nos deixa muito à vontade para fazer nossas próprias decisões, e você deve fazer isto.

Nós criamos um formulário com os três campos que o nosso modelo `User` precisa, e um botão para enviar o formulário. Eu deixei de fora o token `csrf` por simplicidade. Também utilizei a função `url()` para direcionar o formulário para nossa rota POST `/user`. Vamos em frente e criar as rotas necessárias.

```
1  <?php
2
3  // app/routes.php
4
5  Route::get('/', function()
6  {
7      return View::make('hello');
8  });
9
10 Route::get('/user', function()
11 {
12     return View::make('create_user_form');
13 });
14
15
16 Route::post('/user', function()
17 {
18     $user = new User;
19     $user->username = Input::get('username');
20     $user->password = Hash::make(Input::get('password'));
21     $user->email = Input::get('email');
22     $user->save();
23
24     return Response::make('User created! Hurray!');
25 });
```

Nós mantivemos a rota / que vem por padrão, e criamos a rota GET /user que vai ser utilizada para exibir o formulário de criação.

E finalmente nós temos a rota POST /user que vai criar um novo usuário no banco de dados.

Note que nós utilizamos o método `Hash::make()` para encriptar a senha do usuário com o mecanismo de encriptação que o Laravel utiliza (bcrypt) em seu sistema de autenticação. É bom ter em mente que você pode utilizar o método `Hash::make()` em um mesmo valor várias vezes e obter resultados diferentes. Mas sem pânico! Este é o comportamento normal deste mecanismo. Existe muita matemática por trás dele que permite que sua senha possa ser autenticada novamente.

Depois, nós retornamos uma simples resposta da rota apenas para sabermos que a ação foi bem sucedida.

Ótimo! Vamos em frente e visitar a URL /user para criar nosso novo usuário. Se você não apontou seu servidor web para a pasta `public` deste projeto ainda, você pode utilizar o comando `php artisan serve` para iniciar o servidor web do PHP.

```
1 User created! Hurray!
```

Finalmente nós podemos proteger nosso conteúdo secreto. Oh, nós estamos esquecendo de alguma coisa. O conteúdo secreto. Certo... conteúdo secreto... hmm. Certo, o conteúdo secreto vai ser uma lista das celebridades que eu tenho uma queda. Não conte para minha esposa! Nós realmente precisamos que isto esteja protegido.

Primeiro vamos criar a rota GET /crush para exibir uma view.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/crush', function()
6 {
7     return View::make('crush');
8 });
```

Certo, agora nós precisamos criar uma view. Vamos criar o HTML com a lista das celebridades no arquivo app/views/crush.blade.php.

```
1 <ul>
2     <li>Keira Knightley</li>
3     <li>Emma Watson</li>
4 </ul>
```

Temos dois registros, Keira Knightley e Emma Watson. Duas lindas mulheres. Bem, Emma Watson está bem agora, mas eu não sugiro ficar babando por ela e assistir o primeiro filme do Harry Potter. Você vai se sentir muito mal. Não faça isso, uma experiência muito traumatizante.

De qualquer maneira, vamos seguir...

Agora que nós já possuímos conteúdo secreto, nós precisamos protegê-lo. Vamos começar com a mais básica forma de autenticação. A autenticação básica HTTP. É um sistema de autenticação que é uma colaboração entre o navegador e o servidor web. Permite que nós possamos criar um mecanismo de autenticação sem precisar de um formulário de login. É a melhor maneira para começarmos.

Para forçar a autenticação dentro do nosso sistema nós precisamos ‘proteger’ nossa rota. Nós podemos fazer isto utilizando um filtro nela. Vamos utilizar o filtro `auth.basic` para proteger nossa rota GET /crush dos olhos espíões.

Aqui está nosso arquivo de rotas atualizado.

```
1 Route::get('/crush', array(  
2     'before' => 'auth.basic',  
3     function()  
4     {  
5         return View::make('crush');  
6     }  
7 ));
```

Nós alteramos a rota GET /crush adicionando um array como o segundo parâmetro para podermos adicionar informações e configurações adicionais. Nós incluímos um filtro chamado auth.basic. Este é um filtro padrão do Laravel que você pode encontrar dentro do arquivo app/filters.php. Chamar este filtro é tudo que nós precisamos para proteger nossa rota utilizando a autenticação básica do HTTP.

Vamos testar. Primeiro precisamos visitar a URL GET /crush em nosso navegador. Nosso navegador vai exibir um formulário de login HTTP. Pode ser um pouco diferente dependendo do navegador.

Por padrão, o mecanismo básico de autenticação HTTP vai utilizar o campo email do modelo User como o 'username'. Então vamos digitar o email e senha que nós criamos anteriormente.

Foi retornada a nossa lista de mulheres. Esta lista agora está protegida com autenticação básica, e só vai ser exibida se nós digitarmos as credenciais corretas. Nós vamos continuar tendo acesso a esta página até a sessão do nosso navegador expirar.

Que tal deslogar? Nós podemos criar uma rota GET /logout para isto. Vamos dar uma olhada.

```
1 Route::get('/logout', function()  
2 {  
3     Auth::logout();  
4     return Response::make('You are now logged out. :(');  
5 });
```

O método Auth::logout() vai remover nossa sessão de autenticação. Se você visitar GET /logout e retornar para a rota GET /crush você vai perceber que a tela de login será apresentada novamente.

Fazer o login com o email está funcionando, mas e se nós quisermos autenticar utilizando o campo username? Claro! Sem problemas. Simplesmente altere o filtro auth.basic no arquivo app/filters.php como o exemplo abaixo.

```
1 Route::filter('auth.basic', function()  
2 {  
3     return Auth::basic('username');  
4 });
```

Nós passamos uma string como o primeiro parâmetro para o método `Auth::basic()` especificando qual campo nós queremos utilizar como o identificador no processo de login. Em nosso exemplo, nós utilizamos o campo `username` do modelo `User`.

Vamos tentar novamente, visite a rota `GET /logout` para destruir a sessão. Agora visite `GET /crush` novamente. Desta vez utilize o `username` ao invés do email para fazer login.

Ótimo, novamente nós vemos a lista.

Enquanto autenticação básica HTTP é muito rápido para configurar, nós dependemos do formulário de login dos navegadores. Infelizmente, nós não temos controle de como ele é exibido, e é muito pouco flexível. Vamos tentar utilizar o sistema de autenticação do Laravel e criar um formulário de login.

Primeiro nós vamos precisar de uma view com um formulário para digitar nossos dados. Vamos criar na pasta `app/views/login_form.blade.php`.

```
1 <html>
2   <head>
3     <title>Login</title>
4   </head>
5   <body>
6     <form action="{{ url('login') }}" method="post">
7       <p><label for="username">Username:</label></p>
8       <p><input type="text" name="username" placeholder="Username" /></p>
9       <p><label for="password">Password:</label></p>
10      <p><input type="password" name="password" placeholder="Password" /></p>
11    </form>
12    <p><input type="submit" value="Login"></p>
13    <p><input type="checkbox" name="remember" /> <label for="remember">Remember me.</label></p>
14  </body>
15 </html>
```

Aqui temos nosso formulário de login. Ele é muito similar ao nosso formulário de criação de usuários, mas desta vez nós temos apenas os campos `username` e `password`. O formulário vai direcionar para a rota `POST /login`. Nós também adicionamos um checkbox ‘Remember me’ para dizermos se desejamos que nosso login seja mantido por mais tempo. Eu tenho certeza que você já viu algo parecido com isto. Você tem utilizado a internet, certo? :)

Vamos criar a rota para exibir nosso formulário de login. Nós vamos chamá-la `GET /login`. Eu sei, criativo, certo?

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/login', function()
6 {
7     return View::make('login_form');
8 });
```

Certo então, vamos alterar nossa rota secreta GET /crush para utilizar o filtro auth ao invés do auth.basic.

```
1 <?php
2
3 // app/routes.php
4
5 Route::get('/crush', array(
6     'before' => 'auth',
7     function()
8     {
9         return View::make('crush');
10    }
11 ));
```

Agora vamos fazer logout e visitar /crush novamente.

O que aconteceu aqui? Nós fomos redirecionados para o nosso formulário de login. Vamos dar uma olhada no filtro auth no arquivo app/filters.php para descobrir o que aconteceu.

```
1 Route::filter('auth', function()
2 {
3     if (Auth::guest()) return Redirect::guest('login');
4 });
```

Sim, eu vejo. O filtro auth utiliza o método `Auth::guest()` para determinar quando o usuário é um visitante que não está logado no sistema. Se este for o caso, então o filtro vai redirecionar para a rota /login utilizando o método `Redirect::guest()`.

Woah! Foi por isto que nós chamamos nossa rota de login? Não é conveniente? É quase como se nós tivéssemos planejado isto.

Agora que nós temos nosso formulário de login, nós precisamos uma rota para tratar os dados do formulário. Vamos criar a rota POST /login.


```
1  <?php
2
3  // app/routes.php
4
5  Route::post('/login', function()
6  {
7      $credentials = Input::only('username', 'password');
8      if (Auth::attempt($credentials)) {
9          return Redirect::intended('/');
10     }
11     return Redirect::to('login');
12 });
```

Para autenticar utilizando o sistema, nós precisamos primeiro montar um array com as credenciais. Nós pegamos os valores dos campos username e password enviados pelo formulário. Nós não temos que nos preocupar com a encriptação da senha, o Laravel faz isto para nós.

Vale a pena notar que nós podemos adicionar outras opções no array de credenciais. Por exemplo, nós podemos querer adicionar 'enabled' => true para garantir que a conta do usuário esteja ativa.

Uma vez que o array com as credenciais foi montado, nós passamos ele para o método `Auth::attempt()`. Este método vai retornar um valor booleano que vai ser usado para indicar se a tentativa de login foi bem sucedida ou não. Se os dados estiverem corretos, então nossa sessão vai ser criada, e nós estamos logados no sistema.

Perceba que foi utilizado o método `Redirect::intended()` no exemplo acima. Por que isto? Bem, vamos imaginar que você está procurando um carro usado na internet. Por quê? Porque os novos parecem todos iguais. De qualquer maneira, seu amigo Dayle te mandou um link para um carro em que você possa estar interessado. Um antigo WRX Impreza ou algo do tipo. Infelizmente, este site precisa que você faça login para ver os carros. Você acessa o link e é redirecionado para um formulário de login.

Uma vez que você se logou no site, você é redirecionado para a página inicial. Isto não é muito útil, certo? Você queria ver o carro. Agora você precisa clicar no link de novo? O método `Redirect::intended()` pode ser usado para evitar esta situação.

O método `Redirect::intended()` vai redirecionar nossos usuários para a rota que eles estavam tentando acessar antes de serem redirecionados para o formulário de login. Uma vez que o processo de autenticação for completo, nós enviamos o usuário para onde ele queria ir. O primeiro parâmetro para o método é a rota padrão, para garantir que os usuário sejam direcionados para algum lugar caso eles acessem diretamente a tela de login.

Vamos testar nosso novo sistema de autenticação? Primeiro acesse `GET /logout` para fazer logout no sistema. Depois, acesse `GET /crush` para ser redirecionado para a tela de login. Preencha o formulário, envie e você deve ser redirecionado de volta para a página `GET /crush`. Muito bem!

Bem, vamos em frente.. oh espere. Nós esquecemos da funcionalidade ‘Remember me.’! Bem, acho que eu poderia voltar e editar, mas quer saber? Vamos ajustar aqui mesmo.

Vamos alterar nossa rota POST /login novamente.

```
1 <?php
2
3 // app/routes.php
4
5 Route::post('/login', function()
6 {
7     $credentials = Input::only('username', 'password');
8     $remember = Input::has('remember');
9     if (Auth::attempt($credentials, $remember)) {
10         return Redirect::intended('/');
11     }
12     return Redirect::to('login');
13 });
```

Nós adicionamos uma linha para buscar um valor booleano como o segundo parâmetro do método `Auth::attempt()`. Agora se nós marcarmos o checkbox ‘Remember me’ no formulário de login, ele vai lembrar de nossos dados nas próximas vezes que nós quisermos acessar o site. Certo, agora nós podemos seguir em frente.

Uma vez que nós já estamos logados no sistema, seria muito útil poder identificar o usuário que está logado. Não se preocupe! Taylor pensou nisto também. O método `Auth::user()` retorna uma instância do modelo `User` que representa o usuário logado. Isto significa que nós podemos utilizar todo o poder do Eloquent aqui podendo acessar os relacionamentos do modelo, ou quem sabe algo mais simples, como a propriedade `username`.

```
1 $user = Auth::user();
2 echo $user->username;
```

Certo! Senhoras e senhores. Nós temos um truque final para compartilhar com você hoje. Você já viu alguma aplicação que permite que os administradores ‘emulem’ o login com outro usuário? “Trocar para Derek”, por exemplo. Não? Bem, confie em mim, existem aplicações que fazem isto.

De qualquer maneira, eu estou ficando um pouco cansado e minha escrita está ficando um pouco estranha, então eu vou terminar e mostrar como você pode criar esta mesma funcionalidade em suas aplicações.

```
1 $user = User::find(5);  
2 Auth::login($user);  
3 // Now we're Derek!
```

Aqui está! Simplesmente passe a instância do modelo Eloquent do usuário que você deseja fazer login para o método `Auth::login()`. Muito útil!

Onde Vai Isto?

Este pequeno capítulo é baseado em uma pergunta que eu e Taylor recebemos com muita frequência. “Onde eu coloco xxx?”. É uma pergunta interessante. Nós já temos algumas localizações padrões para classes como modelos e controladores, mas no mundo PHP, e em uma aplicação baseada em Composer, nós temos muito mais liberdade do que nós estamos acostumados.

Vamos ver se nós conseguimos achar a resposta. Feche seus olhos.

Droga, eu fiz de novo. Agora como eu vou falar pra você abrir os seus olhos depois? Bem, eu vou assumir que você ficou entediado e abriu seus olhos. O fechar dos olhos é um requerimento da terra da imaginação, um lugar onde eu passo a maior parte do meu tempo. Vamos imaginar que você está em uma cozinha. Eu sei, empolgante, né? Em suas mãos você tem um prato. Onde você coloca o prato?

Eu sei! É uma pergunta realmente difícil. Muitos lugares onde podemos colocar, na pilha de secar louça, mas o prato não está molhado, não é? Eu acho que pode ir ali, mas isto faz sentido para você? Eu acho que nós podemos colocar no balcão da cozinha, iria funcionar, mas você acha isto?

Nós podemos colocar no armário! Agora sim! Longe da vista, organizado. Isto é perfeito. Então vamos colocar este prato no armá... oh. Que armário? Ah não, outra pergunta.

É apenas uma questão de preferência. E não posso te dizer onde você deve colocar seu prato e não posso guardá-lo pra você. Você nunca mais vai achá-lo se eu fizer isto. Eu gosto de adesivar meus pratos no teto da cozinha. É uma mecânismo seguro e organizado.

Mas está não é bem a resposta que você estava esperando, certo?

Eee... obrigado Dayle! OU EU DEVIA TE CHAMAR DE FAYLE!!

Não é a primeira vez que eu escuto isto. Espere um pouco, não precisamos de xingamentos. É verdade, eu não posso te dizer onde você deve colocar o seu código, mas eu posso te ajudar a tomar decisões. Eu posso te mostrar como modelar seu projeto. Parece bom?

Bem... acho que sim.

Agora sim!

Certo, vamos pensar no código do Laravel por um momento. Da maneira que eu vejo, nós temos dois tipos de código. Código bom e código ru... haha, apenas brincando. Nós temos código estrutural e código orientado a objetos.

Espere! Código estrutural? Eu sou um programador moderno, eu não estou escrevendo QBasic aqui. Me dê alguns objetos!

Se acalme um pouco. Você está um pouco mal-humorado hoje, não? O PHP sempre vai ser um pouco estrutural. Tente escrever uma aplicação em PHP com apenas uma classe. Nada mais.

:(

Sim, isso que eu pensei. O PHP não possui o equivalente ao `public static void main()` do Java. Nós precisamos de um pouco de código estrutural antes de nosso código orientado a objetos ser executado.

```
1 <?php
2
3 $a = new MyApplication;
4 $a->init(); // Start our doom machine.
```

Aqui está um exemplo de um código estrutural básico. Nós precisamos criar nossa instância e chamar o primeiro método. Se nós não fizermos isto, nós ficaremos presos com código estrutural.

O Laravel não é diferente. Vá em frente, dê uma olhada no arquivo `public/index.php`. Este é o primeiro arquivo que o nosso servidor chama quando obtemos uma nova requisição.

```
1 <?php
2 /**
3  * Laravel - A PHP Framework For Web Artisans
4  *
5  * @package  Laravel
6  * @author   Taylor Otwell <taylorotwell@gmail.com>
7  */
8
9 require __DIR__.'../bootstrap/autoload.php';
10 $app = require_once __DIR__.'../bootstrap/start.php';
11 $app->run();
```

Eu removi os blocos de comentários, mas você pegou a ideia. Nós temos um pouco de código estrutural, e depois finalmente nós iniciamos a aplicação chamando o método `$app->run()`.

E não é apenas neste arquivo, vá em frente e olhe o arquivo `app/routes.php` ou `app/global/start.php`. É tudo código estrutural. Por que isto?

Bem, na maior parte do tempo, os arquivos com código estrutural dentro do Laravel são usados para configuração ou para registro. Eventos, observadores, rotas, filtros, validações customizadas. A lógica destes componentes é normalmente orientada a objetos, mas nós utilizamos uma linha assim para registrar os componentes...

```
1 <?php
2
3 Validator::extend('foo', 'Me\Validators\Validate@validate');
```

Para o Laravel saber que ele existe.

Se nós não fizéssemos isto, o Laravel iria precisar procurar dentro dos diretórios para achar nossos arquivos. O Laravel precisa que estes arquivos estejam em algum lugar conhecido. E isto não é muito flexível, né? Vamos pensar no seu quarto por um segundo...

Apague as luzes. - E ligue o som com Barry White.

Isto mesmo. É hora do trabalho. Seu quarto não possui paredes de magnolia, um carpete marrom e a cama no canto, certo? Sem chance. É o seu espaço. Você possui posters da Keira Knightley. Fantasias do Star Wars. Você possui um monte de bugigangas por tudo o lugar. Você personalizou o lugar. Representa a sua personalidade.

E é assim que a base do seu código deve ser. Bem... se você está em um time, então deve representar a personalidade e as necessidades do time. Mas vamos pensar em projetos pessoais por enquanto. É a sua aplicação. Você cria as regras. Você é um grande kahuna.

Por isto que o Laravel não procura por validadores, observadores e todas as outras coisas nas localizações padrões. Você têm liberdade para colocá-las onde **você** quiser.

E quanto aos modelos e controladores?

Sim, sim... eu sei que existem diretórios dentro do app para isto. Isto porque todo mundo espera modelos e controladores dos frameworks. Se nós entregarmos o framework com um diretório vazio, as pessoas vão ficar tímidas para começar. Nós precisamos começar em algum lugar.

Isto não significa que você precisa manter esta estrutura de diretórios. Nós não estamos forçando você à utilizá-la. Quebre o molde!

Vamos ver cada tipo de código, e ver o que nós podemos fazer para customizar nosso projeto.

Código Estrutural

Quando nós pensamos em código estrutural, `app/routes.php` aparece em nosso radar. Hmm, eu me pergunto de onde o arquivo `routes.php` é executado. Bem, vamos dar uma olhada no arquivo `vendor/laravel/framework/src/Illuminate/Foundation/start.php`. Apenas um palpite.

```

1  <?php
2
3  // Lots of other bootstrap code here...
4
5  /*
6  |-----
7  | Load The Application Routes
8  |-----
9  |
10 | The Application routes are kept separate from the application starting
11 | just to keep the file a little cleaner. We'll go ahead and load in
12 | all of the routes now and return the application to the callers.
13 |
14 */
15
16 $routes = $app['path'].'/routes.php';
17
18 if (file_exists($routes)) require $routes;
19
20 });

```

Hey, aqui está! Este foi com certeza um palpite de sorte, não foi? Se o arquivo `app/routes.php` existir, é carregado durante o processo de inicialização. Como este código está dentro do diretório `vendor`, e eu espero que você não esteja versionando ele junto com sua aplicação, seria uma má ideia incluir arquivos adicionais aqui.

Vá em frente e dê uma olhada no arquivo `start.php`. Você vê esta seção?

```

1  <?php
2
3  /*
4  |-----
5  | Load The Application Start Script
6  |-----
7  |
8  | The start script gives us the application the opportunity to override
9  | any of the existing IoC bindings, as well as register its own new
10 | bindings for things like repositories, etc. We'll load it here.
11 |
12 */
13
14 $path = $app['path'].'/start/global.php';
15
16 if (file_exists($path)) require $path;

```

Se você olhar para o comentário, ele diz que neste arquivo nós podemos sobrescrever as associações do IoC container. Isto significa que os componentes do Laravel já foram carregados neste ponto, e nós já podemos utilizá-los.

Isto significa que nós podemos utilizar o nosso velho método `require()` para incluir nossos próprios arquivos do `app/start/global.php` e definir nossa própria estrutura para o nosso código estrutural.

Eu acho que isto ficaria melhor com um exemplo.

```
1  <?php
2
3  // More bootstrap code..
4
5  /*
6  |-----
7  | Require The Filters File
8  |-----
9  |
10 | Next we will load the filters file for the application. This gives us
11 | a nice separate location to store our route and application filter
12 | definitions instead of putting them all in the main routes file.
13 |
14 */
15
16 require app_path().'/filters.php';
17
18 // Create our own structure for files.
19
20 require app_path().'/observers.php';    // Model observers.
21 require app_path().'/validators.php';    // Custom validators.
22 require app_path().'/listeners.php';    // Event listeners.
23 require app_path().'/composers.php';    // View composers.
24 require app_path().'/services.php';    // Container bindings.
25 require app_path().'/helpers.php';    // Helper functions.
```

Não se esqueça de realmente criar estes arquivos, caso contrário sua aplicação vai resultar em erros. A função `app_path()` fornece o caminho para o diretório `/app` da sua aplicação. Nós criamos agora alguns arquivos para armazenar o registro do nosso código estrutural dentro. Isto significa que nós não estamos enchendo o arquivo `routes.php` com registro de coisas diferentes.

Não assuma que estes arquivos são a **melhor maneira** de organizar sua aplicação. Crie sua própria estrutura. É o seu quarto. Coloque posters das **suas** bandas favoritas, não as minhas.

Se você quiser, você pode colocar fora do diretório `/app`. Na verdade, vou contar um pequeno segredo. Quando eu estou estruturando minhas aplicações, o diretório `/app` é praticamente vazio. Apenas as configurações ficam por lá.

Quando eu comecei em um novo emprego recentemente, meu colega Anthony, que está agora pulando para cima e para baixo porque eu falei dele neste livro, ou está assinando minha demissão porque eu não pedi permissão para ele, estava assustado ao ver como eu estruturei as minhas aplicações.

Não se parece nada com os exemplos das aplicações de Laravel que você vê no Github. Nada no diretório `app`. Minhas rotas ficam em um arquivo `/src/routes.php`. Isto porque ele quis que eu definisse a estrutura do projeto, e eu fiz do meu jeito. Óbvio que se você estiver trabalhando em um grande time, você vai precisar montar colaborativamente uma estrutura que faça sentido para todo o time. Você não quer confundir ninguém.

Você deve estar se perguntando por que eu não usei esta estrutura nos meus exemplos? Bem, existe um pequeno truque para ser um professor. Quando você sugere uma técnica, as pessoas tendem a transformá-la em um padrão. Eles falam para seus amigos sobre ela, e quando menos percebem ela se transforma na única maneira possível de resolver um problema.

Eu fiquei preocupado que este seria o caso com a estrutura da aplicação. Eu realmente gostaria que isto se torne uma coisa pessoal para você, ou para o seu time. Por isto meus exemplos vão utilizar os locais pré-definidos pelo Laravel.

Agora que nós temos a habilidade de estruturar nosso código estrutural, vamos dar uma olhada onde nós podemos colocar nossas classes.

Código Orientado a Objetos

Sua aplicação Laravel é baseada no Composer. Isto permite que você tenha muita liberdade para estruturar suas classes. Vamos dar uma olhada no arquivo padrão `composer.json` que vem com o Laravel, mais especificamente na seção de `autoload`.

```
1  "autoload": {  
2      "classmap": [  
3          "app/commands",  
4          "app/controllers",  
5          "app/models",  
6          "app/database/migrations",  
7          "app/database/seeds",  
8          "app/tests/TestCase.php"  
9      ]  
10 },
```

Nós já lemos todo o capítulo sobre Composer, então você sabe que as classes dentro do `classmap` e todas as classes nos subdiretórios serão convertidas em um grande mapa com o nome e o caminho das classes toda vez que o comando `composer dump-autoload` for executado.

E é assim que o Laravel encontra nossos arquivos. O problema é que as pessoas não estavam acostumadas com o Composer quando o Laravel 4 foi lançado, e elas freqüentemente esqueciam de utilizar o comando `composer dump-autoload` toda vez que criavam uma nova classe. Isto levou os usuários a criarem muitas postagens perguntado por que “o Laravel está dizendo que minha classe não pode ser encontrada”.

Para evitar este problema, Taylor adicionou um segundo mecanismo de autoload dentro do framework. Vamos dar uma olhada no arquivo `app/start/global.php` novamente.

```
1  <?php
2
3  /*
4  |-----
5  | Register The Laravel Class Loader
6  |-----
7  |
8  | In addition to using Composer, you may use the Laravel class loader to
9  | load your controllers and models. This is useful for keeping all of
10 | your classes in the "global" namespace without Composer updating.
11 |
12 */
13
14 ClassLoader::addDirectories(array(
15
16     app_path().'/commands',
17     app_path().'/controllers',
18     app_path().'/models',
19     app_path().'/database/seeds',
20
21 ));
```

As classes que estão localizadas dentro destes diretórios não precisam que o comando `composer dump-autoload` seja executado toda vez que uma classe é adicionada. Então se você quiser adicionar novas localizações para suas classes, você pode simplesmente adicionar outros diretórios no `composer.json` ou no arquivo `app/start/global.php`.

Isto é legal, mas eu gosto de pensar que o verdadeiro poder do Composer é quando estamos carregando classes que possuem *namespaces*. Vamos adicionar um novo mecanismo de autoload no arquivo `composer.json`.

```
1 "autoload": {  
2     "classmap": [  
3         "app/database/migrations",  
4         "app/database/seeds"  
5     ],  
6     "psr-0": {  
7         "Example": "src"  
8     }  
9 },
```

Para utilizar o carregamento de classes com psr-0, você primeiro precisa pensar em um namespace base. No exemplo acima, eu utilizei o namespace 'Example', que aponta para a pasta src, relativa a raiz do projeto.

Você provavelmente vai utilizar algo diferente de Example como raiz do seu namespace. As pessoas costumam utilizar seus sobrenomes ou o nome da empresa. Qualquer coisa que não seja 'Example', por favor!

Uma vez que nós rodamos o comando `composer update` e o nosso mecanismo de autoload foi registrado, nós podemos criar a estrutura que quisermos dentro do diretório src, contanto que fique dentro do subdiretório Example.

Por exemplo, sua aplicação pode se parecer com isto.

```
1 src/Example/Controllers/UsersController.php  
2 src/Example/Controllers/BooksController.php  
3 src/Example/Models/User.php  
4 src/Example/Models/Book.php  
5 src/Example/Validators/UsersValidator.php  
6 src/Example/Validators/BooksValidator.php  
7 src/Example/Observers/BooksObserver.php  
8 src/Example/SomethingDoer/Other/DoSomething.php
```

Viu? Nós podemos criar nossa própria estrutura de diretórios. Se você não gosta do 'Controllers' como um diretório, você pode chamar como quiser. O Laravel e o Composer não possuem regras para onde as suas classes devem estar. Lembre-se, este é o **seu** quarto.

Apenas tenha em menta que a sua estrutura de pastas deve ser a mesma estrutura do seu namespace.

Por exemplo, no exemplo acima, é assim que vai ser a estrutura do arquivo `src/Example/Validators/UsersValidator.php`.

```
1  <?php
2
3  namespace Example\Validators;
4
5  class UsersValidator
6  {
7      // Your code here...
8  }
```

Quando estiver utilizando classes com namespaces, em qualquer um dos métodos de ‘registro’ do Laravel, não se esqueça de escrever o namespace completo junto com o nome da classe. Por exemplo, é assim que você vai rotear um arquivo pertencente a um namespace.

```
1  Route::get('/', 'Example\Controllers\UsersController@showUsers');
```

Existe mais uma coisa que você deve saber quando estiver utilizando classes com namespace. Eu sempre esqueço disto! Quando sua classe pertence a um namespace, o PHP assume que todas as classes que forem utilizadas dentro dela vão estar localizadas dentro do mesmo namespace. Vamos ver um exemplo.

```
1  <?php
2
3  namespace Example\Controllers;
4
5  class UsersController extends Controller
6  {
7      public function showUsers()
8      {
9          return 'all the users! \n/';
10     }
11 }
```

Nós temos o nosso controller UsersController, que estende o Controller base do Laravel. Certo? Certo!?

Errado.

Como nós estamos dentro do namespace Example\Controllers, o PHP vai assumir que nós estamos estendendo o arquivo Example\Controllers\Controller. Nós precisamos dizer explicitamente que nós queremos utilizar o Controller do namespace global.

```
1  <?php
2
3  namespace Example\Controllers;
4
5  use Controller;
6
7  class UsersController extends Controller
8  {
9      public function showUsers()
10     {
11         return 'all the users! \o/';
12     }
13 }
```

Agora o nosso controller vai funcionar como o esperado.

Mesmo que isto funcione corretamente, eu estou um pouco agitado. Ao invés de utilizar o atalho (alias, em inglês), eu gosto de utilizar a classe real. O quê? Você não sabia que a classe `Controller` é um atalho? Vá em frente e olhe o arquivo `app/config/app.php`.

```
1  'aliases' => array(
2
3      'App'           => 'Illuminate\Support\Facades\App',
4      'Artisan'       => 'Illuminate\Support\Facades\Artisan',
5      'Auth'          => 'Illuminate\Support\Facades\Auth',
6      'Blade'         => 'Illuminate\Support\Facades\Blade',
7
8      .. many other aliases ..
9
10     'Seeder'        => 'Illuminate\Database\Seeder',
11     'Session'       => 'Illuminate\Support\Facades\Session',
12     'SSH'           => 'Illuminate\Support\Facades\SSH',
13     'Str'           => 'Illuminate\Support\Str',
14     'URL'           => 'Illuminate\Support\Facades\URL',
15     'Validator'     => 'Illuminate\Support\Facades\Validator',
16     'View'          => 'Illuminate\Support\Facades\View',
17
18 ),
```

Olhe! Um monte de atalhos. Você não pensou que o Laravel iria criar todas as suas classes no namespace global, pensou? Dentro do array `aliases` nós podemos ver as **reais** classes que os atalhos representam.

Ao escrever minhas classes com namespaces, eu gosto de utilizar as classes **reais**. Eu acho que deixa o código mais descritivo, e também melhora a documentação de qualquer pacote gerador de documentação que você venha a utilizar.

Aqui está um outro exemplo.

```
1  <?php
2
3  namespace Example\Controllers;
4
5  use Illuminate\Routing\Controller;
6  use Illuminate\Support\Facades\Input;
7
8  class UsersController extends Controller
9  {
10     public function showUsers()
11     {
12         return Input::get('name');
13     }
14 }
```

Bem, aqui está. Nós vimos algumas maneiras de estruturar sua aplicação fora do formato ‘padrão’. Por favor, não copie os exemplos deste capítulo ao pé da letra. Experimente, tente criar sua própria estrutura. É o seu quarto.

Então da próxima que você se perguntar...

Onde eu coloco xxx?

Você não deveria estar fazendo download de xxx, seu pervertido! Apenas brincando.

A resposta sempre vai ser... onde você quiser!

Eventos

Querido leitor,

Você foi convidado para a festa dos pandas felizes do Dayle Ress. Por favor, vista-se adequadamente e traga bandeiras coloridas. Mais detalhes no evento.

Atenciosamente,

Lushui.

Tempos empolgantes, certo? Você chega na festa vestindo sua melhor roupa de domingo com uma bandeira colorida em sua mão. No salão principal você está cercado por milhares de outros desenvolvedores. Nas paredes existem pinturas feitas em óleo pelos famosos pandas vermelhos que serviram a família Ress no passado.

Champagne e caviar são servidos pelos pandas vestindo seus melhores smokings. É um belo evento. Eu tenho um bom gosto, não é?

Eu acho que já está na hora de eu me dirigir aos meus convidados.

Sejam bem vindos à festa anual dos pandas. Você foi convidado devido ao seu fantástico gosto pela escrita técnica.

Agora, eu sei que as conversas paralelas neste tipo de evento podem ser um pouco estranhas para os desenvolvedores. Mas não tema! Eu criei um belo sistema para isto. Eu espero que todos vocês tenham trazidos suas bandeiras coloridas.

- um brinde *

Excelente!

Nós vamos trabalhar assim. Se você quiser conversar sobre o tempo, levante sua bandeira vermelha. Se você quiser conversar sobre recentes acontecimentos esportivos, levante sua bandeira azul. E finalmente, se você quiser falar sobre vídeo games, levante sua bandeira verde.

Com este simples sistema nós podemos aproveitar esta noite. Por favor, sirva-se de champagne e vamos brindar aos nossos futuros projetos utilizando Laravel.

Conceito

Por que tudo isso? Bem, apesar da história, serve para ilustrar os conceitos básicos da programação orientada a eventos. Segurando uma bandeira colorida no alto, nós estamos ‘disparando’ um evento. Abaixo um exemplo.

```
1 Event::fire('raise.blue.flag');
```

As outras pessoas podem ‘observar’ este evento. Abaixo um exemplo.

```
1 Event::listen('raise.blue.flag', function()  
2 {  
3     return new SmallTalk::make('sports');  
4 });
```

Com o nosso observador de eventos setado, assim que alguém levantar uma bandeira azul nós automaticamente iniciamos uma conversa sobre esportes com esta pessoa. É um processo muito eficiente. Nós podemos criar quantos observadores nós quisermos. E nós podemos até mesmo criar múltiplos observadores para o mesmo evento.

E como nós podemos implementar este sistema no Laravel? Bem, é exatamente como nos exemplos acima. Ok, eu menti novamente. A classe `SmallTalk` não existe. Mas o resto é exatamente igual! Por que não damos uma olhada mais de perto?

Disparando Eventos

Todos os eventos são disparados com uma ‘chave’. Nós poderemos utilizar esta chave para observar os eventos mais tarde. Para disparar um evento, nós utilizamos o método `fire()` da classe `Event`. O primeiro e único parâmetro obrigatório deste método é a própria chave. Vamos ver um exemplo.

```
1 Event::fire('my.event');
```

Qualquer observador que foi registrado com esta chave será executado assim que esta linha de código for executada.

É muito útil poder ser notificado desta maneira, mas nós podemos fazer mais ainda com o sistema de eventos. Nós podemos passar parâmetros adicionais junto com nossos eventos. E nossos observadores podem responder esta informação, ou talvez até modificá-la.

Vamos ver um exemplo.

```
1 Event::fire('my.event', array($object, $value));
```

Com um segundo parâmetro opcional, nós podemos passar um array de variáveis PHP que serão recebidas pelos nossos observadores. Aqui, as regras padrões do PHP se aplicam. Os objetos são passados diretamente, e os tipos básicos de variáveis são passados como cópias.

Isto significa que se você passar um objeto no segundo parâmetro, o observador vai poder modificá-lo.

Vamos dar uma olhada no outro lado da equação. É hora de registrar alguns observadores de eventos.

Observando Eventos

Observar (ou escutar) um evento é simples. Nós podemos utilizar o método `Event::listen()`. O primeiro parâmetro é a 'chave' do evento que nós desejamos responder e o segundo parâmetro é uma função anônima que vai responder ao evento.

Como sempre, vamos ver um exemplo.

```
1 Event::listen('my.event', function()  
2 {  
3     // Executar alguma ação  
4     // Atualizar a base de dados?  
5 });
```

Dentro desta função anônima nós podemos executar qualquer ação que nós quisermos. Nós podemos fazer um log do evento ocorrido. Nós podemos atualizar o banco de dados. Qualquer coisa é possível. Use sua imaginação!

Se o nosso evento for disparado com parâmetros adicionais, nós podemos capturá-los adicionando os parâmetros em nossa função anônima. Vamos dar uma olhada nisto na prática.

```
1 Event::listen('my.event', function($first, $second)  
2 {  
3     // Use $first and $second.  
4 });
```

Os parâmetros são passados na mesma ordem do array que é passado para o método `fire()`.

Como eu mencionei anteriormente, nós podemos registrar múltiplos observadores para o mesmo evento e todos eles serão executados. Abaixo um exemplo.

```
1 Event::listen('my.event', function()  
2 {  
3     // Primeiro observador  
4 });  
5  
6 Event::listen('my.event', function()  
7 {  
8     // Segundo observador  
9 });  
10  
11 Event::listen('my.event', function()  
12 {
```

```
13     // Terceiro observador
14 });
15
16 Event::fire('my.event');
```

No exemplo acima, todos os observadores serão executados quando o evento `my.event` for disparado.

E se nós quisermos que os outros observadores não sejam executados? Sem problemas! Se nós retornarmos `false` do nosso observador, nenhum outro evento será disparado.

Por exemplo, com este código...

```
1 Event::listen('my.event', function()
2 {
3     // First listener.
4     return false;
5 });
6
7 Event::listen('my.event', function()
8 {
9     // Second listener.
10 });
11
12 Event::listen('my.event', function()
13 {
14     // Third listener.
15 });
16
17 Event::fire('my.event');
```

... apenas o primeiro observador será executado.

Eu tenho certeza que você está ficando cansado comigo repetindo isto, mas sempre que há uma função anônima, você também pode utilizar uma classe PHP. A sintaxe é a mesma de sempre. Primeiro é preciso criar a classe que vai manipular o evento.

```
1 class MyListener
2 {
3     public function process()
4     {
5         // Handle the event here.
6     }
7 }
```

Depois, você registra o observador utilizando a mesma notação de quando você define um controller e uma ação. Abaixo um exemplo.

```
1 Event::listen('my.event', 'MyListener@process');
```

O segundo parâmetro para o método `listen()` é a classe e o método, separados por um `@`. Se você não passar nenhum método, então o sistema de eventos vai utilizar automaticamente o método `handle()`.

Os eventos também podem ser observados com prioridades. Desta maneira nós podemos alterar a ordem em que os observadores são executados. Abaixo um exemplo.

```
1 Event::listen('my.event', function()
2 {
3     // First listener.
4 }, 1);
5
6 Event::listen('my.event', function()
7 {
8     // Second listener.
9 }, 3);
10
11 Event::listen('my.event', function()
12 {
13     // Third listener.
14 }, 5);
15
16 Event::fire('my.event');
```

Especificando um número inteiro como o terceiro parâmetro para o método `listen()`, nós podemos alterar a ordem que os nossos observadores são executados. Os observadores com os maiores números serão executados por primeiro. No exemplo acima, os observadores serão executados na ordem reversa.

Ao registrar eventos, você não precisa ser totalmente preciso com a chave do evento. Se você quiser, você pode utilizar um asterisco (*) dentro do seu observador.

```
1 Event::listen('my.*', function()  
2 {  
3     // ..  
4 });
```

Este observador agora será executado toda vez que um evento que comece com o prefixo `my.` for executado.

Registrando Eventos

No capítulo anterior eu mostrei alguns locais onde você pode criar seus códigos. Com os eventos, você pode ter códigos para colocar no local escolhido. Criando uma classe para registrar eventos (chamada de ‘Event Subscriber’), você pode registrar múltiplos eventos nela.

Abaixo um exemplo de uma classe utilizada para registrar eventos.

```
1 class MyListeners  
2 {  
3     public function firstListener()  
4     {  
5         // First event listener.  
6     }  
7  
8     public function secondListener()  
9     {  
10        // Second event listener.  
11    }  
12  
13    public function thirdEventListener()  
14    {  
15        // Third event listener.  
16    }  
17  
18    public function subscribe($events)  
19    {  
20        $events->listen('first.event', 'MyListeners@firstListener');  
21        $events->listen('second.event', 'MyListeners@secondListener');  
22        $events->listen('third.event', 'MyListeners@thirdListener');  
23    }  
24 }
```

Nossa classe é muito parecida com a classe padrão de eventos. Possui os métodos que serão executados quando os eventos forem disparados, e também possui o método `subscribe()`. Este método recebe uma instância da classe de eventos, que neste caso nós chamamos de `$events`.

Nos exemplos anteriores nós utilizamos a classe `Event` para disparar e observar eventos, lembra? Bem, nossa classe é exatamente a mesma coisa. Nós podemos utilizá-la para registrar nossos observadores de dentro da nossa classe.

Para que nossa nova classe funcione corretamente, nós precisamos apenas realizar uma chamada para o método `subscribe()` da classe `Event`, passando uma nova instância da nossa classe por parâmetro.

```
1 Event::subscribe(new MyListeners);
```

Seja qual for a maneira que você escolher para registrar seus eventos, vai funcionar muito bem. Agora pegue outra taça de champagne. Eu pago!

Eventos Globais

Quer ouvir um outro segredo? Apenas entre nós. Eu não quero compartilhar isto com os outros desenvolvedores, ok? Nós podemos espiar o Laravel.

Isto mesmo. Nós podemos interceptar suas mensagens se quisermos. O Laravel dispara os seus próprios eventos. Por exemplo, quando o Laravel executa um query SQL, o evento `illuminate.query` é disparado. Lembre-se que `illuminate` é o nome da suíte de componentes do Laravel.

O parâmetro passado para o evento `illuminate.query` contém a consulta SQL que esta sendo executada.

O Laravel também dispara o evento `illuminate.log` quando a classe `Log` é chamada. Analisando os parâmetros que são passados para o evento nós podemos interceptar as mensagens que estão sendo registradas no sistema.

Existem outros eventos também, que de tão úteis o Taylor criou alguns atalhos que podem ser utilizados para observar estes eventos.

Se você passar uma função anônima para o método `App::after()`, ela será executada depois que o framework completar seu ciclo de requisição-resposta. Logo antes da resposta ser enviada para o cliente.

Os objetos da requisição (`$request`) e da resposta (`$response`) são enviados como parâmetros para o evento. Isto significa que você possui ‘mais uma chance’ para alterar a resposta antes de enviá-la para o cliente.

```
1 App::after(function ($request, $response) {  
2     $response->headers->set('Access-Control-Allow-Origin', '*');  
3 });
```

No exemplo acima nós modificamos a resposta adicionando o cabeçalho 'Access-Control-Allow-Origin'. Este cabeçalho agora será enviado em todas as respostas do sistema.

Você também possui acesso ao método `App::before()`, um evento que é disparado antes das rotas serem processadas. Este evento apenas passa o objeto da requisição (`$request`) como parâmetro.

Exemplos de uso destes dois métodos podem ser encontrados dentro do arquivo `app/filters.php`. Mesmo que eles fiquem dentro do arquivo de filtros, eles são implementados utilizando os eventos.

Casos de Uso

Eu com certeza encorajo você a utilizar o sistema de eventos de uma maneira criativa para resolver os seus problemas, mas aqui estão alguns casos de uso.

Log e Análise

Quando uma função específica ocorre nas minhas aplicações, eu gosto de disparar um evento. Aqui estão alguns eventos que podem ser disparados.

- `user.created`
- `user.deleted`
- `profile.updated`

Eu posso querer fazer um log desta informação, enviar para um serviço externo ou até executar alguma ação. Apenas é necessário registrar um observador para ser executado quando estes eventos forem disparados.

Disparar um evento não prejudica a performance do sistema em quase nada, então eu costumo disparar eventos sempre que for útil para mim. Nunca se sabe quais os tipos de ações que você vai precisar executar mais tarde.

Hooks

Em CMS's e sistemas mais complexos, você normalmente possui uma maneira de adicionar suas extensões ao código ou até mesmo modificar os objetos que o sistema trabalha.

Os eventos são uma maneira fantástica de permitir que os desenvolvedores possam fazer isto sem precisarem ficar estendendo as suas classes.

Este capítulo é uma preparação para um futuro capítulo sobre o princípio de inversão de controle e o container de serviços ('service container') do Laravel.

O Container

Shauna Gordon rastejou lentamente como um rato até a entrada da mansão Otwell. Ela rastejou pelas sombras, fora da vista. Shauna era muito discreta. Ela não era uma desenvolvedora, e sim uma das mais renomadas assaltantes de gatos dos Estados Unidos da Americalandia. Porém, naquela noite, ela não ia roubar gatos. Não, ela estava de olho em um prêmio muito maior.

Ela pegou seu kit de abrir fechaduras e começou a trabalhar na porta. A mansão Otwell tinha seis fechaduras em sua porta, protegendo os valiosos e raros tesouros que possuía dentro. As fechaduras não foram nenhum problema para os dedos ligeiros da Shauna. Ela foi abrindo todas, uma por uma. E a porta se abriu silenciosamente.

Shauna rastejou lentamente pelos pisos de madeira da entrada principal. Seu sapato de cinco dedos não fez nenhum barulho quando entraram em contato com o piso. Tudo em sua volta eram tesouros e quinquilharias. Peças com valores inestimáveis das antigas versões do Laravel. Troféus de batalhas de desenvolvimento e de cerimônias de premiação. Nenhum destes tesouros chamou a atenção de Shauna. Ela estava de olho no tesouro maior. E lá estava ele, com toda a sua glória!

Eu imagino que seja algo parecido com a Arca da Aliança dos filmes do Indiana Jones. Sabe? Uma grande caixa de ouro. Eu sei que você sabe.

Era o container. O coração do Laravel! Este é o mais valioso tesouro da mansão. Shauna deslizou até o container silenciosamente e colocou suas mãos sobre ele. Ela não podia conter sua excitação. Ela precisava saber o que tinha dentro.

Devagar, ela deslizou a tampa do container e olhou para as profundezas dele. Com uma explosão de luz e gritos, um grande número de entidades preencheu a sala. Funções anônimas e instâncias de objetos giravam em torno da mansão Otwell. Os gritos e os ventos eram tão violentos que a família Otwell foi acordada.

Taylor se inclinou sobre a varanda da mansão, segurando uma espingarda de cano serrado em cada mão.

“Shauna!? Você de novo? Saia já da minha casa! Simplesmente faça um clone do repositório do Laravel se você quiser ter o seu container!”

Ok, eu sei o que você deve estar pensando.

Dayle, você escreveu sobre a arquitetura do container no capítulo sobre Arquitetura.

Sim, eu escrevi. Mas você acha que lá tinha tudo o que há para você saber sobre o container? Que o misterioso coração pulsante do Laravel tinha apenas aquele truque? Não, meu caro leitor, você está muito enganado.

O container é a parte **mais importante** do framework. Ele junta todos os componentes do framework. Sem o container, não existiria o Laravel 4. Não existiria o Code Bright. Todo mundo estaria triste, e os desenvolvedores não seriam tão belos.

No capítulo que fala sobre arquitetura eu mencionei como o objeto `$app` estende a classe `Illuminate\Container\Container`. Eu expliquei como nós podemos armazenar objetos e valores utilizando uma sintaxe similar a de um array. O container se transforma em uma caixa mágica, que pode armazenar qualquer coisa que você precisar. Porém, ele é um pouco misterioso e pode ser um pouco confuso para quem está começando. Mais ou menos como a Arca da Aliança. Eu já mencionei o quanto eu gosto desses filmes?

Bem, vamos começar antes que eu receba um email do senhor Lucas me xingando.

Inversão de Controle

Você já deve ter ouvido o container ser chamado de ‘IoC’ container. Você consegue adivinhar o que ‘IoC’ significa?

Inversão de Controle (Inversion of Control, em inglês).

Não seja um espertinho, você já tinha visto no título desta seção, não é?! Se você não fosse tão legal eu teria que ficar brabo com você. Inversão de Controle é auto-explicativo, certo? Vamos em frete.

Estou brincando! Na verdade é um termo terrível. Eu não gosto quando os desenvolvedores inventam termos complicados para parecerem mais espertos do que eles realmente são. Isso realmente me irrita!

Vamos simplificar este termo. IoC basicamente significa:

- 1 Permitir que o framework monte as coisas para **nós**.

Nós damos ao framework uma receita para criar objetos e valores para nós. Isto significa que nós estamos transferindo (ou invertendo) o controle da instanciação destes objetos para o framework. Nós estamos confiando nele para montar a estrutura para nós.

Antes, eu expliquei como nós podemos utilizar a instância `$app` como um array para armazenar nossos objetos dentro do container. Bem, no Laravel nós também temos o facade `App` que pode ser utilizado para deixar o container (e o objeto da aplicação) disponível em qualquer local do framework.

Nós podemos utilizar o facade para armazenar e recuperar nossos objetos e valores de dentro do container, compartilhando eles entre as diversas classes para montar nossa aplicação.

Primeiro, vamos ver um exemplo sem utilizar o facade.


```
1  <?php
2
3  // app/routes.php
4
5  // Get hold of the app object.
6  $app = app();
7
8  // Bind the integer 3 into the container.
9  $app['three'] = 3;
10
11 // Output the response.
12 echo $app['three'];
```

Isto funciona perfeitamente bem. Nós recebemos o valor 3 da maneira que nós esperamos. Não é nem um pouco diferente do que trabalhar com um array normal.

Mas a verdade é que, o container está fazendo algo muito inteligente por debaixo dos panos. Vamos dar uma olhada na instanciação de um componente de SMS fictício. Normalmente, você vai armazenar os parâmetros em arquivos de configuração, mas colocar diretamente neste exemplo por questões de simplicidade.

```
1  <?php
2
3  // app/routes.php
4
5  // Get hold of the app object.
6  $app = app();
7
8  // Create a new SMS component.
9  $sms = new SMSClient;
10 $sms->setUsername('shauna');
11 $sms->setPassword('1_5t341_c4t5!');
12
13 // Bind the component.
14 $app['sms'] = $sms;
15
16 // Send an SMS.
17 $app['sms']->send('+55121212153', 'Yo dawg.');
```

Nós setamos o índice sms do nosso array \$app com a instância da nossa classe de SMS, mas por debaixo dos panos, o container do Laravel está fazendo algo muito mais inteligente por nós.

O container do Laravel implementa a interface `ArrayAccess`, que permite que uma classe funcione da mesma maneira que um array. Por exemplo, acessando diretamente seus valores pelos índices.

Ao implementar a interface `ArrayAccess`, você deve implementar dois métodos, `offsetGet()` e `offsetSet()`, para que o PHP saiba o que fazer quando aquela classe for utilizada como um array.

Vamos dar uma olhada no código fonte da classe do container. Sim, eu sei que é uma classe complicada, mas eu prometo que isso vai ser útil. Nós vamos olhar para os dois métodos que eu mencionei anteriormente, `offsetGet()` e `offsetSet()`.

```
1  <?php
2
3  // Container.php
4
5  /**
6   * Get the value at a given offset.
7   *
8   * @param string $key
9   * @return mixed
10  */
11  public function offsetGet($key)
12  {
13      return $this->make($key);
14  }
15
16  /**
17   * Set the value at a given offset.
18   *
19   * @param string $key
20   * @param mixed $value
21   * @return void
22   */
23  public function offsetSet($key, $value)
24  {
25      // If the value is not a Closure, we will make it one. This simply gives
26      // more "drop-in" replacement functionality for the Pimple which this
27      // container's simplest functions are base modeled and built after.
28      if ( ! $value instanceof Closure)
29      {
30          $value = function() use ($value)
31          {
32              return $value;
33          };
34      }
35
36      $this->bind($key, $value);
```

```
37 }
```

Confuso? Sem problemas. Deixe-me explicar. Vamos dar uma olhada no conteúdo do método `offsetGet()`.

```
1  <?php
2
3  // Container.php
4
5  return $this->make($key);
```

Aqui o container está dizendo, que quando você tentar acessar qualquer índice do array, para a classe utilizar o método `make()` com o índice que foi passado por parâmetro.

O método `make` pode ser utilizado para montar os objetos e valores que estão armazenados dentro no container. Ele faz algumas outras coisas bem espertas, mas nós vamos ver isto depois.

O método `offsetSet()` é um pouco mais confuso. Abaixo está ele se você estiver interessado.

```
1  <?php
2
3  // Container.php
4
5  if ( ! $value instanceof Closure)
6  {
7      $value = function() use ($value)
8      {
9          return $value;
10     };
11 }
12
13 $this->bind($key, $value);
```

Quando você tenta setar um valor dentro do container utilizando a sintaxe de array, primeiro é feita uma verificação para ver se o valor passado é uma função anônima (closure, em inglês). Caso não seja, é setado o valor passado dentro de uma closure que apenas retorna o valor original.

Por que isso? Bem, assim como o método `make()` é utilizado para buscar internamente os valores do container, o método `bind()` é utilizado para setar. O método `bind()` recebe uma função anônima como parâmetro, por isso que nós precisamos armazenar nossos valores em uma função anônima antes de passá-lo para o método `bind()`.

Uma vez que o valor está dentro de uma função anônima, é utilizado o método `bind()` para armazená-lo internamente no container.

Olhando para o nosso exemplo passo por passo, ficaria assim...

```
1  <?php
2
3  // app/routes.php
4
5  // Bind the component.
6  $app['sms'] = $sms;
```

... se transforma em...

```
1  <?php
2
3  // app/routes.php
4
5  $app['sms'] = function () use ($sms) {
6      return $sms;
7  };
```

... quando armazenado dentro do container.

Espero que você ainda esteja comigo. Agora nós sabemos que tudo que está dentro do container está dentro de uma função anônima. Então quando nós tentarmos pegar o componente de SMS de dentro do container...

```
1  <?php
2
3  // app/routes.php
4
5  $sms = $app['sms'];
```

Nós sabemos que uma função anônima está sendo executada internamente, e ela retorna uma instância do objeto \$sms.

Agora, chega do objeto \$app. Ele é confuso. Vamos utilizar o facade. Nós não podemos utilizar a sintaxe de array no facade, visto que ele é simplesmente um meio de acesso a instância do \$app. Mas ao invés disto, nós vamos utilizar os métodos bind() e make() diretamente.

Agora que nós já sabemos que o método bind() recebe uma função anônima, nós podemos utilizá-lo para armazenar nossos valores e objetos utilizando o facade. Vamos armazenar um número novamente.

```
1  <?php
2
3  // app/routes.php
4
5  // Bind an integer into the container.
6  App::bind('three', function () {
7      return 3;
8  });
9
10 // Retrieve the integer from the container.
11 echo App::make('three');
```

Na verdade nós não estamos fazendo nada de novo aqui. Nós simplesmente mascaramos nosso outro código por este novo, mais belo. O método `bind` recebe o índice que vai ser utilizado no container como o primeiro parâmetro, e como o segundo parâmetro, nós passamos a função anônima que retorna o nosso valor.

E então nós podemos utilizar o método `make()`, passando o índice do container que nós utilizamos para setar o valor, e nos é retornado o resultado da função anônima.

Vamos ver um exemplo parecido utilizando o nosso componente de SMS.

```
1  <?php
2
3  // app/routes.php
4
5  // Create a new SMS component.
6  $sms = new SMSClient;
7  $sms->setUsername('shauna');
8  $sms->setPassword('1_5t341_c4t5!');
9
10 // Bind the component.
11 App::bind('sms', function () use ($sms) {
12     return $sms;
13 });
14
15 // Send an SMS.
16 App::make('sms')->send('+55121212153', 'Yo dawg.');
```

Primeiro nós criamos o componente. Depois, nós chamamos o método `bind` passando o índice e a função anônima. E finalmente, nós buscamos o objeto do container passando o índice para o método `make`.

Porém, existe um problema aqui. Você já conseguiu identificá-lo? Ele pode ser um pouco difícil de identificar, mas você precisa pensar no fluxo de execução de sua aplicação para descobrir.

Nós estamos criando a instância do SMS e passando ele para o container. Mas e se nós não quisermos utilizar este objeto em todas as requisições. É muito provável que apenas algumas rotas enviem mensagens SMS. Em todas as outras requisições, nós apenas estamos desperdiçando tempo e recursos criando o componente de SMS.

E é aqui que a função anônima se torna útil. A função anônima dentro do container apenas é executada quando você utiliza o método `make` do container.

Então por que nós não passamos toda a criação do componente de SMS para dentro da função anônima? Vamos ver como nós podemos fazer isto.

```
1  <?php
2
3  // app/routes.php
4
5  // Bind the component.
6  App::bind('sms', function () {
7
8      // Create a new SMS component.
9      $sms = new SMSClient;
10     $sms->setUsername('shauna');
11     $sms->setPassword('1_5t341_c4t5!');
12
13     // Return the created SMS component.
14     return $sms;
15 });
16
17 // Send an SMS.
18 App::make('sms')->send('+55121212153', 'Yo dawg.');
```

Não é uma grande alteração, não é? Nós simplesmente colocamos a criação do componente de SMS para dentro da função anônima. Nós estamos invertendo os controles aqui. Nós estamos dando ao framework a habilidade de criar o componente de SMS apenas quando for necessário. Nós sabemos que a instância do `$sms` vai apenas ser criada quando nós chamarmos o método `App::make('sms')`.

Viu, inversão de controle não é tão assustador quanto parece, não é? Talvez devesse ser chamado ‘Permissão para que o framework crie coisas para você apenas quando você precisa delas’. Muito melhor!

Existe, porém, uma falha final ao armazenar nosso componente de SMS. Se você já descobriu, vá em frente e sirva-se de algumas das estrelas douradas do Dayle. Elas tem o gosto de plástico com brilho.

Dê uma olhada no exemplo abaixo, você consegue ver o problema?

```
1  <?php
2
3  // app/routes.php
4
5  // Bind the component.
6  App::bind('sms', function () {
7
8      // Create a new SMS component.
9      $sms = new SMSClient;
10     $sms->setUsername('shauna');
11     $sms->setPassword('1_5t341_c4t5!');
12
13     // Return the created SMS component.
14     return $sms;
15 });
16
17 // Send an SMS.
18 App::make('sms')->send('+55121212153', 'Yo doge. ');
19
20 // Send another SMS.
21 App::make('sms')->send('+55121223432', 'SUCH TEXT. WOW');
```

Aqui, nós adicionamos uma segunda linha para enviar outra mensagem de texto. Isto não deve ser um problema, certo? Bem, nosso código vai funcionar corretamente. O problema é que ele não é muito eficiente. Para descobrir a falha, nós precisamos pensar sobre o fluxo de execução novamente.

Nós armazenamos a criação do componente de SMS dentro do container. Depois, nós resolvemos o componente para enviar uma mensagem de texto, e depois fazemos isso novamente.

Nós resolvemos o componente duas vezes. Nós executamos a função anônima duas vezes. Isto significa que nós estamos instanciando o componente de SMS duas vezes. Nós estamos setando as credenciais duas vezes. Está cansado de ouvir duas vezes? Eu também estou começando a cansar de escrever.

Deixe eu tirar isto para fora de mim. DUAS VEZES DUAS VEZES DUAS VEZES DUAS VEZES.

Certo, agora nós estamos bem. Esta resolução dupla é um problema. É um desperdício, não é? O componente é utilizado exatamente da mesma maneira na segunda vez, com as mesmas credenciais. Por que nós estamos nos incomodando de instanciá-lo novamente? É um desperdício de memória e de processamento do servidor.

Provavelmente o esperto e poderoso Otwell pensou em alguma saída para este problema. E, claro que ele pensou. E como sempre, o Laravel vai te salvar.

Nós podemos utilizar o método mágico `share()` do container. Ele permite que o resultado de uma função anônima seja colocado no cache do container, na requisição atual, e assim qualquer próxima

resolução que for necessária vai retornar o valor contido no cache, sem executar novamente a função anônima.

Vamos dar uma olhada em como isto pode ser usado.

```
1  <?php
2
3  // app/routes.php
4
5  // Bind the component as a singleton.
6  App::singleton('sms', function () {
7
8      // Create a new SMS component.
9      $sms = new SMSClient;
10     $sms->setUsername('shauna');
11     $sms->setPassword('1_5t341_c4t5!');
12
13     // Return the created SMS component.
14     return $sms;
15 });
16
17 // Send an SMS.
18 App::make('sms')->send('+55121212153', 'Yo doge. ');
19
20 // Send another SMS.
21 App::make('sms')->send('+55121223432', 'SUCH TEXT. WOW');
```

Tudo o que nós precisamos é trocar o método `bind()` pelo método `singleton()`, que possui exatamente os mesmos parâmetros. Você já deve ter ouvido falar do padrão de projeto ‘Singleton’. Ele garante que exista apenas uma instância de uma classe, e não permite que ela seja duplicada. Este aqui não é um verdadeiro singleton, mas possui um processo similar. A função anônima vai ser executada apenas uma vez e você vai receber a mesma instância do componente de SMS todas as vezes.

E por que eu deveria acreditar em você?

Mas você está mal-humorado hoje, hein? Você pensou que eu mentiria pra você? Bem, eu não um bom histórico, né? Mas deixe-me provar para você.

Vamos voltar aos números! Dê uma olhada no seguinte código.


```
1 <?php
2
3 // app/routes.php
4
5 App::bind('rand', function () {
6     return rand();
7 });
8
9 var_dump(App::make('rand'));
10 var_dump(App::make('rand'));
11 var_dump(App::make('rand'));
```

Nós sabemos que a função `rand()` do PHP pode ser usada para gerar um número randômico. Vamos executar este código e ver o que nos é retornado.

```
1 int 1742870120
2 int 385800925
3 int 1573429337
```

Wow. Muito randômico. Quantos números. É exatamente o que nós esperávamos. Utilizando o método `bind()`, a função anônima é executada todas as vezes em que o método `make()` é chamado. E cada vez, nós recebemos um novo número.

Vamos tentar utilizando o método `singleton()`?

```
1 <?php
2
3 // app/routes.php
4
5 App::singleton('rand', function () {
6     return rand();
7 });
8
9 var_dump(App::make('rand'));
10 var_dump(App::make('rand'));
11 var_dump(App::make('rand'));
```

É uma pequena alteração, mas eu acho que vai mudar muito o resultado. Vamos dar uma olhada nos valores retornados.

```
1 int 903162162
2 int 903162162
3 int 903162162
```

A função anônima é executada apenas uma vez. O número 903162162 que nós recebemos da função `rand()` foi armazenado no cache, e é retornado em todas as próximas chamadas do método `make()`.

Quando nós utilizamos o container, nós precisamos decidir quais componentes devem ser compartilhados ou não. As vezes, nós precisamos um novo componente instanciado todas as vezes, e as vezes, isto é simplesmente um desperdício de recursos. Como sempre, faça o seu melhor julgamento como desenvolvedor para escolher o mais apropriado para cada situação.

Você acha que nós já exploramos todos os mistérios do container? Claro que não! Na verdade eu tenho mais um seção inteira dedicada ao meu ‘truque’ favorito do container. Prepare-se para ficar maravilhado!

Injeção de Dependências

Ah não! Outro termo assustador, né? Se você está começando a programar orientado a objetos você já deve ter ouvido falar desta técnica. Você deve estar pensando “Ah não, é aquela coisa relacionada a testes!”. Eu sei que testar o código pode ser intimidador, mas mesmo sem escrever nenhum teste, eu prometo que injeção de dependências (Dependency Injection, em inglês) pode ser muito útil para você. Não vai ser assustador. Eu prometo!

Uma vez eu vi uma citação sobre injeção de dependências que eu gostei muito. Infelizmente, eu não consigo me lembrar quem falou. Eu gosto de pensar que o autor semi anônimo gostaria que eu utilizasse dela, então aqui vai.

Injeção de dependências... ou como eu gosto de chamar.. ‘setar coisas’. **Desenvolvedor Filósofo Anônimo.**

É fantástico porque é absolutamente verdade. E aqui está mais um exemplo dos desenvolvedores utilizando nomes complicados, para fazer a técnica parecer muito mais complicada e avançada do que ela realmente é.

Vamos começar com um pequeno exemplo. Antes de começarmos com injeção de dependências, vamos dar uma olhada na resolução baseada em reflexão (reflection, em inglês) utilizando o container. Vamos dar uma olhada no exemplo que vai ajudar a clarear este termo horrível.

```
1 <?php
2
3 // app/routes.php
4
5 $collection = App::make('Illuminate\Support\Collection');
```

Este código não é muito útil, pois ele apenas nos retorna uma coleção vazia. Por que isso? Bem, eu não lembro de ter adicionado nenhum item no container, você lembra?

Mas o container é muito esperto. Se nós tentarmos resolver um índice do container que não existe, e uma classe existir com este nome, então obviamente o que nós queremos é a classe, certo?

O container fala pra si mesmo “Hey, uma definição de classe `Illuminate\Support\Collection` aqui, por que eu não crio uma nova instância dela?”.

Esta conversa não está realmente dentro do código, infelizmente. Alguém bem que podia enviar um pull request para isto.

Vamos dar uma rápida olhada nos parâmetros da classe `Collection`.

```
1 <?php
2
3 // Collection.php
4
5 public function __construct(array $items = array())
```

Você pode passar um array de itens para o construtor para setar os itens iniciais da coleção. Porém, ele possui um valor padrão, um array vazio.

Ao resolver classes pelos seus nomes utilizando o container, ele vai utilizar a API de reflexão do PHP para verificar quais os parâmetros necessários para criar uma nova instância daquele objeto. Visto que o parâmetro possui um valor padrão, a classe pode ser instanciada sem passar nenhum parâmetro.

Vamos criar a nossa própria classe para ver o que acontece quando nossos parâmetros não possuem um valor padrão.

```
1  <?php
2
3  // app/routes.php
4
5  // Uma classe de exemplo, que recebe uma instância de 'something'.
6  class Example
7  {
8      protected $something;
9
10     public function __construct($something)
11     {
12         $this->something = $something;
13     }
14 }
15
16 // Tentando resolver a classe Example pelo container.
17 $example = App::make('Example');
```

Aqui nós temos uma classe `Example` que recebe uma instância de `$something` em seu construtor. Nós estamos injetando esta dependência dentro da classe. Nós estamos ‘setando’. E é isto que injeção de dependências é.

Infelizmente, o container não faz a mínima ideia do que o parâmetro `$something` é. Então, o que acontece quando nós executamos este código? Bem, nós recebemos um erro. Recebemos a exceção `Illuminate\Container\BindingResolutionException` com a mensagem ‘Unresolvable dependency resolving [Parameter #0 [

Pelo menos ele tentou, certo? Então como nós podemos ajudar o container com este problema? Bem, a melhor coisa que nós podemos fazer é descrever qual instância que nós estamos injetando. Nós precisamos que o container saiba qual a classe que nós precisamos.

Nós podemos fazer isto utilizando uma técnica chamada de ‘type-hinting’ nos parâmetros. Vamos criar a classe `Something`, e depois dizer que o parâmetro `$something` que a classe `Example` recebe deve ser uma instância da classe `Something`. Parece complicado, mas fica muito mais fácil com um exemplo!

```
1  <?php
2
3  // app/routes.php
4
5  // A Something class.
6  class Something
7  {
8      // ...
9  }
10
11 // An example class, injecting a 'something' instance.
12 class Example
13 {
14     protected $something;
15
16     public function __construct(Something $something)
17     {
18         $this->something = $something;
19     }
20 }
21
22 // Attempt to resolve the example class by name from the container.
23 $example = App::make('Example');
```

Desta vez, a classe `Example` foi instanciada corretamente. Mas como isto funciona?

Bem, o container olhou quais os parâmetros necessários para instanciar a classe `Example`, e percebeu que ela necessita de uma instância de classe `Something`. Com isto, ele simplesmente criou uma nova instância da classe `Something` e passou por parâmetro para a classe `Example`. Aqui, o Laravel gerenciou a nossa hierarquia de dependências.

E o que é mais impressionante neste processo, é que tudo isto acontece recursivamente. Vamos dar uma olhada em um exemplo um pouco mais complicado.

```
1  <?php
2
3  // app/routes.php
4
5  class First
6  {
7      public function __construct(Second $second) {}
8  }
9
```

```
10 class Second
11 {
12     public function __construct(Third $third, Fourth $fourth) {}
13 }
14
15 class Third
16 {
17     public function __construct(
18         Fourth $fourth,
19         Illuminate\Container\Container $container
20     ) {}
21 }
22
23 class Fourth
24 {
25     public function __construct(array $myArray = array()) {}
26 }
27
28 $first = App::make('First');
```

Aqui nós temos 4 níveis em nossa hierarquia de dependências, que o container vai instanciar para nós com muito prazer utilizando o método `make()`. Eu não vou passar por cada etapa do exemplo, mas como uma pequena lição de casa, por que você não tenta identificar o seguinte:

- Injeção dos quatro níveis de classes.
- Injeção de um valor padrão.
- Injeção de múltiplos parâmetros.
- Injeção do container do Laravel

E por que você não tenta desenhar um diagrama de dependências? Mande um tweet para @daylerees com ele e eu dou a nota! :)

A melhor parte disto, é que a maioria das classes utilizadas pelo framework são resolvidas utilizando o container.

Em qualquer lugar da aplicação onde você especifica o par de ‘classe@ação’ no seguinte formato...

```
1 Some\Namespaced\Class@andMethod
```

... a classe vai ser resolvida pelo container. Isto significa que qualquer dependência, e qualquer dependência das dependências vão ser resolvidas automaticamente. Seus controladores, filtros e qualquer outra classe podem tirar vantagem deste mecanismo.

No próximo capítulo nós vamos criar provedores de serviço (Service Providers, como é chamado no Laravel) para manipular e configurar bibliotecas de terceiros.

Em Breve

Hey, cadê o resto do meu livro?

Como eu claramente escrevi na página de descrição do livro, ele foi publicado durante sua escrita. Se você está vendo esta página, significa que ele ainda não está pronto.

Mas não se preocupe, eu tenho grandes planos para este livro. Eu vou continuar escrevendo capítulos, e adicionar correções e atualizações até eu considere que o livro cubra tudo o que você precisa sobre o framework. Todos os futuros capítulos e atualizações estarão disponíveis gratuitamente para quem comprou este livro. Apenas acesse o Leanpub quando você receber um e-mail sobre com as atualizações. Simples assim!

Eu também gostaria de aproveitar este tempo e agradecer cada um de vocês por apoiar minha escrita. Eu realmente gostei de escrever os dois livros, e pretendo escrever mais no futuro. Se vocês não tivessem me apoiado, e enviados emails com feedback, eu provavelmente não teria achado este belo hobby (trabalho?) meu.

Se o Code Happy e o Code Bright ajudaram você, então eu realmente vou apreciar se você compartilhar a URL do livro com seus amigos. É <https://leanpub.com/codebright-ptbr>³⁰, caso você tenha perdido. :)

Meu plano para o livro é o seguinte.

- Descrever os conceitos e componentes básicos do framework em detalhes.
- Escrever um capítulo ‘construa-uma-aplicação’ para uma aplicação simples, construída com o que nós aprendemos.
- Continuar explorando os recursos avançados do framework.
- Mais capítulos ‘construa-uma-aplicação’ baseados em funcionalidades avançadas.
- Melhores práticas e dicas úteis.

Eu também estou considerando um capítulo de perguntas e respostas baseado no retorno que eu recebi deste livro, então se você tem alguma pergunta que não vai ser respondida por algum futuro capítulo, me avise.

Qualquer coisa que você queira conversar sobre você pode entrar em contato comigo pelo email me@daylerees.com³¹ ou [daylerees no Twitter](#)³². Você também pode me encontrar no canal #laravel

³⁰<https://leanpub.com/codebright-ptbr>

³¹<mailto:me@daylerees.com>

³²<http://twitter.com/daylerees>

no Freenode IRC, mas por favor não fique ofendido se eu não responder na hora, pois eu tenho um trabalho durante o dia também!

Obrigado novamente por ser parte do Code Bright.

Com amor,

Dayle e seu exército de pandas vermelhos.

xxx